BestBuy Books Project Documentation

Generated on: 2026-03-14 21:04:15

Project Overview

BestBuy_Books is an e-commerce platform built with Laravel framework supporting multi-role users (Administrators, Merchants, Customers) for managing books, orders, chats, and issue reports. The project implements real-time communication between users and administrators, merchant registration approval process, and role-based access control.

Directory Structure

```
■■ BestBuy_Books - Copy/
■    ■■ .editorconfig
■    ■■ .env
■    ■■ .env.example
■    ■■ .gitattributes
■    ■■ .gitignore
■    ■■ .styleci.yml
■    ■■ artisan
■    ■■ CHANGELOG.md
■    ■■ composer.json
■    ■■ composer.lock
■    ■■ generate_pdf.py
■    ■■ generate_pdf_optimized.py
■    ■■ generate_project_pdf.py
■    ■■ jsconfig.json
■    ■■ package-lock.json
■    ... and more files
■    ■■ .git/
■    ■    ■■ COMMIT_EDITMSG
■    ■    ■■ config
■    ■    ■■ description
■    ■    ■■ HEAD
■    ■    ■■ index
■    ■    ■■ ORIG_HEAD
■    ■    ■■ packed-refs
■    ■    ■■ hooks/
■    ■    ■    ■■ applypatch-msg.sample
■    ■    ■    ■■ commit-msg.sample
■    ■    ■    ■■ fsmonitor-watchman.sample
■    ■    ■    ■■ post-update.sample
■    ■    ■    ■■ pre-applypatch.sample
■    ■    ■    ■■ pre-commit.sample
■    ■    ■    ■■ pre-merge-commit.sample
■    ■    ■    ■■ pre-push.sample
■    ■    ■    ■■ pre-rebase.sample
■    ■    ■    ■■ pre-receive.sample
■    ■    ■    ■■ prepare-commit-msg.sample
■    ■    ■    ■■ push-to-checkout.sample
■    ■    ■    ■■ sendemail-validate.sample
■    ■    ■    ■■ update.sample
■    ■    ■■ info/
■    ■    ■    ■■ exclude
■    ■    ■    ■■ refs
■    ■    ■■ logs/
■    ■    ■    ■■ HEAD
■    ■    ■    ■■ refs/
■    ■    ■    ■    ■■ heads/
■    ■    ■    ■    ■    ■■ 12.x
■    ■    ■    ■    ■    ■■ main
■    ■    ■    ■    ■■ remotes/
■    ■    ■    ■    ■    ■■ origin/
■    ■    ■    ■    ■    ■    ■■ HEAD
■    ■    ■    ■    ■    ■    ■■ main
■    ■    ■■ objects/
■    ■    ■    ■■ 02/
■    ■    ■    ■    ■■ f39f492c6ba85bd27580cf456baa038b9684f1
```

```
■    ■    ■    ■■ 03/
■    ■    ■    ■    ■■ 413767ed9981cf7bdcf6f060622080ba86cd62
■    ■    ■    ■    ■■ d76ecc0c854b3b90db68e1c8bb0fd1a7b17d98
■    ■    ■    ■■ 04/
■    ■    ■    ■    ■■ 463798a2eeec60d99908a19061369823149408
■    ■    ■    ■    ■■ f3ad109065671c3b47e17994da0bda3bfed240
■    ■    ■    ■■ 06/
■    ■    ■    ■    ■■ 565a15d5371a9ee855a24e57baa51f6a25c291
■    ■    ■    ■■ 07/
■    ■    ■    ■    ■■ 0e39ec16038738e70844e42724472ddf4d603a
■    ■    ■    ■    ■■ ba3337733c41881f35073e7e2635344757b2ad
■    ■    ■    ■■ 08/
■    ■    ■    ■    ■■ bfefaf5e26d94b9d0efe1616dd8d9789ddd92d
■    ■    ■    ■■ 09/
■    ■    ■    ■    ■■ 51b27e00ebcc1eb04df52ca31892e24ae65fab
■    ■    ■    ■    ■■ b1578dd3db1e0881b073c2f01083308bf2c07f
■    ■    ■    ■    ■■ cb7006a309676414e6aff583339584bcab3d47
■    ■    ■    ■■ 0a/
■    ■    ■    ■    ■■ 2aa7e459bc58e7e6b01913f2821563390b3141
■    ■    ■    ■    ■■ eb520f54efd7acb94d540e410117ae0bc27ada
■    ■    ■    ■■ 0c/
■    ■    ■    ■    ■■ 5afcbcd15285ac36af761196424210bf3f3ddc
■    ■    ■    ■    ■■ 5dd9dca723cb8c8d0503fb16f30d619cb60cbb
■    ■    ■    ■■ 0d/
■    ■    ■    ■    ■■ 593e8c1a34656c96e4f7ede0c2a2b6d44e7705
■    ■    ■    ■    ■■ daa664404faa031a2ec18da94f1574acc0e1de
■    ■    ■    ■■ 0e/
■    ■    ■    ■    ■■ 58c32fe810a72a486b3f9d41e74fe397db555c
■    ■    ■    ■■ 0f/
■    ■    ■    ■    ■■ 00accc49e455624b3365cbdf32c9c28e45314f
■    ■    ■    ■■ 10/
■    ■    ■    ■    ■■ 5bde0fd05cf767f5d7af762485fd10d31c6085
■    ■    ■    ■■ 11/
■    ■    ■    ■    ■■ 5d615be08e70cfeb5e81fdccf6fc1b498d5ed2
■    ■    ■    ■■ 12/
■    ■    ■    ■    ■■ dd58447dcc561bd7a51c05ceaa16377e7ccd5b
■    ■    ■    ■■ 13/
■    ■    ■    ■    ■■ 05196626978ecdc11946b48dd6699de0c7a4f7
■    ■    ■    ■    ■■ 7852f4c354a8ccaea68a87939f66c7e3591b6c
■    ■    ■    ■    ■■ a5fa40e44e8eb40298c2b9ac51f110bb5edea0
■    ■    ■    ■    ■■ dcb7ce7eb6c9d3a75bc95ddfdcd024337f1418
■    ■    ■    ■■ 14/
■    ■    ■    ■    ■■ 89d0e0f1d7b358509b0226a279d62081c11410
■    ■    ■    ■    ■■ b73c704521c7a790eb7a560bd2b0f425c53e91
■    ■    ■    ■■ 16/
■    ■    ■    ■    ■■ 0fb623d1627047d23015441c5f5af199afe001
■    ■    ■    ■    ■■ 5e4f3dca9a6862c328dcb42f15614e73a561bb
■    ■    ■    ■■ 17/
■    ■    ■    ■    ■■ 833d33c06f53d23d49df064c90207c3e7ea5e0
■    ■    ■    ■    ■■ a1bd7a6a24d8b44476d965ff7417293050ac2d
■    ■    ■    ■■ 18/
■    ■    ■    ■    ■■ a1d45defc0202ce9e940318da3e99d8ead351d
■    ■    ■    ■    ■■ f6b83584a62601e9b6a8c4295fd62159571d36
■    ■    ■    ■■ 19/
■    ■    ■    ■    ■■ 5e14c2e4745fd855a029405ec60b6987dea8ae
■    ■    ■    ■    ■■ ae3e3639cec7d1a86c8ae4c6bf505bff8d9a16
■    ■    ■    ■    ■■ e56b74941a88148ed121018fb176b0e1f2d57d
■    ■    ■    ■    ■■ f290861d160514a70ff12148612ff44be8ebbe
■    ■    ■    ■■ 1a/
■    ■    ■    ■    ■■ ceac01e810c1d68e05641f8eb24abb393ec944
■    ■    ■    ■■ 1b/
■    ■    ■    ■    ■■ 2a4521ef9f5ef0b7595682e9ff37ce83e9bb28
■    ■    ■    ■■ 1c/
■    ■    ■    ■    ■■ 3e1536922f51a07e91337d3ae047de90b69710
■    ■    ■    ■■ 1e/
■    ■    ■    ■    ■■ 7a7c6501e5656272f4a3e23639f194b23971f3
■    ■    ■    ■■ 1f/
■    ■    ■    ■    ■■ 3502c5e23899a613a7821ccd8f392c38ec0dbd
■    ■    ■    ■■ 20/
■    ■    ■    ■    ■■ a90cf0dc75c1b9b410be6d679848e2a5c9b4d6
■    ■    ■    ■■ 21/
■    ■    ■    ■    ■■ 097df19065ca6535c28dd82e0faef8244fc330
■    ■    ■    ■■ 22/
■    ■    ■    ■    ■■ dfe5265db4da856b65c828c89ea49c62a1c7d8
■    ■    ■    ■■ 23/
■    ■    ■    ■    ■■ 55c492409c6b7f4338820676ce18c7f25520cd
■    ■    ■    ■    ■■ 7ae2504312a352049847ce3889c7126baede93
■    ■    ■    ■■ 24/
■    ■    ■    ■    ■■ 3f9d1dc6751c5fd64dacf4def7be51cddf5744
■    ■    ■    ■    ■■ be0215a8e0bda4c80356b0a8ae1abcfd0de6dc
■    ■    ■    ■■ 25/
```

■ ■ ■ ■ ■■ 173dc980a616040b7cd8d8efbc432dd8bf6700
■ ■ ■ ■ ■■ 746424580a1206e6995f034ecbf8b9cc7cd607
■ ■ ■ ■ ■■ 7b6f3ac68c675cc6434aa7af8fdada7c3d2fa1
■ ■ ■ ■■ 26/
■ ■ ■ ■ ■■ 5bb0e742b7b0b2852638c11400d4316a770a1c
■ ■ ■ ■■ 29/
■ ■ ■ ■ ■■ ac8a17ac763983f3c799d60d3a123bb2e61c6c
■ ■ ■ ■■ 2a/
■ ■ ■ ■ ■■ 0c89f199544e7fa6c9d5200ba367725d1ad2bb
■ ■ ■ ■■ 2b/
■ ■ ■ ■ ■■ 50391ef5e4175c312cf5fbe9e643aa02c18c5c
■ ■ ■ ■ ■■ a35d0ee6c5e3eb40c766eafee5db51ea85bf25
■ ■ ■ ■■ 2f/
■ ■ ■ ■ ■■ 6f21b8671c3be04d589237645db5f8bc25757c
■ ■ ■ ■ ■■ d0d9003dcfbb293f801ffdc97c4f134106748d
■ ■ ■ ■■ 32/
■ ■ ■ ■ ■■ 734e7e98caaaee8a89b3bd2795fb3064ae17d4
■ ■ ■ ■ ■■ 8a96a446daddf375b04ab3c82b1d5e6599d422
■ ■ ■ ■ ■■ e586b3ac6550c27a325b647bf4d02df4838961
■ ■ ■ ■■ 33/
■ ■ ■ ■ ■■ cf5363236b6cdb798416a2de3bc009db9091cd
■ ■ ■ ■■ 34/
■ ■ ■ ■ ■■ 42ab935f2013aaa51b8c42967f04b9c5bdcdfd
■ ■ ■ ■ ■■ a31feb16a6e9dc5495823793cb4d217394192d
■ ■ ■ ■ ■■ cc58fb2311fe3e7dec52aa73ab811251957598
■ ■ ■ ■■ 35/
■ ■ ■ ■ ■■ 0f9da9d9469c62a012da7bd539c6983e206349
■ ■ ■ ■ ■■ 20d0438e169e9924f24790f26151a947e9dd26
■ ■ ■ ■■ 36/
■ ■ ■ ■ ■■ 22a8f37c2accfa385618240df70ac74fea342a
■ ■ ■ ■■ 38/
■ ■ ■ ■ ■■ 67f225c7425a2ed06c23c1f5b883e449dae51d
■ ■ ■ ■ ■■ 71c98187defa540d4507393be4372f3e725da9
■ ■ ■ ■■ 39/
■ ■ ■ ■ ■■ 26ecf72a8b0ceb1d796215be602d6b6accf846
■ ■ ■ ■ ■■ 4cc4af4690c25742488bcdc387625172671646
■ ■ ■ ■■ 3b/
■ ■ ■ ■ ■■ 1e4b7df1a4dc0044fe18ea23d9522954664b94
■ ■ ■ ■ ■■ 4723849f81e2b5a579a2feee9b168e3b264441
■ ■ ■ ■■ 3c/
■ ■ ■ ■ ■■ 04917c6297beebd2b891dd8d0afdd8d1b67a44
■ ■ ■ ■■ 3d/
■ ■ ■ ■ ■■ 0eb93d883d9ca3a5f4a7afe382b8416d889292
■ ■ ■ ■ ■■ a8b946dc65904b99a3a8df2e5583b2fda489c5
■ ■ ■ ■ ■■ f32f942cf7f250a77d4a15840df83fa291f70e
■ ■ ■ ■■ 3f/
■ ■ ■ ■ ■■ 07a199cee2cfcb12a2b8528967a02cc149ccc9
■ ■ ■ ■■ 41/
■ ■ ■ ■ ■■ ecf05abf23b96e9f4e61ba48a225d4fec5cc4a
■ ■ ■ ■ ■■ f13c40151b764f5d7b343a29ecf791078aa20e
■ ■ ■ ■■ 42/
■ ■ ■ ■ ■■ 0e5793b32ba6641b2f0fb22fe1f784143232c9
■ ■ ■ ■ ■■ 525794debcfb9a89a2f9e2032f73c15a68168f
■ ■ ■ ■■ 43/
■ ■ ■ ■ ■■ ea815cbb839b6a8a5529d7d74d3c328f59c30b
■ ■ ■ ■■ 44/
■ ■ ■ ■ ■■ f58bf3bfd02cc65a01d8a27ddaf6a4abd5a5ba
■ ■ ■ ■■ 45/
■ ■ ■ ■ ■■ 73162568490eece6f91b1140c76f10378f6455
■ ■ ■ ■ ■■ aae23d9e5bc755664f09982176022bfb874fff
■ ■ ■ ■■ 47/
■ ■ ■ ■ ■■ e0bcae7db46ee12fab0b37600b449f49cd388c
■ ■ ■ ■■ 48/
■ ■ ■ ■ ■■ 2d6ebf4624f4429e418eb4587847c9cb19a349
■ ■ ■ ■ ■■ 332e8decff1882df027fcee679df0c04e5b475
■ ■ ■ ■ ■■ 5edc7cbcb951a33294a81707ac4ed49f685186
■ ■ ■ ■■ 49/
■ ■ ■ ■ ■■ 79fa9181c0c3c83be522d899b0e5b739d8c9a4
■ ■ ■ ■ ■■ 7daba9d275beaadb61add8fa86958ef4fce822
■ ■ ■ ■ ■■ 886c3e90a7d6463222afe15d33dfeb9359fe31
■ ■ ■ ■■ 4b/
■ ■ ■ ■ ■■ ba803e60517f4597280d42f006da8ddf31b081
■ ■ ■ ■■ 4c/
■ ■ ■ ■ ■■ 66ab677a37eb2ade2824ede0b9d3f65b866232
■ ■ ■ ■■ 4e/
■ ■ ■ ■ ■■ 02e4a5c211ab9a27a7e38de0c1acd8bbdff705
■ ■ ■ ■ ■■ 7016660c9c905914a5c8c4f7953c90b7dafc20
■ ■ ■ ■■ 4f/
■ ■ ■ ■ ■■ a3a83c440dd6cf8e2009a5db55fca9dfee3462
■ ■ ■ ■ ■■ eda76c957177bccbdc8bfde2d83ddc0e0d65d5
■ ■ ■ ■■ 50/

■ ■ ■ ■ ■■ 39051a42f8cd76266ed674d329946a0fe43db5
■ ■ ■ ■■ 52/
■ ■ ■ ■ ■■ 09c5ca969e3bd75e0b7c8426682b557f7138e1
■ ■ ■ ■ ■■ a920516ef96180d7e6a4d09bb504c97bc4613f
■ ■ ■ ■■ 55/
■ ■ ■ ■ ■■ 1eedbd27d7729c0fca4778131aa44b9e6d8efe
■ ■ ■ ■■ 56/
■ ■ ■ ■ ■■ f47e7aa90f83ad136741fcd09ac2987642c9ab
■ ■ ■ ■■ 57/
■ ■ ■ ■ ■■ a82b58e0ca6e3140a3c2472408731327f43d94
■ ■ ■ ■ ■■ e2f7afee32eebfb826c5ed425ee6f19ee61a8c
■ ■ ■ ■ ■■ ea18c75664fb18703d053e190ba9444ac29315
■ ■ ■ ■■ 58/
■ ■ ■ ■ ■■ 76c49c28baa628e03b5a6055132b4da5357269
■ ■ ■ ■ ■■ e1959a710d841e91399eb9414e0dd67b1cf85d
■ ■ ■ ■■ 5a/
■ ■ ■ ■ ■■ d730b405ab24f6453e086d0b39961ba79eb301
■ ■ ■ ■■ 5b/
■ ■ ■ ■ ■■ 800412677c2c4a3085367d5b177438ec4f8ea8
■ ■ ■ ■■ 5d/
■ ■ ■ ■ ■■ d371c2c4d8ec4bbc290dd1352198c22291d0a0
■ ■ ■ ■■ 5e/
■ ■ ■ ■ ■■ b9326ed748b8c0f683f3c119e653998ada26bc
■ ■ ■ ■■ 5f/
■ ■ ■ ■ ■■ 037ef4dccaec082c63fe83736e9a7ea0bb783f
■ ■ ■ ■■ 60/
■ ■ ■ ■ ■■ 9a88fff9d0f80199dd2c66c6bd340b6fa9dc0e
■ ■ ■ ■■ 61/
■ ■ ■ ■ ■■ 7a9b9c6ce57d155ac8aaede47996fbc157e451
■ ■ ■ ■ ■■ e0bca70fd2f09553fe1ce5c23562fa1d4238df
■ ■ ■ ■■ 62/
■ ■ ■ ■ ■■ 693547966b41887f52722877147db6f1f83823
■ ■ ■ ■ ■■ 96801a4b2d8524359b72762c2cbcce35c6b405
■ ■ ■ ■■ 63/
■ ■ ■ ■ ■■ 17c4e0258a8e27b15ca8f9b813c38b986a3884
■ ■ ■ ■■ 64/
■ ■ ■ ■ ■■ 7f0be5776e155fdd39123714fefb22cded7dbb
■ ■ ■ ■ ■■ b9bdd7138ace5359f9b14e89b19994022b62d0
■ ■ ■ ■■ 68/
■ ■ ■ ■ ■■ 7ac841ce19250fb2cff70247ff081a1cf97267
■ ■ ■ ■ ■■ 86e110af24e89a2ede4c57429e82e45757b7f6
■ ■ ■ ■■ 69/
■ ■ ■ ■ ■■ 2c5a8c065a74968e09163864d8ccbb2f35bf97
■ ■ ■ ■■ 6a/
■ ■ ■ ■ ■■ 2cf8d4f34aefa399ab5430aaed3ecb9fc843e0
■ ■ ■ ■■ 6b/
■ ■ ■ ■ ■■ 337eb84891970de554a5dd05a863a4e4d91404
■ ■ ■ ■■ 6c/
■ ■ ■ ■ ■■ 58b2c140ed78392080b4e262c1ac780e1e7a77
■ ■ ■ ■ ■■ e33faa01e5f2f13d1064b9e218a43d8791a068
■ ■ ■ ■■ 6d/
■ ■ ■ ■ ■■ 9c3b7bacc984241436aee7000a5b27f276480a
■ ■ ■ ■■ 70/
■ ■ ■ ■ ■■ 5570b43f3a048623759a159020d7ee0739cc57
■ ■ ■ ■ ■■ c858826864b46ce5fde9be54a877ae0456c836
■ ■ ■ ■ ■■ fe203ed3722e44b650f98db9fc04602dc8a729
■ ■ ■ ■■ 71/
■ ■ ■ ■ ■■ 1839d0a8a29e2da42ca5426434445769e6bb49
■ ■ ■ ■ ■■ 23b17f99b7c773a18e31f057904a9d144b7ebd
■ ■ ■ ■■ 75/
■ ■ ■ ■ ■■ 0a5770cbf714168ec1291e28ef94e5eb58460f
■ ■ ■ ■ ■■ 1306afe1b05cb5cf75b2d2919238d519f335e4
■ ■ ■ ■ ■■ 2bd109ae4f67fc0976f8ae1ede1d9dfaf0e71b
■ ■ ■ ■ ■■ 60cffeba5c5d6a4e7570a42bf1d12715e4b6d8
■ ■ ■ ■ ■■ d00519c3e93c56718b7ef66822fe7a5ca4839e
■ ■ ■ ■■ 76/
■ ■ ■ ■ ■■ b288c5c0bc6535dae3a00b8e8db0d416c76ef8
■ ■ ■ ■■ 77/
■ ■ ■ ■ ■■ d0e40da9b6b095f0f28ed34ce799e2d9ab9a08
■ ■ ■ ■■ 78/
■ ■ ■ ■ ■■ 02f91d4c24a22ea0172c653e6d7ecc1fb318f2
■ ■ ■ ■ ■■ 1354894e23d18547d032803776543b417ae77b
■ ■ ■ ■ ■■ 255b2873d317950909b7c56e4c52d636b8d495
■ ■ ■ ■ ■■ 41286f53c3b6cd3fd804ff03c4e31b684fd404
■ ■ ■ ■ ■■ 4765e3a5961b9293b74158d8c270088ef4738a
■ ■ ■ ■ ■■ 6c0576c339948e678cede636a1095f8193a72d
■ ■ ■ ■■ 79/
■ ■ ■ ■ ■■ aab2a2d2c0707ecb8389ba39c5d19892c99db9
■ ■ ■ ■■ 7a/
■ ■ ■ ■ ■■ 3c01383edbd9352511744aa5b6389a736d9e0a
■ ■ ■ ■■ 7d/

■ ■ ■ ■ ■■ 9664eb401b37cc50017dcdbc5ae456aed3c0fa
■ ■ ■ ■■ 7f/
■ ■ ■ ■ ■■ 234d1d8393472b4445828365979c561219177e
■ ■ ■ ■■ 80/
■ ■ ■ ■ ■■ 1ff18d531ebcbda17a307c27189cc4e0ffb4f7
■ ■ ■ ■ ■■ f68f55036ed4ef280784fb93eb6096f9fd4b8c
■ ■ ■ ■■ 81/
■ ■ ■ ■ ■■ 76ff98b0d94c1f873deca7f860402e25ec5191
■ ■ ■ ■ ■■ f35dfcfe070c7320e3aabc9e12fa457ecb06ed
■ ■ ■ ■■ 82/
■ ■ ■ ■ ■■ 0808a89003a7bb9f8552522ca8436ec9fa5899
■ ■ ■ ■ ■■ 591c4f7827b11d9ed8f595236abe003dd98242
■ ■ ■ ■■ 83/
■ ■ ■ ■ ■■ 39c2a705c6009f650fdb321a80e269bc3a08e2
■ ■ ■ ■ ■■ 7fbdbf752bebaa2064305559e20277e9812e03
■ ■ ■ ■ ■■ 8094854230f9a43f520da5b78336232a8fd7b2
■ ■ ■ ■■ 85/
■ ■ ■ ■ ■■ 2c2c23db65b2949d51e707d80394b3c80fd6c6
■ ■ ■ ■ ■■ 6bcf22702527840557f0e92ef95cc3f16d12a4
■ ■ ■ ■■ 86/
■ ■ ■ ■ ■■ 3f7586b3e712ec8b0f32249b732dce4664ee57
■ ■ ■ ■■ 87/
■ ■ ■ ■ ■■ 244f815d13569cf4ad7bfd934a69c576d18e08
■ ■ ■ ■ ■■ 3b4f7d7e0c53e0a8fad5b830889a059f7da203
■ ■ ■ ■■ 8b/
■ ■ ■ ■ ■■ 6d915261d3c811dbbd20fc9ce20ee476bb72c6
■ ■ ■ ■■ 8d/
■ ■ ■ ■ ■■ 9023e23a8646ed2413f051e2af28a36982ea26
■ ■ ■ ■ ■■ 98433a460e1c4cd07a1a66cdae660ebc7af1cf
■ ■ ■ ■ ■■ c3a2447b83f5c5323bb21d2903b4e8408beccc
■ ■ ■ ■ ■■ c63cec0029dcbcfdc229ccd415761f787c0d9e
■ ■ ■ ■■ 8e/
■ ■ ■ ■ ■■ 4092a9101513c681ebf8c258c8b55640ef95eb
■ ■ ■ ■■ 91/
■ ■ ■ ■ ■■ 06337c213dba29c00877f42674df4aeb471beb
■ ■ ■ ■ ■■ 3a0d12b95840da050bce323bcf2f51c28ffeab
■ ■ ■ ■ ■■ 869d2a7257421495f79915b258db649cf23e9b
■ ■ ■ ■■ 92/
■ ■ ■ ■ ■■ bb8f70e7bf9527df505136c14a8543732de42b
■ ■ ■ ■■ 93/
■ ■ ■ ■ ■■ 64f9d77b016b45f8c29da03b14c8f48bc35785
■ ■ ■ ■■ 94/
■ ■ ■ ■ ■■ 7ab9ad804ebf65e95a5bbe3406e954f7550658
■ ■ ■ ■ ■■ 8ee1f86986b3051b30b7ba61b3abbaae349955
■ ■ ■ ■ ■■ e0aa062c824080d391574b226dd7df8419a8e4
■ ■ ■ ■ ■■ f22c25f1b913f3079af3b376576fa1cce11966
■ ■ ■ ■■ 95/
■ ■ ■ ■ ■■ 342000dfca7745084def92ff57fd9dfe85e4e5
■ ■ ■ ■■ 96/
■ ■ ■ ■ ■■ e9f6c0e00b95b8fb79ede08bb230ec1828531d
■ ■ ■ ■■ 98/
■ ■ ■ ■ ■■ 734432a0be7d6635e7b7dd09c43508ba9cb9be
■ ■ ■ ■■ 9a/
■ ■ ■ ■ ■■ 05b18a03f348138d2a14fc77acd5a8fb7638d9
■ ■ ■ ■ ■■ 0d684ad15e61d891eb745bd1bbaf85a2647631
■ ■ ■ ■ ■■ 1d2b2febf1765d7421e5c56e525101137699d8
■ ■ ■ ■■ 9b/
■ ■ ■ ■ ■■ 8181ca241a0fecda6fa48a74ca453b31cd7f78
■ ■ ■ ■■ 9c/
■ ■ ■ ■ ■■ 105928675b0415b9ee254ea08a068cfbc90740
■ ■ ■ ■ ■■ 63735f910f31aaf3e4311119308ddf6795b9fe
■ ■ ■ ■■ 9d/
■ ■ ■ ■ ■■ 3684a8355c6a45fd047592f49e808ad14be9c4
■ ■ ■ ■■ 9f/
■ ■ ■ ■ ■■ 00218966101dfc86e82cc8319292a06896d9fc
■ ■ ■ ■ ■■ 011db15679c5f5cc06a07acfbd8774247676c8
■ ■ ■ ■ ■■ 054bfcceaeb5ef99d4017c46ac76b1cd65f259
■ ■ ■ ■ ■■ 8ae087fce0ef607a732cb4ef921a8c86acadec
■ ■ ■ ■ ■■ a8197ee0036686f33e456f0463e75d07d86e10
■ ■ ■ ■■ a0/
■ ■ ■ ■ ■■ 65a7be7c298d04100b7905121387d184cc4c1d
■ ■ ■ ■■ a1/
■ ■ ■ ■ ■■ 5f244171214e9bdc098f1f85ed7e33fd374e92
■ ■ ■ ■ ■■ decf533b8dfb870c7d98dc3f35d21a76dee151
■ ■ ■ ■■ a4/
■ ■ ■ ■ ■■ c98b84291910a18c9b6360c5e3b83ef976556a
■ ■ ■ ■■ a5/
■ ■ ■ ■ ■■ f55db3c3fa76987250e2744e04b97b7602cfb1
■ ■ ■ ■ ■■ fecf694c30f675873ac22302bf8651b47583ed
■ ■ ■ ■■ a6/
■ ■ ■ ■ ■■ eb55489bf43de69a8fc04034f9b0d52011d4ff

■ ■ ■ ■ ■■ eb5d62db3cf26e4b8650b806e7b79028f2d9f4
■ ■ ■ ■■ a7/
■ ■ ■ ■ ■■ dfd14142e91ca062f445ae9ae8ba7d7bfd1c46
■ ■ ■ ■■ aa/
■ ■ ■ ■ ■■ 50350588345237d55315d5e59f8fe4fd93ad6f
■ ■ ■ ■ ■■ f61355e6ab819df6e78001f0fc9f2b29eee311
■ ■ ■ ■■ ab/
■ ■ ■ ■ ■■ 136d75d7c415ac601e03b4c24b2657dadeef71
■ ■ ■ ■ ■■ 27b282fb0922f0516e1616c6ffe2565db9d4d0
■ ■ ■ ■ ■■ 91b506a5a5c1c475b5b839b20a2a21c38548aa
■ ■ ■ ■■ ae/
■ ■ ■ ■ ■■ 16c68ffa5ede883f2ef7ca335384da1add248e
■ ■ ■ ■■ af/
■ ■ ■ ■ ■■ 04cf4b7636b5f94592fcc97023a1ad38ff892d
■ ■ ■ ■ ■■ f8c31737327aa6585332e2785f6b987170e755
■ ■ ■ ■■ b0/
■ ■ ■ ■ ■■ 929897dd25c699cf4a8fde9af3a7baace11f0c
■ ■ ■ ■ ■■ f8ec35a093b81622607e87db0b2a3a2db9e80d
■ ■ ■ ■■ b1/
■ ■ ■ ■ ■■ 9fb40e05609fefb76beb8cfb9f1906b3eebbe9
■ ■ ■ ■ ■■ fe21bd51a931da50aac354b7ab0b8f32f3b99b
■ ■ ■ ■■ b2/
■ ■ ■ ■ ■■ 2c5441407542f12ab36c18658280609c806396
■ ■ ■ ■■ b3/
■ ■ ■ ■ ■■ 44a83063f7ab650ee05861ad680bc4ef62838d
■ ■ ■ ■ ■■ a1a9fe4f3b2952f912c48e950a017b69d0c744
■ ■ ■ ■■ b4/
■ ■ ■ ■ ■■ 2e0d5346216eb6a0e0e635e0dc9d0627f2c46b
■ ■ ■ ■ ■■ 57a73cf15cdc076a1d582285c5adffcdc77002
■ ■ ■ ■ ■■ c480912a0f80fa4319497d0777787498eacffe
■ ■ ■ ■■ b5/
■ ■ ■ ■ ■■ 5c6b2236141de80e4debbb508b81947f09a76e
■ ■ ■ ■ ■■ fd35a68cd72697b8e99a714a337d2eb2823501
■ ■ ■ ■■ b6/
■ ■ ■ ■ ■■ 12071f4c851c9bca8f3d1c920b5fc47c1e9e0a
■ ■ ■ ■ ■■ 8348841c8a182b8e45505579508547a27b2347
■ ■ ■ ■■ b7/
■ ■ ■ ■ ■■ 8e47525bc7e0ec2445395704614555fd04ba39
■ ■ ■ ■■ b9/
■ ■ ■ ■ ■■ 1f060a7d73d919a33ca7b4e9d524eaa72fb4e6
■ ■ ■ ■ ■■ f1ca7bebddabc62c3ce7bb058dc1f6bda07933
■ ■ ■ ■■ ba/
■ ■ ■ ■ ■■ c11797c5ee9db4c43e459920c761e873076402
■ ■ ■ ■■ bb/
■ ■ ■ ■ ■■ f079d2c45bf2a2e5b5b0e760c0a3e3cd4ef077
■ ■ ■ ■■ bc/
■ ■ ■ ■ ■■ cb0a22b596b30dc99c95ea0dff08898fe19c49
■ ■ ■ ■■ bd/
■ ■ ■ ■ ■■ 2883a3e3471ba68eaa5fb37e6c42d73676477a
■ ■ ■ ■ ■■ 441609f9047194831c6993722856a0ac3825b8
■ ■ ■ ■ ■■ e8427f38848b62bc103c545938b41abb08ccad
■ ■ ■ ■■ be/
■ ■ ■ ■ ■■ ba927c0c6630a5eb39086e9a20ed6c56d47767
■ ■ ■ ■ ■■ ca639852b93b79dfc3baa42af01e1f95ae3c28
■ ■ ■ ■■ bf/
■ ■ ■ ■ ■■ eece9daf4e61d11decf876065e991080da79e4
■ ■ ■ ■■ c0/
■ ■ ■ ■ ■■ 350728d8751c977ab3e2421063cc170dce49bd
■ ■ ■ ■ ■■ d02db7dabe66d9cbaf14d0c2d240c71914f87d
■ ■ ■ ■ ■■ e88c389e3b14f7129abe50b78781aa14f14d27
■ ■ ■ ■■ c1/
■ ■ ■ ■ ■■ 410d46a096800d7ce80c6deed6c8f93044f0d0
■ ■ ■ ■■ c2/
■ ■ ■ ■ ■■ a7a03c551ce8a29e6bf88f273cb7ab9409e21b
■ ■ ■ ■■ c4/
■ ■ ■ ■ ■■ 2ef436abb1c3fd6070e92baa1e8c832687263a
■ ■ ■ ■ ■■ c48ce0f1f91be8e459a8e919887a776c3be7d9
■ ■ ■ ■■ c5/
■ ■ ■ ■ ■■ b955e790d0eaf246bb45e7b8486a72cbc7be03
■ ■ ■ ■■ c6/
■ ■ ■ ■ ■■ 5fe6e796e9002fd0c3be19047fe84191d66635
■ ■ ■ ■■ c8/
■ ■ ■ ■ ■■ b83128c8cc479789037bcf584a51f0d2e17742
■ ■ ■ ■ ■■ de4cff4e06ad358822fc1319bf924e68e9eac1
■ ■ ■ ■ ■■ e559081a39b35e4b47cabef9776fd4c47c9c01
■ ■ ■ ■■ cb/
■ ■ ■ ■ ■■ 978b4962da5e357af90ec7cd2a96c42e324850
■ ■ ■ ■ ■■ ea347a0ed0afb1622797a91a0bc1e8f522b367
■ ■ ■ ■■ cc/
■ ■ ■ ■ ■■ 9a45aa9a6480f15c882af8df53cc70b2291496
■ ■ ■ ■■ ce/

```
■    ■    ■    ■    ■■ 8b42d88bc36c7c300f31b93f75f4420618b714
■    ■    ■    ■    ■■ ac8b4a5076d71077f2c04d5ce114b38b397287
■    ■    ■    ■■ cf/
■    ■    ■    ■    ■■ 8e9e14b2ddea481a8acad1818b323e732de525
■    ■    ■    ■    ■■ aa5049408d7ea704f55458dbedecc64f805d8b
■    ■    ■    ■■ d0/
■    ■    ■    ■    ■■ 2519f4ade965c8c8b914efd36b2cc91bd8ea60
■    ■    ■    ■    ■■ 9d9bda67ff7768796681b3b33d1aed7a1f214c
■    ■    ■    ■    ■■ abfa42c92e3da0f606054309226c28ddd8401d
■    ■    ■    ■■ d1/
■    ■    ■    ■    ■■ 0b1b5f0b1012909403d4779b96a2971deb4f1f
■    ■    ■    ■■ d2/
■    ■    ■    ■    ■■ 103e3b741e39f2b6c8bd9e0e5d2972c82d80e5
■    ■    ■    ■    ■■ 9fd7ebfdc6771b8a15b8120e677f78c310f0b7
■    ■    ■    ■    ■■ b1947c1224603aa7220c526327d8adb8bc6999
■    ■    ■    ■    ■■ b1f14be72b75e5c11f6bd7178c63c17a1bb9b6
■    ■    ■    ■■ d4/
■    ■    ■    ■    ■■ 12ddc5c22c402c71a84569bbf2b65b9c979a75
■    ■    ■    ■    ■■ 9de10e4de63d4fe64cf029ad22d6a2d0250180
■    ■    ■    ■    ■■ dce35d3edce8a0059999aa3eb44875cf4da72c
■    ■    ■    ■■ d5/
■    ■    ■    ■    ■■ 1318eb68dd5e764d14ec3df6749a944561fa8b
■    ■    ■    ■    ■■ 576a17d570c66e01c449d85d781bf92317c149
■    ■    ■    ■    ■■ 7723e4e9cc23e41e9440d80e6b79ca214a4c77
■    ■    ■    ■■ d6/
■    ■    ■    ■    ■■ 76af2dfc6f05a2bf1ead4435c695e750e231a3
■    ■    ■    ■    ■■ 966116a373e5a6c00d340f147c25649be25df3
■    ■    ■    ■■ d7/
■    ■    ■    ■    ■■ 4f7693d4d670d5adaac69c5aa67afed89c31cc
■    ■    ■    ■    ■■ ef252f8267cacb90d7a9ac816bedb6ec9d5b6a
■    ■    ■    ■■ d8/
■    ■    ■    ■    ■■ d185b284ceadc4641ee3a36aab6be90251b08c
■    ■    ■    ■■ d9/
■    ■    ■    ■    ■■ 0fd7434887f8093c6f35c310a5bb98f97b4e19
■    ■    ■    ■    ■■ 8c9cf96662e087501338b0e5d3a8443e900f4e
■    ■    ■    ■    ■■ 9ea10d048a179431cc9400edf48724e4229d46
■    ■    ■    ■■ da/
■    ■    ■    ■    ■■ 12957c354eb0bc76275da0967a2e9bb261b9be
■    ■    ■    ■    ■■ 6016faf325fd91c5323f04c8851fec1dd8bf50
```

File: .env

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:TZ63Sc+475o3BZpoQCRWlEywb+/1xFp6H1unr/fH3hA=
APP_DEBUG=true
APP_URL=http://localhost

APP_LOCALE=en
APP_FALLBACK_LOCALE=en
APP_FAKER_LOCALE=en_US

APP_MAINTENANCE_DRIVER=file
# APP_MAINTENANCE_STORE=database

# PHP_CLI_SERVER_WORKERS=4

BCRYPT_ROUNDS=12

LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=bestbuy_book
DB_USERNAME=root
DB_PASSWORD=

SESSION_DRIVER=database
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null

BROADCAST_CONNECTION=log
FILESYSTEM_DISK=local
QUEUE_CONNECTION=database

CACHE_STORE=database
# CACHE_PREFIX=

MEMCACHED_HOST=127.0.0.1

REDIS_CLIENT=phpredis
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379

MAIL_MAILER=log
MAIL_SCHEME=null
MAIL_HOST=127.0.0.1
MAIL_PORT=2525
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_FROM_ADDRESS="hello@example.com"
MAIL_FROM_NAME="${APP_NAME}"

AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=
AWS_USE_PATH_STYLE_ENDPOINT=false

VITE_APP_NAME="${APP_NAME}"
```

File: .styleci.yml

```
php:
  preset: laravel
  disabled:
    - no_unused_imports
  finder:
    not-name:
      - index.php
js: true
css: true
```

File: CHANGELOG.md

```
# Release Notes

## [Unreleased](https://github.com/laravel/laravel/compare/v12.11.1...12.x)

## [v12.11.1](https://github.com/laravel/laravel/compare/v12.11.0...v12.11.1) - 2025-12-23

* Use environment variable for `DB_SSLMODE` - Postgres by [@robsontenorio](https://github.com/robsontenorio) in https://
* fix: ensure APP_URL does not have trailing slash in filesystem by [@msamgan](https://github.com/msamgan) in https://gi

## [v12.11.0](https://github.com/laravel/laravel/compare/v12.10.1...v12.11.0) - 2025-11-25

* fix: cookies are not available for subdomains by default by [@joostdebruijn](https://github.com/joostdebruijn) in http
* Fix PHP 8.5 PDO Driver Specific Constant Deprecation by [@RyanSchaefer](https://github.com/RyanSchaefer) in https://gi
* Ignore Laravel compiled views for Vite  by [@QistiAmal1212](https://github.com/QistiAmal1212) in https://github.com/la

## [v12.10.1](https://github.com/laravel/laravel/compare/v12.10.0...v12.10.1) - 2025-11-06

* Update schema URL in package.json by [@robinmiau](https://github.com/robinmiau) in https://github.com/laravel/laravel/

## [v12.10.0](https://github.com/laravel/laravel/compare/v12.9.1...v12.10.0) - 2025-11-04

* Add background driver by [@barryvdh](https://github.com/barryvdh) in https://github.com/laravel/laravel/pull/6699

## [v12.9.1](https://github.com/laravel/laravel/compare/v12.9.0...v12.9.1) - 2025-10-23

* [12.x] Replace Bootcamp with Laravel Learn by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611) in https://github.com
* [12.x] Comment out CLI workers for fresh applications by [@timacdonald](https://github.com/timacdonald) in https://git

## [v12.9.0](https://github.com/laravel/laravel/compare/v12.8.0...v12.9.0) - 2025-10-21

**Full Changelog**: https://github.com/laravel/laravel/compare/v12.8.0...v12.9.0

## [v12.8.0](https://github.com/laravel/laravel/compare/v12.7.1...v12.8.0) - 2025-10-20

* [12.x] Makes test suite using broadcast's `null` driver by [@nunomaduro](https://github.com/nunomaduro) in https://git

## [v12.7.1](https://github.com/laravel/laravel/compare/v12.7.0...v12.7.1) - 2025-10-15

* Added `failover` driver to the `queue` config comment.  by [@sajjadhossainshohag](https://github.com/sajjadhossainshoh

## [v12.7.0](https://github.com/laravel/laravel/compare/v12.6.0...v12.7.0) - 2025-10-14

**Full Changelog**: https://github.com/laravel/laravel/compare/v12.6.0...v12.7.0

## [v12.6.0](https://github.com/laravel/laravel/compare/v12.5.0...v12.6.0) - 2025-10-02

* Fix setup script by [@goldmont](https://github.com/goldmont) in https://github.com/laravel/laravel/pull/6682

## [v12.5.0](https://github.com/laravel/laravel/compare/v12.4.0...v12.5.0) - 2025-09-30

* [12.x] Fix type casting for environment variables in config files by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611
* Fix CVEs affecting vite by [@faissaloux](https://github.com/faissaloux) in https://github.com/laravel/laravel/pull/667
* Update .editorconfig to target compose.yaml by [@fredikaputra](https://github.com/fredikaputra) in https://github.com/
* Add pre-package-uninstall script to composer.json by [@cosmastech](https://github.com/cosmastech) in https://github.co

## [v12.4.0](https://github.com/laravel/laravel/compare/v12.3.1...v12.4.0) - 2025-08-29

* [12.x] Add default Redis retry configuration by [@mateusjatenee](https://github.com/mateusjatenee) in https://github.c

## [v12.3.1](https://github.com/laravel/laravel/compare/v12.3.0...v12.3.1) - 2025-08-21

* [12.x] Bump Pint version by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611) in https://github.com/laravel/laravel/p
* [12.x] Making sure all related processed are closed when terminating the currently command by [@AhmedAlaa4611](https:/
* [12.x] Use application name from configuration by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611) in https://github
* Bring back postAutoloadDump script by [@jasonvarga](https://github.com/jasonvarga) in https://github.com/laravel/larav

## [v12.3.0](https://github.com/laravel/laravel/compare/v12.2.0...v12.3.0) - 2025-08-03

* Fix Critical Security Vulnerability in form-data Dependency by [@izzygld](https://github.com/izzygld) in https://githu
* Revert "fix" by [@RobertBoes](https://github.com/RobertBoes) in https://github.com/laravel/laravel/pull/6646
* Change composer post-autoload-dump script to Artisan command by [@lmjhs](https://github.com/lmjhs) in https://github.c

## [v12.2.0](https://github.com/laravel/laravel/compare/v12.1.0...v12.2.0) - 2025-07-11

* Add Vite 7 support by [@timacdonald](https://github.com/timacdonald) in https://github.com/laravel/laravel/pull/6639
## [v12.1.0](https://github.com/laravel/laravel/compare/v12.0.11...v12.1.0) - 2025-07-03
```

```
* [12.x] Disable nightwatch in testing by [@laserhybiz](https://github.com/laserhybiz) in https://github.com/laravel/lar
* [12.x] Reorder environment variables in phpunit.xml for logical grouping by [@AhmedAlaa4611](https://github.com/AhmedA
* Change to hyphenate prefixes and cookie names by [@u01jmg3](https://github.com/u01jmg3) in https://github.com/laravel/
* [12.x] Fix type casting for environment variables in config files by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611

## [v12.0.11](https://github.com/laravel/laravel/compare/v12.0.10...v12.0.11) - 2025-06-10

**Full Changelog**: https://github.com/laravel/laravel/compare/v12.0.10...v12.0.11

## [v12.0.10](https://github.com/laravel/laravel/compare/v12.0.9...v12.0.10) - 2025-06-09

* fix alphabetical order by [@Khuthaily](https://github.com/Khuthaily) in https://github.com/laravel/laravel/pull/6627
* [12.x] Reduce redundancy and keeps the .gitignore file cleaner by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611) i
* [12.x] Fix: Add void return type to satisfy Rector analysis by [@Aluisio-Pires](https://github.com/Aluisio-Pires) in h

## [v12.0.9](https://github.com/laravel/laravel/compare/v12.0.8...v12.0.9) - 2025-05-26

* [12.x] Remove apc by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611) in https://github.com/laravel/laravel/pull/661
* [12.x] Add JSON Schema to package.json by [@martinbean](https://github.com/martinbean) in https://github.com/laravel/l
* Minor language update by [@woganmay](https://github.com/woganmay) in https://github.com/laravel/laravel/pull/6615
* Enhance .gitignore to exclude common OS and log files by [@mohammadRezaei1380](https://github.com/mohammadRezaei1380)

## [v12.0.8](https://github.com/laravel/laravel/compare/v12.0.7...v12.0.8) - 2025-05-12

* [12.x] Clean up URL formatting in README by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611) in https://github.com/l

## [v12.0.7](https://github.com/laravel/laravel/compare/v12.0.6...v12.0.7) - 2025-04-15

* Add `composer run test` command by [@crynobone](https://github.com/crynobone) in https://github.com/laravel/laravel/pu
* Partner Directory Changes in ReadME by [@joshcirre](https://github.com/joshcirre) in https://github.com/laravel/larave

## [v12.0.6](https://github.com/laravel/laravel/compare/v12.0.5...v12.0.6) - 2025-04-08

**Full Changelog**: https://github.com/laravel/laravel/compare/v12.0.5...v12.0.6

## [v12.0.5](https://github.com/laravel/laravel/compare/v12.0.4...v12.0.5) - 2025-04-02

* [12.x] Update `config/mail.php` to match the latest core configuration by [@AhmedAlaa4611](https://github.com/AhmedAla

## [v12.0.4](https://github.com/laravel/laravel/compare/v12.0.3...v12.0.4) - 2025-03-31

* Bump vite from 6.0.11 to 6.2.3 - Vulnerability patch by [@abdel-aouby](https://github.com/abdel-aouby) in https://gith
* Bump vite from 6.2.3 to 6.2.4 by [@thinkverse](https://github.com/thinkverse) in https://github.com/laravel/laravel/pu

## [v12.0.3](https://github.com/laravel/laravel/compare/v12.0.2...v12.0.3) - 2025-03-17

* Remove reverted change from CHANGELOG.md by [@AJenbo](https://github.com/AJenbo) in https://github.com/laravel/laravel
* Improves clarity in app.css file by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611) in https://github.com/laravel/l
* [12.x] Refactor: Structural improvement for clarity by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611) in https://g
* Bump axios from 1.7.9 to 1.8.2 - Vulnerability patch by [@abdel-aouby](https://github.com/abdel-aouby) in https://gith
* [12.x] Remove Unnecessarily [@source](https://github.com/source) by [@AhmedAlaa4611](https://github.com/AhmedAlaa4611)

## [v12.0.2](https://github.com/laravel/laravel/compare/v12.0.1...v12.0.2) - 2025-03-04

* Make the github test action run out of the box independent of the choice of testing framework by [@ndeblauw](https://g

## [v12.0.1](https://github.com/laravel/laravel/compare/v12.0.0...v12.0.1) - 2025-02-24

* [12.x] prefer stable stability by [@pataar](https://github.com/pataar) in https://github.com/laravel/laravel/pull/6548

## [v12.0.0 (2025-??-??)](https://github.com/laravel/laravel/compare/v11.0.2...v12.0.0)

Laravel 12 includes a variety of changes to the application skeleton. Please consult the diff to see what's new.
```

File: composer.json

```
{
    "$schema": "https://getcomposer.org/schema.json",
    "name": "laravel/laravel",
    "type": "project",
    "description": "The skeleton application for the Laravel framework.",
    "keywords": ["laravel", "framework"],
    "license": "MIT",
    "require": {
        "php": "^8.2",
        "inertiajs/inertia-laravel": "^2.0",
        "laravel/framework": "^12.0",
        "laravel/sanctum": "^4.0",
        "laravel/tinker": "^2.10.1",
```

```
        "tightenco/ziggy": "^2.0"
    },
    "require-dev": {
        "fakerphp/faker": "^1.23",
        "laravel/breeze": "*",
        "laravel/pail": "^1.2.2",
        "laravel/pint": "^1.24",
        "laravel/sail": "^1.41",
        "mockery/mockery": "^1.6",
        "nunomaduro/collision": "^8.6",
        "phpunit/phpunit": "^11.5.3"
    },
    "autoload": {
        "psr-4": {
            "App\\": "app/",
            "Database\\Factories\\": "database/factories/",
            "Database\\Seeders\\": "database/seeders/"
        }
    },
    "autoload-dev": {
        "psr-4": {
            "Tests\\": "tests/"
        }
    },
    "scripts": {
        "setup": [
            "composer install",
            "@php -r \"file_exists('.env') || copy('.env.example', '.env');\"",
            "@php artisan key:generate",
            "@php artisan migrate --force",
            "npm install",
            "npm run build"
        ],
        "dev": [
            "Composer\\Config::disableProcessTimeout",
            "npx concurrently -c \"#93c5fd,#c4b5fd,#fb7185,#fdba74\" \"php artisan serve\" \"php artisan queue:listen --
        ],
        "test": [
            "@php artisan config:clear --ansi",
            "@php artisan test"
        ],
        "post-autoload-dump": [
            "Illuminate\\Foundation\\ComposerScripts::postAutoloadDump",
            "@php artisan package:discover --ansi"
        ],
        "post-update-cmd": [
            "@php artisan vendor:publish --tag=laravel-assets --ansi --force"
        ],
        "post-root-package-install": [
            "@php -r \"file_exists('.env') || copy('.env.example', '.env');\""
        ],
        "post-create-project-cmd": [
            "@php artisan key:generate --ansi",
            "@php -r \"file_exists('database/database.sqlite') || touch('database/database.sqlite');\"",
            "@php artisan migrate --graceful --ansi"
        ],
        "pre-package-uninstall": [
            "Illuminate\\Foundation\\ComposerScripts::prePackageUninstall"
        ]
    },
    "extra": {
        "laravel": {
            "dont-discover": []
        }
    },
    "config": {
        "optimize-autoloader": true,
        "preferred-install": "dist",
        "sort-packages": true,
        "allow-plugins": {
            "pestphp/pest-plugin": true,
            "php-http/discovery": true
        }
    },
    "minimum-stability": "stable",
    "prefer-stable": true
}
```

File: generate_pdf.py

```
from reportlab.lib.pagesizes import letter, A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Preformatted, PageBreak
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.platypus.tables import Table
from reportlab.platypus.tableofcontents import TableOfContents
from reportlab.lib.enums import TA_LEFT, TA_CENTER
from reportlab.lib import colors
import os
import time
from datetime import datetime

def generate_project_pdf():
    # Output PDF file
    output_path = "project_documentation.pdf"
    doc = SimpleDocTemplate(
        output_path,
        pagesize=A4,
        rightMargin=72,
        leftMargin=72,
        topMargin=72,
        bottomMargin=72
    )

    # Get styles
    styles = getSampleStyleSheet()
    title_style = ParagraphStyle(
        'CustomTitle',
        parent=styles['Heading1'],
        fontSize=24,
        spaceAfter=30,
        alignment=TA_CENTER,
        textColor=(0.2, 0.2, 0.6)
    )

    subtitle_style = ParagraphStyle(
        'Subtitle',
        parent=styles['Normal'],
        fontSize=14,
        spaceAfter=20,
        alignment=TA_CENTER,
        textColor=(0.3, 0.3, 0.3)
    )

    header_style = ParagraphStyle(
        'Header',
        parent=styles['Heading2'],
        fontSize=16,
        spaceAfter=12,
        borderWidth=1,
        borderColor='black',
        borderPadding=10
    )

    file_header_style = ParagraphStyle(
        'FileHeader',
        parent=styles['Heading3'],
        fontSize=12,
        spaceAfter=6,
        borderWidth=1,
        borderColor='gray',
        borderPadding=6,
        backColor=colors.lightgrey
    )

    # Create story
    story = []

    # Title
    story.append(Paragraph('BestBuy Books Project Documentation', title_style))
    story.append(Paragraph(f'Generated on: {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}', subtitle_style))
    story.append(Spacer(1, 20))

    # Project overview
    story.append(Paragraph('Project Overview', header_style))
    story.append(Paragraph(
        'BestBuy_Books is an e-commerce platform built with Laravel framework supporting multi-role users '
        '(Administrators, Merchants, Customers) for managing books, orders, chats, and issue reports. '
        'The project implements real-time communication between users and administrators, merchant '
        'registration approval process, and role-based access control.',
        styles['Normal']
```

```
))
story.append(Spacer(1, 20))

# Directory structure
story.append(Paragraph('Directory Structure', header_style))
story.append(Spacer(1, 12))

# Generate directory tree
dir_tree = generate_directory_tree(r"c:\xampp\htdocs\BestBuy_Books - Copy")
story.append(Preformatted(dir_tree, styles['Code']))
story.append(PageBreak())

# Add table of contents placeholder
toc = TableOfContents()
toc.levelStyles = [
    ParagraphStyle(
        fontName='Times-Bold',
        fontSize=14,
        name='TOCHeading1',
        leftIndent=20,
        firstLineIndent=-20,
        spaceBefore=10,
        leading=16
    ),
    ParagraphStyle(
        fontName='Times-Roman',
        fontSize=12,
        name='TOCHeading2',
        leftIndent=40,
        firstLineIndent=-20,
        spaceBefore=5,
        leading=12
    )
]
story.append(toc)
story.append(PageBreak())

# Process files with size limits
dirs_to_exclude = [
    'vendor', 'node_modules', '.git', 'storage', 'tests',
    'bootstrap/cache', 'storage/framework', 'public/build',
    'resources/js', 'resources/css', 'database/factories',
    'database/seeders'
]

extensions_to_include = [
    '.php', '.blade.php', '.js', '.jsx', '.json', '.md',
    '.css', '.py', '.txt', '.env', '.sql', '.xml', '.yml',
    '.yaml', '.html', '.twig'
]

# Process core application files
root_path = r"c:\xampp\htdocs\BestBuy_Books - Copy"
for root, dirs, files in os.walk(root_path):
    # Remove excluded directories
    dirs[:] = [d for d in dirs if d not in dirs_to_exclude]

    for file in files:
        if any(file.lower().endswith(ext) for ext in extensions_to_include):
            file_path = os.path.join(root, file)

            # Calculate relative path
            rel_path = os.path.relpath(file_path, root_path)

            # Skip very large files
            try:
                file_size = os.path.getsize(file_path)
                if file_size > 100000:  # Skip files larger than 100KB
                    continue
            except OSError:
                continue

            # Add file header
            story.append(Paragraph(f'File: {rel_path}', file_header_style))

            # Read and add file content with limit
            try:
                with open(file_path, 'r', encoding='utf-8', errors='replace') as f:
                    content = f.read(50000)  # Limit to first 50k chars per file

                    # Add content
```

```
                        story.append(Preformatted(content, styles['Code']))

                        # Add size warning if needed
                        if len(content) >= 50000:
                            story.append(Paragraph('(Content truncated due to size)',
                                                   ParagraphStyle(
                                                       'TruncateNote',
                                                       parent=styles['Normal'],
                                                       fontSize=8,
                                                       textColor='red'
                                                   )))
                # except Exception as e:
                    story.append(Paragraph(f'[Could not read file: {str(e)}]',
                                           ParagraphStyle(
                                               'ErrorNote',
                                               parent=styles['Normal'],
                                               textColor='red'
                                           )))

                story.append(Spacer(1, 20))

    # Build document
    doc.multiBuild(story)
    print(f"PDF generated successfully at {output_path}")
    print(f"File size: {os.path.getsize(output_path) / (1024*1024):.2f} MB")


def generate_directory_tree(startpath):
    """Generate a tree-like representation of the directory structure"""
    output = []
    # Normalize path separators
    startpath = os.path.normpath(startpath)
    for root, dirs, files in os.walk(startpath):
        # Get the level by counting path separators
        level = root.replace(startpath, '').count(os.sep)
        indent = ' ■   ' * level

        # Get basename of the folder
        folder_name = os.path.basename(root)
        output.append(f'{indent} ■■ {folder_name}/')

        subindent = ' ■   ' * (level + 1)
        for i, file in enumerate(files):
            if i == len(files) - 1:
                output.append(f'{subindent} ■■ {file}')
            else:
                output.append(f'{subindent} ■■ {file}')

    return '\n'.join(output[:1000])  # Limit to prevent huge outputs


def split_large_content(content, max_chars_per_part=50000):
    """Split large content into smaller parts"""
    parts = []
    for i in range(0, len(content), max_chars_per_part):
        parts.append(content[i:i+max_chars_per_part])
    return parts


if __name__ == "__main__":
    generate_project_pdf()
```

File: generate_pdf_optimized.py

```
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Preformatted, PageBreak
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_LEFT, TA_CENTER
from reportlab.lib import colors
import os
from datetime import datetime

def generate_project_pdf_optimized():
    # Output PDF file
    output_path = "project_documentation_optimized.pdf"
    doc = SimpleDocTemplate(
        output_path,
        pagesize=A4,
        rightMargin=50,
```

```
    leftMargin=50,
    topMargin=50,
    bottomMargin=50
)

# Get styles
styles = getSampleStyleSheet()
title_style = ParagraphStyle(
    'CustomTitle',
    parent=styles['Heading1'],
    fontSize=18,
    spaceAfter=20,
    alignment=TA_CENTER,
    textColor=(0.2, 0.2, 0.6)
)

subtitle_style = ParagraphStyle(
    'Subtitle',
    parent=styles['Normal'],
    fontSize=12,
    spaceAfter=15,
    alignment=TA_CENTER,
    textColor=(0.3, 0.3, 0.3)
)

header_style = ParagraphStyle(
    'Header',
    parent=styles['Heading2'],
    fontSize=14,
    spaceAfter=10
)

file_header_style = ParagraphStyle(
    'FileHeader',
    parent=styles['Heading3'],
    fontSize=10,
    spaceAfter=5,
    borderWidth=0.5,
    borderColor='gray',
    borderPadding=3,
    backColor=colors.lightgrey
)

code_style = ParagraphStyle(
    'Code',
    parent=styles['Code'],
    fontSize=8,
    leading=10
)

# Create story
story = []

# Title
story.append(Paragraph('BestBuy Books Project Documentation', title_style))
story.append(Paragraph(f'Generated on: {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}', subtitle_style))
story.append(Spacer(1, 15))

# Project overview
story.append(Paragraph('Project Overview', header_style))
story.append(Paragraph(
    'BestBuy_Books is an e-commerce platform built with Laravel framework supporting multi-role users '
    '(Administrators, Merchants, Customers) for managing books, orders, chats, and issue reports. '
    'The project implements real-time communication between users and administrators, merchant '
    'registration approval process, and role-based access control.',
    styles['Normal']
))
story.append(Spacer(1, 15))

# Directory structure
story.append(Paragraph('Directory Structure', header_style))
story.append(Spacer(1, 10))

# Generate directory tree
dir_tree = generate_directory_tree(r"c:\xampp\htdocs\BestBuy_Books - Copy")
story.append(Preformatted(dir_tree, code_style))
story.append(PageBreak())

# Process files with stricter size limits
dirs_to_exclude = [
    'vendor', 'node_modules', '.git', 'storage', 'tests',
```

```
        'bootstrap/cache', 'storage/framework', 'public/build',
        'resources/js', 'resources/css', 'database/factories',
        'database/seeders', '__pycache__', '.vscode', '.idea'
    ]

    extensions_to_include = [
        '.php', '.blade.php', '.py', '.txt', '.env', '.sql',
        '.xml', '.yml', '.yaml', '.md', '.json'
    ]

    # Process core application files with stricter limits
    root_path = r"c:\xampp\htdocs\BestBuy_Books - Copy"
    total_chars_added = 0

    for root, dirs, files in os.walk(root_path):
        # Remove excluded directories
        dirs[:] = [d for d in dirs if d not in dirs_to_exclude]

        for file in files:
            if any(file.lower().endswith(ext) for ext in extensions_to_include):
                file_path = os.path.join(root, file)

                # Calculate relative path
                rel_path = os.path.relpath(file_path, root_path)

                # Skip very large files
                try:
                    file_size = os.path.getsize(file_path)
                    if file_size > 50000:  # Skip files larger than 50KB
                        continue
                except OSError:
                    continue

                # Check if we're approaching size limit
                if total_chars_added > 800000:  # Stop before reaching 10MB
                    story.append(Paragraph('Document size limit reached. Remaining files not included.',
                                           ParagraphStyle('LimitNote', parent=styles['Normal'], textColor='red')))
                    break

                # Add file header
                story.append(Paragraph(f'File: {rel_path}', file_header_style))

                # Read and add file content with stricter limit
                try:
                    with open(file_path, 'r', encoding='utf-8', errors='replace') as f:
                        content = f.read(20000)  # Limit to first 20k chars per file

                        # Add content
                        story.append(Preformatted(content, code_style))

                        # Add size warning if needed
                        if len(content) >= 20000:
                            story.append(Paragraph('(Content truncated due to size)',
                                                   ParagraphStyle(
                                                       'TruncateNote',
                                                       parent=styles['Normal'],
                                                       fontSize=6,
                                                       textColor='red'
                                                   )))

                        total_chars_added += len(content)

                except Exception as e:
                    story.append(Paragraph(f'[Could not read file: {str(e)}]',
                                           ParagraphStyle(
                                               'ErrorNote',
                                               parent=styles['Normal'],
                                               textColor='red'
                                           )))

                story.append(Spacer(1, 10))

        if total_chars_added > 800000:  # Stop before reaching 10MB
            break

    # Build document
    doc.build(story)
    print(f"Optimized PDF generated successfully at {output_path}")
    print(f"File size: {os.path.getsize(output_path) / (1024*1024):.2f} MB")
def generate_directory_tree(startpath):
    """Generate a tree-like representation of the directory structure"""
```

```
    output = []
    for root, dirs, files in os.walk(startpath):
        # Limit depth to prevent extremely deep trees
        depth = root.replace(startpath, '').count(os.sep)
        if depth > 5:  # Limit depth to 5 levels
            continue

        # Get the level by counting path separators
        level = depth
        indent = ' ■   ' * level

        # Get basename of the folder
        folder_name = os.path.basename(root)
        if folder_name:  # Skip empty names
            output.append(f'{indent} ■■ {folder_name}/')

        subindent = ' ■   ' * (level + 1)
        count = 0
        for file in files:
            if count >= 20:  # Limit files listed per directory
                output.append(f'{subindent} ... and more files')
                break
            output.append(f'{subindent} ■■ {file}')
            count += 1

    return '\n'.join(output[:500])  # Limit to prevent huge outputs


if __name__ == "__main__":
    generate_project_pdf_optimized()
```

File: generate_project_pdf.py

```
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Preformatted, PageBreak
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_LEFT, TA_CENTER
from reportlab.lib import colors
import os
from datetime import datetime

def generate_project_pdf():
    """
    Generates a PDF containing the project code and directory structure,
    keeping the file size under 10MB as per project specifications.
    """
    # Output PDF file
    output_path = "project_documentation.pdf"
    doc = SimpleDocTemplate(
        output_path,
        pagesize=A4,
        rightMargin=50,
        leftMargin=50,
        topMargin=50,
        bottomMargin=50
    )

    # Get styles
    styles = getSampleStyleSheet()

    # Custom styles for better compactness
    title_style = ParagraphStyle(
        'CustomTitle',
        parent=styles['Heading1'],
        fontSize=16,
        spaceAfter=15,
        alignment=TA_CENTER,
        textColor=(0.2, 0.2, 0.6)
    )

    subtitle_style = ParagraphStyle(
        'Subtitle',
        parent=styles['Normal'],
        fontSize=10,
        spaceAfter=12,
        alignment=TA_CENTER,
        textColor=(0.3, 0.3, 0.3)
    )
    header_style = ParagraphStyle(
```

```
    'Header',
    parent=styles['Heading2'],
    fontSize=12,
    spaceAfter=8
)

file_header_style = ParagraphStyle(
    'FileHeader',
    parent=styles['Heading3'],
    fontSize=9,
    spaceAfter=4,
    borderWidth=0.5,
    borderColor='gray',
    borderPadding=2,
    backColor=colors.lightgrey
)

code_style = ParagraphStyle(
    'Code',
    parent=styles['Code'],
    fontSize=7,
    leading=9
)

truncate_style = ParagraphStyle(
    'TruncateNote',
    parent=styles['Normal'],
    fontSize=6,
    textColor=colors.red
)

# Create story
story = []

# Title
story.append(Paragraph('BestBuy Books Project Documentation', title_style))
story.append(Paragraph(f'Generated on: {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}', subtitle_style))
story.append(Spacer(1, 15))

# Project overview
story.append(Paragraph('Project Overview', header_style))
story.append(Paragraph(
    'BestBuy_Books is an e-commerce platform built with Laravel framework supporting multi-role users '
    '(Administrators, Merchants, Customers) for managing books, orders, chats, and issue reports. '
    'The project implements real-time communication between users and administrators, merchant '
    'registration approval process, and role-based access control.',
    styles['Normal']
))
story.append(Spacer(1, 15))

# Directory structure
story.append(Paragraph('Directory Structure', header_style))
story.append(Spacer(1, 10))

# Generate directory tree
dir_tree = generate_directory_tree(r"c:\xampp\htdocs\BestBuy_Books - Copy")
story.append(Preformatted(dir_tree, code_style))
story.append(PageBreak())

# Process files with strict size limits as per project specifications
dirs_to_exclude = [
    'vendor', 'node_modules', '.git', 'storage', 'tests',
    'bootstrap/cache', 'storage/framework', 'public/build',
    'resources/js', 'resources/css', 'database/factories',
    'database/seeders', '__pycache__', '.vscode', '.idea'
]

extensions_to_include = [
    '.php', '.blade.php', '.py', '.txt', '.env', '.sql',
    '.xml', '.yml', '.yaml', '.md', '.json'
]

# Process core application files with size limits per specification
root_path = r"c:\xampp\htdocs\BestBuy_Books - Copy"
total_chars_added = 0
files_processed = 0

for root, dirs, files in os.walk(root_path):
    # Remove excluded directories
    dirs[:] = [d for d in dirs if d not in dirs_to_exclude]
    for file in files:
```

```
            if any(file.lower().endswith(ext) for ext in extensions_to_include):
                file_path = os.path.join(root, file)

                # Calculate relative path
                rel_path = os.path.relpath(file_path, root_path)

                # Skip very large files per specification (>50KB)
                try:
                    file_size = os.path.getsize(file_path)
                    if file_size > 50000:  # Skip files larger than 50KB
                        continue
                except OSError:
                    continue

                # Check if we're approaching size limit to stay under 10MB
                if total_chars_added > 800000:  # Stop before reaching 10MB
                    story.append(Paragraph(
                        'Document size limit reached. Remaining files not included.',
                        ParagraphStyle('LimitNote', parent=styles['Normal'], textColor=colors.red)
                    ))
                    break

                # Add file header
                story.append(Paragraph(f'File: {rel_path}', file_header_style))

                # Read and add file content with limit per specification (max 20KB)
                try:
                    with open(file_path, 'r', encoding='utf-8', errors='replace') as f:
                        content = f.read(20000)  # Limit to first 20k chars per file per spec

                        # Add content
                        story.append(Preformatted(content, code_style))

                        # Add size warning if needed
                        if len(content) >= 20000:
                            story.append(Paragraph(
                                '(Content truncated due to size - showing first 20KB)',
                                truncate_style
                            ))

                        total_chars_added += len(content)
                        files_processed += 1

                except Exception as e:
                    story.append(Paragraph(
                        f'[Could not read file: {str(e)}]',
                        ParagraphStyle('ErrorNote', parent=styles['Normal'], textColor=colors.red)
                    ))

                story.append(Spacer(1, 8))

        if total_chars_added > 800000:  # Stop before reaching 10MB
            break

    # Add summary
    story.append(PageBreak())
    story.append(Paragraph('Summary', header_style))
    story.append(Paragraph(f'Total files processed: {files_processed}', styles['Normal']))
    story.append(Paragraph(f'Total characters added: {total_chars_added:,}', styles['Normal']))

    # Build document
    doc.build(story)
    print(f"Project PDF generated successfully at {output_path}")
    print(f"File size: {os.path.getsize(output_path) / (1024*1024):.2f} MB")
    print(f"Files processed: {files_processed}")


def generate_directory_tree(startpath):
    """
    Generate a tree-like representation of the directory structure
    with depth and file count limits to keep the PDF compact
    """
    output = []
    for root, dirs, files in os.walk(startpath):
        # Limit depth to prevent extremely deep trees
        depth = root.replace(startpath, '').count(os.sep)
        if depth > 5:  # Limit depth to 5 levels
            continue

        # Get the level by counting path separators
        level = depth
```

```
        indent = ' ■   ' * level

        # Get basename of the folder
        folder_name = os.path.basename(root)
        if folder_name:  # Skip empty names
            output.append(f'{indent} ■■ {folder_name}/')

        subindent = ' ■   ' * (level + 1)
        count = 0
        for file in files:
            if count >= 15:  # Limit files listed per directory to keep compact
                output.append(f'{subindent} ... and more files')
                break
            output.append(f'{subindent} ■■ {file}')
            count += 1

    return '\n'.join(output[:500])  # Limit to prevent huge outputs


if __name__ == "__main__":
    generate_project_pdf()
```

File: jsconfig.json

```
{
    "compilerOptions": {
        "baseUrl": ".",
        "paths": {
            "@/*": ["resources/js/*"],
            "ziggy-js": ["./vendor/tightenco/ziggy"]
        }
    },
    "exclude": ["node_modules", "public"]
}
```

File: package.json

```
{
    "$schema": "https://www.schemastore.org/package.json",
    "private": true,
    "type": "module",
    "scripts": {
        "build": "vite build",
        "dev": "vite"
    },
    "devDependencies": {
        "@headlessui/react": "^2.0.0",
        "@inertiajs/react": "^2.0.0",
        "@tailwindcss/forms": "^0.5.3",
        "@tailwindcss/vite": "^4.0.0",
        "@vitejs/plugin-react": "^4.2.0",
        "autoprefixer": "^10.4.12",
        "axios": "^1.11.0",
        "concurrently": "^9.0.1",
        "laravel-vite-plugin": "^2.0.0",
        "postcss": "^8.4.31",
        "react": "^18.2.0",
        "react-dom": "^18.2.0",
        "tailwindcss": "^3.2.1",
        "vite": "^7.0.7"
    }
}
```

File: phpunit.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
         bootstrap="vendor/autoload.php"
         colors="true"
>
    <testsuites>
        <testsuite name="Unit">
            <directory>tests/Unit</directory>
        </testsuite>
        <testsuite name="Feature">
            <directory>tests/Feature</directory>
```

```
        </testsuite>
    </testsuites>
    <source>
        <include>
            <directory>app</directory>
        </include>
    </source>
    <php>
        <env name="APP_ENV" value="testing"/>
        <env name="APP_MAINTENANCE_DRIVER" value="file"/>
        <env name="BCRYPT_ROUNDS" value="4"/>
        <env name="BROADCAST_CONNECTION" value="null"/>
        <env name="CACHE_STORE" value="array"/>
        <env name="DB_CONNECTION" value="sqlite"/>
        <env name="DB_DATABASE" value=":memory:"/>
        <env name="MAIL_MAILER" value="array"/>
        <env name="QUEUE_CONNECTION" value="sync"/>
        <env name="SESSION_DRIVER" value="array"/>
        <env name="PULSE_ENABLED" value="false"/>
        <env name="TELESCOPE_ENABLED" value="false"/>
        <env name="NIGHTWATCH_ENABLED" value="false"/>
    </php>
</phpunit>
```

File: README.md

```
<p align="center"><a href="https://laravel.com" target="_blank"><img src="https://raw.githubusercontent.com/laravel/art/

<p align="center">
<a href="https://github.com/laravel/framework/actions"><img src="https://github.com/laravel/framework/workflows/tests/ba
<a href="https://packagist.org/packages/laravel/framework"><img src="https://img.shields.io/packagist/dt/laravel/framewo
<a href="https://packagist.org/packages/laravel/framework"><img src="https://img.shields.io/packagist/v/laravel/framewor
<a href="https://packagist.org/packages/laravel/framework"><img src="https://img.shields.io/packagist/l/laravel/framewor
</p>

## About Laravel

Laravel is a web application framework with expressive, elegant syntax. We believe development must be an enjoyable and

- [Simple, fast routing engine](https://laravel.com/docs/routing).
- [Powerful dependency injection container](https://laravel.com/docs/container).
- Multiple back-ends for [session](https://laravel.com/docs/session) and [cache](https://laravel.com/docs/cache) storage
- Expressive, intuitive [database ORM](https://laravel.com/docs/eloquent).
- Database agnostic [schema migrations](https://laravel.com/docs/migrations).
- [Robust background job processing](https://laravel.com/docs/queues).
- [Real-time event broadcasting](https://laravel.com/docs/broadcasting).

Laravel is accessible, powerful, and provides tools required for large, robust applications.

## Learning Laravel

Laravel has the most extensive and thorough [documentation](https://laravel.com/docs) and video tutorial library of all

If you don't feel like reading, [Laracasts](https://laracasts.com) can help. Laracasts contains thousands of video tutor

## Laravel Sponsors

We would like to extend our thanks to the following sponsors for funding Laravel development. If you are interested in b

### Premium Partners

- **[Vehikl](https://vehikl.com)**
- **[Tighten Co.](https://tighten.co)**
- **[Kirschbaum Development Group](https://kirschbaumdevelopment.com)**
- **[64 Robots](https://64robots.com)**
- **[Curotec](https://www.curotec.com/services/technologies/laravel)**
- **[DevSquad](https://devsquad.com/hire-laravel-developers)**
- **[Redberry](https://redberry.international/laravel-development)**
- **[Active Logic](https://activelogic.com)**

## Contributing

Thank you for considering contributing to the Laravel framework! The contribution guide can be found in the [Laravel doc

## Code of Conduct

In order to ensure that the Laravel community is welcoming to all, please review and abide by the [Code of Conduct](http

## Security Vulnerabilities
```

```
If you discover a security vulnerability within Laravel, please send an e-mail to Taylor Otwell via [taylor@laravel.com]

## License

The Laravel framework is open-sourced software licensed under the [MIT license](https://opensource.org/licenses/MIT).
"# BestBuy_Books"
"# BestBuy_Books"
```

File: .github\workflows\issues.yml

```yaml
name: Issues

on:
  issues:
    types: [labeled]

permissions:
  issues: write

jobs:
  help-wanted:
    uses: laravel/.github/.github/workflows/issues.yml@main
```

File: .github\workflows\pull-requests.yml

```yaml
name: Pull Requests

on:
  pull_request_target:
    types: [opened]

permissions:
  pull-requests: write

jobs:
  uneditable:
    uses: laravel/.github/.github/workflows/pull-requests.yml@main
```

File: .github\workflows\tests.yml

```yaml
name: Tests

on:
  push:
    branches:
      - master
      - '*.x'
  pull_request:
  schedule:
    - cron: '0 0 * * *'

permissions:
  contents: read

jobs:
  tests:
    runs-on: ubuntu-latest

    strategy:
      fail-fast: true
      matrix:
        php: [8.2, 8.3, 8.4]

    name: PHP ${{ matrix.php }}

    steps:
      - name: Checkout code
        uses: actions/checkout@v6

      - name: Setup PHP
        uses: shivammathur/setup-php@v2
        with:
          php-version: ${{ matrix.php }}
          extensions: dom, curl, libxml, mbstring, zip, pcntl, pdo, sqlite, pdo_sqlite
          coverage: none

      - name: Install Composer dependencies
```

```
        run: composer install --prefer-dist --no-interaction --no-progress

      - name: Copy environment file
        run: cp .env.example .env

      - name: Generate app key
        run: php artisan key:generate

      - name: Execute tests
        run: php artisan test
```

File: .github\workflows\update-changelog.yml

```
name: Update Changelog

on:
  release:
    types: [released]

permissions: {}

jobs:
  update:
    permissions:
      contents: write
    uses: laravel/.github/.github/workflows/update-changelog.yml@main
```

File: .venv\Lib\site-packages\imagesize.py

```
import io
import os
import re
import struct
from xml.etree import ElementTree


__all__ = ["get", "getDPI", "__version__"]


__version__ = "1.4.1"


_UNIT_KM = -3
_UNIT_100M = -2
_UNIT_10M = -1
_UNIT_1M = 0
_UNIT_10CM = 1
_UNIT_CM = 2
_UNIT_MM = 3
_UNIT_0_1MM = 4
_UNIT_0_01MM = 5
_UNIT_UM = 6
_UNIT_INCH = 6

_TIFF_TYPE_SIZES = {
  1: 1,
  2: 1,
  3: 2,
  4: 4,
  5: 8,
  6: 1,
  7: 1,
  8: 2,
  9: 4,
  10: 8,
  11: 4,
  12: 8,
}


def _convertToDPI(density, unit):
    if unit == _UNIT_KM:
        return int(density * 0.0000254 + 0.5)
    elif unit == _UNIT_100M:
        return int(density * 0.000254 + 0.5)
    elif unit == _UNIT_10M:
        return int(density * 0.00254 + 0.5)
    elif unit == _UNIT_1M:
```

```
        return int(density * 0.0254 + 0.5)
    elif unit == _UNIT_10CM:
        return int(density * 0.254 + 0.5)
    elif unit == _UNIT_CM:
        return int(density * 2.54 + 0.5)
    elif unit == _UNIT_MM:
        return int(density * 25.4 + 0.5)
    elif unit == _UNIT_0_1MM:
        return density * 254
    elif unit == _UNIT_0_01MM:
        return density * 2540
    elif unit == _UNIT_UM:
        return density * 25400
    return density


def _convertToPx(value):
    matched = re.match(r"(\d+(?:\.\d+)?)?([a-z]*)$", value)
    if not matched:
        raise ValueError("unknown length value: %s" % value)

    length, unit = matched.groups()
    if unit == "":
        return float(length)
    elif unit == "cm":
        return float(length) * 96 / 2.54
    elif unit == "mm":
        return float(length) * 96 / 2.54 / 10
    elif unit == "in":
        return float(length) * 96
    elif unit == "pc":
        return float(length) * 96 / 6
    elif unit == "pt":
        return float(length) * 96 / 6
    elif unit == "px":
        return float(length)

    raise ValueError("unknown unit type: %s" % unit)


def get(filepath):
    """
    Return (width, height) for a given img file content
    no requirements
    :type filepath: Union[bytes, str, pathlib.Path]
    :rtype Tuple[int, int]
    """
    height = -1
    width = -1

    if isinstance(filepath, io.BytesIO):  # file-like object
        fhandle = filepath
    else:
        fhandle = open(filepath, 'rb')

    try:
        head = fhandle.read(31)
        size = len(head)
        # handle GIFs
        if size >= 10 and head[:6] in (b'GIF87a', b'GIF89a'):
            # Check to see if content_type is correct
            try:
                width, height = struct.unpack("<hh", head[6:10])
            except struct.error:
                raise ValueError("Invalid GIF file")
        # see png edition spec bytes are below chunk length then and finally the
        elif size >= 24 and head.startswith(b'\211PNG\r\n\032\n') and head[12:16] == b'IHDR':
            try:
                width, height = struct.unpack(">LL", head[16:24])
            except struct.error:
                raise ValueError("Invalid PNG file")
        # Maybe this is for an older PNG version.
        elif size >= 16 and head.startswith(b'\211PNG\r\n\032\n'):
            # Check to see if we have the right content type
            try:
                width, height = struct.unpack(">LL", head[8:16])
            except struct.error:
                raise ValueError("Invalid PNG file")
        # handle JPEGs
        elif size >= 2 and head.startswith(b'\377\330'):
            try:
```

```
        fhandle.seek(0)  # Read 0xff next
        size = 2
        ftype = 0
        while not 0xc0 <= ftype <= 0xcf or ftype in [0xc4, 0xc8, 0xcc]:
            fhandle.seek(size, 1)
            byte = fhandle.read(1)
            while ord(byte) == 0xff:
                byte = fhandle.read(1)
            ftype = ord(byte)
            size = struct.unpack('>H', fhandle.read(2))[0] - 2
        # We are at a SOFn block
        fhandle.seek(1, 1)  # Skip `precision' byte.
        height, width = struct.unpack('>HH', fhandle.read(4))
    except (struct.error, TypeError):
        raise ValueError("Invalid JPEG file")
# handle JPEG2000s
elif size >= 12 and head.startswith(b'\x00\x00\x00\x0cjP  \r\n\x87\n'):
    fhandle.seek(48)
    try:
        height, width = struct.unpack('>LL', fhandle.read(8))
    except struct.error:
        raise ValueError("Invalid JPEG2000 file")
# handle big endian TIFF
elif size >= 8 and head.startswith(b"\x4d\x4d\x00\x2a"):
    offset = struct.unpack('>L', head[4:8])[0]
    fhandle.seek(offset)
    ifdsize = struct.unpack(">H", fhandle.read(2))[0]
    for i in range(ifdsize):
        tag, datatype, count, data = struct.unpack(">HHLL", fhandle.read(12))
        if tag == 256:
            if datatype == 3:
                width = int(data / 65536)
            elif datatype == 4:
                width = data
            else:
                raise ValueError("Invalid TIFF file: width column data type should be SHORT/LONG.")
        elif tag == 257:
            if datatype == 3:
                height = int(data / 65536)
            elif datatype == 4:
                height = data
            else:
                raise ValueError("Invalid TIFF file: height column data type should be SHORT/LONG.")
        if width != -1 and height != -1:
            break
    if width == -1 or height == -1:
        raise ValueError("Invalid TIFF file: width and/or height IDS entries are missing.")
elif size >= 8 and head.startswith(b"\x49\x49\x2a\x00"):
    offset = struct.unpack('<L', head[4:8])[0]
    fhandle.seek(offset)
    ifdsize = struct.unpack("<H", fhandle.read(2))[0]
    for i in range(ifdsize):
        tag, datatype, count, data = struct.unpack("<HHLL", fhandle.read(12))
        if tag == 256:
            width = data
        elif tag == 257:
            height = data
        if width != -1 and height != -1:
            break
    if width == -1 or height == -1:
        raise ValueError("Invalid TIFF file: width and/or height IDS entries are missing.")
# handle little endian BigTiff
elif size >= 8 and head.startswith(b"\x49\x49\x2b\x00"):
    bytesize_offset = struct.unpack('<L', head[4:8])[0]
    if bytesize_offset != 8:
        raise ValueError('Invalid BigTIFF file: Expected offset to be 8, found {} instead.'.format(offset))
    offset = struct.unpack('<Q', head[8:16])[0]
    fhandle.seek(offset)
    ifdsize = struct.unpack("<Q", fhandle.read(8))[0]
    for i in range(ifdsize):
        tag, datatype, count, data = struct.unpack("<HHQQ", fhandle.read(20))
        if tag == 256:
            width = data
        elif tag == 257:
            height = data
        if width != -1 and height != -1:
            break
    if width == -1 or height == -1:
        raise ValueError("Invalid BigTIFF file: width and/or height IDS entries are missing.")

# handle SVGs
```

```
        elif size >= 5 and (head.startswith(b'<?xml') or head.startswith(b'<svg')):
            fhandle.seek(0)
            data = fhandle.read(1024)
            try:
                data = data.decode('utf-8')
                width = re.search(r'[^-]width="(.*?)"', data).group(1)
                height = re.search(r'[^-]height="(.*?)"', data).group(1)
            except Exception:
                raise ValueError("Invalid SVG file")
            width = _convertToPx(width)
            height = _convertToPx(height)

        # handle Netpbm
        elif head[:1] == b"P" and head[1:2] in b"123456":
            fhandle.seek(2)
            sizes = []

            while True:
                next_chr = fhandle.read(1)

                if next_chr.isspace():
                    continue

                if next_chr == b"":
                    raise ValueError("Invalid Netpbm file")

                if next_chr == b"#":
                    fhandle.readline()
                    continue

                if not next_chr.isdigit():
                    raise ValueError("Invalid character found on Netpbm file")

                size = next_chr
                next_chr = fhandle.read(1)

                while next_chr.isdigit():
                    size += next_chr
                    next_chr = fhandle.read(1)

                sizes.append(int(size))

                if len(sizes) == 2:
                    break

                fhandle.seek(-1, os.SEEK_CUR)
            width, height = sizes
        elif head.startswith(b"RIFF") and head[8:12] == b"WEBP":
            if head[12:16] == b"VP8 ":
                width, height = struct.unpack("<HH", head[26:30])
            elif head[12:16] == b"VP8X":
                width = struct.unpack("<I", head[24:27] + b"\0")[0]
                height = struct.unpack("<I", head[27:30] + b"\0")[0]
            elif head[12:16] == b"VP8L":
                b = head[21:25]
                width = (((b[1] & 63) << 8) | b[0]) + 1
                height = (((b[3] & 15) << 10) | (b[2] << 2) | ((b[1] & 192) >> 6)) + 1
            else:
                raise ValueError("Unsupported WebP file")

    finally:
        fhandle.close()

    return width, height


def getDPI(filepath):
    """
    Return (x DPI, y DPI) for a given img file content
    no requirements
    :type filepath: Union[bytes, str, pathlib.Path]
    :rtype Tuple[int, int]
    """
    xDPI = -1
    yDPI = -1

    if not isinstance(filepath, bytes):
        filepath = str(filepath)

    with open(filepath, 'rb') as fhandle:
        head = fhandle.read(24)
```

```
        size = len(head)
        # handle GIFs
        # GIFs doesn't have density
        if size >= 10 and head[:6] in (b'GIF87a', b'GIF89a'):
            pass
        # see png edition spec bytes are below chunk length then and finally the
        elif size >= 24 and head.startswith(b'\211PNG\r\n\032\n'):
            chunkOffset = 8
            chunk = head[8:]
            while True:
                chunkType = chunk[4:8]
                if chunkType == b'pHYs':
                    try:
                        xDensity, yDensity, unit = struct.unpack(">LLB", chunk[8:])
                    except struct.error:
                        raise ValueError("Invalid PNG file")
                    if unit:
                        xDPI = _convertToDPI(xDensity, _UNIT_1M)
                        yDPI = _convertToDPI(yDensity, _UNIT_1M)
                    else:  # no unit
                        xDPI = xDensity
                        yDPI = yDensity
                    break
                elif chunkType == b'IDAT':
                    break
                else:
                    try:
                        dataSize, = struct.unpack(">L", chunk[0:4])
                    except struct.error:
                        raise ValueError("Invalid PNG file")
                    chunkOffset += dataSize + 12
                    fhandle.seek(chunkOffset)
                    chunk = fhandle.read(17)
        # handle JPEGs
        elif size >= 2 and head.startswith(b'\377\330'):
            try:
                fhandle.seek(0)  # Read 0xff next
                size = 2
                ftype = 0
                while not 0xc0 <= ftype <= 0xcf:
                    if ftype == 0xe0:  # APP0 marker
                        fhandle.seek(7, 1)
                        unit, xDensity, yDensity = struct.unpack(">BHH", fhandle.read(5))
                        if unit == 1 or unit == 0:
                            xDPI = xDensity
                            yDPI = yDensity
                        elif unit == 2:
                            xDPI = _convertToDPI(xDensity, _UNIT_CM)
                            yDPI = _convertToDPI(yDensity, _UNIT_CM)
                        break
                    fhandle.seek(size, 1)
                    byte = fhandle.read(1)
                    while ord(byte) == 0xff:
                        byte = fhandle.read(1)
                    ftype = ord(byte)
                    size = struct.unpack('>H', fhandle.read(2))[0] - 2
            except struct.error:
                raise ValueError("Invalid JPEG file")
        # handle JPEG2000s
        elif size >= 12 and head.startswith(b'\x00\x00\x00\x0cjP  \r\n\x87\n'):
            fhandle.seek(32)
            # skip JP2 image header box
            headerSize = struct.unpack('>L', fhandle.read(4))[0] - 8
            fhandle.seek(4, 1)
            foundResBox = False
            try:
                while headerSize > 0:
                    boxHeader = fhandle.read(8)
                    boxType = boxHeader[4:]
                    if boxType == b'res ':  # find resolution super box
                        foundResBox = True
                        headerSize -= 8
                        break
                    boxSize, = struct.unpack('>L', boxHeader[:4])
                    fhandle.seek(boxSize - 8, 1)
                    headerSize -= boxSize
                if foundResBox:
                    while headerSize > 0:
                        boxHeader = fhandle.read(8)
                        boxType = boxHeader[4:]
                        if boxType == b'resd':  # Display resolution box
```

```
                            yDensity, xDensity, yUnit, xUnit = struct.unpack(">HHBB", fhandle.read(10))
                            xDPI = _convertToDPI(xDensity, xUnit)
                            yDPI = _convertToDPI(yDensity, yUnit)
                            break
                        boxSize, = struct.unpack('>L', boxHeader[:4])
                        fhandle.seek(boxSize - 8, 1)
                        headerSize -= boxSize
                except struct.error as e:
                    raise ValueError("Invalid JPEG2000 file")
    return xDPI, yDPI
```

File: .venv\Lib\site-packages\alabaster\support.py

```
from pygments.style import Style
from pygments.token import (
    Comment,
    Error,
    Generic,
    Keyword,
    Literal,
    Name,
    Number,
    Operator,
    Other,
    Punctuation,
    String,
    Whitespace,
)


# Originally based on FlaskyStyle which was based on 'tango'.
class Alabaster(Style):
    background_color = "#f8f8f8"  # doesn't seem to override CSS 'pre' styling?
    default_style = ""

    styles = {
        # No corresponding class for the following:
        # Text:                      "", # class:  ''
        Whitespace: "#f8f8f8",  # class: 'w'
        Error: "#a40000 border:#ef2929",  # class: 'err'
        Other: "#000000",  # class 'x'
        Comment: "italic #8f5902",  # class: 'c'
        Comment.Preproc: "noitalic",  # class: 'cp'
        Keyword: "bold #004461",  # class: 'k'
        Keyword.Constant: "bold #004461",  # class: 'kc'
        Keyword.Declaration: "bold #004461",  # class: 'kd'
        Keyword.Namespace: "bold #004461",  # class: 'kn'
        Keyword.Pseudo: "bold #004461",  # class: 'kp'
        Keyword.Reserved: "bold #004461",  # class: 'kr'
        Keyword.Type: "bold #004461",  # class: 'kt'
        Operator: "#582800",  # class: 'o'
        Operator.Word: "bold #004461",  # class: 'ow' - like keywords
        Punctuation: "bold #000000",  # class: 'p'
        # because special names such as Name.Class, Name.Function, etc.
        # are not recognized as such later in the parsing, we choose them
        # to look the same as ordinary variables.
        Name: "#000000",  # class: 'n'
        Name.Attribute: "#c4a000",  # class: 'na' - to be revised
        Name.Builtin: "#004461",  # class: 'nb'
        Name.Builtin.Pseudo: "#3465a4",  # class: 'bp'
        Name.Class: "#000000",  # class: 'nc' - to be revised
        Name.Constant: "#000000",  # class: 'no' - to be revised
        Name.Decorator: "#888",  # class: 'nd' - to be revised
        Name.Entity: "#ce5c00",  # class: 'ni'
        Name.Exception: "bold #cc0000",  # class: 'ne'
        Name.Function: "#000000",  # class: 'nf'
        Name.Property: "#000000",  # class: 'py'
        Name.Label: "#f57900",  # class: 'nl'
        Name.Namespace: "#000000",  # class: 'nn' - to be revised
        Name.Other: "#000000",  # class: 'nx'
        Name.Tag: "bold #004461",  # class: 'nt' - like a keyword
        Name.Variable: "#000000",  # class: 'nv' - to be revised
        Name.Variable.Class: "#000000",  # class: 'vc' - to be revised
        Name.Variable.Global: "#000000",  # class: 'vg' - to be revised
        Name.Variable.Instance: "#000000",  # class: 'vi' - to be revised
        Number: "#990000",  # class: 'm'
        Literal: "#000000",  # class: 'l'
        Literal.Date: "#000000",  # class: 'ld'
        String: "#4e9a06",  # class: 's'
```

```
        String.Backtick: "#4e9a06",  # class: 'sb'
        String.Char: "#4e9a06",  # class: 'sc'
        String.Doc: "italic #8f5902",  # class: 'sd' - like a comment
        String.Double: "#4e9a06",  # class: 's2'
        String.Escape: "#4e9a06",  # class: 'se'
        String.Heredoc: "#4e9a06",  # class: 'sh'
        String.Interpol: "#4e9a06",  # class: 'si'
        String.Other: "#4e9a06",  # class: 'sx'
        String.Regex: "#4e9a06",  # class: 'sr'
        String.Single: "#4e9a06",  # class: 's1'
        String.Symbol: "#4e9a06",  # class: 'ss'
        Generic: "#000000",  # class: 'g'
        Generic.Deleted: "#a40000",  # class: 'gd'
        Generic.Emph: "italic #000000",  # class: 'ge'
        Generic.Error: "#ef2929",  # class: 'gr'
        Generic.Heading: "bold #000080",  # class: 'gh'
        Generic.Inserted: "#00A000",  # class: 'gi'
        Generic.Output: "#888",  # class: 'go'
        Generic.Prompt: "#745334",  # class: 'gp'
        Generic.Strong: "bold #000000",  # class: 'gs'
        Generic.Subheading: "bold #800080",  # class: 'gu'
        Generic.Traceback: "bold #a40000",  # class: 'gt'
    }
```

File: .venv\Lib\site-packages\alabaster__init__.py

```
import os

__version_info__ = (0, 7, 16)
__version__ = "0.7.16"


def get_path():
    """
    Shortcut for users whose theme is next to their conf.py.
    """
    # Theme directory is defined as our parent directory
    return os.path.abspath(os.path.dirname(os.path.dirname(__file__)))


def update_context(app, pagename, templatename, context, doctree):
    context["alabaster_version"] = __version__
    context["alabaster_version_info"] = __version_info__

    # Convert 'show_powered_by' in the theme options to
    # the preferred option, html_show_sphinx.
    html_theme_options = app.config.html_theme_options
    if "show_powered_by" in html_theme_options:
        show_powered_by = html_theme_options["show_powered_by"]
        if isinstance(show_powered_by, str):
            context["show_sphinx"] = show_powered_by.lower() == "true"
        else:
            context["show_sphinx"] = bool(show_powered_by)  # to allow int values


def setup(app):
    app.require_sphinx("3.4")
    theme_path = os.path.abspath(os.path.dirname(__file__))
    app.add_html_theme("alabaster", theme_path)
    app.connect("html-page-context", update_context)
    return {"version": __version__, "parallel_read_safe": True}
```

File: .venv\Lib\site-packages\alabaster-0.7.16.dist-info\entry_points.txt

```
[sphinx.html_themes]
alabaster=alabaster
```

File: .venv\Lib\site-packages\babel\core.py

```
"""
    babel.core
    ~~~~~~~~~~

    Core locale representation and locale data access.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
```

```
"""

from __future__ import annotations

import os
import pickle
from collections.abc import Iterable, Mapping
from typing import TYPE_CHECKING, Any

from babel import localedata
from babel.plural import PluralRule

__all__ = ['UnknownLocaleError', 'Locale', 'default_locale', 'negotiate_locale',
           'parse_locale']

if TYPE_CHECKING:
    from typing_extensions import Literal, TypeAlias

    _GLOBAL_KEY: TypeAlias = Literal[
        "all_currencies",
        "currency_fractions",
        "language_aliases",
        "likely_subtags",
        "meta_zones",
        "parent_exceptions",
        "script_aliases",
        "territory_aliases",
        "territory_currencies",
        "territory_languages",
        "territory_zones",
        "variant_aliases",
        "windows_zone_mapping",
        "zone_aliases",
        "zone_territories",
    ]

    _global_data: Mapping[_GLOBAL_KEY, Mapping[str, Any]] | None

_global_data = None
_default_plural_rule = PluralRule({})


def _raise_no_data_error():
    raise RuntimeError('The babel data files are not available. '
                       'This usually happens because you are using '
                       'a source checkout from Babel and you did '
                       'not build the data files.  Just make sure '
                       'to run "python setup.py import_cldr" before '
                       'installing the library.')


def get_global(key: _GLOBAL_KEY) -> Mapping[str, Any]:
    """Return the dictionary for the given key in the global data.

    The global data is stored in the ``babel/global.dat`` file and contains
    information independent of individual locales.

    >>> get_global('zone_aliases')['UTC']
    u'Etc/UTC'
    >>> get_global('zone_territories')['Europe/Berlin']
    u'DE'

    The keys available are:

    - ``all_currencies``
    - ``currency_fractions``
    - ``language_aliases``
    - ``likely_subtags``
    - ``parent_exceptions``
    - ``script_aliases``
    - ``territory_aliases``
    - ``territory_currencies``
    - ``territory_languages``
    - ``territory_zones``
    - ``variant_aliases``
    - ``windows_zone_mapping``
    - ``zone_aliases``
    - ``zone_territories``

    .. note:: The internal structure of the data may change between versions.
    .. versionadded:: 0.9
```

```
    :param key: the data key
    """
    global _global_data
    if _global_data is None:
        dirname = os.path.join(os.path.dirname(__file__))
        filename = os.path.join(dirname, 'global.dat')
        if not os.path.isfile(filename):
            _raise_no_data_error()
        with open(filename, 'rb') as fileobj:
            _global_data = pickle.load(fileobj)
            assert _global_data is not None
    return _global_data.get(key, {})


LOCALE_ALIASES = {
    'ar': 'ar_SY', 'bg': 'bg_BG', 'bs': 'bs_BA', 'ca': 'ca_ES', 'cs': 'cs_CZ',
    'da': 'da_DK', 'de': 'de_DE', 'el': 'el_GR', 'en': 'en_US', 'es': 'es_ES',
    'et': 'et_EE', 'fa': 'fa_IR', 'fi': 'fi_FI', 'fr': 'fr_FR', 'gl': 'gl_ES',
    'he': 'he_IL', 'hu': 'hu_HU', 'id': 'id_ID', 'is': 'is_IS', 'it': 'it_IT',
    'ja': 'ja_JP', 'km': 'km_KH', 'ko': 'ko_KR', 'lt': 'lt_LT', 'lv': 'lv_LV',
    'mk': 'mk_MK', 'nl': 'nl_NL', 'nn': 'nn_NO', 'no': 'nb_NO', 'pl': 'pl_PL',
    'pt': 'pt_PT', 'ro': 'ro_RO', 'ru': 'ru_RU', 'sk': 'sk_SK', 'sl': 'sl_SI',
    'sv': 'sv_SE', 'th': 'th_TH', 'tr': 'tr_TR', 'uk': 'uk_UA',
}


class UnknownLocaleError(Exception):
    """Exception thrown when a locale is requested for which no locale data
    is available.
    """

    def __init__(self, identifier: str) -> None:
        """Create the exception.

        :param identifier: the identifier string of the unsupported locale
        """
        Exception.__init__(self, f"unknown locale {identifier!r}")

        #: The identifier of the locale that could not be found.
        self.identifier = identifier


class Locale:
    """Representation of a specific locale.

    >>> locale = Locale('en', 'US')
    >>> repr(locale)
    "Locale('en', territory='US')"
    >>> locale.display_name
    u'English (United States)'

    A `Locale` object can also be instantiated from a raw locale string:

    >>> locale = Locale.parse('en-US', sep='-')
    >>> repr(locale)
    "Locale('en', territory='US')"

    `Locale` objects provide access to a collection of locale data, such as
    territory and language names, number and date format patterns, and more:

    >>> locale.number_symbols['latn']['decimal']
    u'.'

    If a locale is requested for which no locale data is available, an
    `UnknownLocaleError` is raised:

    >>> Locale.parse('en_XX')
    Traceback (most recent call last):
        ...
    UnknownLocaleError: unknown locale 'en_XX'

    For more information see :rfc:`3066`.
    """

    def __init__(
        self,
        language: str,
        territory: str | None = None,
        script: str | None = None,
        variant: str | None = None,
        modifier: str | None = None,
```

```
) -> None:
    """Initialize the locale object from the given identifier components.

    >>> locale = Locale('en', 'US')
    >>> locale.language
    'en'
    >>> locale.territory
    'US'

    :param language: the language code
    :param territory: the territory (country or region) code
    :param script: the script code
    :param variant: the variant code
    :param modifier: a modifier (following the '@' symbol, sometimes called '@variant')
    :raise `UnknownLocaleError`: if no locale data is available for the
                                 requested locale
    """
    #: the language code
    self.language = language
    #: the territory (country or region) code
    self.territory = territory
    #: the script code
    self.script = script
    #: the variant code
    self.variant = variant
    #: the modifier
    self.modifier = modifier
    self.__data: localedata.LocaleDataDict | None = None

    identifier = str(self)
    identifier_without_modifier = identifier.partition('@')[0]
    if not localedata.exists(identifier_without_modifier):
        raise UnknownLocaleError(identifier)

@classmethod
def default(cls, category: str | None = None, aliases: Mapping[str, str] = LOCALE_ALIASES) -> Locale:
    """Return the system default locale for the specified category.

    >>> for name in ['LANGUAGE', 'LC_ALL', 'LC_CTYPE', 'LC_MESSAGES']:
    ...     os.environ[name] = ''
    >>> os.environ['LANG'] = 'fr_FR.UTF-8'
    >>> Locale.default('LC_MESSAGES')
    Locale('fr', territory='FR')

    The following fallbacks to the variable are always considered:

    - ``LANGUAGE``
    - ``LC_ALL``
    - ``LC_CTYPE``
    - ``LANG``

    :param category: one of the ``LC_XXX`` environment variable names
    :param aliases: a dictionary of aliases for locale identifiers
    """
    # XXX: use likely subtag expansion here instead of the
    # aliases dictionary.
    locale_string = default_locale(category, aliases=aliases)
    return cls.parse(locale_string)

@classmethod
def negotiate(
    cls,
    preferred: Iterable[str],
    available: Iterable[str],
    sep: str = '_',
    aliases: Mapping[str, str] = LOCALE_ALIASES,
) -> Locale | None:
    """Find the best match between available and requested locale strings.

    >>> Locale.negotiate(['de_DE', 'en_US'], ['de_DE', 'de_AT'])
    Locale('de', territory='DE')
    >>> Locale.negotiate(['de_DE', 'en_US'], ['en', 'de'])
    Locale('de')
    >>> Locale.negotiate(['de_DE', 'de'], ['en_US'])

    You can specify the character used in the locale identifiers to separate
    the different components. This separator is applied to both lists. Also,
    case is ignored in the comparison:

    >>> Locale.negotiate(['de-DE', 'de'], ['en-us', 'de-de'], sep='-')
    Locale('de', territory='DE')
```

```
    :param preferred: the list of locale identifiers preferred by the user
    :param available: the list of locale identifiers available
    :param aliases: a dictionary of aliases for locale identifiers
    """
    identifier = negotiate_locale(preferred, available, sep=sep,
                                  aliases=aliases)
    if identifier:
        return Locale.parse(identifier, sep=sep)
    return None

@classmethod
def parse(
    cls,
    identifier: str | Locale | None,
    sep: str = '_',
    resolve_likely_subtags: bool = True,
) -> Locale:
    """Create a `Locale` instance for the given locale identifier.

    >>> l = Locale.parse('de-DE', sep='-')
    >>> l.display_name
    u'Deutsch (Deutschland)'

    If the `identifier` parameter is not a string, but actually a `Locale`
    object, that object is returned:

    >>> Locale.parse(l)
    Locale('de', territory='DE')

    If the `identifier` parameter is neither of these, such as `None`
    e.g. because a default locale identifier could not be determined,
    a `TypeError` is raised:

    >>> Locale.parse(None)
    Traceback (most recent call last):
        ...
    TypeError: ...

    This also can perform resolving of likely subtags which it does
    by default.  This is for instance useful to figure out the most
    likely locale for a territory you can use ``'und'`` as the
    language tag:

    >>> Locale.parse('und_AT')
    Locale('de', territory='AT')

    Modifiers are optional, and always at the end, separated by "@":

    >>> Locale.parse('de_AT@euro')
    Locale('de', territory='AT', modifier='euro')

    :param identifier: the locale identifier string
    :param sep: optional component separator
    :param resolve_likely_subtags: if this is specified then a locale will
                                   have its likely subtag resolved if the
                                   locale otherwise does not exist.  For
                                   instance ``zh_TW`` by itself is not a
                                   locale that exists but Babel can
                                   automatically expand it to the full
                                   form of ``zh_hant_TW``.  Note that this
                                   expansion is only taking place if no
                                   locale exists otherwise.  For instance
                                   there is a locale ``en`` that can exist
                                   by itself.
    :raise `ValueError`: if the string does not appear to be a valid locale
                         identifier
    :raise `UnknownLocaleError`: if no locale data is available for the
                                 requested locale
    :raise `TypeError`: if the identifier is not a string or a `Locale`
    """
    if isinstance(identifier, Locale):
        return identifier
    elif not isinstance(identifier, str):
        raise TypeError(f"Unexpected value for identifier: {identifier!r}")

    parts = parse_locale(identifier, sep=sep)
    input_id = get_locale_identifier(parts)

    def _try_load(parts):
        try:
            return cls(*parts)
```

```
        except UnknownLocaleError:
            return None

    def _try_load_reducing(parts):
        # Success on first hit, return it.
        locale = _try_load(parts)
        if locale is not None:
            return locale

        # Now try without script and variant
        locale = _try_load(parts[:2])
        if locale is not None:
            return locale

    locale = _try_load(parts)
    if locale is not None:
        return locale
    if not resolve_likely_subtags:
        raise UnknownLocaleError(input_id)

    # From here onwards is some very bad likely subtag resolving.  This
    # whole logic is not entirely correct but good enough (tm) for the
    # time being.  This has been added so that zh_TW does not cause
    # errors for people when they upgrade.  Later we should properly
    # implement ICU like fuzzy locale objects and provide a way to
    # maximize and minimize locale tags.

    if len(parts) == 5:
        language, territory, script, variant, modifier = parts
    else:
        language, territory, script, variant = parts
        modifier = None
    language = get_global('language_aliases').get(language, language)
    territory = get_global('territory_aliases').get(territory or '', (territory,))[0]
    script = get_global('script_aliases').get(script or '', script)
    variant = get_global('variant_aliases').get(variant or '', variant)

    if territory == 'ZZ':
        territory = None
    if script == 'Zzzz':
        script = None

    parts = language, territory, script, variant, modifier

    # First match: try the whole identifier
    new_id = get_locale_identifier(parts)
    likely_subtag = get_global('likely_subtags').get(new_id)
    if likely_subtag is not None:
        locale = _try_load_reducing(parse_locale(likely_subtag))
        if locale is not None:
            return locale

    # If we did not find anything so far, try again with a
    # simplified identifier that is just the language
    likely_subtag = get_global('likely_subtags').get(language)
    if likely_subtag is not None:
        parts2 = parse_locale(likely_subtag)
        if len(parts2) == 5:
            language2, _, script2, variant2, modifier2 = parts2
        else:
            language2, _, script2, variant2 = parts2
            modifier2 = None
        locale = _try_load_reducing((language2, territory, script2, variant2, modifier2))
        if locale is not None:
            return locale

    raise UnknownLocaleError(input_id)

def __eq__(self, other: object) -> bool:
    for key in ('language', 'territory', 'script', 'variant', 'modifier'):
        if not hasattr(other, key):
            return False
    return (
        self.language == getattr(other, 'language') and  # noqa: B009
        self.territory == getattr(other, 'territory') and  # noqa: B009
        self.script == getattr(other, 'script') and  # noqa: B009
        self.variant == getattr(other, 'variant') and  # noqa: B009
        self.modifier == getattr(other, 'modifier')  # noqa: B009
    )

def __ne__(self, other: object) -> bool:
```

```
    return not self.__eq__(other)

def __hash__(self) -> int:
    return hash((self.language, self.territory, self.script,
                 self.variant, self.modifier))

def __repr__(self) -> str:
    parameters = ['']
    for key in ('territory', 'script', 'variant', 'modifier'):
        value = getattr(self, key)
        if value is not None:
            parameters.append(f"{key}={value!r}")
    return f"Locale({self.language!r}{', '.join(parameters)})"

def __str__(self) -> str:
    return get_locale_identifier((self.language, self.territory,
                                  self.script, self.variant,
                                  self.modifier))

@property
def _data(self) -> localedata.LocaleDataDict:
    if self.__data is None:
        self.__data = localedata.LocaleDataDict(localedata.load(str(self)))
    return self.__data

def get_display_name(self, locale: Locale | str | None = None) -> str | None:
    """Return the display name of the locale using the given locale.

    The display name will include the language, territory, script, and
    variant, if those are specified.

    >>> Locale('zh', 'CN', script='Hans').get_display_name('en')
    u'Chinese (Simplified, China)'

    Modifiers are currently passed through verbatim:

    >>> Locale('it', 'IT', modifier='euro').get_display_name('en')
    u'Italian (Italy, euro)'

    :param locale: the locale to use
    """
    if locale is None:
        locale = self
    locale = Locale.parse(locale)
    retval = locale.languages.get(self.language)
    if retval and (self.territory or self.script or self.variant):
        details = []
        if self.script:
            details.append(locale.scripts.get(self.script))
        if self.territory:
            details.append(locale.territories.get(self.territory))
        if self.variant:
            details.append(locale.variants.get(self.variant))
        if self.modifier:
            details.append(self.modifier)
        detail_string = ', '.join(atom for atom in details if atom)
        if detail_string:
            retval += f" ({detail_string})"
    return retval

display_name = property(get_display_name, doc="""\
    The localized display name of the locale.

    >>> Locale('en').display_name
    u'English'
    >>> Locale('en', 'US').display_name
    u'English (United States)'
    >>> Locale('sv').display_name
    u'svenska'

    :type: `unicode`
    """)

def get_language_name(self, locale: Locale | str | None = None) -> str | None:
    """Return the language of this locale in the given locale.

    >>> Locale('zh', 'CN', script='Hans').get_language_name('de')
    u'Chinesisch'

    .. versionadded:: 1.0
    :param locale: the locale to use
```

```
        """
        if locale is None:
            locale = self
        locale = Locale.parse(locale)
        return locale.languages.get(self.language)

    language_name = property(get_language_name, doc="""\
        The localized language name of the locale.

        >>> Locale('en', 'US').language_name
        u'English'
    """)

    def get_territory_name(self, locale: Locale | str | None = None) -> str | None:
        """Return the territory name in the given locale."""
        if locale is None:
            locale = self
        locale = Locale.parse(locale)
        return locale.territories.get(self.territory or '')

    territory_name = property(get_territory_name, doc="""\
        The localized territory name of the locale if available.

        >>> Locale('de', 'DE').territory_name
        u'Deutschland'
    """)

    def get_script_name(self, locale: Locale | str | None = None) -> str | None:
        """Return the script name in the given locale."""
        if locale is None:
            locale = self
        locale = Locale.parse(locale)
        return locale.scripts.get(self.script or '')

    script_name = property(get_script_name, doc="""\
        The localized script name of the locale if available.

        >>> Locale('sr', 'ME', script='Latn').script_name
        u'latinica'
    """)

    @property
    def english_name(self) -> str | None:
        """The english display name of the locale.

        >>> Locale('de').english_name
        u'German'
        >>> Locale('de', 'DE').english_name
        u'German (Germany)'

        :type: `unicode`"""
        return self.get_display_name(Locale('en'))

    # { General Locale Display Names

    @property
    def languages(self) -> localedata.LocaleDataDict:
        """Mapping of language codes to translated language names.

        >>> Locale('de', 'DE').languages['ja']
        u'Japanisch'

        See `ISO 639 <http://www.loc.gov/standards/iso639-2/>`_ for
        more information.
        """
        return self._data['languages']

    @property
    def scripts(self) -> localedata.LocaleDa
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\babel\languages.py

```
from __future__ import annotations

from babel.core import get_global


def get_official_languages(territory: str, regional: bool = False, de_facto: bool = False) -> tuple[str, ...]:
```

```
    """
    Get the official language(s) for the given territory.

    The language codes, if any are known, are returned in order of descending popularity.

    If the `regional` flag is set, then languages which are regionally official are also returned.

    If the `de_facto` flag is set, then languages which are "de facto" official are also returned.

    .. warning:: Note that the data is as up to date as the current version of the CLDR used
                 by Babel.  If you need scientifically accurate information, use another source!

    :param territory: Territory code
    :type territory: str
    :param regional: Whether to return regionally official languages too
    :type regional: bool
    :param de_facto: Whether to return de-facto official languages too
    :type de_facto: bool
    :return: Tuple of language codes
    :rtype: tuple[str]
    """

    territory = str(territory).upper()
    allowed_stati = {"official"}
    if regional:
        allowed_stati.add("official_regional")
    if de_facto:
        allowed_stati.add("de_facto_official")

    languages = get_global("territory_languages").get(territory, {})
    pairs = [
        (info['population_percent'], language)
        for language, info in languages.items()
        if info.get('official_status') in allowed_stati
    ]
    pairs.sort(reverse=True)
    return tuple(lang for _, lang in pairs)


def get_territory_language_info(territory: str) -> dict[str, dict[str, float | str | None]]:
    """
    Get a dictionary of language information for a territory.

    The dictionary is keyed by language code; the values are dicts with more information.

    The following keys are currently known for the values:

    * `population_percent`: The percentage of the territory's population speaking the
                            language.
    * `official_status`: An optional string describing the officiality status of the language.
                         Known values are "official", "official_regional" and "de_facto_official".

    .. warning:: Note that the data is as up to date as the current version of the CLDR used
                 by Babel.  If you need scientifically accurate information, use another source!

    .. note:: Note that the format of the dict returned may change between Babel versions.

    See https://www.unicode.org/cldr/charts/latest/supplemental/territory_language_information.html

    :param territory: Territory code
    :type territory: str
    :return: Language information dictionary
    :rtype: dict[str, dict]
    """
    territory = str(territory).upper()
    return get_global("territory_languages").get(territory, {}).copy()
```

File: .venv\Lib\site-packages\babel\lists.py

```
"""
    babel.lists
    ~~~~~~~~~~~

    Locale dependent formatting of lists.

    The default locale for the functions in this module is determined by the
    following environment variables, in that order:

     * ``LC_ALL``, and
```

```
     * ``LANG``

    :copyright: (c) 2015-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

from collections.abc import Sequence
from typing import TYPE_CHECKING

from babel.core import Locale, default_locale

if TYPE_CHECKING:
    from typing_extensions import Literal

DEFAULT_LOCALE = default_locale()


def format_list(lst: Sequence[str],
                style: Literal['standard', 'standard-short', 'or', 'or-short', 'unit', 'unit-short', 'unit-narrow'] = 's
                locale: Locale | str | None = DEFAULT_LOCALE) -> str:
    """
    Format the items in `lst` as a list.

    >>> format_list(['apples', 'oranges', 'pears'], locale='en')
    u'apples, oranges, and pears'
    >>> format_list(['apples', 'oranges', 'pears'], locale='zh')
    u'apples\u3001oranges\u548cpears'
    >>> format_list(['omena', 'peruna', 'aplari'], style='or', locale='fi')
    u'omena, peruna tai aplari'

    These styles are defined, but not all are necessarily available in all locales.
    The following text is verbatim from the Unicode TR35-49 spec [1].

    * standard:
      A typical 'and' list for arbitrary placeholders.
      eg. "January, February, and March"
    * standard-short:
      A short version of an 'and' list, suitable for use with short or abbreviated placeholder values.
      eg. "Jan., Feb., and Mar."
    * or:
      A typical 'or' list for arbitrary placeholders.
      eg. "January, February, or March"
    * or-short:
      A short version of an 'or' list.
      eg. "Jan., Feb., or Mar."
    * unit:
      A list suitable for wide units.
      eg. "3 feet, 7 inches"
    * unit-short:
      A list suitable for short units
      eg. "3 ft, 7 in"
    * unit-narrow:
      A list suitable for narrow units, where space on the screen is very limited.
      eg. "3′ 7″"

    [1]: https://www.unicode.org/reports/tr35/tr35-49/tr35-general.html#ListPatterns

    :param lst: a sequence of items to format in to a list
    :param style: the style to format the list with. See above for description.
    :param locale: the locale
    """
    locale = Locale.parse(locale)
    if not lst:
        return ''
    if len(lst) == 1:
        return lst[0]

    if style not in locale.list_patterns:
        raise ValueError(
            f'Locale {locale} does not support list formatting style {style!r} '
            f'(supported are {sorted(locale.list_patterns)})',
        )
    patterns = locale.list_patterns[style]

    if len(lst) == 2:
        return patterns['2'].format(*lst)

    result = patterns['start'].format(lst[0], lst[1])
    for elem in lst[2:-1]:
        result = patterns['middle'].format(result, elem)
```

```
    result = patterns['end'].format(result, lst[-1])

    return result
```

File: .venv\Lib\site-packages\babel\localedata.py

```
"""
    babel.localedata
    ~~~~~~~~~~~~~~~~

    Low-level locale data access.

    :note: The `Locale` class, which uses this module under the hood, provides a
           more convenient interface for accessing the locale data.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""

from __future__ import annotations

import os
import pickle
import re
import sys
import threading
from collections import abc
from collections.abc import Iterator, Mapping, MutableMapping
from functools import lru_cache
from itertools import chain
from typing import Any

_cache: dict[str, Any] = {}
_cache_lock = threading.RLock()
_dirname = os.path.join(os.path.dirname(__file__), 'locale-data')
_windows_reserved_name_re = re.compile("^(con|prn|aux|nul|com[0-9]|lpt[0-9])$", re.I)


def normalize_locale(name: str) -> str | None:
    """Normalize a locale ID by stripping spaces and apply proper casing.

    Returns the normalized locale ID string or `None` if the ID is not
    recognized.
    """
    if not name or not isinstance(name, str):
        return None
    name = name.strip().lower()
    for locale_id in chain.from_iterable([_cache, locale_identifiers()]):
        if name == locale_id.lower():
            return locale_id


def resolve_locale_filename(name: os.PathLike[str] | str) -> str:
    """
    Resolve a locale identifier to a `.dat` path on disk.
    """

    # Clean up any possible relative paths.
    name = os.path.basename(name)

    # Ensure we're not left with one of the Windows reserved names.
    if sys.platform == "win32" and _windows_reserved_name_re.match(os.path.splitext(name)[0]):
        raise ValueError(f"Name {name} is invalid on Windows")

    # Build the path.
    return os.path.join(_dirname, f"{name}.dat")


def exists(name: str) -> bool:
    """Check whether locale data is available for the given locale.

    Returns `True` if it exists, `False` otherwise.

    :param name: the locale identifier string
    """
    if not name or not isinstance(name, str):
        return False
    if name in _cache:
        return True
```

```
    file_found = os.path.exists(resolve_locale_filename(name))
    return True if file_found else bool(normalize_locale(name))


@lru_cache(maxsize=None)
def locale_identifiers() -> list[str]:
    """Return a list of all locale identifiers for which locale data is
    available.

    This data is cached after the first invocation.
    You can clear the cache by calling `locale_identifiers.cache_clear()`.

    .. versionadded:: 0.8.1

    :return: a list of locale identifiers (strings)
    """
    return [
        stem
        for stem, extension in
        (os.path.splitext(filename) for filename in os.listdir(_dirname))
        if extension == '.dat' and stem != 'root'
    ]


def load(name: os.PathLike[str] | str, merge_inherited: bool = True) -> dict[str, Any]:
    """Load the locale data for the given locale.

    The locale data is a dictionary that contains much of the data defined by
    the Common Locale Data Repository (CLDR). This data is stored as a
    collection of pickle files inside the ``babel`` package.

    >>> d = load('en_US')
    >>> d['languages']['sv']
    u'Swedish'

    Note that the results are cached, and subsequent requests for the same
    locale return the same dictionary:

    >>> d1 = load('en_US')
    >>> d2 = load('en_US')
    >>> d1 is d2
    True

    :param name: the locale identifier string (or "root")
    :param merge_inherited: whether the inherited data should be merged into
                            the data of the requested locale
    :raise `IOError`: if no locale data file is found for the given locale
                      identifier, or one of the locales it inherits from
    """
    name = os.path.basename(name)
    _cache_lock.acquire()
    try:
        data = _cache.get(name)
        if not data:
            # Load inherited data
            if name == 'root' or not merge_inherited:
                data = {}
            else:
                from babel.core import get_global
                parent = get_global('parent_exceptions').get(name)
                if not parent:
                    parts = name.split('_')
                    parent = "root" if len(parts) == 1 else "_".join(parts[:-1])
                data = load(parent).copy()
            filename = resolve_locale_filename(name)
            with open(filename, 'rb') as fileobj:
                if name != 'root' and merge_inherited:
                    merge(data, pickle.load(fileobj))
                else:
                    data = pickle.load(fileobj)
            _cache[name] = data
        return data
    finally:
        _cache_lock.release()


def merge(dict1: MutableMapping[Any, Any], dict2: Mapping[Any, Any]) -> None:
    """Merge the data from `dict2` into the `dict1` dictionary, making copies
    of nested dictionaries.

    >>> d = {1: 'foo', 3: 'baz'}
```

```
    >>> merge(d, {1: 'Foo', 2: 'Bar'})
    >>> sorted(d.items())
    [(1, 'Foo'), (2, 'Bar'), (3, 'baz')]

    :param dict1: the dictionary to merge into
    :param dict2: the dictionary containing the data that should be merged
    """
    for key, val2 in dict2.items():
        if val2 is not None:
            val1 = dict1.get(key)
            if isinstance(val2, dict):
                if val1 is None:
                    val1 = {}
                if isinstance(val1, Alias):
                    val1 = (val1, val2)
                elif isinstance(val1, tuple):
                    alias, others = val1
                    others = others.copy()
                    merge(others, val2)
                    val1 = (alias, others)
                else:
                    val1 = val1.copy()
                    merge(val1, val2)
            else:
                val1 = val2
            dict1[key] = val1


class Alias:
    """Representation of an alias in the locale data.

    An alias is a value that refers to some other part of the locale data,
    as specified by the `keys`.
    """

    def __init__(self, keys: tuple[str, ...]) -> None:
        self.keys = tuple(keys)

    def __repr__(self) -> str:
        return f"<{type(self).__name__} {self.keys!r}>"

    def resolve(self, data: Mapping[str | int | None, Any]) -> Mapping[str | int | None, Any]:
        """Resolve the alias based on the given data.

        This is done recursively, so if one alias resolves to a second alias,
        that second alias will also be resolved.

        :param data: the locale data
        :type data: `dict`
        """
        base = data
        for key in self.keys:
            data = data[key]
        if isinstance(data, Alias):
            data = data.resolve(base)
        elif isinstance(data, tuple):
            alias, others = data
            data = alias.resolve(base)
        return data


class LocaleDataDict(abc.MutableMapping):
    """Dictionary wrapper that automatically resolves aliases to the actual
    values.
    """

    def __init__(self, data: MutableMapping[str | int | None, Any], base: Mapping[str | int | None, Any] | None = None):
        self._data = data
        if base is None:
            base = data
        self.base = base

    def __len__(self) -> int:
        return len(self._data)

    def __iter__(self) -> Iterator[str | int | None]:
        return iter(self._data)

    def __getitem__(self, key: str | int | None) -> Any:
        orig = val = self._data[key]
        if isinstance(val, Alias):  # resolve an alias
```

```
            val = val.resolve(self.base)
        if isinstance(val, tuple):  # Merge a partial dict with an alias
            alias, others = val
            val = alias.resolve(self.base).copy()
            merge(val, others)
        if isinstance(val, dict):  # Return a nested alias-resolving dict
            val = LocaleDataDict(val, base=self.base)
        if val is not orig:
            self._data[key] = val
        return val

    def __setitem__(self, key: str | int | None, value: Any) -> None:
        self._data[key] = value

    def __delitem__(self, key: str | int | None) -> None:
        del self._data[key]

    def copy(self) -> LocaleDataDict:
        return LocaleDataDict(self._data.copy(), base=self.base)
```

File: .venv\Lib\site-packages\babel\plural.py

```
"""
    babel.numbers
    ~~~~~~~~~~~~~

    CLDR Plural support.  See UTS #35.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

import decimal
import re
from collections.abc import Iterable, Mapping
from typing import TYPE_CHECKING, Any, Callable

if TYPE_CHECKING:
    from typing_extensions import Literal

_plural_tags = ('zero', 'one', 'two', 'few', 'many', 'other')
_fallback_tag = 'other'


def extract_operands(source: float | decimal.Decimal) -> tuple[decimal.Decimal | int, int, int, int, int, int, Literal[0
    """Extract operands from a decimal, a float or an int, according to `CLDR rules`_.

    The result is an 8-tuple (n, i, v, w, f, t, c, e), where those symbols are as follows:

    ====== ===============================================================
    Symbol Value
    ------ ---------------------------------------------------------------
    n      absolute value of the source number (integer and decimals).
    i      integer digits of n.
    v      number of visible fraction digits in n, with trailing zeros.
    w      number of visible fraction digits in n, without trailing zeros.
    f      visible fractional digits in n, with trailing zeros.
    t      visible fractional digits in n, without trailing zeros.
    c      compact decimal exponent value: exponent of the power of 10 used in compact decimal formatting.
    e      currently, synonym for ‘c’. however, may be redefined in the future.
    ====== ===============================================================

    .. _`CLDR rules`: https://www.unicode.org/reports/tr35/tr35-61/tr35-numbers.html#Operands

    :param source: A real number
    :type source: int|float|decimal.Decimal
    :return: A n-i-v-w-f-t-c-e tuple
    :rtype: tuple[decimal.Decimal, int, int, int, int, int, int, int]
    """
    n = abs(source)
    i = int(n)
    if isinstance(n, float):
        if i == n:
            n = i
        else:
            # Cast the `float` to a number via the string representation.
            # This is required for Python 2.6 anyway (it will straight out fail to
            # do the conversion otherwise), and it's highly unlikely that the user
```

```
            # actually wants the lossless conversion behavior (quoting the Python
            # documentation):
            # > If value is a float, the binary floating point value is losslessly
            # > converted to its exact decimal equivalent.
            # > This conversion can often require 53 or more digits of precision.
            # Should the user want that behavior, they can simply pass in a pre-
            # converted `Decimal` instance of desired accuracy.
            n = decimal.Decimal(str(n))

    if isinstance(n, decimal.Decimal):
        dec_tuple = n.as_tuple()
        exp = dec_tuple.exponent
        fraction_digits = dec_tuple.digits[exp:] if exp < 0 else ()
        trailing = ''.join(str(d) for d in fraction_digits)
        no_trailing = trailing.rstrip('0')
        v = len(trailing)
        w = len(no_trailing)
        f = int(trailing or 0)
        t = int(no_trailing or 0)
    else:
        v = w = f = t = 0
    c = e = 0  # TODO: c and e are not supported
    return n, i, v, w, f, t, c, e


class PluralRule:
    """Represents a set of language pluralization rules.  The constructor
    accepts a list of (tag, expr) tuples or a dict of `CLDR rules`_. The
    resulting object is callable and accepts one parameter with a positive or
    negative number (both integer and float) for the number that indicates the
    plural form for a string and returns the tag for the format:

    >>> rule = PluralRule({'one': 'n is 1'})
    >>> rule(1)
    'one'
    >>> rule(2)
    'other'

    Currently the CLDR defines these tags: zero, one, two, few, many and
    other where other is an implicit default.  Rules should be mutually
    exclusive; for a given numeric value, only one rule should apply (i.e.
    the condition should only be true for one of the plural rule elements.

    .. _`CLDR rules`: https://www.unicode.org/reports/tr35/tr35-33/tr35-numbers.html#Language_Plural_Rules
    """

    __slots__ = ('abstract', '_func')

    def __init__(self, rules: Mapping[str, str] | Iterable[tuple[str, str]]) -> None:
        """Initialize the rule instance.

        :param rules: a list of ``(tag, expr)``) tuples with the rules
                      conforming to UTS #35 or a dict with the tags as keys
                      and expressions as values.
        :raise RuleError: if the expression is malformed
        """
        if isinstance(rules, Mapping):
            rules = rules.items()
        found = set()
        self.abstract: list[tuple[str, Any]] = []
        for key, expr in sorted(rules):
            if key not in _plural_tags:
                raise ValueError(f"unknown tag {key!r}")
            elif key in found:
                raise ValueError(f"tag {key!r} defined twice")
            found.add(key)
            ast = _Parser(expr).ast
            if ast:
                self.abstract.append((key, ast))

    def __repr__(self) -> str:
        rules = self.rules
        args = ", ".join([f"{tag}: {rules[tag]}" for tag in _plural_tags if tag in rules])
        return f"<{type(self).__name__} {args!r}>"

    @classmethod
    def parse(cls, rules: Mapping[str, str] | Iterable[tuple[str, str]] | PluralRule) -> PluralRule:
        """Create a `PluralRule` instance for the given rules.  If the rules
        are a `PluralRule` object, that object is returned.

        :param rules: the rules as list or dict, or a `PluralRule` object
```

```
        :raise RuleError: if the expression is malformed
        """
        if isinstance(rules, PluralRule):
            return rules
        return cls(rules)

    @property
    def rules(self) -> Mapping[str, str]:
        """The `PluralRule` as a dict of unicode plural rules.

        >>> rule = PluralRule({'one': 'n is 1'})
        >>> rule.rules
        {'one': 'n is 1'}
        """
        _compile = _UnicodeCompiler().compile
        return {tag: _compile(ast) for tag, ast in self.abstract}

    @property
    def tags(self) -> frozenset[str]:
        """A set of explicitly defined tags in this rule.  The implicit default
        ``'other'`` rules is not part of this set unless there is an explicit
        rule for it.
        """
        return frozenset(i[0] for i in self.abstract)

    def __getstate__(self) -> list[tuple[str, Any]]:
        return self.abstract

    def __setstate__(self, abstract: list[tuple[str, Any]]) -> None:
        self.abstract = abstract

    def __call__(self, n: float | decimal.Decimal) -> str:
        if not hasattr(self, '_func'):
            self._func = to_python(self)
        return self._func(n)


def to_javascript(rule: Mapping[str, str] | Iterable[tuple[str, str]] | PluralRule) -> str:
    """Convert a list/dict of rules or a `PluralRule` object into a JavaScript
    function.  This function depends on no external library:

    >>> to_javascript({'one': 'n is 1'})
    "(function(n) { return (n == 1) ? 'one' : 'other'; })"

    Implementation detail: The function generated will probably evaluate
    expressions involved into range operations multiple times.  This has the
    advantage that external helper functions are not required and is not a
    big performance hit for these simple calculations.

    :param rule: the rules as list or dict, or a `PluralRule` object
    :raise RuleError: if the expression is malformed
    """
    to_js = _JavaScriptCompiler().compile
    result = ['(function(n) { return ']
    for tag, ast in PluralRule.parse(rule).abstract:
        result.append(f"{to_js(ast)} ? {tag!r} : ")
    result.append('%r; })' % _fallback_tag)
    return ''.join(result)


def to_python(rule: Mapping[str, str] | Iterable[tuple[str, str]] | PluralRule) -> Callable[[float | decimal.Decimal], s
    """Convert a list/dict of rules or a `PluralRule` object into a regular
    Python function.  This is useful in situations where you need a real
    function and don't are about the actual rule object:

    >>> func = to_python({'one': 'n is 1', 'few': 'n in 2..4'})
    >>> func(1)
    'one'
    >>> func(3)
    'few'
    >>> func = to_python({'one': 'n in 1,11', 'few': 'n in 3..10,13..19'})
    >>> func(11)
    'one'
    >>> func(15)
    'few'

    :param rule: the rules as list or dict, or a `PluralRule` object
    :raise RuleError: if the expression is malformed
    """
    namespace = {
        'IN': in_range_list,
```

```
        'WITHIN': within_range_list,
        'MOD': cldr_modulo,
        'extract_operands': extract_operands,
    }
    to_python_func = _PythonCompiler().compile
    result = [
        'def evaluate(n):',
        ' n, i, v, w, f, t, c, e = extract_operands(n)',
    ]
    for tag, ast in PluralRule.parse(rule).abstract:
        # the str() call is to coerce the tag to the native string.  It's
        # a limited ascii restricted set of tags anyways so that is fine.
        result.append(f" if ({to_python_func(ast)}): return {str(tag)!r}")
    result.append(f" return {_fallback_tag!r}")
    code = compile('\n'.join(result), '<rule>', 'exec')
    eval(code, namespace)
    return namespace['evaluate']


def to_gettext(rule: Mapping[str, str] | Iterable[tuple[str, str]] | PluralRule) -> str:
    """The plural rule as gettext expression.  The gettext expression is
    technically limited to integers and returns indices rather than tags.

    >>> to_gettext({'one': 'n is 1', 'two': 'n is 2'})
    'nplurals=3; plural=((n == 1) ? 0 : (n == 2) ? 1 : 2);'

    :param rule: the rules as list or dict, or a `PluralRule` object
    :raise RuleError: if the expression is malformed
    """
    rule = PluralRule.parse(rule)

    used_tags = rule.tags | {_fallback_tag}
    _compile = _GettextCompiler().compile
    _get_index = [tag for tag in _plural_tags if tag in used_tags].index

    result = [f"nplurals={len(used_tags)}; plural=("]
    for tag, ast in rule.abstract:
        result.append(f"{_compile(ast)} ? {_get_index(tag)} : ")
    result.append(f"{_get_index(_fallback_tag)});")
    return ''.join(result)


def in_range_list(num: float | decimal.Decimal, range_list: Iterable[Iterable[float | decimal.Decimal]]) -> bool:
    """Integer range list test.  This is the callback for the "in" operator
    of the UTS #35 pluralization rule language:

    >>> in_range_list(1, [(1, 3)])
    True
    >>> in_range_list(3, [(1, 3)])
    True
    >>> in_range_list(3, [(1, 3), (5, 8)])
    True
    >>> in_range_list(1.2, [(1, 4)])
    False
    >>> in_range_list(10, [(1, 4)])
    False
    >>> in_range_list(10, [(1, 4), (6, 8)])
    False
    """
    return num == int(num) and within_range_list(num, range_list)


def within_range_list(num: float | decimal.Decimal, range_list: Iterable[Iterable[float | decimal.Decimal]]) -> bool:
    """Float range test.  This is the callback for the "within" operator
    of the UTS #35 pluralization rule language:

    >>> within_range_list(1, [(1, 3)])
    True
    >>> within_range_list(1.0, [(1, 3)])
    True
    >>> within_range_list(1.2, [(1, 4)])
    True
    >>> within_range_list(8.8, [(1, 4), (7, 15)])
    True
    >>> within_range_list(10, [(1, 4)])
    False
    >>> within_range_list(10.5, [(1, 4), (20, 30)])
    False
    """
    return any(num >= min_ and num <= max_ for min_, max_ in range_list)
def cldr_modulo(a: float, b: float) -> float:
```

```
    """Javaish modulo.  This modulo operator returns the value with the sign
    of the dividend rather than the divisor like Python does:

    >>> cldr_modulo(-3, 5)
    -3
    >>> cldr_modulo(-3, -5)
    -3
    >>> cldr_modulo(3, 5)
    3
    """
    reverse = 0
    if a < 0:
        a *= -1
        reverse = 1
    if b < 0:
        b *= -1
    rv = a % b
    if reverse:
        rv *= -1
    return rv


class RuleError(Exception):
    """Raised if a rule is malformed."""


_VARS = {
    'n',  # absolute value of the source number.
    'i',  # integer digits of n.
    'v',  # number of visible fraction digits in n, with trailing zeros.*
    'w',  # number of visible fraction digits in n, without trailing zeros.*
    'f',  # visible fraction digits in n, with trailing zeros.*
    't',  # visible fraction digits in n, without trailing zeros.*
    'c',  # compact decimal exponent value: exponent of the power of 10 used in compact decimal formatting.
    'e',  # currently, synonym for `c`. however, may be redefined in the future.
}

_RULES: list[tuple[str | None, re.Pattern[str]]] = [
    (None, re.compile(r'\s+', re.UNICODE)),
    ('word', re.compile(fr'\b(and|or|is|(?:with)?in|not|mod|[{"".join(_VARS)}])\b')),
    ('value', re.compile(r'\d+')),
    ('symbol', re.compile(r'%|,|!=|=')),
    ('ellipsis', re.compile(r'\.{2,3}|\u2026', re.UNICODE)),  # U+2026: ELLIPSIS
]


def tokenize_rule(s: str) -> list[tuple[str, str]]:
    s = s.split('@')[0]
    result: list[tuple[str, str]] = []
    pos = 0
    end = len(s)
    while pos < end:
        for tok, rule in _RULES:
            match = rule.match(s, pos)
            if match is not None:
                pos = match.end()
                if tok:
                    result.append((tok, match.group()))
                break
        else:
            raise RuleError('malformed CLDR pluralization rule.  '
                            'Got unexpected %r' % s[pos])
    return result[::-1]


def test_next_token(
    tokens: list[tuple[str, str]],
    type_: str,
    value: str | None = None,
) -> list[tuple[str, str]] | bool:
    return tokens and tokens[-1][0] == type_ and \
        (value is None or tokens[-1][1] == value)


def skip_token(tokens: list[tuple[str, str]], type_: str, value: str | None = None):
    if test_next_token(tokens, type_, value):
        return tokens.pop()


def value_node(value: int) -> tuple[Literal['value'], tuple[int]]:
    return 'value', (value, )
```

```
def ident_node(name: str) -> tuple[str, tuple[()]]:
    return name, ()


def range_list_node(
    range_list: Iterable[Iterable[float | decimal.Decimal]],
) -> tuple[Literal['range_list'], Iterable[Iterable[float | decimal.Decimal]]]:
    return 'range_list', range_list


def negate(rv: tuple[Any, ...]) -> tuple[Literal['not'], tuple[tuple[Any, ...]]]:
    return 'not', (rv,)


class _Parser:
    """Internal parser.  This class can translate a single rule into an abstract
    tree of tuples. It implements the following grammar::

        condition     = and_condition ('or' and_condition)*
                        ('@integer' samples)?
                        ('@decimal' samples)?
        and_condition = relation ('and' relation)*
        relation      = is_relation | in_relation | within_relation
        is_relation   = expr 'is' ('not')? value
        in_relation   = expr (('not')? 'in' | '=' | '!=') range_list
        within_relation = expr ('not')? 'within' range_list
        expr          = operand (('mod' | '%') value)?
        operand       = 'n' | 'i' | 'f' | 't' | 'v' | 'w'
        range_list    = (range | value) (',' range_list)*
        value         = digit+
        digit         = 0|1|2|3|4|5|6|7|8|9
        range         = value'..'value
        samples       = sampleRange (',' sampleRange)* (',' ('…'|'...'))?
        sampleRange   = decimalValue '~' decimalValue
        decimalValue  = value ('.' value)?

    - Whitespace can occur between or around any of the above tokens.
    - Rules should be mutually exclusive; for a given numeric value, only one
      rule should apply (i.e. the condition should only be true for one of
      the plural rule elements).
    - The in and within relations can take comma-separated lists, such as:
      'n in 3,5,7..15'.
    - Samples are ignored.

    The translator parses the expression on instantiation into an attribute
    called `ast`.
    """

    def __init__(self, string):
        self.tokens = tokenize_rule(string)
        if not self.tokens:
            # If the pattern is only samples, it's entirely possible
            # no stream of tokens whatsoever is generated.
            self.ast = None
            return
        self.ast = self.condition()
        if self.tokens:
            raise RuleError(f"Expected end of rule, got {self.tokens[-1][1]!r}")

    def expect(self, type_, value=None, term=None):
        token = skip_token(self.tokens, type_, value)
        if token is not None:
            return token
        if term is None:
            term = repr(value is None and type_ or value)
        if not self.tokens:
            raise RuleError(f"expected {term} but end of rule reached")
        raise RuleError(f"expected {term} but got {self.tokens[-1][1]!r}")

    def condition(self):
        op = self.and_condition()
        while skip_token(self.tokens, 'word', 'or'):
            op = 'or', (op, self.and_condition())
        return op

    def and_condition(self):
        op = self.relation()
        while skip_token(self.tokens, 'word', 'and'):
            op = 'and', (op, self.relation())
        return op
    def relation(self):
```

```
        left = self.expr()
        if skip_token(self.tokens, 'word', 'is'):
            return skip_token(self.tokens, 'word', 'not') and 'isnot' or 'is', \
                (left, self.value())
        negated = skip_token(self.tokens, 'word', 'not')
        method = 'in'
        if skip_token(self.tokens, 'word', 'within'):
            method = 'within'
        else:
            if not skip_token(self.tokens, 'word', 'in'):
                if negated:
                    raise RuleError('Cannot negate operator based rules.')
                return self.newfangled_relation(left)
        rv = 'relation', (method, left, self.range_list())
        return negate(rv) if negated else rv

    def newfangled_relation(self, left):
        if skip_token(self.tokens, 'symbol', '='):
            negated = False
        elif skip_token(self.tokens, 'symbol', '!='):
            negated = True
        else:
            raise RuleError('Expected "=" or "!=" or legacy relation')
        rv = 'relation', ('in', left, self.range_list())
        return negate(rv) if negated else rv

    def range_or_value(self):
        left = self.value()
        if skip_token(self.tokens, 'ellipsis'):
            return left, self.value()
        else:
            return left, left

    def range_list(self):
        range_list = [self.range_or_value()]
        while skip_token(self.tokens, 'symbol', ','):
            range_list.append(self.range_or_value())
        return range_list_node(range_list)

    def expr(self):
        word = skip_token(self.tokens, 'word')
        if word is None or word[1] not in _VARS:
            raise RuleError('Expected identifier variable')
        name = word[1]
        if skip_token(self.tokens, 'word', 'mod'):
            return 'mod', ((name, ()), self.value())
        elif skip_token(self.tokens, 'symbol', '%'):
            return 'mod', ((name, ()), self.value())
        return ident_node(name)

    def value(self):
        return value_node(int(self.expect('value')[1]))


def _binary_compiler(tmpl):
    """Compiler factory for the `_Compiler`."""
    return lambda self, left, right: tmpl % (self.compile(left), self.compile(right))


def _unary_compiler(tmpl):
    """Compiler factory for the `_Compiler`."""
    return lambda self, x: tmpl % self.compile(x)


compile_zero = lambda x: '0'


class _Compiler:
    """The compilers are able to transform the expressions into multiple
    output formats.
    """

    def compile(self, arg):
        op, args = arg
        return getattr(self, f"compile_{op}")(*args)

    compile_n = lambda x: 'n'
    compile_i = lambda x: 'i'
    compile_v = lambda x: 'v'
    compile_w = lambda x: 'w'
    compile_f = lambda x: 'f'
```

```
    compile_t = lambda x: 't'
    compile_c = lambda x: 'c'
    compile_e = lambda x: 'e'
    compile_value = lambda x, v: str(v)
    compile_and = _binary_compiler('(%s && %s
```
(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\babel\support.py

```
"""
    babel.support
    ~~~~~~~~~~~~~~

    Several classes and functions that help with integrating and using Babel
    in applications.

    .. note: the code in this module is not used by Babel itself

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

import decimal
import gettext
import locale
import os
from collections.abc import Iterator
from typing import TYPE_CHECKING, Any, Callable, Iterable

from babel.core import Locale
from babel.dates import format_date, format_datetime, format_time, format_timedelta
from babel.numbers import (
    format_compact_currency,
    format_compact_decimal,
    format_currency,
    format_decimal,
    format_percent,
    format_scientific,
)

if TYPE_CHECKING:
    from typing_extensions import Literal

    from babel.dates import _PredefinedTimeFormat


class Format:
    """Wrapper class providing the various date and number formatting functions
    bound to a specific locale and time-zone.

    >>> from babel.util import UTC
    >>> from datetime import date
    >>> fmt = Format('en_US', UTC)
    >>> fmt.date(date(2007, 4, 1))
    u'Apr 1, 2007'
    >>> fmt.decimal(1.2345)
    u'1.234'
    """

    def __init__(
        self,
        locale: Locale | str,
        tzinfo: datetime.tzinfo | None = None,
        *,
        numbering_system: Literal["default"] | str = "latn",
    ) -> None:
        """Initialize the formatter.

        :param locale: the locale identifier or `Locale` instance
        :param tzinfo: the time-zone info (a `tzinfo` instance or `None`)
        :param numbering_system: The numbering system used for formatting number symbols. Defaults to "latn".
                                 The special value "default" will use the default numbering system of the locale.
        """
        self.locale = Locale.parse(locale)
        self.tzinfo = tzinfo
        self.numbering_system = numbering_system

    def date(
```

```
    self,
    date: datetime.date | None = None,
    format: _PredefinedTimeFormat | str = 'medium',
) -> str:
    """Return a date formatted according to the given pattern.

    >>> from datetime import date
    >>> fmt = Format('en_US')
    >>> fmt.date(date(2007, 4, 1))
    u'Apr 1, 2007'
    """
    return format_date(date, format, locale=self.locale)

def datetime(
    self,
    datetime: datetime.date | None = None,
    format: _PredefinedTimeFormat | str = 'medium',
) -> str:
    """Return a date and time formatted according to the given pattern.

    >>> from datetime import datetime
    >>> from babel.dates import get_timezone
    >>> fmt = Format('en_US', tzinfo=get_timezone('US/Eastern'))
    >>> fmt.datetime(datetime(2007, 4, 1, 15, 30))
    u'Apr 1, 2007, 11:30:00\u202fAM'
    """
    return format_datetime(datetime, format, tzinfo=self.tzinfo, locale=self.locale)

def time(
    self,
    time: datetime.time | datetime.datetime | None = None,
    format: _PredefinedTimeFormat | str = 'medium',
) -> str:
    """Return a time formatted according to the given pattern.

    >>> from datetime import datetime
    >>> from babel.dates import get_timezone
    >>> fmt = Format('en_US', tzinfo=get_timezone('US/Eastern'))
    >>> fmt.time(datetime(2007, 4, 1, 15, 30))
    u'11:30:00\u202fAM'
    """
    return format_time(time, format, tzinfo=self.tzinfo, locale=self.locale)

def timedelta(
    self,
    delta: datetime.timedelta | int,
    granularity: Literal["year", "month", "week", "day", "hour", "minute", "second"] = "second",
    threshold: float = 0.85,
    format: Literal["narrow", "short", "medium", "long"] = "long",
    add_direction: bool = False,
) -> str:
    """Return a time delta according to the rules of the given locale.

    >>> from datetime import timedelta
    >>> fmt = Format('en_US')
    >>> fmt.timedelta(timedelta(weeks=11))
    u'3 months'
    """
    return format_timedelta(delta, granularity=granularity,
                            threshold=threshold,
                            format=format, add_direction=add_direction,
                            locale=self.locale)

def number(self, number: float | decimal.Decimal | str) -> str:
    """Return an integer number formatted for the locale.

    >>> fmt = Format('en_US')
    >>> fmt.number(1099)
    u'1,099'
    """
    return format_decimal(number, locale=self.locale, numbering_system=self.numbering_system)

def decimal(self, number: float | decimal.Decimal | str, format: str | None = None) -> str:
    """Return a decimal number formatted for the locale.

    >>> fmt = Format('en_US')
    >>> fmt.decimal(1.2345)
    u'1.234'
    """
    return format_decimal(number, format, locale=self.locale, numbering_system=self.numbering_system)
def compact_decimal(
```

```
        self,
        number: float | decimal.Decimal | str,
        format_type: Literal['short', 'long'] = 'short',
        fraction_digits: int = 0,
    ) -> str:
        """Return a number formatted in compact form for the locale.

        >>> fmt = Format('en_US')
        >>> fmt.compact_decimal(123456789)
        u'123M'
        >>> fmt.compact_decimal(1234567, format_type='long', fraction_digits=2)
        '1.23 million'
        """
        return format_compact_decimal(
            number,
            format_type=format_type,
            fraction_digits=fraction_digits,
            locale=self.locale,
            numbering_system=self.numbering_system,
        )

    def currency(self, number: float | decimal.Decimal | str, currency: str) -> str:
        """Return a number in the given currency formatted for the locale.
        """
        return format_currency(number, currency, locale=self.locale, numbering_system=self.numbering_system)

    def compact_currency(
        self,
        number: float | decimal.Decimal | str,
        currency: str,
        format_type: Literal['short'] = 'short',
        fraction_digits: int = 0,
    ) -> str:
        """Return a number in the given currency formatted for the locale
        using the compact number format.

        >>> Format('en_US').compact_currency(1234567, "USD", format_type='short', fraction_digits=2)
        '$1.23M'
        """
        return format_compact_currency(number, currency, format_type=format_type, fraction_digits=fraction_digits,
                                       locale=self.locale, numbering_system=self.numbering_system)

    def percent(self, number: float | decimal.Decimal | str, format: str | None = None) -> str:
        """Return a number formatted as percentage for the locale.

        >>> fmt = Format('en_US')
        >>> fmt.percent(0.34)
        u'34%'
        """
        return format_percent(number, format, locale=self.locale, numbering_system=self.numbering_system)

    def scientific(self, number: float | decimal.Decimal | str) -> str:
        """Return a number formatted using scientific notation for the locale.
        """
        return format_scientific(number, locale=self.locale, numbering_system=self.numbering_system)


class LazyProxy:
    """Class for proxy objects that delegate to a specified function to evaluate
    the actual object.

    >>> def greeting(name='world'):
    ...     return 'Hello, %s!' % name
    >>> lazy_greeting = LazyProxy(greeting, name='Joe')
    >>> print(lazy_greeting)
    Hello, Joe!
    >>> u'  ' + lazy_greeting
    u'  Hello, Joe!'
    >>> u'(%s)' % lazy_greeting
    u'(Hello, Joe!)'

    This can be used, for example, to implement lazy translation functions that
    delay the actual translation until the string is actually used. The
    rationale for such behavior is that the locale of the user may not always
    be available. In web applications, you only know the locale when processing
    a request.

    The proxy implementation attempts to be as complete as possible, so that
    the lazy objects should mostly work as expected, for example for sorting:

    >>> greetings = [
```

```
...     LazyProxy(greeting, 'world'),
...     LazyProxy(greeting, 'Joe'),
...     LazyProxy(greeting, 'universe'),
... ]
>>> greetings.sort()
>>> for greeting in greetings:
...     print(greeting)
Hello, Joe!
Hello, universe!
Hello, world!
"""
__slots__ = ['_func', '_args', '_kwargs', '_value', '_is_cache_enabled', '_attribute_error']

if TYPE_CHECKING:
    _func: Callable[..., Any]
    _args: tuple[Any, ...]
    _kwargs: dict[str, Any]
    _is_cache_enabled: bool
    _value: Any
    _attribute_error: AttributeError | None

def __init__(self, func: Callable[..., Any], *args: Any, enable_cache: bool = True, **kwargs: Any) -> None:
    # Avoid triggering our own __setattr__ implementation
    object.__setattr__(self, '_func', func)
    object.__setattr__(self, '_args', args)
    object.__setattr__(self, '_kwargs', kwargs)
    object.__setattr__(self, '_is_cache_enabled', enable_cache)
    object.__setattr__(self, '_value', None)
    object.__setattr__(self, '_attribute_error', None)

@property
def value(self) -> Any:
    if self._value is None:
        try:
            value = self._func(*self._args, **self._kwargs)
        except AttributeError as error:
            object.__setattr__(self, '_attribute_error', error)
            raise

        if not self._is_cache_enabled:
            return value
        object.__setattr__(self, '_value', value)
    return self._value

def __contains__(self, key: object) -> bool:
    return key in self.value

def __bool__(self) -> bool:
    return bool(self.value)

def __dir__(self) -> list[str]:
    return dir(self.value)

def __iter__(self) -> Iterator[Any]:
    return iter(self.value)

def __len__(self) -> int:
    return len(self.value)

def __str__(self) -> str:
    return str(self.value)

def __add__(self, other: object) -> Any:
    return self.value + other

def __radd__(self, other: object) -> Any:
    return other + self.value

def __mod__(self, other: object) -> Any:
    return self.value % other

def __rmod__(self, other: object) -> Any:
    return other % self.value

def __mul__(self, other: object) -> Any:
    return self.value * other

def __rmul__(self, other: object) -> Any:
    return other * self.value

def __call__(self, *args: Any, **kwargs: Any) -> Any:
```

```
        return self.value(*args, **kwargs)

    def __lt__(self, other: object) -> bool:
        return self.value < other

    def __le__(self, other: object) -> bool:
        return self.value <= other

    def __eq__(self, other: object) -> bool:
        return self.value == other

    def __ne__(self, other: object) -> bool:
        return self.value != other

    def __gt__(self, other: object) -> bool:
        return self.value > other

    def __ge__(self, other: object) -> bool:
        return self.value >= other

    def __delattr__(self, name: str) -> None:
        delattr(self.value, name)

    def __getattr__(self, name: str) -> Any:
        if self._attribute_error is not None:
            raise self._attribute_error
        return getattr(self.value, name)

    def __setattr__(self, name: str, value: Any) -> None:
        setattr(self.value, name, value)

    def __delitem__(self, key: Any) -> None:
        del self.value[key]

    def __getitem__(self, key: Any) -> Any:
        return self.value[key]

    def __setitem__(self, key: Any, value: Any) -> None:
        self.value[key] = value

    def __copy__(self) -> LazyProxy:
        return LazyProxy(
            self._func,
            enable_cache=self._is_cache_enabled,
            *self._args,  # noqa: B026
            **self._kwargs,
        )

    def __deepcopy__(self, memo: Any) -> LazyProxy:
        from copy import deepcopy
        return LazyProxy(
            deepcopy(self._func, memo),
            enable_cache=deepcopy(self._is_cache_enabled, memo),
            *deepcopy(self._args, memo),  # noqa: B026
            **deepcopy(self._kwargs, memo),
        )


class NullTranslations(gettext.NullTranslations):

    if TYPE_CHECKING:
        _info: dict[str, str]
        _fallback: NullTranslations | None

    DEFAULT_DOMAIN = None

    def __init__(self, fp: gettext._TranslationsReader | None = None) -> None:
        """Initialize a simple translations class which is not backed by a
        real catalog. Behaves similar to gettext.NullTranslations but also
        offers Babel's on *gettext methods (e.g. 'dgettext()').

        :param fp: a file-like object (ignored in this class)
        """
        # These attributes are set by gettext.NullTranslations when a catalog
        # is parsed (fp != None). Ensure that they are always present because
        # some *gettext methods (including '.gettext()') rely on the attributes.
        self._catalog: dict[tuple[str, Any] | str, str] = {}
        self.plural: Callable[[float | decimal.Decimal], int] = lambda n: int(n != 1)
        super().__init__(fp=fp)
        self.files = list(filter(None, [getattr(fp, 'name', None)]))
        self.domain = self.DEFAULT_DOMAIN
```

```
    self._domains: dict[str, NullTranslations] = {}

def dgettext(self, domain: str, message: str) -> str:
    """Like ``gettext()``, but look the message up in the specified
    domain.
    """
    return self._domains.get(domain, self).gettext(message)

def ldgettext(self, domain: str, message: str) -> str:
    """Like ``lgettext()``, but look the message up in the specified
    domain.
    """
    import warnings
    warnings.warn(
        'ldgettext() is deprecated, use dgettext() instead',
        DeprecationWarning,
        stacklevel=2,
    )
    return self._domains.get(domain, self).lgettext(message)

def udgettext(self, domain: str, message: str) -> str:
    """Like ``ugettext()``, but look the message up in the specified
    domain.
    """
    return self._domains.get(domain, self).ugettext(message)
# backward compatibility with 0.9
dugettext = udgettext

def dngettext(self, domain: str, singular: str, plural: str, num: int) -> str:
    """Like ``ngettext()``, but look the message up in the specified
    domain.
    """
    return self._domains.get(domain, self).ngettext(singular, plural, num)

def ldngettext(self, domain: str, singular: str, plural: str, num: int) -> str:
    """Like ``lngettext()``, but look the message up in the specified
    domain.
    """
    import warnings
    warnings.warn(
        'ldngettext() is deprecated, use dngettext() instead',
        DeprecationWarning,
        stacklevel=2,
    )
    return self._domains.get(domain, self).lngettext(singular, plural, num)

def udngettext(self, domain: str, singular: str, plural: str, num: int) -> str:
    """Like ``ungettext()`` but look the message up in the specified
    domain.
    """
    return self._domains.get(domain, self).ungettext(singular, plural, num)
# backward compatibility with 0.9
dungettext = udngettext

# Most of the downwards code, until it gets included in stdlib, from:
#    https://bugs.python.org/file10036/gettext-pgettext.patch
#
# The encoding of a msgctxt and a msgid in a .mo file is
# msgctxt + "\x04" + msgid (gettext version >= 0.15)
CONTEXT_ENCODING = '%s\x04%s'

def pgettext(self, context: str, message: str) -> str | object:
    """Look up the `context` and `message` id in the catalog and return the
    corresponding message string, as an 8-bit string encoded with the
    catalog's charset encoding, if known.  If there is no entry in the
    catalog for the `message` id and `context` , and a fallback has been
    set, the look up is forwarded to the fallback's ``pgettext()``
    method. Otherwise, the `message` id is returned.
    """
    ctxt_msg_id = self.CONTEXT_ENCODING % (context, message)
    missing = object()
    tmsg = self._catalog.get(ctxt_msg_id, missing)
    if tmsg is missing:
        if self._fallback:
            return self._fallback.pgettext(context, message)
        return message
    return tmsg

def lpgettext(self, context: str, message: str) -> str | bytes | object:
    """Equivalent to ``pgettext()``, but the translation is returned in the
    preferred system encoding, if no other encoding was explicitly set with
```

```
        ``bind_textdomain_codeset()``.
        """
        import warnings
        warnings.warn(
            'lpgettext() is deprecated, use pgettext() instead',
            DeprecationWarning,
            stacklevel=2,
        )
        tmsg = self.pgettext(context, message)
        encoding = getattr(self, "_output_charset", None) or locale.getpreferredencoding()
        return tmsg.encode(encoding) if isinstance(tmsg, str) else tmsg

    def npgettext(self, context: str, singular: str, plural: str, num: int) -> str:
        """Do a plural-forms lookup of a message id.  `singular` is used as the
        message id for purposes of lookup in the catalog, while `num` is used to
        determine which plural form to use.  The returned message string is an
        8-bit string encoded with the catalog's charset encoding, if known.

        If the message id for `context` is not found in the catalog, and a
        fallback is specified, the request is forwarded to the fallback's
        ``npgettext()`` method.  Otherwise, when ``num`` is 1 ``singular`` is
        returned, and ``plural`` is returned in all other cases.
        """
        ctxt_msg_id = self.CONTEXT_ENCODING % (context, singular)
        try:
            tmsg = self._catalog[(ctxt_msg_id, self.plural(num))]
            return tmsg
        except KeyError:
            if self._fallback:
                return self._fallback.npgettext(context, singular, plural, num)
            if num == 1:
                return singular
            else:
                return plural

    def lnpgettext(self, context: str, singular: str, plural: str, num: int) -> str | bytes:
        """Equivalent to ``npgettext()``, but the translation is returned in the
        preferred system encoding, if no other encoding was explicitly set with
        ``bind_textdomain_codeset()``.
        """
        import warnings
        warnings.warn(
            'lnpgettext() is deprecated, use npgettext() instead',
            DeprecationWarning,
            stacklevel=2,
        )
        ctxt_msg_id = self.CONTEXT_ENCODING % (context, singular)
        try:
            tmsg = self._catalog[(ctxt_msg_id, self.plural(num))]
            encoding = getattr(self, "_output_charset", None) or locale.getpreferredencoding()
            return tmsg.encode(encoding)
        except KeyError:
            if self._fallback:
                return self._fallback.lnpgettext(context, singular, plural, num)
            if num == 1:
                return singular
            else:
                return plural

    def upgettext(self, context: str, message: str) -> str:
        """Look up the `context` and `message` id in the catalog and return the
        corresponding message string, as a Unicode string.  If there is no entry
        in the catalog for the `message` id and `context`, and a fallback has
        been set, the look up is forwarded to the fallback's ``upgettext()``
        method.  Otherwise, the `message` id is returned.
        """
        ctxt_message_id = self.CONTEXT_ENCODING % (context, message)
        missing = object()
        tmsg = self._catalog.get(ctxt_message
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\babel\units.py

```
from __future__ import annotations

import decimal
from typing import TYPE_CHECKING

from babel.core import Locale
```

```
from babel.numbers import LC_NUMERIC, format_decimal

if TYPE_CHECKING:
    from typing_extensions import Literal


class UnknownUnitError(ValueError):
    def __init__(self, unit: str, locale: Locale) -> None:
        ValueError.__init__(self, f"{unit} is not a known unit in {locale}")


def get_unit_name(
    measurement_unit: str,
    length: Literal['short', 'long', 'narrow'] = 'long',
    locale: Locale | str | None = LC_NUMERIC,
) -> str | None:
    """
    Get the display name for a measurement unit in the given locale.

    >>> get_unit_name("radian", locale="en")
    'radians'

    Unknown units will raise exceptions:

    >>> get_unit_name("battery", locale="fi")
    Traceback (most recent call last):
        ...
    UnknownUnitError: battery/long is not a known unit/length in fi

    :param measurement_unit: the code of a measurement unit.
                             Known units can be found in the CLDR Unit Validity XML file:
                             https://unicode.org/repos/cldr/tags/latest/common/validity/unit.xml

    :param length: "short", "long" or "narrow"
    :param locale: the `Locale` object or locale identifier
    :return: The unit display name, or None.
    """
    locale = Locale.parse(locale)
    unit = _find_unit_pattern(measurement_unit, locale=locale)
    if not unit:
        raise UnknownUnitError(unit=measurement_unit, locale=locale)
    return locale.unit_display_names.get(unit, {}).get(length)


def _find_unit_pattern(unit_id: str, locale: Locale | str | None = LC_NUMERIC) -> str | None:
    """
    Expand a unit into a qualified form.

    Known units can be found in the CLDR Unit Validity XML file:
    https://unicode.org/repos/cldr/tags/latest/common/validity/unit.xml

    >>> _find_unit_pattern("radian", locale="en")
    'angle-radian'

    Unknown values will return None.

    >>> _find_unit_pattern("horse", locale="en")

    :param unit_id: the code of a measurement unit.
    :return: A key to the `unit_patterns` mapping, or None.
    """
    locale = Locale.parse(locale)
    unit_patterns = locale._data["unit_patterns"]
    if unit_id in unit_patterns:
        return unit_id
    for unit_pattern in sorted(unit_patterns, key=len):
        if unit_pattern.endswith(unit_id):
            return unit_pattern
    return None


def format_unit(
    value: str | float | decimal.Decimal,
    measurement_unit: str,
    length: Literal['short', 'long', 'narrow'] = 'long',
    format: str | None = None,
    locale: Locale | str | None = LC_NUMERIC,
    *,
    numbering_system: Literal["default"] | str = "latn",
) -> str:
    """Format a value of a given unit.
```

```
    Values are formatted according to the locale's usual pluralization rules
    and number formats.

    >>> format_unit(12, 'length-meter', locale='ro_RO')
    u'12 metri'
    >>> format_unit(15.5, 'length-mile', locale='fi_FI')
    u'15,5 mailia'
    >>> format_unit(1200, 'pressure-millimeter-ofhg', locale='nb')
    u'1\\xa0200 millimeter kvikks\\xf8lv'
    >>> format_unit(270, 'ton', locale='en')
    u'270 tons'
    >>> format_unit(1234.5, 'kilogram', locale='ar_EG', numbering_system='default')
    u'1■234■5 ■■■■■■■■'

    Number formats may be overridden with the ``format`` parameter.

    >>> import decimal
    >>> format_unit(decimal.Decimal("-42.774"), 'temperature-celsius', 'short', format='#.0', locale='fr')
    u'-42,8\\u202f\\xb0C'

    The locale's usual pluralization rules are respected.

    >>> format_unit(1, 'length-meter', locale='ro_RO')
    u'1 metru'
    >>> format_unit(0, 'length-mile', locale='cy')
    u'0 mi'
    >>> format_unit(1, 'length-mile', locale='cy')
    u'1 filltir'
    >>> format_unit(3, 'length-mile', locale='cy')
    u'3 milltir'

    >>> format_unit(15, 'length-horse', locale='fi')
    Traceback (most recent call last):
        ...
    UnknownUnitError: length-horse is not a known unit in fi

    .. versionadded:: 2.2.0

    :param value: the value to format. If this is a string, no number formatting will be attempted.
    :param measurement_unit: the code of a measurement unit.
                             Known units can be found in the CLDR Unit Validity XML file:
                             https://unicode.org/repos/cldr/tags/latest/common/validity/unit.xml
    :param length: "short", "long" or "narrow"
    :param format: An optional format, as accepted by `format_decimal`.
    :param locale: the `Locale` object or locale identifier
    :param numbering_system: The numbering system used for formatting number symbols. Defaults to "latn".
                             The special value "default" will use the default numbering system of the locale.
    :raise `UnsupportedNumberingSystemError`: If the numbering system is not supported by the locale.
    """
    locale = Locale.parse(locale)

    q_unit = _find_unit_pattern(measurement_unit, locale=locale)
    if not q_unit:
        raise UnknownUnitError(unit=measurement_unit, locale=locale)
    unit_patterns = locale._data["unit_patterns"][q_unit].get(length, {})

    if isinstance(value, str):  # Assume the value is a preformatted singular.
        formatted_value = value
        plural_form = "one"
    else:
        formatted_value = format_decimal(value, format, locale, numbering_system=numbering_system)
        plural_form = locale.plural_form(value)

    if plural_form in unit_patterns:
        return unit_patterns[plural_form].format(formatted_value)

    # Fall back to a somewhat bad representation.
    # nb: This is marked as no-cover, as the current CLDR seemingly has no way for this to happen.
    fallback_name = get_unit_name(measurement_unit, length=length, locale=locale)  # pragma: no cover
    return f"{formatted_value} {fallback_name or measurement_unit}"  # pragma: no cover


def _find_compound_unit(
    numerator_unit: str,
    denominator_unit: str,
    locale: Locale | str | None = LC_NUMERIC,
) -> str | None:
    """
    Find a predefined compound unit pattern.

    Used internally by format_compound_unit.
```

```
    >>> _find_compound_unit("kilometer", "hour", locale="en")
    'speed-kilometer-per-hour'

    >>> _find_compound_unit("mile", "gallon", locale="en")
    'consumption-mile-per-gallon'

    If no predefined compound pattern can be found, `None` is returned.

    >>> _find_compound_unit("gallon", "mile", locale="en")

    >>> _find_compound_unit("horse", "purple", locale="en")

    :param numerator_unit: The numerator unit's identifier
    :param denominator_unit: The denominator unit's identifier
    :param locale: the `Locale` object or locale identifier
    :return: A key to the `unit_patterns` mapping, or None.
    :rtype: str|None
    """
    locale = Locale.parse(locale)

    # Qualify the numerator and denominator units.  This will turn possibly partial
    # units like "kilometer" or "hour" into actual units like "length-kilometer" and
    # "duration-hour".

    resolved_numerator_unit = _find_unit_pattern(numerator_unit, locale=locale)
    resolved_denominator_unit = _find_unit_pattern(denominator_unit, locale=locale)

    # If either was not found, we can't possibly build a suitable compound unit either.
    if not (resolved_numerator_unit and resolved_denominator_unit):
        return None

    # Since compound units are named "speed-kilometer-per-hour", we'll have to slice off
    # the quantities (i.e. "length", "duration") from both qualified units.

    bare_numerator_unit = resolved_numerator_unit.split("-", 1)[-1]
    bare_denominator_unit = resolved_denominator_unit.split("-", 1)[-1]

    # Now we can try and rebuild a compound unit specifier, then qualify it:

    return _find_unit_pattern(f"{bare_numerator_unit}-per-{bare_denominator_unit}", locale=locale)


def format_compound_unit(
    numerator_value: str | float | decimal.Decimal,
    numerator_unit: str | None = None,
    denominator_value: str | float | decimal.Decimal = 1,
    denominator_unit: str | None = None,
    length: Literal["short", "long", "narrow"] = "long",
    format: str | None = None,
    locale: Locale | str | None = LC_NUMERIC,
    *,
    numbering_system: Literal["default"] | str = "latn",
) -> str | None:
    """
    Format a compound number value, i.e. "kilometers per hour" or similar.

    Both unit specifiers are optional to allow for formatting of arbitrary values still according
    to the locale's general "per" formatting specifier.

    >>> format_compound_unit(7, denominator_value=11, length="short", locale="pt")
    '7/11'

    >>> format_compound_unit(150, "kilometer", denominator_unit="hour", locale="sv")
    '150 kilometer per timme'

    >>> format_compound_unit(150, "kilowatt", denominator_unit="year", locale="fi")
    '150 kilowattia / vuosi'

    >>> format_compound_unit(32.5, "ton", 15, denominator_unit="hour", locale="en")
    '32.5 tons per 15 hours'

    >>> format_compound_unit(1234.5, "ton", 15, denominator_unit="hour", locale="ar_EG", numbering_system="arab")
    '1■234■5 ■■ ■■■ 15 ■■■■'

    >>> format_compound_unit(160, denominator_unit="square-meter", locale="fr")
    '160 par m\\xe8tre carr\\xe9'

    >>> format_compound_unit(4, "meter", "ratakisko", length="short", locale="fi")
    '4 m/ratakisko'

    >>> format_compound_unit(35, "minute", denominator_unit="fathom", locale="sv")
```

```
'35 minuter per famn'

>>> from babel.numbers import format_currency
>>> format_compound_unit(format_currency(35, "JPY", locale="de"), denominator_unit="liter", locale="de")
'35\\xa0\\xa5 pro Liter'

See https://www.unicode.org/reports/tr35/tr35-general.html#perUnitPatterns

:param numerator_value: The numerator value. This may be a string,
                        in which case it is considered preformatted and the unit is ignored.
:param numerator_unit: The numerator unit. See `format_unit`.
:param denominator_value: The denominator value. This may be a string,
                          in which case it is considered preformatted and the unit is ignored.
:param denominator_unit: The denominator unit. See `format_unit`.
:param length: The formatting length. "short", "long" or "narrow"
:param format: An optional format, as accepted by `format_decimal`.
:param locale: the `Locale` object or locale identifier
:param numbering_system: The numbering system used for formatting number symbols. Defaults to "latn".
                         The special value "default" will use the default numbering system of the locale.
:return: A formatted compound value.
:raise `UnsupportedNumberingSystemError`: If the numbering system is not supported by the locale.
"""
locale = Locale.parse(locale)

# Look for a specific compound unit first...

if numerator_unit and denominator_unit and denominator_value == 1:
    compound_unit = _find_compound_unit(numerator_unit, denominator_unit, locale=locale)
    if compound_unit:
        return format_unit(
            numerator_value,
            compound_unit,
            length=length,
            format=format,
            locale=locale,
            numbering_system=numbering_system,
        )

# ... failing that, construct one "by hand".

if isinstance(numerator_value, str):  # Numerator is preformatted
    formatted_numerator = numerator_value
elif numerator_unit:  # Numerator has unit
    formatted_numerator = format_unit(
        numerator_value,
        numerator_unit,
        length=length,
        format=format,
        locale=locale,
        numbering_system=numbering_system,
    )
else:  # Unitless numerator
    formatted_numerator = format_decimal(
        numerator_value,
        format=format,
        locale=locale,
        numbering_system=numbering_system,
    )

if isinstance(denominator_value, str):  # Denominator is preformatted
    formatted_denominator = denominator_value
elif denominator_unit:  # Denominator has unit
    if denominator_value == 1:  # support perUnitPatterns when the denominator is 1
        denominator_unit = _find_unit_pattern(denominator_unit, locale=locale)
        per_pattern = locale._data["unit_patterns"].get(denominator_unit, {}).get(length, {}).get("per")
        if per_pattern:
            return per_pattern.format(formatted_numerator)
        # See TR-35's per-unit pattern algorithm, point 3.2.
        # For denominator 1, we replace the value to be formatted with the empty string;
        # this will make `format_unit` return " second" instead of "1 second".
        denominator_value = ""

    formatted_denominator = format_unit(
        denominator_value,
        measurement_unit=(denominator_unit or ""),
        length=length,
        format=format,
        locale=locale,
        numbering_system=numbering_system,
    ).strip()
else:  # Bare denominator
```

```
        formatted_denominator = format_decimal(
            denominator_value,
            format=format,
            locale=locale,
            numbering_system=numbering_system,
        )

    # TODO: this doesn't support "compound_variations" (or "prefix"), and will fall back to the "x/y" representation
    per_pattern = locale._data["compound_unit_patterns"].get("per", {}).get(length, {}).get("compound", "{0}/{1}")

    return per_pattern.format(formatted_numerator, formatted_denominator)
```

File: .venv\Lib\site-packages\babel\util.py

```
"""
    babel.util
    ~~~~~~~~~~

    Various utility classes and functions.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

import codecs
import collections
import datetime
import os
import re
import textwrap
from collections.abc import Generator, Iterable
from typing import IO, Any, TypeVar

from babel import dates, localtime

missing = object()

_T = TypeVar("_T")


def distinct(iterable: Iterable[_T]) -> Generator[_T, None, None]:
    """Yield all items in an iterable collection that are distinct.

    Unlike when using sets for a similar effect, the original ordering of the
    items in the collection is preserved by this function.

    >>> print(list(distinct([1, 2, 1, 3, 4, 4])))
    [1, 2, 3, 4]
    >>> print(list(distinct('foobar')))
    ['f', 'o', 'b', 'a', 'r']

    :param iterable: the iterable collection providing the data
    """
    seen = set()
    for item in iter(iterable):
        if item not in seen:
            yield item
            seen.add(item)


# Regexp to match python magic encoding line
PYTHON_MAGIC_COMMENT_re = re.compile(
    br'[ \t\f]* \# .* coding[=:][ \t]*([-\w.]+)', re.VERBOSE)


def parse_encoding(fp: IO[bytes]) -> str | None:
    """Deduce the encoding of a source file from magic comment.

    It does this in the same way as the `Python interpreter`__

    .. __: https://docs.python.org/3.4/reference/lexical_analysis.html#encoding-declarations

    The ``fp`` argument should be a seekable file object.

    (From Jeff Dairiki)
    """
    pos = fp.tell()
    fp.seek(0)
```

```
    try:
        line1 = fp.readline()
        has_bom = line1.startswith(codecs.BOM_UTF8)
        if has_bom:
            line1 = line1[len(codecs.BOM_UTF8):]

        m = PYTHON_MAGIC_COMMENT_re.match(line1)
        if not m:
            try:
                import ast
                ast.parse(line1.decode('latin-1'))
            except (ImportError, SyntaxError, UnicodeEncodeError):
                # Either it's a real syntax error, in which case the source is
                # not valid python source, or line2 is a continuation of line1,
                # in which case we don't want to scan line2 for a magic
                # comment.
                pass
            else:
                line2 = fp.readline()
                m = PYTHON_MAGIC_COMMENT_re.match(line2)

        if has_bom:
            if m:
                magic_comment_encoding = m.group(1).decode('latin-1')
                if magic_comment_encoding != 'utf-8':
                    raise SyntaxError(f"encoding problem: {magic_comment_encoding} with BOM")
            return 'utf-8'
        elif m:
            return m.group(1).decode('latin-1')
        else:
            return None
    finally:
        fp.seek(pos)


PYTHON_FUTURE_IMPORT_re = re.compile(
    r'from\s+__future__\s+import\s+\(*(.+)\)*')


def parse_future_flags(fp: IO[bytes], encoding: str = 'latin-1') -> int:
    """Parse the compiler flags by :mod:`__future__` from the given Python
    code.
    """
    import __future__
    pos = fp.tell()
    fp.seek(0)
    flags = 0
    try:
        body = fp.read().decode(encoding)

        # Fix up the source to be (hopefully) parsable by regexpen.
        # This will likely do untoward things if the source code itself is broken.

        # (1) Fix `import (\n...` to be `import (...`.
        body = re.sub(r'import\s*\([\r\n]+', 'import (', body)
        # (2) Join line-ending commas with the next line.
        body = re.sub(r',\s*[\r\n]+', ', ', body)
        # (3) Remove backslash line continuations.
        body = re.sub(r'\\\s*[\r\n]+', ' ', body)

        for m in PYTHON_FUTURE_IMPORT_re.finditer(body):
            names = [x.strip().strip('()') for x in m.group(1).split(',')]
            for name in names:
                feature = getattr(__future__, name, None)
                if feature:
                    flags |= feature.compiler_flag
    finally:
        fp.seek(pos)
    return flags


def pathmatch(pattern: str, filename: str) -> bool:
    """Extended pathname pattern matching.

    This function is similar to what is provided by the ``fnmatch`` module in
    the Python standard library, but:

     * can match complete (relative or absolute) path names, and not just file
       names, and
     * also supports a convenience pattern ("**") to match files at any
       directory level.
```

```
    Examples:

    >>> pathmatch('**.py', 'bar.py')
    True
    >>> pathmatch('**.py', 'foo/bar/baz.py')
    True
    >>> pathmatch('**.py', 'templates/index.html')
    False

    >>> pathmatch('./foo/**.py', 'foo/bar/baz.py')
    True
    >>> pathmatch('./foo/**.py', 'bar/baz.py')
    False

    >>> pathmatch('^foo/**.py', 'foo/bar/baz.py')
    True
    >>> pathmatch('^foo/**.py', 'bar/baz.py')
    False

    >>> pathmatch('**/templates/*.html', 'templates/index.html')
    True
    >>> pathmatch('**/templates/*.html', 'templates/foo/bar.html')
    False

    :param pattern: the glob pattern
    :param filename: the path name of the file to match against
    """
    symbols = {
        '?': '[^/]',
        '?/': '[^/]/',
        '*': '[^/]+',
        '*/': '[^/]+/',
        '**/': '(?:.+/)*?',
        '**': '(?:.+/)*?[^/]+',
    }

    if pattern.startswith('^'):
        buf = ['^']
        pattern = pattern[1:]
    elif pattern.startswith('./'):
        buf = ['^']
        pattern = pattern[2:]
    else:
        buf = []

    for idx, part in enumerate(re.split('([?*]+/?)', pattern)):
        if idx % 2:
            buf.append(symbols[part])
        elif part:
            buf.append(re.escape(part))
    match = re.match(f"{''.join(buf)}$", filename.replace(os.sep, "/"))
    return match is not None


class TextWrapper(textwrap.TextWrapper):
    wordsep_re = re.compile(
        r'(\s+|'                                  # any whitespace
        r'(?<=[\w\!\"\'\&\.\,\?])-{2,}(?=\w))',   # em-dash
    )


def wraptext(text: str, width: int = 70, initial_indent: str = '', subsequent_indent: str = '') -> list[str]:
    """Simple wrapper around the ``textwrap.wrap`` function in the standard
    library. This version does not wrap lines on hyphens in words.

    :param text: the text to wrap
    :param width: the maximum line width
    :param initial_indent: string that will be prepended to the first line of
                           wrapped output
    :param subsequent_indent: string that will be prepended to all lines save
                              the first of wrapped output
    """
    wrapper = TextWrapper(width=width, initial_indent=initial_indent,
                          subsequent_indent=subsequent_indent,
                          break_long_words=False)
    return wrapper.wrap(text)


# TODO (Babel 3.x): Remove this re-export
odict = collections.OrderedDict
class FixedOffsetTimezone(datetime.tzinfo):
```

```
    """Fixed offset in minutes east from UTC."""

    def __init__(self, offset: float, name: str | None = None) -> None:

        self._offset = datetime.timedelta(minutes=offset)
        if name is None:
            name = 'Etc/GMT%+d' % offset
        self.zone = name

    def __str__(self) -> str:
        return self.zone

    def __repr__(self) -> str:
        return f'<FixedOffset "{self.zone}" {self._offset}>'

    def utcoffset(self, dt: datetime.datetime) -> datetime.timedelta:
        return self._offset

    def tzname(self, dt: datetime.datetime) -> str:
        return self.zone

    def dst(self, dt: datetime.datetime) -> datetime.timedelta:
        return ZERO


# Export the localtime functionality here because that's
# where it was in the past.
# TODO(3.0): remove these aliases
UTC = dates.UTC
LOCALTZ = dates.LOCALTZ
get_localzone = localtime.get_localzone
STDOFFSET = localtime.STDOFFSET
DSTOFFSET = localtime.DSTOFFSET
DSTDIFF = localtime.DSTDIFF
ZERO = localtime.ZERO


def _cmp(a: Any, b: Any):
    return (a > b) - (a < b)
```

File: .venv\Lib\site-packages\babel__init__.py

```
"""
    babel
    ~~~~~

    Integrated collection of utilities that assist in internationalizing and
    localizing applications.

    This package is basically composed of two major parts:

     * tools to build and work with ``gettext`` message catalogs
     * a Python interface to the CLDR (Common Locale Data Repository), providing
       access to various locale display names, localized number and date
       formatting, etc.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""

from babel.core import (
    Locale,
    UnknownLocaleError,
    default_locale,
    get_locale_identifier,
    negotiate_locale,
    parse_locale,
)

__version__ = '2.14.0'

__all__ = [
    'Locale',
    'UnknownLocaleError',
    'default_locale',
    'get_locale_identifier',
    'negotiate_locale',
    'parse_locale',
]
```

File: .venv\Lib\site-packages\babel\localtime_fallback.py

```
"""
    babel.localtime._fallback
    ~~~~~~~~~~~~~~~~~~~~~~~~~

    Emulated fallback local timezone when all else fails.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""

import datetime
import time

STDOFFSET = datetime.timedelta(seconds=-time.timezone)
DSTOFFSET = datetime.timedelta(seconds=-time.altzone) if time.daylight else STDOFFSET

DSTDIFF = DSTOFFSET - STDOFFSET
ZERO = datetime.timedelta(0)


class _FallbackLocalTimezone(datetime.tzinfo):

    def utcoffset(self, dt: datetime.datetime) -> datetime.timedelta:
        if self._isdst(dt):
            return DSTOFFSET
        else:
            return STDOFFSET

    def dst(self, dt: datetime.datetime) -> datetime.timedelta:
        if self._isdst(dt):
            return DSTDIFF
        else:
            return ZERO

    def tzname(self, dt: datetime.datetime) -> str:
        return time.tzname[self._isdst(dt)]

    def _isdst(self, dt: datetime.datetime) -> bool:
        tt = (dt.year, dt.month, dt.day,
              dt.hour, dt.minute, dt.second,
              dt.weekday(), 0, -1)
        stamp = time.mktime(tt)
        tt = time.localtime(stamp)
        return tt.tm_isdst > 0
```

File: .venv\Lib\site-packages\babel\localtime_helpers.py

```
try:
    import pytz
except ModuleNotFoundError:
    pytz = None
    import zoneinfo


def _get_tzinfo(tzenv: str):
    """Get the tzinfo from `zoneinfo` or `pytz`

    :param tzenv: timezone in the form of Continent/City
    :return: tzinfo object or None if not found
    """
    if pytz:
        try:
            return pytz.timezone(tzenv)
        except pytz.UnknownTimeZoneError:
            pass
    else:
        try:
            return zoneinfo.ZoneInfo(tzenv)
        except zoneinfo.ZoneInfoNotFoundError:
            pass

    return None


def _get_tzinfo_or_raise(tzenv: str):
```

```
    tzinfo = _get_tzinfo(tzenv)
    if tzinfo is None:
        raise LookupError(
            f"Can not find timezone {tzenv}. \n"
            "Timezone names are generally in the form `Continent/City`.",
        )
    return tzinfo


def _get_tzinfo_from_file(tzfilename: str):
    with open(tzfilename, 'rb') as tzfile:
        if pytz:
            return pytz.tzfile.build_tzinfo('local', tzfile)
        else:
            return zoneinfo.ZoneInfo.from_file(tzfile)
```

File: .venv\Lib\site-packages\babel\localtime_unix.py

```
import datetime
import os
import re

from babel.localtime._helpers import (
    _get_tzinfo,
    _get_tzinfo_from_file,
    _get_tzinfo_or_raise,
)


def _tz_from_env(tzenv: str) -> datetime.tzinfo:
    if tzenv[0] == ':':
        tzenv = tzenv[1:]

    # TZ specifies a file
    if os.path.exists(tzenv):
        return _get_tzinfo_from_file(tzenv)

    # TZ specifies a zoneinfo zone.
    return _get_tzinfo_or_raise(tzenv)


def _get_localzone(_root: str = '/') -> datetime.tzinfo:
    """Tries to find the local timezone configuration.
    This method prefers finding the timezone name and passing that to
    zoneinfo or pytz, over passing in the localtime file, as in the later
    case the zoneinfo name is unknown.
    The parameter _root makes the function look for files like /etc/localtime
    beneath the _root directory. This is primarily used by the tests.
    In normal usage you call the function without parameters.
    """

    tzenv = os.environ.get('TZ')
    if tzenv:
        return _tz_from_env(tzenv)

    # This is actually a pretty reliable way to test for the local time
    # zone on operating systems like OS X.  On OS X especially this is the
    # only one that actually works.
    try:
        link_dst = os.readlink('/etc/localtime')
    except OSError:
        pass
    else:
        pos = link_dst.find('/zoneinfo/')
        if pos >= 0:
            zone_name = link_dst[pos + 10:]
            tzinfo = _get_tzinfo(zone_name)
            if tzinfo is not None:
                return tzinfo

    # Now look for distribution specific configuration files
    # that contain the timezone name.
    tzpath = os.path.join(_root, 'etc/timezone')
    if os.path.exists(tzpath):
        with open(tzpath, 'rb') as tzfile:
            data = tzfile.read()

            # Issue #3 in tzlocal was that /etc/timezone was a zoneinfo file.
            # That's a misconfiguration, but we need to handle it gracefully:
```

```
            if data[:5] != b'TZif2':
                etctz = data.strip().decode()
                # Get rid of host definitions and comments:
                if ' ' in etctz:
                    etctz, dummy = etctz.split(' ', 1)
                if '#' in etctz:
                    etctz, dummy = etctz.split('#', 1)

                return _get_tzinfo_or_raise(etctz.replace(' ', '_'))

    # CentOS has a ZONE setting in /etc/sysconfig/clock,
    # OpenSUSE has a TIMEZONE setting in /etc/sysconfig/clock and
    # Gentoo has a TIMEZONE setting in /etc/conf.d/clock
    # We look through these files for a timezone:
    timezone_re = re.compile(r'\s*(TIME)?ZONE\s*=\s*"(?P<etctz>.+)"')

    for filename in ('etc/sysconfig/clock', 'etc/conf.d/clock'):
        tzpath = os.path.join(_root, filename)
        if not os.path.exists(tzpath):
            continue
        with open(tzpath) as tzfile:
            for line in tzfile:
                match = timezone_re.match(line)
                if match is not None:
                    # We found a timezone
                    etctz = match.group("etctz")
                    return _get_tzinfo_or_raise(etctz.replace(' ', '_'))

    # No explicit setting existed. Use localtime
    for filename in ('etc/localtime', 'usr/local/etc/localtime'):
        tzpath = os.path.join(_root, filename)

        if not os.path.exists(tzpath):
            continue
        return _get_tzinfo_from_file(tzpath)

    raise LookupError('Can not find any timezone configuration')
```

File: .venv\Lib\site-packages\babel\localtime_win32.py

```
from __future__ import annotations

try:
    import winreg
except ImportError:
    winreg = None

import datetime
from typing import Any, Dict, cast

from babel.core import get_global
from babel.localtime._helpers import _get_tzinfo_or_raise

# When building the cldr data on windows this module gets imported.
# Because at that point there is no global.dat yet this call will
# fail.  We want to catch it down in that case then and just assume
# the mapping was empty.
try:
    tz_names: dict[str, str] = cast(Dict[str, str], get_global('windows_zone_mapping'))
except RuntimeError:
    tz_names = {}


def valuestodict(key) -> dict[str, Any]:
    """Convert a registry key's values to a dictionary."""
    dict = {}
    size = winreg.QueryInfoKey(key)[1]
    for i in range(size):
        data = winreg.EnumValue(key, i)
        dict[data[0]] = data[1]
    return dict


def get_localzone_name() -> str:
    # Windows is special. It has unique time zone names (in several
    # meanings of the word) available, but unfortunately, they can be
    # translated to the language of the operating system, so we need to
    # do a backwards lookup, by going through all time zones and see which
    # one matches.
```

```
    handle = winreg.ConnectRegistry(None, winreg.HKEY_LOCAL_MACHINE)

    TZLOCALKEYNAME = r'SYSTEM\CurrentControlSet\Control\TimeZoneInformation'
    localtz = winreg.OpenKey(handle, TZLOCALKEYNAME)
    keyvalues = valuestodict(localtz)
    localtz.Close()
    if 'TimeZoneKeyName' in keyvalues:
        # Windows 7 (and Vista?)

        # For some reason this returns a string with loads of NUL bytes at
        # least on some systems. I don't know if this is a bug somewhere, I
        # just work around it.
        tzkeyname = keyvalues['TimeZoneKeyName'].split('\x00', 1)[0]
    else:
        # Windows 2000 or XP

        # This is the localized name:
        tzwin = keyvalues['StandardName']

        # Open the list of timezones to look up the real name:
        TZKEYNAME = r'SOFTWARE\Microsoft\Windows NT\CurrentVersion\Time Zones'
        tzkey = winreg.OpenKey(handle, TZKEYNAME)

        # Now, match this value to Time Zone information
        tzkeyname = None
        for i in range(winreg.QueryInfoKey(tzkey)[0]):
            subkey = winreg.EnumKey(tzkey, i)
            sub = winreg.OpenKey(tzkey, subkey)
            data = valuestodict(sub)
            sub.Close()
            if data.get('Std', None) == tzwin:
                tzkeyname = subkey
                break

        tzkey.Close()
        handle.Close()

    if tzkeyname is None:
        raise LookupError('Can not find Windows timezone configuration')

    timezone = tz_names.get(tzkeyname)
    if timezone is None:
        # Nope, that didn't work. Try adding 'Standard Time',
        # it seems to work a lot of times:
        timezone = tz_names.get(f"{tzkeyname} Standard Time")

    # Return what we have.
    if timezone is None:
        raise LookupError(f"Can not find timezone {tzkeyname}")

    return timezone


def _get_localzone() -> datetime.tzinfo:
    if winreg is None:
        raise LookupError(
            'Runtime support not available')

    return _get_tzinfo_or_raise(get_localzone_name())
```

File: .venv\Lib\site-packages\babel\localtime__init__.py

```
"""
    babel.localtime
    ~~~~~~~~~~~~~~~

    Babel specific fork of tzlocal to determine the local timezone
    of the system.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""

import datetime
import sys

if sys.platform == 'win32':
    from babel.localtime._win32 import _get_localzone
else:
```

```
    from babel.localtime._unix import _get_localzone


# TODO(3.0): the offset constants are not part of the public API
#            and should be removed
from babel.localtime._fallback import (
    DSTDIFF,  # noqa: F401
    DSTOFFSET,  # noqa: F401
    STDOFFSET,  # noqa: F401
    ZERO,  # noqa: F401
    _FallbackLocalTimezone,
)


def get_localzone() -> datetime.tzinfo:
    """Returns the current underlying local timezone object.
    Generally this function does not need to be used, it's a
    better idea to use the :data:`LOCALTZ` singleton instead.
    """
    return _get_localzone()


try:
    LOCALTZ = get_localzone()
except LookupError:
    LOCALTZ = _FallbackLocalTimezone()
```

File: .venv\Lib\site-packages\babel\messages\catalog.py

```
"""
    babel.messages.catalog
    ~~~~~~~~~~~~~~~~~~~~~~

    Data structures for message catalogs.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

import datetime
import re
from collections import OrderedDict
from collections.abc import Iterable, Iterator
from copy import copy
from difflib import SequenceMatcher
from email import message_from_string
from heapq import nlargest
from typing import TYPE_CHECKING

from babel import __version__ as VERSION
from babel.core import Locale, UnknownLocaleError
from babel.dates import format_datetime
from babel.messages.plurals import get_plural
from babel.util import LOCALTZ, FixedOffsetTimezone, _cmp, distinct

if TYPE_CHECKING:
    from typing_extensions import TypeAlias

    _MessageID: TypeAlias = str | tuple[str, ...] | list[str]

__all__ = ['Message', 'Catalog', 'TranslationError']

def get_close_matches(word, possibilities, n=3, cutoff=0.6):
    """A modified version of ``difflib.get_close_matches``.

    It just passes ``autojunk=False`` to the ``SequenceMatcher``, to work
    around https://github.com/python/cpython/issues/90825.
    """
    if not n > 0:  # pragma: no cover
        raise ValueError(f"n must be > 0: {n!r}")
    if not 0.0 <= cutoff <= 1.0:  # pragma: no cover
        raise ValueError(f"cutoff must be in [0.0, 1.0]: {cutoff!r}")
    result = []
    s = SequenceMatcher(autojunk=False) # only line changed from difflib.py
    s.set_seq2(word)
    for x in possibilities:
        s.set_seq1(x)
        if s.real_quick_ratio() >= cutoff and \
```

```
            s.quick_ratio() >= cutoff and \
            s.ratio() >= cutoff:
             result.append((s.ratio(), x))

    # Move the best scorers to head of list
    result = nlargest(n, result)
    # Strip scores for the best n matches
    return [x for score, x in result]


PYTHON_FORMAT = re.compile(r'''
    \%
        (?:\(([\w]*)\))?
        (
            [-#0\ +]?(?:\*|[\d]+)?
            (?:\.(?:\*|[\d]+))?
            [hlL]?
        )
        ([diouxXeEfFgGcrs%])
''', re.VERBOSE)


def _parse_datetime_header(value: str) -> datetime.datetime:
    match = re.match(r'^(?P<datetime>.*?)(?P<tzoffset>[+-]\d{4})?$', value)

    dt = datetime.datetime.strptime(match.group('datetime'), '%Y-%m-%d %H:%M')

    # Separate the offset into a sign component, hours, and # minutes
    tzoffset = match.group('tzoffset')
    if tzoffset is not None:
        plus_minus_s, rest = tzoffset[0], tzoffset[1:]
        hours_offset_s, mins_offset_s = rest[:2], rest[2:]

        # Make them all integers
        plus_minus = int(f"{plus_minus_s}1")
        hours_offset = int(hours_offset_s)
        mins_offset = int(mins_offset_s)

        # Calculate net offset
        net_mins_offset = hours_offset * 60
        net_mins_offset += mins_offset
        net_mins_offset *= plus_minus

        # Create an offset object
        tzoffset = FixedOffsetTimezone(net_mins_offset)

        # Store the offset in a datetime object
        dt = dt.replace(tzinfo=tzoffset)

    return dt


class Message:
    """Representation of a single message in a catalog."""

    def __init__(
        self,
        id: _MessageID,
        string: _MessageID | None = '',
        locations: Iterable[tuple[str, int]] = (),
        flags: Iterable[str] = (),
        auto_comments: Iterable[str] = (),
        user_comments: Iterable[str] = (),
        previous_id: _MessageID = (),
        lineno: int | None = None,
        context: str | None = None,
    ) -> None:
        """Create the message object.

        :param id: the message ID, or a ``(singular, plural)`` tuple for
                   pluralizable messages
        :param string: the translated message string, or a
                       ``(singular, plural)`` tuple for pluralizable messages
        :param locations: a sequence of ``(filename, lineno)`` tuples
        :param flags: a set or sequence of flags
        :param auto_comments: a sequence of automatic comments for the message
        :param user_comments: a sequence of user comments for the message
        :param previous_id: the previous message ID, or a ``(singular, plural)``
                            tuple for pluralizable messages
        :param lineno: the line number on which the msgid line was found in the
                       PO file, if any
```

```
        :param context: the message context
        """
        self.id = id
        if not string and self.pluralizable:
            string = ('', '')
        self.string = string
        self.locations = list(distinct(locations))
        self.flags = set(flags)
        if id and self.python_format:
            self.flags.add('python-format')
        else:
            self.flags.discard('python-format')
        self.auto_comments = list(distinct(auto_comments))
        self.user_comments = list(distinct(user_comments))
        if isinstance(previous_id, str):
            self.previous_id = [previous_id]
        else:
            self.previous_id = list(previous_id)
        self.lineno = lineno
        self.context = context

    def __repr__(self) -> str:
        return f"<{type(self).__name__} {self.id!r} (flags: {list(self.flags)!r})>"

    def __cmp__(self, other: object) -> int:
        """Compare Messages, taking into account plural ids"""
        def values_to_compare(obj):
            if isinstance(obj, Message) and obj.pluralizable:
                return obj.id[0], obj.context or ''
            return obj.id, obj.context or ''
        return _cmp(values_to_compare(self), values_to_compare(other))

    def __gt__(self, other: object) -> bool:
        return self.__cmp__(other) > 0

    def __lt__(self, other: object) -> bool:
        return self.__cmp__(other) < 0

    def __ge__(self, other: object) -> bool:
        return self.__cmp__(other) >= 0

    def __le__(self, other: object) -> bool:
        return self.__cmp__(other) <= 0

    def __eq__(self, other: object) -> bool:
        return self.__cmp__(other) == 0

    def __ne__(self, other: object) -> bool:
        return self.__cmp__(other) != 0

    def is_identical(self, other: Message) -> bool:
        """Checks whether messages are identical, taking into account all
        properties.
        """
        assert isinstance(other, Message)
        return self.__dict__ == other.__dict__

    def clone(self) -> Message:
        return Message(*map(copy, (self.id, self.string, self.locations,
                                   self.flags, self.auto_comments,
                                   self.user_comments, self.previous_id,
                                   self.lineno, self.context)))

    def check(self, catalog: Catalog | None = None) -> list[TranslationError]:
        """Run various validation checks on the message.  Some validations
        are only performed if the catalog is provided.  This method returns
        a sequence of `TranslationError` objects.

        :rtype: ``iterator``
        :param catalog: A catalog instance that is passed to the checkers
        :see: `Catalog.check` for a way to perform checks for all messages
              in a catalog.
        """
        from babel.messages.checkers import checkers
        errors: list[TranslationError] = []
        for checker in checkers:
            try:
                checker(catalog, self)
            except TranslationError as e:
                errors.append(e)
        return errors
```

```
    @property
    def fuzzy(self) -> bool:
        """Whether the translation is fuzzy.

        >>> Message('foo').fuzzy
        False
        >>> msg = Message('foo', 'foo', flags=['fuzzy'])
        >>> msg.fuzzy
        True
        >>> msg
        <Message 'foo' (flags: ['fuzzy'])>

        :type:  `bool`"""
        return 'fuzzy' in self.flags

    @property
    def pluralizable(self) -> bool:
        """Whether the message is plurizable.

        >>> Message('foo').pluralizable
        False
        >>> Message(('foo', 'bar')).pluralizable
        True

        :type:  `bool`"""
        return isinstance(self.id, (list, tuple))

    @property
    def python_format(self) -> bool:
        """Whether the message contains Python-style parameters.

        >>> Message('foo %(name)s bar').python_format
        True
        >>> Message(('foo %(name)s', 'foo %(name)s')).python_format
        True

        :type:  `bool`"""
        ids = self.id
        if not isinstance(ids, (list, tuple)):
            ids = [ids]
        return any(PYTHON_FORMAT.search(id) for id in ids)


class TranslationError(Exception):
    """Exception thrown by translation checkers when invalid message
    translations are encountered."""


DEFAULT_HEADER = """\
# Translations template for PROJECT.
# Copyright (C) YEAR ORGANIZATION
# This file is distributed under the same license as the PROJECT project.
# FIRST AUTHOR <EMAIL@ADDRESS>, YEAR.
#"""


def parse_separated_header(value: str) -> dict[str, str]:
    # Adapted from https://peps.python.org/pep-0594/#cgi
    from email.message import Message
    m = Message()
    m['content-type'] = value
    return dict(m.get_params())


class Catalog:
    """Representation of a message catalog."""

    def __init__(
        self,
        locale: str | Locale | None = None,
        domain: str | None = None,
        header_comment: str | None = DEFAULT_HEADER,
        project: str | None = None,
        version: str | None = None,
        copyright_holder: str | None = None,
        msgid_bugs_address: str | None = None,
        creation_date: datetime.datetime | str | None = None,
        revision_date: datetime.datetime | datetime.time | float | str | None = None,
        last_translator: str | None = None,
        language_team: str | None = None,
        charset: str | None = None,
```

```
    fuzzy: bool = True,
) -> None:
    """Initialize the catalog object.

    :param locale: the locale identifier or `Locale` object, or `None`
                   if the catalog is not bound to a locale (which basically
                   means it's a template)
    :param domain: the message domain
    :param header_comment: the header comment as string, or `None` for the
                           default header
    :param project: the project's name
    :param version: the project's version
    :param copyright_holder: the copyright holder of the catalog
    :param msgid_bugs_address: the email address or URL to submit bug
                               reports to
    :param creation_date: the date the catalog was created
    :param revision_date: the date the catalog was revised
    :param last_translator: the name and email of the last translator
    :param language_team: the name and email of the language team
    :param charset: the encoding to use in the output (defaults to utf-8)
    :param fuzzy: the fuzzy bit on the catalog header
    """
    self.domain = domain
    self.locale = locale
    self._header_comment = header_comment
    self._messages: OrderedDict[str | tuple[str, str], Message] = OrderedDict()

    self.project = project or 'PROJECT'
    self.version = version or 'VERSION'
    self.copyright_holder = copyright_holder or 'ORGANIZATION'
    self.msgid_bugs_address = msgid_bugs_address or 'EMAIL@ADDRESS'

    self.last_translator = last_translator or 'FULL NAME <EMAIL@ADDRESS>'
    """Name and email address of the last translator."""
    self.language_team = language_team or 'LANGUAGE <LL@li.org>'
    """Name and email address of the language team."""

    self.charset = charset or 'utf-8'

    if creation_date is None:
        creation_date = datetime.datetime.now(LOCALTZ)
    elif isinstance(creation_date, datetime.datetime) and not creation_date.tzinfo:
        creation_date = creation_date.replace(tzinfo=LOCALTZ)
    self.creation_date = creation_date
    if revision_date is None:
        revision_date = 'YEAR-MO-DA HO:MI+ZONE'
    elif isinstance(revision_date, datetime.datetime) and not revision_date.tzinfo:
        revision_date = revision_date.replace(tzinfo=LOCALTZ)
    self.revision_date = revision_date
    self.fuzzy = fuzzy

    # Dictionary of obsolete messages
    self.obsolete: OrderedDict[str | tuple[str, str], Message] = OrderedDict()
    self._num_plurals = None
    self._plural_expr = None

def _set_locale(self, locale: Locale | str | None) -> None:
    if locale is None:
        self._locale_identifier = None
        self._locale = None
        return

    if isinstance(locale, Locale):
        self._locale_identifier = str(locale)
        self._locale = locale
        return

    if isinstance(locale, str):
        self._locale_identifier = str(locale)
        try:
            self._locale = Locale.parse(locale)
        except UnknownLocaleError:
            self._locale = None
        return

    raise TypeError(f"`locale` must be a Locale, a locale identifier string, or None; got {locale!r}")

def _get_locale(self) -> Locale | None:
    return self._locale

def _get_locale_identifier(self) -> str | None:
```

```
    return self._locale_identifier

locale = property(_get_locale, _set_locale)
locale_identifier = property(_get_locale_identifier)

def _get_header_comment(self) -> str:
    comment = self._header_comment
    year = datetime.datetime.now(LOCALTZ).strftime('%Y')
    if hasattr(self.revision_date, 'strftime'):
        year = self.revision_date.strftime('%Y')
    comment = comment.replace('PROJECT', self.project) \
                     .replace('VERSION', self.version) \
                     .replace('YEAR', year) \
                     .replace('ORGANIZATION', self.copyright_holder)
    locale_name = (self.locale.english_name if self.locale else self.locale_identifier)
    if locale_name:
        comment = comment.replace("Translations template", f"{locale_name} translations")
    return comment

def _set_header_comment(self, string: str | None) -> None:
    self._header_comment = string

header_comment = property(_get_header_comment, _set_header_comment, doc="""\
The header comment for the catalog.

>>> catalog = Catalog(project='Foobar', version='1.0',
...                   copyright_holder='Foo Company')
>>> print(catalog.header_comment) #doctest: +ELLIPSIS
# Translations template for Foobar.
# Copyright (C) ... Foo Company
# This file is distributed under the same license as the Foobar project.
# FIRST AUTHOR <EMAIL@ADDRESS>, ....
#

The header can also be set from a string. Any known upper-case variables
will be replaced when the header is retrieved again:

>>> catalog = Catalog(project='Foobar', version='1.0',
...                   copyright_holder='Foo Company')
>>> catalog.header_comment = '''\\
... # The POT for my really cool PROJECT project.
... # Copyright (C) 1990-2003 ORGANIZATION
... # This file is distributed under the same license as the PROJECT
... # project.
... #'''
>>> print(catalog.header_comment)
# The POT for my really cool Foobar project.
# Copyright (C) 1990-2003 Foo Company
# This file is distributed under the same license as the Foobar
# project.
#

:type: `unicode`
""")

def _get_mime_headers(self) -> list[tuple[str, str]]:
    headers: list[tuple[str, str]] = []
    headers.append(("Project-Id-Version", f"{self.project} {self.version}"))
    headers.append(('Report-Msgid-Bugs-To', self.msgid_bugs_address))
    headers.append(('POT-Creation-Date',
                    format_datetime(self.creation_date, 'yyyy-MM-dd HH:mmZ',
                                    locale='en')))
    if isinstance(self.revision_date, (datetime.datetime, datetime.time, int, float)):
        headers.append(('PO-Revision-Date',
                        format_datetime(self.revision_date,
                                        'yyyy-MM-dd HH:mmZ', locale='en')))
    else:
        headers.append(('PO-Revision-Date', self.revision_date))
    headers.append(('Last-Translator', self.last_translator))
    if self.locale_identifier:
        headers.append(('Language', str(self.locale_identifier)))
    if self.locale_identifier and ('LANGUAGE' in self.language_team):
        headers.append(('Language-Team',
                        self.language_team.replace('LANGUAGE',
                                                   str(self.locale_identifier))))
    else:
        headers.append(('Language-Team', self.language_team))
    if self.locale is not None:
        headers.append(('Plural-Forms', self.plural_forms))
    headers.append(('MIME-Version', '1.0'))
    headers.append(("Content-Type", f"text/plain; charset={self.charset}"))
```

```
        headers.append(('Content-Transfer-Encoding', '8bit'))
        headers.append(("Generated-By", f"Babel {VERSION}\n"))
        return headers

    def _force_text(self, s: str | bytes, encoding: str = 'utf-8', errors: str = 'strict') -> str:
        if isinstance(s, str):
            return s
        if isinstance(s, bytes):
            return s.decode(encoding, errors)
        return str(s)

    def _set_mime_headers(self, headers: Iterable[tuple[str, str]]) -> None:
        for name, value in headers:
            name = self._force_text(name.lower(), encoding=self.charset)
            value = self._force_text(value, encoding=self.charset)
            if name == 'project-id-version':
                parts = value.split(' ')
                self.project = ' '.join(parts[:-1])
                self.version = parts[-1]
            elif name == 'report-msgid-bugs-to':
                self.msgid_bugs_address = value
            elif name == 'last-translator':
                self.last_translator = value
            elif name == 'language':
                value = value.replace('-', '_')
                self._set_locale(value)
            elif name == 'language-team':
                self.language_team = value
            elif name == 'content-type':
                params = parse_separated_header(value)
                if 'charset' in params:
                    self.charset = params['charset'].lower()
            elif name == 'plural-forms':
                params = parse_separated_header(f" ;{value}")
                self._num_plurals = int(params.get('nplurals', 2))
                self._plural_expr = params.get('plural', '(n != 1)')
            elif name == 'pot-creation-date':
                self.creation_date = _parse_datetime_header(value)
            elif name == 'po-revision-date':
                # Keep the value if it's not the default one
                if 'YEAR' not in value:
                    self.revision_date = _parse_datetime_header(value)

    mime_headers = property(_get_mime_headers, _set_mime_headers, doc="""\
    The MIME headers of the catalog, used for the special ``msgid ""`` entry.

    The behavior of this property changes slightly depending on whether a locale
    is set or not, the latter indicating that the catalog is actually a template
    for actual translations.

    Here's an example of the output for such a catalog template:

    >>> from babel.dates import UTC
    >>> from datetime import datetime
    >>> created = datetime(1990, 4, 1, 15, 30, tzinfo=UTC)
    >>> catalog = Catalog(project='Foobar', version='1.0',
    ...                   creation_date=created)
    >>> for name, value in catalog.mime_headers:
    ...     print('%s: %s' % (name, value))
    Project-Id-Version: Foobar 1.0
    Report-Msgid-Bugs-To: EMAIL@ADDRESS
    POT-Creation-Date: 1990-04-01 15:30+0000
    PO-Revision-Date: YEAR-MO-DA HO:MI+ZONE
    Last-Translator: FULL NAM
```
(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\babel\messages\checkers.py

```
"""
    babel.messages.checkers
    ~~~~~~~~~~~~~~~~~~~~~~~

    Various routines that help with validation of translations.

    :since: version 0.9

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
```

```
from __future__ import annotations

from collections.abc import Callable

from babel.messages.catalog import PYTHON_FORMAT, Catalog, Message, TranslationError

#: list of format chars that are compatible to each other
_string_format_compatibilities = [
    {'i', 'd', 'u'},
    {'x', 'X'},
    {'f', 'F', 'g', 'G'},
]


def num_plurals(catalog: Catalog | None, message: Message) -> None:
    """Verify the number of plurals in the translation."""
    if not message.pluralizable:
        if not isinstance(message.string, str):
            raise TranslationError("Found plural forms for non-pluralizable "
                                   "message")
        return

    # skip further tests if no catalog is provided.
    elif catalog is None:
        return

    msgstrs = message.string
    if not isinstance(msgstrs, (list, tuple)):
        msgstrs = (msgstrs,)
    if len(msgstrs) != catalog.num_plurals:
        raise TranslationError("Wrong number of plural forms (expected %d)" %
                               catalog.num_plurals)


def python_format(catalog: Catalog | None, message: Message) -> None:
    """Verify the format string placeholders in the translation."""
    if 'python-format' not in message.flags:
        return
    msgids = message.id
    if not isinstance(msgids, (list, tuple)):
        msgids = (msgids,)
    msgstrs = message.string
    if not isinstance(msgstrs, (list, tuple)):
        msgstrs = (msgstrs,)

    for msgid, msgstr in zip(msgids, msgstrs):
        if msgstr:
            _validate_format(msgid, msgstr)


def _validate_format(format: str, alternative: str) -> None:
    """Test format string `alternative` against `format`.  `format` can be the
    msgid of a message and `alternative` one of the `msgstr`\\s.  The two
    arguments are not interchangeable as `alternative` may contain less
    placeholders if `format` uses named placeholders.

    The behavior of this function is undefined if the string does not use
    string formatting.

    If the string formatting of `alternative` is compatible to `format` the
    function returns `None`, otherwise a `TranslationError` is raised.

    Examples for compatible format strings:

    >>> _validate_format('Hello %s!', 'Hallo %s!')
    >>> _validate_format('Hello %i!', 'Hallo %d!')

    Example for an incompatible format strings:

    >>> _validate_format('Hello %(name)s!', 'Hallo %s!')
    Traceback (most recent call last):
      ...
    TranslationError: the format strings are of different kinds

    This function is used by the `python_format` checker.

    :param format: The original format string
    :param alternative: The alternative format string that should be checked
                        against format
    :raises TranslationError: on formatting errors
    """
```

```
    def _parse(string: str) -> list[tuple[str, str]]:
        result: list[tuple[str, str]] = []
        for match in PYTHON_FORMAT.finditer(string):
            name, format, typechar = match.groups()
            if typechar == '%' and name is None:
                continue
            result.append((name, str(typechar)))
        return result

    def _compatible(a: str, b: str) -> bool:
        if a == b:
            return True
        for set in _string_format_compatibilities:
            if a in set and b in set:
                return True
        return False

    def _check_positional(results: list[tuple[str, str]]) -> bool:
        positional = None
        for name, _char in results:
            if positional is None:
                positional = name is None
            else:
                if (name is None) != positional:
                    raise TranslationError('format string mixes positional '
                                           'and named placeholders')
        return bool(positional)

    a, b = map(_parse, (format, alternative))

    # now check if both strings are positional or named
    a_positional, b_positional = map(_check_positional, (a, b))
    if a_positional and not b_positional and not b:
        raise TranslationError('placeholders are incompatible')
    elif a_positional != b_positional:
        raise TranslationError('the format strings are of different kinds')

    # if we are operating on positional strings both must have the
    # same number of format chars and those must be compatible
    if a_positional:
        if len(a) != len(b):
            raise TranslationError('positional format placeholders are '
                                   'unbalanced')
        for idx, ((_, first), (_, second)) in enumerate(zip(a, b)):
            if not _compatible(first, second):
                raise TranslationError('incompatible format for placeholder '
                                       '%d: %r and %r are not compatible' %
                                       (idx + 1, first, second))

    # otherwise the second string must not have names the first one
    # doesn't have and the types of those included must be compatible
    else:
        type_map = dict(a)
        for name, typechar in b:
            if name not in type_map:
                raise TranslationError(f'unknown named placeholder {name!r}')
            elif not _compatible(typechar, type_map[name]):
                raise TranslationError(
                    f'incompatible format for placeholder {name!r}: '
                    f'{typechar!r} and {type_map[name]!r} are not compatible',
                )


def _find_checkers() -> list[Callable[[Catalog | None, Message], object]]:
    checkers: list[Callable[[Catalog | None, Message], object]] = []
    try:
        from pkg_resources import working_set
    except ImportError:
        pass
    else:
        for entry_point in working_set.iter_entry_points('babel.checkers'):
            checkers.append(entry_point.load())
    if len(checkers) == 0:
        # if pkg_resources is not available or no usable egg-info was found
        # (see #230), just resort to hard-coded checkers
        return [num_plurals, python_format]
    return checkers


checkers: list[Callable[[Catalog | None, Message], object]] = _find_checkers()
```

File: .venv\Lib\site-packages\babel\messages\extract.py

```
"""
    babel.messages.extract
    ~~~~~~~~~~~~~~~~~~~~~~

    Basic infrastructure for extracting localizable messages from source files.

    This module defines an extensible system for collecting localizable message
    strings from a variety of sources. A native extractor for Python source
    files is builtin, extractors for other sources can be added using very
    simple plugins.

    The main entry points into the extraction functionality are the functions
    `extract_from_dir` and `extract_from_file`.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

import ast
import io
import os
import sys
import tokenize
from collections.abc import (
    Callable,
    Collection,
    Generator,
    Iterable,
    Mapping,
    MutableSequence,
)
from os.path import relpath
from textwrap import dedent
from tokenize import COMMENT, NAME, OP, STRING, generate_tokens
from typing import TYPE_CHECKING, Any

from babel.util import parse_encoding, parse_future_flags, pathmatch

if TYPE_CHECKING:
    from typing import IO, Protocol

    from _typeshed import SupportsItems, SupportsRead, SupportsReadline
    from typing_extensions import Final, TypeAlias, TypedDict

    class _PyOptions(TypedDict, total=False):
        encoding: str

    class _JSOptions(TypedDict, total=False):
        encoding: str
        jsx: bool
        template_string: bool
        parse_template_string: bool

    class _FileObj(SupportsRead[bytes], SupportsReadline[bytes], Protocol):
        def seek(self, __offset: int, __whence: int = ...) -> int: ...
        def tell(self) -> int: ...

    _SimpleKeyword: TypeAlias = tuple[int | tuple[int, int] | tuple[int, str], ...] | None
    _Keyword: TypeAlias = dict[int | None, _SimpleKeyword] | _SimpleKeyword

    # 5-tuple of (filename, lineno, messages, comments, context)
    _FileExtractionResult: TypeAlias = tuple[str, int, str | tuple[str, ...], list[str], str | None]

    # 4-tuple of (lineno, message, comments, context)
    _ExtractionResult: TypeAlias = tuple[int, str | tuple[str, ...], list[str], str | None]

    # Required arguments: fileobj, keywords, comment_tags, options
    # Return value: Iterable of (lineno, message, comments, context)
    _CallableExtractionMethod: TypeAlias = Callable[
        [_FileObj | IO[bytes], Mapping[str, _Keyword], Collection[str], Mapping[str, Any]],
        Iterable[_ExtractionResult],
    ]

    _ExtractionMethod: TypeAlias = _CallableExtractionMethod | str

GROUP_NAME: Final[str] = 'babel.extractors'

DEFAULT_KEYWORDS: dict[str, _Keyword] = {
```

```
    '_': None,
    'gettext': None,
    'ngettext': (1, 2),
    'ugettext': None,
    'ungettext': (1, 2),
    'dgettext': (2,),
    'dngettext': (2, 3),
    'N_': None,
    'pgettext': ((1, 'c'), 2),
    'npgettext': ((1, 'c'), 2, 3),
}

DEFAULT_MAPPING: list[tuple[str, str]] = [('**.py', 'python')]

# New tokens in Python 3.12, or None on older versions
FSTRING_START = getattr(tokenize, "FSTRING_START", None)
FSTRING_MIDDLE = getattr(tokenize, "FSTRING_MIDDLE", None)
FSTRING_END = getattr(tokenize, "FSTRING_END", None)


def _strip_comment_tags(comments: MutableSequence[str], tags: Iterable[str]):
    """Helper function for `extract` that strips comment tags from strings
    in a list of comment lines.  This functions operates in-place.
    """
    def _strip(line: str):
        for tag in tags:
            if line.startswith(tag):
                return line[len(tag):].strip()
        return line
    comments[:] = map(_strip, comments)


def default_directory_filter(dirpath: str | os.PathLike[str]) -> bool:
    subdir = os.path.basename(dirpath)
    # Legacy default behavior: ignore dot and underscore directories
    return not (subdir.startswith('.') or subdir.startswith('_'))


def extract_from_dir(
    dirname: str | os.PathLike[str] | None = None,
    method_map: Iterable[tuple[str, str]] = DEFAULT_MAPPING,
    options_map: SupportsItems[str, dict[str, Any]] | None = None,
    keywords: Mapping[str, _Keyword] = DEFAULT_KEYWORDS,
    comment_tags: Collection[str] = (),
    callback: Callable[[str, str, dict[str, Any]], object] | None = None,
    strip_comment_tags: bool = False,
    directory_filter: Callable[[str], bool] | None = None,
) -> Generator[_FileExtractionResult, None, None]:
    """Extract messages from any source files found in the given directory.

    This function generates tuples of the form ``(filename, lineno, message,
    comments, context)``.

    Which extraction method is used per file is determined by the `method_map`
    parameter, which maps extended glob patterns to extraction method names.
    For example, the following is the default mapping:

    >>> method_map = [
    ...     ('**.py', 'python')
    ... ]

    This basically says that files with the filename extension ".py" at any
    level inside the directory should be processed by the "python" extraction
    method. Files that don't match any of the mapping patterns are ignored. See
    the documentation of the `pathmatch` function for details on the pattern
    syntax.

    The following extended mapping would also use the "genshi" extraction
    method on any file in "templates" subdirectory:

    >>> method_map = [
    ...     ('**/templates/**.*', 'genshi'),
    ...     ('**.py', 'python')
    ... ]

    The dictionary provided by the optional `options_map` parameter augments
    these mappings. It uses extended glob patterns as keys, and the values are
    dictionaries mapping options names to option values (both strings).

    The glob patterns of the `options_map` do not necessarily need to be the
    same as those used in the method mapping. For example, while all files in
```

```
    the ``templates`` folders in an application may be Genshi applications, the
    options for those files may differ based on extension:

    >>> options_map = {
    ...     '**/templates/**.txt': {
    ...         'template_class': 'genshi.template:TextTemplate',
    ...         'encoding': 'latin-1'
    ...     },
    ...     '**/templates/**.html': {
    ...         'include_attrs': ''
    ...     }
    ... }

    :param dirname: the path to the directory to extract messages from.  If
                    not given the current working directory is used.
    :param method_map: a list of ``(pattern, method)`` tuples that maps of
                       extraction method names to extended glob patterns
    :param options_map: a dictionary of additional options (optional)
    :param keywords: a dictionary mapping keywords (i.e. names of functions
                     that should be recognized as translation functions) to
                     tuples that specify which of their arguments contain
                     localizable strings
    :param comment_tags: a list of tags of translator comments to search for
                         and include in the results
    :param callback: a function that is called for every file that message are
                     extracted from, just before the extraction itself is
                     performed; the function is passed the filename, the name
                     of the extraction method and and the options dictionary as
                     positional arguments, in that order
    :param strip_comment_tags: a flag that if set to `True` causes all comment
                               tags to be removed from the collected comments.
    :param directory_filter: a callback to determine whether a directory should
                             be recursed into. Receives the full directory path;
                             should return True if the directory is valid.
    :see: `pathmatch`
    """
    if dirname is None:
        dirname = os.getcwd()
    if options_map is None:
        options_map = {}
    if directory_filter is None:
        directory_filter = default_directory_filter

    absname = os.path.abspath(dirname)
    for root, dirnames, filenames in os.walk(absname):
        dirnames[:] = [
            subdir for subdir in dirnames
            if directory_filter(os.path.join(root, subdir))
        ]
        dirnames.sort()
        filenames.sort()
        for filename in filenames:
            filepath = os.path.join(root, filename).replace(os.sep, '/')

            yield from check_and_call_extract_file(
                filepath,
                method_map,
                options_map,
                callback,
                keywords,
                comment_tags,
                strip_comment_tags,
                dirpath=absname,
            )


def check_and_call_extract_file(
    filepath: str | os.PathLike[str],
    method_map: Iterable[tuple[str, str]],
    options_map: SupportsItems[str, dict[str, Any]],
    callback: Callable[[str, str, dict[str, Any]], object] | None,
    keywords: Mapping[str, _Keyword],
    comment_tags: Collection[str],
    strip_comment_tags: bool,
    dirpath: str | os.PathLike[str] | None = None,
) -> Generator[_FileExtractionResult, None, None]:
    """Checks if the given file matches an extraction method mapping, and if so, calls extract_from_file.

    Note that the extraction method mappings are based relative to dirpath.
    So, given an absolute path to a file `filepath`, we want to check using
    just the relative path from `dirpath` to `filepath`.
```

```
    Yields 5-tuples (filename, lineno, messages, comments, context).

    :param filepath: An absolute path to a file that exists.
    :param method_map: a list of ``(pattern, method)`` tuples that maps of
                       extraction method names to extended glob patterns
    :param options_map: a dictionary of additional options (optional)
    :param callback: a function that is called for every file that message are
                     extracted from, just before the extraction itself is
                     performed; the function is passed the filename, the name
                     of the extraction method and and the options dictionary as
                     positional arguments, in that order
    :param keywords: a dictionary mapping keywords (i.e. names of functions
                     that should be recognized as translation functions) to
                     tuples that specify which of their arguments contain
                     localizable strings
    :param comment_tags: a list of tags of translator comments to search for
                         and include in the results
    :param strip_comment_tags: a flag that if set to `True` causes all comment
                               tags to be removed from the collected comments.
    :param dirpath: the path to the directory to extract messages from.
    :return: iterable of 5-tuples (filename, lineno, messages, comments, context)
    :rtype: Iterable[tuple[str, int, str|tuple[str], list[str], str|None]
    """
    # filename is the relative path from dirpath to the actual file
    filename = relpath(filepath, dirpath)

    for pattern, method in method_map:
        if not pathmatch(pattern, filename):
            continue

        options = {}
        for opattern, odict in options_map.items():
            if pathmatch(opattern, filename):
                options = odict
        if callback:
            callback(filename, method, options)
        for message_tuple in extract_from_file(
            method, filepath,
            keywords=keywords,
            comment_tags=comment_tags,
            options=options,
            strip_comment_tags=strip_comment_tags,
        ):
            yield (filename, *message_tuple)

        break


def extract_from_file(
    method: _ExtractionMethod,
    filename: str | os.PathLike[str],
    keywords: Mapping[str, _Keyword] = DEFAULT_KEYWORDS,
    comment_tags: Collection[str] = (),
    options: Mapping[str, Any] | None = None,
    strip_comment_tags: bool = False,
) -> list[_ExtractionResult]:
    """Extract messages from a specific file.

    This function returns a list of tuples of the form ``(lineno, message, comments, context)``.

    :param filename: the path to the file to extract messages from
    :param method: a string specifying the extraction method (.e.g. "python")
    :param keywords: a dictionary mapping keywords (i.e. names of functions
                     that should be recognized as translation functions) to
                     tuples that specify which of their arguments contain
                     localizable strings
    :param comment_tags: a list of translator tags to search for and include
                         in the results
    :param strip_comment_tags: a flag that if set to `True` causes all comment
                               tags to be removed from the collected comments.
    :param options: a dictionary of additional options (optional)
    :returns: list of tuples of the form ``(lineno, message, comments, context)``
    :rtype: list[tuple[int, str|tuple[str], list[str], str|None]
    """
    if method == 'ignore':
        return []

    with open(filename, 'rb') as fileobj:
        return list(extract(method, fileobj, keywords, comment_tags,
                            options, strip_comment_tags))
def _match_messages_against_spec(lineno: int, messages: list[str|None], comments: list[str],
```

```
                                fileobj: _FileObj, spec: tuple[int|tuple[int, str], ...]):
    translatable = []
    context = None

    # last_index is 1 based like the keyword spec
    last_index = len(messages)
    for index in spec:
        if isinstance(index, tuple): # (n, 'c')
            context = messages[index[0] - 1]
            continue
        if last_index < index:
            # Not enough arguments
            return
        message = messages[index - 1]
        if message is None:
            return
        translatable.append(message)

    # keyword spec indexes are 1 based, therefore '-1'
    if isinstance(spec[0], tuple):
        # context-aware *gettext method
        first_msg_index = spec[1] - 1
    else:
        first_msg_index = spec[0] - 1
    # An empty string msgid isn't valid, emit a warning
    if not messages[first_msg_index]:
        filename = (getattr(fileobj, "name", None) or "(unknown)")
        sys.stderr.write(
            f"{filename}:{lineno}: warning: Empty msgid.  It is reserved by GNU gettext: gettext(\"\") "
            f"returns the header entry with meta information, not the empty string.\n",
        )
        return

    translatable = tuple(translatable)
    if len(translatable) == 1:
        translatable = translatable[0]

    return lineno, translatable, comments, context


def extract(
    method: _ExtractionMethod,
    fileobj: _FileObj,
    keywords: Mapping[str, _Keyword] = DEFAULT_KEYWORDS,
    comment_tags: Collection[str] = (),
    options: Mapping[str, Any] | None = None,
    strip_comment_tags: bool = False,
) -> Generator[_ExtractionResult, None, None]:
    """Extract messages from the given file-like object using the specified
    extraction method.

    This function returns tuples of the form ``(lineno, message, comments, context)``.

    The implementation dispatches the actual extraction to plugins, based on the
    value of the ``method`` parameter.

    >>> source = b'''# foo module
    ... def run(argv):
    ...    print(_('Hello, world!'))
    ... '''

    >>> from io import BytesIO
    >>> for message in extract('python', BytesIO(source)):
    ...     print(message)
    (3, u'Hello, world!', [], None)

    :param method: an extraction method (a callable), or
                   a string specifying the extraction method (.e.g. "python");
                   if this is a simple name, the extraction function will be
                   looked up by entry point; if it is an explicit reference
                   to a function (of the form ``package.module:funcname`` or
                   ``package.module.funcname``), the corresponding function
                   will be imported and used
    :param fileobj: the file-like object the messages should be extracted from
    :param keywords: a dictionary mapping keywords (i.e. names of functions
                     that should be recognized as translation functions) to
                     tuples that specify which of their arguments contain
                     localizable strings
    :param comment_tags: a list of translator tags to search for and include
                         in the results
    :param options: a dictionary of additional options (optional)
```

```
    :param strip_comment_tags: a flag that if set to `True` causes all comment
                               tags to be removed from the collected comments.
    :raise ValueError: if the extraction method is not registered
    :returns: iterable of tuples of the form ``(lineno, message, comments, context)``
    :rtype: Iterable[tuple[int, str|tuple[str], list[str], str|None]
    """
    func = None
    if callable(method):
        func = method
    elif ':' in method or '.' in method:
        if ':' not in method:
            lastdot = method.rfind('.')
            module, attrname = method[:lastdot], method[lastdot + 1:]
        else:
            module, attrname = method.split(':', 1)
        func = getattr(__import__(module, {}, {}, [attrname]), attrname)
    else:
        try:
            from pkg_resources import working_set
        except ImportError:
            pass
        else:
            for entry_point in working_set.iter_entry_points(GROUP_NAME,
                                                             method):
                func = entry_point.load(require=True)
                break
        if func is None:
            # if pkg_resources is not available or no usable egg-info was found
            # (see #230), we resort to looking up the builtin extractors
            # directly
            builtin = {
                'ignore': extract_nothing,
                'python': extract_python,
                'javascript': extract_javascript,
            }
            func = builtin.get(method)

    if func is None:
        raise ValueError(f"Unknown extraction method {method!r}")

    results = func(fileobj, keywords.keys(), comment_tags,
                   options=options or {})

    for lineno, funcname, messages, comments in results:
        if not isinstance(messages, (list, tuple)):
            messages = [messages]
        if not messages:
            continue

        specs = keywords[funcname] or None if funcname else None
        # {None: x} may be collapsed into x for backwards compatibility.
        if not isinstance(specs, dict):
            specs = {None: specs}

        if strip_comment_tags:
            _strip_comment_tags(comments, comment_tags)

        # None matches all arities.
        for arity in (None, len(messages)):
            try:
                spec = specs[arity]
            except KeyError:
                continue
            if spec is None:
                spec = (1,)
            result = _match_messages_against_spec(lineno, messages, comments, fileobj, spec)
            if result is not None:
                yield result


def extract_nothing(
    fileobj: _FileObj,
    keywords: Mapping[str, _Keyword],
    comment_tags: Collection[str],
    options: Mapping[str, Any],
) -> list[_ExtractionResult]:
    """Pseudo extractor that does not actually extract anything, but simply
    returns an empty list.
    """
    return []
def extract_python(
```

```
    fileobj: IO[bytes],
    keywords: Mapping[str, _Keyword],
    comment_tags: Collection[str],
    options: _PyOptions,
) -> Generator[_ExtractionResult, None, None]:
    """Extract messages from Python source code.

    It returns an iterator yielding tuples in the following form ``(lineno,
    funcname, message, comments)``.

    :param fileobj: the seekable, file-like object the messages should be
                    extracted from
    :param keywords: a list of keywords (i.e. function names) that should be
                     recognized as tra
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\babel\messages\frontend.py

```
"""
    babel.messages.frontend
    ~~~~~~~~~~~~~~~~~~~~~~~

    Frontends for the message extraction functionality.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""

from __future__ import annotations

import datetime
import fnmatch
import logging
import optparse
import os
import re
import shutil
import sys
import tempfile
from collections import OrderedDict
from configparser import RawConfigParser
from io import StringIO
from typing import Iterable

from babel import Locale, localedata
from babel import __version__ as VERSION
from babel.core import UnknownLocaleError
from babel.messages.catalog import DEFAULT_HEADER, Catalog
from babel.messages.extract import (
    DEFAULT_KEYWORDS,
    DEFAULT_MAPPING,
    check_and_call_extract_file,
    extract_from_dir,
)
from babel.messages.mofile import write_mo
from babel.messages.pofile import read_po, write_po
from babel.util import LOCALTZ

log = logging.getLogger('babel')


class BaseError(Exception):
    pass


class OptionError(BaseError):
    pass


class SetupError(BaseError):
    pass


def listify_value(arg, split=None):
    """
    Make a list out of an argument.

    Values from `distutils` argument parsing are always single strings;
    values from `optparse` parsing may be lists of strings that may need
```

```
    to be further split.

    No matter the input, this function returns a flat list of whitespace-trimmed
    strings, with `None` values filtered out.

    >>> listify_value("foo bar")
    ['foo', 'bar']
    >>> listify_value(["foo bar"])
    ['foo', 'bar']
    >>> listify_value([["foo"], "bar"])
    ['foo', 'bar']
    >>> listify_value([["foo"], ["bar", None, "foo"]])
    ['foo', 'bar', 'foo']
    >>> listify_value("foo, bar, quux", ",")
    ['foo', 'bar', 'quux']

    :param arg: A string or a list of strings
    :param split: The argument to pass to `str.split()`.
    :return:
    """
    out = []

    if not isinstance(arg, (list, tuple)):
        arg = [arg]

    for val in arg:
        if val is None:
            continue
        if isinstance(val, (list, tuple)):
            out.extend(listify_value(val, split=split))
            continue
        out.extend(s.strip() for s in str(val).split(split))
    assert all(isinstance(val, str) for val in out)
    return out


class CommandMixin:
    # This class is a small shim between Distutils commands and
    # optparse option parsing in the frontend command line.

    #: Option name to be input as `args` on the script command line.
    as_args = None

    #: Options which allow multiple values.
    #: This is used by the `optparse` transmogrification code.
    multiple_value_options = ()

    #: Options which are booleans.
    #: This is used by the `optparse` transmogrification code.
    # (This is actually used by distutils code too, but is never
    # declared in the base class.)
    boolean_options = ()

    #: Option aliases, to retain standalone command compatibility.
    #: Distutils does not support option aliases, but optparse does.
    #: This maps the distutils argument name to an iterable of aliases
    #: that are usable with optparse.
    option_aliases = {}

    #: Choices for options that needed to be restricted to specific
    #: list of choices.
    option_choices = {}

    #: Log object. To allow replacement in the script command line runner.
    log = log

    def __init__(self, dist=None):
        # A less strict version of distutils' `__init__`.
        self.distribution = dist
        self.initialize_options()
        self._dry_run = None
        self.verbose = False
        self.force = None
        self.help = 0
        self.finalized = 0

    def initialize_options(self):
        pass

    def ensure_finalized(self):
        if not self.finalized:
```

```
            self.finalize_options()
        self.finalized = 1

    def finalize_options(self):
        raise RuntimeError(
            f"abstract method -- subclass {self.__class__} must override",
        )


class CompileCatalog(CommandMixin):
    description = 'compile message catalogs to binary MO files'
    user_options = [
        ('domain=', 'D',
         "domains of PO files (space separated list, default 'messages')"),
        ('directory=', 'd',
         'path to base directory containing the catalogs'),
        ('input-file=', 'i',
         'name of the input file'),
        ('output-file=', 'o',
         "name of the output file (default "
         "'<output_dir>/<locale>/LC_MESSAGES/<domain>.mo')"),
        ('locale=', 'l',
         'locale of the catalog to compile'),
        ('use-fuzzy', 'f',
         'also include fuzzy translations'),
        ('statistics', None,
         'print statistics about translations'),
    ]
    boolean_options = ['use-fuzzy', 'statistics']

    def initialize_options(self):
        self.domain = 'messages'
        self.directory = None
        self.input_file = None
        self.output_file = None
        self.locale = None
        self.use_fuzzy = False
        self.statistics = False

    def finalize_options(self):
        self.domain = listify_value(self.domain)
        if not self.input_file and not self.directory:
            raise OptionError('you must specify either the input file or the base directory')
        if not self.output_file and not self.directory:
            raise OptionError('you must specify either the output file or the base directory')

    def run(self):
        n_errors = 0
        for domain in self.domain:
            for errors in self._run_domain(domain).values():
                n_errors += len(errors)
        if n_errors:
            self.log.error('%d errors encountered.', n_errors)
        return (1 if n_errors else 0)

    def _run_domain(self, domain):
        po_files = []
        mo_files = []

        if not self.input_file:
            if self.locale:
                po_files.append((self.locale,
                                 os.path.join(self.directory, self.locale,
                                              'LC_MESSAGES',
                                              f"{domain}.po")))
                mo_files.append(os.path.join(self.directory, self.locale,
                                             'LC_MESSAGES',
                                             f"{domain}.mo"))
            else:
                for locale in os.listdir(self.directory):
                    po_file = os.path.join(self.directory, locale,
                                           'LC_MESSAGES', f"{domain}.po")
                    if os.path.exists(po_file):
                        po_files.append((locale, po_file))
                        mo_files.append(os.path.join(self.directory, locale,
                                                     'LC_MESSAGES',
                                                     f"{domain}.mo"))
        else:
            po_files.append((self.locale, self.input_file))
            if self.output_file:
                mo_files.append(self.output_file)
```

```
            else:
                mo_files.append(os.path.join(self.directory, self.locale,
                                             'LC_MESSAGES',
                                             f"{domain}.mo"))

        if not po_files:
            raise OptionError('no message catalogs found')

        catalogs_and_errors = {}

        for idx, (locale, po_file) in enumerate(po_files):
            mo_file = mo_files[idx]
            with open(po_file, 'rb') as infile:
                catalog = read_po(infile, locale)

            if self.statistics:
                translated = 0
                for message in list(catalog)[1:]:
                    if message.string:
                        translated += 1
                percentage = 0
                if len(catalog):
                    percentage = translated * 100 // len(catalog)
                self.log.info(
                    '%d of %d messages (%d%%) translated in %s',
                    translated, len(catalog), percentage, po_file,
                )

            if catalog.fuzzy and not self.use_fuzzy:
                self.log.info('catalog %s is marked as fuzzy, skipping', po_file)
                continue

            catalogs_and_errors[catalog] = catalog_errors = list(catalog.check())
            for message, errors in catalog_errors:
                for error in errors:
                    self.log.error(
                        'error: %s:%d: %s', po_file, message.lineno, error,
                    )

            self.log.info('compiling catalog %s to %s', po_file, mo_file)

            with open(mo_file, 'wb') as outfile:
                write_mo(outfile, catalog, use_fuzzy=self.use_fuzzy)

        return catalogs_and_errors


def _make_directory_filter(ignore_patterns):
    """
    Build a directory_filter function based on a list of ignore patterns.
    """

    def cli_directory_filter(dirname):
        basename = os.path.basename(dirname)
        return not any(
            fnmatch.fnmatch(basename, ignore_pattern)
            for ignore_pattern
            in ignore_patterns
        )

    return cli_directory_filter


class ExtractMessages(CommandMixin):
    description = 'extract localizable strings from the project code'
    user_options = [
        ('charset=', None,
         'charset to use in the output file (default "utf-8")'),
        ('keywords=', 'k',
         'space-separated list of keywords to look for in addition to the '
         'defaults (may be repeated multiple times)'),
        ('no-default-keywords', None,
         'do not include the default keywords'),
        ('mapping-file=', 'F',
         'path to the mapping configuration file'),
        ('no-location', None,
         'do not include location comments with filename and line number'),
        ('add-location=', None,
         'location lines format. If it is not given or "full", it generates '
         'the lines with both file name and line number. If it is "file", '
         'the line number part is omitted. If it is "never", it completely '
```

```
     'suppresses the lines (same as --no-location).'),
    ('omit-header', None,
     'do not include msgid "" entry in header'),
    ('output-file=', 'o',
     'name of the output file'),
    ('width=', 'w',
     'set output line width (default 76)'),
    ('no-wrap', None,
     'do not break long message lines, longer than the output line width, '
     'into several lines'),
    ('sort-output', None,
     'generate sorted output (default False)'),
    ('sort-by-file', None,
     'sort output by file location (default False)'),
    ('msgid-bugs-address=', None,
     'set report address for msgid'),
    ('copyright-holder=', None,
     'set copyright holder in output'),
    ('project=', None,
     'set project name in output'),
    ('version=', None,
     'set project version in output'),
    ('add-comments=', 'c',
     'place comment block with TAG (or those preceding keyword lines) in '
     'output file. Separate multiple TAGs with commas(,)'),  # TODO: Support repetition of this argument
    ('strip-comments', 's',
     'strip the comment TAGs from the comments.'),
    ('input-paths=', None,
     'files or directories that should be scanned for messages. Separate multiple '
     'files or directories with commas(,)'),  # TODO: Support repetition of this argument
    ('input-dirs=', None,  # TODO (3.x): Remove me.
     'alias for input-paths (does allow files as well as directories).'),
    ('ignore-dirs=', None,
     'Patterns for directories to ignore when scanning for messages. '
     'Separate multiple patterns with spaces (default ".* ._")'),
    ('header-comment=', None,
     'header comment for the catalog'),
    ('last-translator=', None,
     'set the name and email of the last translator in output'),
]
boolean_options = [
    'no-default-keywords', 'no-location', 'omit-header', 'no-wrap',
    'sort-output', 'sort-by-file', 'strip-comments',
]
as_args = 'input-paths'
multiple_value_options = (
    'add-comments',
    'keywords',
    'ignore-dirs',
)
option_aliases = {
    'keywords': ('--keyword',),
    'mapping-file': ('--mapping',),
    'output-file': ('--output',),
    'strip-comments': ('--strip-comment-tags',),
    'last-translator': ('--last-translator',),
}
option_choices = {
    'add-location': ('full', 'file', 'never'),
}

def initialize_options(self):
    self.charset = 'utf-8'
    self.keywords = None
    self.no_default_keywords = False
    self.mapping_file = None
    self.no_location = False
    self.add_location = None
    self.omit_header = False
    self.output_file = None
    self.input_dirs = None
    self.input_paths = None
    self.width = None
    self.no_wrap = False
    self.sort_output = False
    self.sort_by_file = False
    self.msgid_bugs_address = None
    self.copyright_holder = None
    self.project = None
    self.version = None
    self.add_comments = None
```

```
        self.strip_comments = False
        self.include_lineno = True
        self.ignore_dirs = None
        self.header_comment = None
        self.last_translator = None

    def finalize_options(self):
        if self.input_dirs:
            if not self.input_paths:
                self.input_paths = self.input_dirs
            else:
                raise OptionError(
                    'input-dirs and input-paths are mutually exclusive',
                )

        keywords = {} if self.no_default_keywords else DEFAULT_KEYWORDS.copy()

        keywords.update(parse_keywords(listify_value(self.keywords)))

        self.keywords = keywords

        if not self.keywords:
            raise OptionError(
                'you must specify new keywords if you disable the default ones',
            )

        if not self.output_file:
            raise OptionError('no output file specified')
        if self.no_wrap and self.width:
            raise OptionError(
                "'--no-wrap' and '--width' are mutually exclusive",
            )
        if not self.no_wrap and not self.width:
            self.width = 76
        elif self.width is not None:
            self.width = int(self.width)

        if self.sort_output and self.sort_by_file:
            raise OptionError(
                "'--sort-output' and '--sort-by-file' are mutually exclusive",
            )

        if self.input_paths:
            if isinstance(self.input_paths, str):
                self.input_paths = re.split(r',\s*', self.input_paths)
        elif self.distribution is not None:
            self.input_paths = dict.fromkeys([
                k.split('.', 1)[0]
                for k in (self.distribution.packages or ())
            ]).keys()
        else:
            self.input_paths = []

        if not self.input_paths:
            raise OptionError("no input files or directories specified")

        for path in self.input_paths:
            if not os.path.exists(path):
                raise OptionError(f"Input path: {path} does not exist")

        self.add_comments = listify_value(self.add_comments or (), ",")

        if self.distribution:
            if not self.project:
                self.project = self.distribution.get_name()
            if not self.version:
                self.version = self.distribution.get_version()

        if self.add_location == 'never':
            self.no_location = True
        elif self.add_location == 'file':
            self.include_lineno = False

        ignore_dirs = listify_value(self.ignore_dirs)
        if ignore_dirs:
            self.directory_filter = _make_directory_filter(self.ignore_dirs)
        else:
            self.directory_filter = None

    def _build_callback(self, path: str):
        def callback(filename: str, method: str, options: dict):
```

```
        if method == 'ignore':
            return

        # If we explicitly provide a full filepath, just use that.
        # Otherwise, path will be the directory path and filename
        # is the relative path from that dir to the file.
        # So we can join those to get the full filepath.
        if os.path.isfile(path):
            filepath = path
        else:
            filepath = os.path.normpath(os.path.join(path, filename))

        optstr = ''
        if options:
            opt_values = ", ".join(f'{k}="{v}"' for k, v in options.items())
            optstr = f" ({opt_values})"
        self.log.info('extracting messages from %s%s', filepath, optstr)

    return callback

def run(self):
    mappings = self._get_mappings()
    with open(self.output_file, 'wb') as outfile:
        catalog = Catalog(project=self.project,
                          version=self.version,
                          msgid_bugs_address=self.msgid_bugs_address,
                          copyright_holder=self.copyright_holder,
                          charset=self.charset,
                          header_comment=(self.header_comment or DEFAULT_HEADER),
                          last_translator=self.last_translator)

        for path, method_map, options_map in mappings:
            callback = self._build_callback(path)
            if os.path.isfile(path):
                current_dir = os.getcwd()
                extracted = check_and_call_extract_file(
                    path, method_map, options_map,
                    callback, self.keywords, self.add_comments,
                    self.strip_comments, current_dir,
                )
            else:
                extracted = extract_from_dir(
                    path, method_map, options_map,
                    keywords=self.keywords,
                    comment_tags=self.add_comments,
                    callback=callback,
                    strip_comment_tags=self.strip_comments,
                    directory_filter=self.directory_filter,
                )
            for filename, lineno, message, comments, context in extracted:
                if os.path.isfile(path):
                    filepath = filename  # already normalized
                else:
                    filepath = os.path.normpath(os.path.join(path, filename))

                catalog.add(message, None, [(filepath, lineno)],
                            auto_comments=comments, context=context)

        self.log.info('writing PO template file to %s', self.output_file)
        write_po(outfile, catalog, width=self.width,
                 no_location=self.no_location,
                 omit_header=self.omit_header,
                 sort_output=self.sort_output,
                 sort_by_file=self.sort_by_file,
                 include_lineno=self.include_lineno)

def _get_mappings(self):
    mappings = []

    if self.mapping_file:
        with open(self.mapping_file) as fileobj:
            method_map, options_map = parse_mapping(fileobj)
        for path in self.input_paths:
            mappings.append((path, method_map, options_map))

    elif getattr(self.distribution, 'message_extractors', None):
        message_extractors = self.distribution.message_extractors
        for path, mapping in message_extractors.items():
            if isinstance(mapping, str):
                method_map, options_map = parse_mapping(StringIO(mapping))
            else:
```

```
                        method_map, options_map = [], {}
                        for pattern, method, options in mapping:
                            method_map.append((pattern, method))
                            options_map[pattern] = option
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\babel\messages\jslexer.py

```
"""
    babel.messages.jslexer
    ~~~~~~~~~~~~~~~~~~~~~~

    A simple JavaScript 1.5 lexer which is used for the JavaScript
    extractor.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

import re
from collections.abc import Generator
from typing import NamedTuple

operators: list[str] = sorted([
    '+', '-', '*', '%', '!=', '==', '<', '>', '<=', '>=', '=',
    '+=', '-=', '*=', '%=', '<<', '>>', '>>>', '<<=', '>>=',
    '>>>=', '&', '&=', '|', '|=', '&&', '||', '^', '^=', '(', ')',
    '[', ']', '{', '}', '!', '--', '++', '~', ',', ';', '.', ':',
], key=len, reverse=True)

escapes: dict[str, str] = {'b': '\b', 'f': '\f', 'n': '\n', 'r': '\r', 't': '\t'}

name_re = re.compile(r'[\w$_][\w\d$_]*', re.UNICODE)
dotted_name_re = re.compile(r'[\w$_][\w\d$_.]*[\w\d$_.]', re.UNICODE)
division_re = re.compile(r'/=?')
regex_re = re.compile(r'/(?:[^/\\]*(?:\\.[^/\\]*)*)/[a-zA-Z]*', re.DOTALL)
line_re = re.compile(r'(\r\n|\n|\r)')
line_join_re = re.compile(r'\\' + line_re.pattern)
uni_escape_re = re.compile(r'[a-fA-F0-9]{1,4}')
hex_escape_re = re.compile(r'[a-fA-F0-9]{1,2}')


class Token(NamedTuple):
    type: str
    value: str
    lineno: int


_rules: list[tuple[str | None, re.Pattern[str]]] = [
    (None, re.compile(r'\s+', re.UNICODE)),
    (None, re.compile(r'<!--.*')),
    ('linecomment', re.compile(r'//.*')),
    ('multilinecomment', re.compile(r'/\*.*?\*/', re.UNICODE | re.DOTALL)),
    ('dotted_name', dotted_name_re),
    ('name', name_re),
    ('number', re.compile(r'''(
        (?:0|[1-9]\d*)
        (\.\d+)?
        ([eE][-+]?\d+)? |
        (0x[a-fA-F0-9]+)
    )''', re.VERBOSE)),
    ('jsx_tag', re.compile(r'(?:</?[^>\s]+|/>)', re.I)),  # May be mangled in `get_rules`
    ('operator', re.compile(r'(%s)' % '|'.join(map(re.escape, operators)))),
    ('template_string', re.compile(r'''`(?:[^`\\]*(?:\\.[^`\\]*)*)`''', re.UNICODE)),
    ('string', re.compile(r'''(
        '(?:[^'\\]*(?:\\.[^'\\]*)*)'  |
        "(?:[^"\\]*(?:\\.[^"\\]*)*)"
    )''', re.VERBOSE | re.DOTALL)),
]


def get_rules(jsx: bool, dotted: bool, template_string: bool) -> list[tuple[str | None, re.Pattern[str]]]:
    """
    Get a tokenization rule list given the passed syntax options.

    Internal to this module.
    """
    rules = []
```

```
    for token_type, rule in _rules:
        if not jsx and token_type and 'jsx' in token_type:
            continue
        if not template_string and token_type == 'template_string':
            continue
        if token_type == 'dotted_name':
            if not dotted:
                continue
            token_type = 'name'
        rules.append((token_type, rule))
    return rules


def indicates_division(token: Token) -> bool:
    """A helper function that helps the tokenizer to decide if the current
    token may be followed by a division operator.
    """
    if token.type == 'operator':
        return token.value in (')', ']', '}', '++', '--')
    return token.type in ('name', 'number', 'string', 'regexp')


def unquote_string(string: str) -> str:
    """Unquote a string with JavaScript rules.  The string has to start with
    string delimiters (``'``, ``"`` or the back-tick/grave accent (for template strings).)
    """
    assert string and string[0] == string[-1] and string[0] in '"\'``', \
        'string provided is not properly delimited'
    string = line_join_re.sub('\\1', string[1:-1])
    result: list[str] = []
    add = result.append
    pos = 0

    while True:
        # scan for the next escape
        escape_pos = string.find('\\', pos)
        if escape_pos < 0:
            break
        add(string[pos:escape_pos])

        # check which character is escaped
        next_char = string[escape_pos + 1]
        if next_char in escapes:
            add(escapes[next_char])

        # unicode escapes.  trie to consume up to four characters of
        # hexadecimal characters and try to interpret them as unicode
        # character point.  If there is no such character point, put
        # all the consumed characters into the string.
        elif next_char in 'uU':
            escaped = uni_escape_re.match(string, escape_pos + 2)
            if escaped is not None:
                escaped_value = escaped.group()
                if len(escaped_value) == 4:
                    try:
                        add(chr(int(escaped_value, 16)))
                    except ValueError:
                        pass
                    else:
                        pos = escape_pos + 6
                        continue
                add(next_char + escaped_value)
                pos = escaped.end()
                continue
            else:
                add(next_char)

        # hex escapes. conversion from 2-digits hex to char is infallible
        elif next_char in 'xX':
            escaped = hex_escape_re.match(string, escape_pos + 2)
            if escaped is not None:
                escaped_value = escaped.group()
                add(chr(int(escaped_value, 16)))
                pos = escape_pos + 2 + len(escaped_value)
                continue
            else:
                add(next_char)

        # bogus escape.  Just remove the backslash.
        else:
            add(next_char)
```

```
        pos = escape_pos + 2

    if pos < len(string):
        add(string[pos:])

    return ''.join(result)


def tokenize(source: str, jsx: bool = True, dotted: bool = True, template_string: bool = True, lineno: int = 1) -> Gener
    """
    Tokenize JavaScript/JSX source.  Returns a generator of tokens.

    :param jsx: Enable (limited) JSX parsing.
    :param dotted: Read dotted names as single name token.
    :param template_string: Support ES6 template strings
    :param lineno: starting line number (optional)
    """
    may_divide = False
    pos = 0
    end = len(source)
    rules = get_rules(jsx=jsx, dotted=dotted, template_string=template_string)

    while pos < end:
        # handle regular rules first
        for token_type, rule in rules:  # noqa: B007
            match = rule.match(source, pos)
            if match is not None:
                break
        # if we don't have a match we don't give up yet, but check for
        # division operators or regular expression literals, based on
        # the status of `may_divide` which is determined by the last
        # processed non-whitespace token using `indicates_division`.
        else:
            if may_divide:
                match = division_re.match(source, pos)
                token_type = 'operator'
            else:
                match = regex_re.match(source, pos)
                token_type = 'regexp'
            if match is None:
                # woops. invalid syntax. jump one char ahead and try again.
                pos += 1
                continue

        token_value = match.group()
        if token_type is not None:
            token = Token(token_type, token_value, lineno)
            may_divide = indicates_division(token)
            yield token
        lineno += len(line_re.findall(token_value))
        pos = match.end()
```

File: .venv\Lib\site-packages\babel\messages\mofile.py

```
"""
    babel.messages.mofile
    ~~~~~~~~~~~~~~~~~~~~~

    Writing of files in the ``gettext`` MO (machine object) format.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

import array
import struct
from typing import TYPE_CHECKING

from babel.messages.catalog import Catalog, Message

if TYPE_CHECKING:
    from _typeshed import SupportsRead, SupportsWrite

LE_MAGIC: int = 0x950412de
BE_MAGIC: int = 0xde120495


def read_mo(fileobj: SupportsRead[bytes]) -> Catalog:
```

```
    """Read a binary MO file from the given file-like object and return a
    corresponding `Catalog` object.

    :param fileobj: the file-like object to read the MO file from

    :note: The implementation of this function is heavily based on the
           ``GNUTranslations._parse`` method of the ``gettext`` module in the
           standard library.
    """
    catalog = Catalog()
    headers = {}

    filename = getattr(fileobj, 'name', '')

    buf = fileobj.read()
    buflen = len(buf)
    unpack = struct.unpack

    # Parse the .mo file header, which consists of 5 little endian 32
    # bit words.
    magic = unpack('<I', buf[:4])[0]  # Are we big endian or little endian?
    if magic == LE_MAGIC:
        version, msgcount, origidx, transidx = unpack('<4I', buf[4:20])
        ii = '<II'
    elif magic == BE_MAGIC:
        version, msgcount, origidx, transidx = unpack('>4I', buf[4:20])
        ii = '>II'
    else:
        raise OSError(0, 'Bad magic number', filename)

    # Now put all messages from the .mo file buffer into the catalog
    # dictionary
    for _i in range(msgcount):
        mlen, moff = unpack(ii, buf[origidx:origidx + 8])
        mend = moff + mlen
        tlen, toff = unpack(ii, buf[transidx:transidx + 8])
        tend = toff + tlen
        if mend < buflen and tend < buflen:
            msg = buf[moff:mend]
            tmsg = buf[toff:tend]
        else:
            raise OSError(0, 'File is corrupt', filename)

        # See if we're looking at GNU .mo conventions for metadata
        if mlen == 0:
            # Catalog description
            lastkey = key = None
            for item in tmsg.splitlines():
                item = item.strip()
                if not item:
                    continue
                if b':' in item:
                    key, value = item.split(b':', 1)
                    lastkey = key = key.strip().lower()
                    headers[key] = value.strip()
                elif lastkey:
                    headers[lastkey] += b'\n' + item

        if b'\x04' in msg:  # context
            ctxt, msg = msg.split(b'\x04')
        else:
            ctxt = None

        if b'\x00' in msg:  # plural forms
            msg = msg.split(b'\x00')
            tmsg = tmsg.split(b'\x00')
            if catalog.charset:
                msg = [x.decode(catalog.charset) for x in msg]
                tmsg = [x.decode(catalog.charset) for x in tmsg]
        else:
            if catalog.charset:
                msg = msg.decode(catalog.charset)
                tmsg = tmsg.decode(catalog.charset)
        catalog[msg] = Message(msg, tmsg, context=ctxt)

        # advance to next entry in the seek tables
        origidx += 8
        transidx += 8

    catalog.mime_headers = headers.items()
    return catalog
```

```
def write_mo(fileobj: SupportsWrite[bytes], catalog: Catalog, use_fuzzy: bool = False) -> None:
    """Write a catalog to the specified file-like object using the GNU MO file
    format.

    >>> import sys
    >>> from babel.messages import Catalog
    >>> from gettext import GNUTranslations
    >>> from io import BytesIO

    >>> catalog = Catalog(locale='en_US')
    >>> catalog.add('foo', 'Voh')
    <Message ...>
    >>> catalog.add((u'bar', u'baz'), (u'Bahr', u'Batz'))
    <Message ...>
    >>> catalog.add('fuz', 'Futz', flags=['fuzzy'])
    <Message ...>
    >>> catalog.add('Fizz', '')
    <Message ...>
    >>> catalog.add(('Fuzz', 'Fuzzes'), ('', ''))
    <Message ...>
    >>> buf = BytesIO()

    >>> write_mo(buf, catalog)
    >>> x = buf.seek(0)
    >>> translations = GNUTranslations(fp=buf)
    >>> if sys.version_info[0] >= 3:
    ...     translations.ugettext = translations.gettext
    ...     translations.ungettext = translations.ngettext
    >>> translations.ugettext('foo')
    u'Voh'
    >>> translations.ungettext('bar', 'baz', 1)
    u'Bahr'
    >>> translations.ungettext('bar', 'baz', 2)
    u'Batz'
    >>> translations.ugettext('fuz')
    u'fuz'
    >>> translations.ugettext('Fizz')
    u'Fizz'
    >>> translations.ugettext('Fuzz')
    u'Fuzz'
    >>> translations.ugettext('Fuzzes')
    u'Fuzzes'

    :param fileobj: the file-like object to write to
    :param catalog: the `Catalog` instance
    :param use_fuzzy: whether translations marked as "fuzzy" should be included
                      in the output
    """
    messages = list(catalog)
    messages[1:] = [m for m in messages[1:]
                    if m.string and (use_fuzzy or not m.fuzzy)]
    messages.sort()

    ids = strs = b''
    offsets = []

    for message in messages:
        # For each string, we need size and file offset.  Each string is NUL
        # terminated; the NUL does not count into the size.
        if message.pluralizable:
            msgid = b'\x00'.join([
                msgid.encode(catalog.charset) for msgid in message.id
            ])
            msgstrs = []
            for idx, string in enumerate(message.string):
                if not string:
                    msgstrs.append(message.id[min(int(idx), 1)])
                else:
                    msgstrs.append(string)
            msgstr = b'\x00'.join([
                msgstr.encode(catalog.charset) for msgstr in msgstrs
            ])
        else:
            msgid = message.id.encode(catalog.charset)
            msgstr = message.string.encode(catalog.charset)
        if message.context:
            msgid = b'\x04'.join([message.context.encode(catalog.charset),
                                  msgid])
        offsets.append((len(ids), len(msgid), len(strs), len(msgstr)))
        ids += msgid + b'\x00'
        strs += msgstr + b'\x00'
```

```
    # The header is 7 32-bit unsigned integers.  We don't use hash tables, so
    # the keys start right after the index tables.
    keystart = 7 * 4 + 16 * len(messages)
    valuestart = keystart + len(ids)

    # The string table first has the list of keys, then the list of values.
    # Each entry has first the size of the string, then the file offset.
    koffsets = []
    voffsets = []
    for o1, l1, o2, l2 in offsets:
        koffsets += [l1, o1 + keystart]
        voffsets += [l2, o2 + valuestart]
    offsets = koffsets + voffsets

    fileobj.write(struct.pack('Iiiiiii',
                              LE_MAGIC,                   # magic
                              0,                          # version
                              len(messages),              # number of entries
                              7 * 4,                      # start of key index
                              7 * 4 + len(messages) * 8,  # start of value index
                              0, 0,                       # size and offset of hash table
                              ) + array.array.tobytes(array.array("i", offsets)) + ids + strs)
```

File: .venv\Lib\site-packages\babel\messages\plurals.py

```
"""
    babel.messages.plurals
    ~~~~~~~~~~~~~~~~~~~~~~

    Plural form definitions.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

from operator import itemgetter

from babel.core import Locale, default_locale

# XXX: remove this file, duplication with babel.plural


LC_CTYPE: str | None = default_locale('LC_CTYPE')


PLURALS: dict[str, tuple[int, str]] = {
    # Afar
    # 'aa': (),
    # Abkhazian
    # 'ab': (),
    # Avestan
    # 'ae': (),
    # Afrikaans - From Pootle's PO's
    'af': (2, '(n != 1)'),
    # Akan
    # 'ak': (),
    # Amharic
    # 'am': (),
    # Aragonese
    # 'an': (),
    # Arabic - From Pootle's PO's
    'ar': (6, '(n==0 ? 0 : n==1 ? 1 : n==2 ? 2 : n%100>=3 && n%100<=10 ? 3 : n%100>=0 && n%100<=2 ? 4 : 5)'),
    # Assamese
    # 'as': (),
    # Avaric
    # 'av': (),
    # Aymara
    # 'ay': (),
    # Azerbaijani
    # 'az': (),
    # Bashkir
    # 'ba': (),
    # Belarusian
    'be': (3, '(n%10==1 && n%100!=11 ? 0 : n%10>=2 && n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2)'),
    # Bulgarian - From Pootle's PO's
    'bg': (2, '(n != 1)'),
    # Bihari
    # 'bh': (),
```

```
# Bislama
# 'bi': (),
# Bambara
# 'bm': (),
# Bengali - From Pootle's PO's
'bn': (2, '(n != 1)'),
# Tibetan - as discussed in private with Andrew West
'bo': (1, '0'),
# Breton
'br': (
    6,
    '(n==1 ? 0 : n%10==1 && n%100!=11 && n%100!=71 && n%100!=91 ? 1 : n%10==2 && n%100!=12 && n%100!=72 && '
    'n%100!=92 ? 2 : (n%10==3 || n%10==4 || n%10==9) && n%100!=13 && n%100!=14 && n%100!=19 && n%100!=73 && '
    'n%100!=74 && n%100!=79 && n%100!=93 && n%100!=94 && n%100!=99 ? 3 : n%1000000==0 ? 4 : 5)',
),
# Bosnian
'bs': (3, '(n%10==1 && n%100!=11 ? 0 : n%10>=2 && n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2)'),
# Catalan - From Pootle's PO's
'ca': (2, '(n != 1)'),
# Chechen
# 'ce': (),
# Chamorro
# 'ch': (),
# Corsican
# 'co': (),
# Cree
# 'cr': (),
# Czech
'cs': (3, '((n==1) ? 0 : (n>=2 && n<=4) ? 1 : 2)'),
# Church Slavic
# 'cu': (),
# Chuvash
'cv': (1, '0'),
# Welsh
'cy': (5, '(n==1 ? 1 : n==2 ? 2 : n==3 ? 3 : n==6 ? 4 : 0)'),
# Danish
'da': (2, '(n != 1)'),
# German
'de': (2, '(n != 1)'),
# Divehi
# 'dv': (),
# Dzongkha
'dz': (1, '0'),
# Greek
'el': (2, '(n != 1)'),
# English
'en': (2, '(n != 1)'),
# Esperanto
'eo': (2, '(n != 1)'),
# Spanish
'es': (2, '(n != 1)'),
# Estonian
'et': (2, '(n != 1)'),
# Basque - From Pootle's PO's
'eu': (2, '(n != 1)'),
# Persian - From Pootle's PO's
'fa': (1, '0'),
# Finnish
'fi': (2, '(n != 1)'),
# French
'fr': (2, '(n > 1)'),
# Friulian - From Pootle's PO's
'fur': (2, '(n > 1)'),
# Irish
'ga': (5, '(n==1 ? 0 : n==2 ? 1 : n>=3 && n<=6 ? 2 : n>=7 && n<=10 ? 3 : 4)'),
# Galician - From Pootle's PO's
'gl': (2, '(n != 1)'),
# Hausa - From Pootle's PO's
'ha': (2, '(n != 1)'),
# Hebrew
'he': (2, '(n != 1)'),
# Hindi - From Pootle's PO's
'hi': (2, '(n != 1)'),
# Croatian
'hr': (3, '(n%10==1 && n%100!=11 ? 0 : n%10>=2 && n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2)'),
# Hungarian
'hu': (1, '0'),
# Armenian - From Pootle's PO's
'hy': (1, '0'),
# Icelandic - From Pootle's PO's
'is': (2, '(n%10==1 && n%100!=11 ? 0 : 1)'),
```

```
    # Italian
    'it': (2, '(n != 1)'),
    # Japanese
    'ja': (1, '0'),
    # Georgian - From Pootle's PO's
    'ka': (1, '0'),
    # Kongo - From Pootle's PO's
    'kg': (2, '(n != 1)'),
    # Khmer - From Pootle's PO's
    'km': (1, '0'),
    # Korean
    'ko': (1, '0'),
    # Kurdish - From Pootle's PO's
    'ku': (2, '(n != 1)'),
    # Lao - Another member of the Tai language family, like Thai.
    'lo': (1, '0'),
    # Lithuanian
    'lt': (3, '(n%10==1 && n%100!=11 ? 0 : n%10>=2 && (n%100<10 || n%100>=20) ? 1 : 2)'),
    # Latvian
    'lv': (3, '(n%10==1 && n%100!=11 ? 0 : n != 0 ? 1 : 2)'),
    # Maltese - From Pootle's PO's
    'mt': (4, '(n==1 ? 0 : n==0 || ( n%100>=1 && n%100<=10) ? 1 : (n%100>10 && n%100<20 ) ? 2 : 3)'),
    # Norwegian Bokmål
    'nb': (2, '(n != 1)'),
    # Dutch
    'nl': (2, '(n != 1)'),
    # Norwegian Nynorsk
    'nn': (2, '(n != 1)'),
    # Norwegian
    'no': (2, '(n != 1)'),
    # Punjabi - From Pootle's PO's
    'pa': (2, '(n != 1)'),
    # Polish
    'pl': (3, '(n==1 ? 0 : n%10>=2 && n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2)'),
    # Portuguese
    'pt': (2, '(n != 1)'),
    # Brazilian
    'pt_BR': (2, '(n > 1)'),
    # Romanian - From Pootle's PO's
    'ro': (3, '(n==1 ? 0 : (n==0 || (n%100 > 0 && n%100 < 20)) ? 1 : 2)'),
    # Russian
    'ru': (3, '(n%10==1 && n%100!=11 ? 0 : n%10>=2 && n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2)'),
    # Slovak
    'sk': (3, '((n==1) ? 0 : (n>=2 && n<=4) ? 1 : 2)'),
    # Slovenian
    'sl': (4, '(n%100==1 ? 0 : n%100==2 ? 1 : n%100==3 || n%100==4 ? 2 : 3)'),
    # Serbian - From Pootle's PO's
    'sr': (3, '(n%10==1 && n%100!=11 ? 0 : n%10>=2 && n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2)'),
    # Southern Sotho - From Pootle's PO's
    'st': (2, '(n != 1)'),
    # Swedish
    'sv': (2, '(n != 1)'),
    # Thai
    'th': (1, '0'),
    # Turkish
    'tr': (1, '0'),
    # Ukrainian
    'uk': (3, '(n%10==1 && n%100!=11 ? 0 : n%10>=2 && n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2)'),
    # Venda - From Pootle's PO's
    've': (2, '(n != 1)'),
    # Vietnamese - From Pootle's PO's
    'vi': (1, '0'),
    # Xhosa - From Pootle's PO's
    'xh': (2, '(n != 1)'),
    # Chinese - From Pootle's PO's (modified)
    'zh': (1, '0'),
}


DEFAULT_PLURAL: tuple[int, str] = (2, '(n != 1)')


class _PluralTuple(tuple):
    """A tuple with plural information."""

    __slots__ = ()
    num_plurals = property(itemgetter(0), doc="""
    The number of plurals used by the locale.""")
    plural_expr = property(itemgetter(1), doc="""
    The plural expression used by the locale.""")
    plural_forms = property(lambda x: 'nplurals={}; plural={};'.format(*x), doc="""
```

```
    The plural expression used by the catalog or locale.""")

    def __str__(self) -> str:
        return self.plural_forms


def get_plural(locale: str | None = LC_CTYPE) -> _PluralTuple:
    """A tuple with the information catalogs need to perform proper
    pluralization.  The first item of the tuple is the number of plural
    forms, the second the plural expression.

    >>> get_plural(locale='en')
    (2, '(n != 1)')
    >>> get_plural(locale='ga')
    (5, '(n==1 ? 0 : n==2 ? 1 : n>=3 && n<=6 ? 2 : n>=7 && n<=10 ? 3 : 4)')

    The object returned is a special tuple with additional members:

    >>> tup = get_plural("ja")
    >>> tup.num_plurals
    1
    >>> tup.plural_expr
    '0'
    >>> tup.plural_forms
    'nplurals=1; plural=0;'

    Converting the tuple into a string prints the plural forms for a
    gettext catalog:

    >>> str(tup)
    'nplurals=1; plural=0;'
    """
    locale = Locale.parse(locale)
    try:
        tup = PLURALS[str(locale)]
    except KeyError:
        try:
            tup = PLURALS[locale.language]
        except KeyError:
            tup = DEFAULT_PLURAL
    return _PluralTuple(tup)
```

File: .venv\Lib\site-packages\babel\messages\pofile.py

```
"""
    babel.messages.pofile
    ~~~~~~~~~~~~~~~~~~~~~

    Reading and writing of files in the ``gettext`` PO (portable object)
    format.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""
from __future__ import annotations

import os
import re
from collections.abc import Iterable
from typing import TYPE_CHECKING

from babel.core import Locale
from babel.messages.catalog import Catalog, Message
from babel.util import _cmp, wraptext

if TYPE_CHECKING:
    from typing import IO, AnyStr

    from _typeshed import SupportsWrite
    from typing_extensions import Literal


def unescape(string: str) -> str:
    r"""Reverse `escape` the given string.

    >>> print(unescape('"Say:\\n  \\"hello, world!\\"\\n"'))
    Say:
      "hello, world!"
    <BLANKLINE>
```

```
    :param string: the string to unescape
    """
    def replace_escapes(match):
        m = match.group(1)
        if m == 'n':
            return '\n'
        elif m == 't':
            return '\t'
        elif m == 'r':
            return '\r'
        # m is \ or "
        return m
    return re.compile(r'\\([\\trn"])').sub(replace_escapes, string[1:-1])


def denormalize(string: str) -> str:
    r"""Reverse the normalization done by the `normalize` function.

    >>> print(denormalize(r'''"""
    ... "Say:\n"
    ... "  \"hello, world!\"\n"'''))
    Say:
      "hello, world!"
    <BLANKLINE>

    >>> print(denormalize(r'''"""
    ... "Say:\n"
    ... "  \"Lorem ipsum dolor sit "
    ... "amet, consectetur adipisicing"
    ... " elit, \"\n"'''))
    Say:
      "Lorem ipsum dolor sit amet, consectetur adipisicing elit, "
    <BLANKLINE>

    :param string: the string to denormalize
    """
    if '\n' in string:
        escaped_lines = string.splitlines()
        if string.startswith('"""'):
            escaped_lines = escaped_lines[1:]
        lines = map(unescape, escaped_lines)
        return ''.join(lines)
    else:
        return unescape(string)


class PoFileError(Exception):
    """Exception thrown by PoParser when an invalid po file is encountered."""

    def __init__(self, message: str, catalog: Catalog, line: str, lineno: int) -> None:
        super().__init__(f'{message} on {lineno}')
        self.catalog = catalog
        self.line = line
        self.lineno = lineno


class _NormalizedString:

    def __init__(self, *args: str) -> None:
        self._strs: list[str] = []
        for arg in args:
            self.append(arg)

    def append(self, s: str) -> None:
        self._strs.append(s.strip())

    def denormalize(self) -> str:
        return ''.join(map(unescape, self._strs))

    def __bool__(self) -> bool:
        return bool(self._strs)

    def __repr__(self) -> str:
        return os.linesep.join(self._strs)

    def __cmp__(self, other: object) -> int:
        if not other:
            return 1

        return _cmp(str(self), str(other))
    def __gt__(self, other: object) -> bool:
```

```
        return self.__cmp__(other) > 0

    def __lt__(self, other: object) -> bool:
        return self.__cmp__(other) < 0

    def __ge__(self, other: object) -> bool:
        return self.__cmp__(other) >= 0

    def __le__(self, other: object) -> bool:
        return self.__cmp__(other) <= 0

    def __eq__(self, other: object) -> bool:
        return self.__cmp__(other) == 0

    def __ne__(self, other: object) -> bool:
        return self.__cmp__(other) != 0


class PoFileParser:
    """Support class to  read messages from a ``gettext`` PO (portable object) file
    and add them to a `Catalog`

    See `read_po` for simple cases.
    """

    _keywords = [
        'msgid',
        'msgstr',
        'msgctxt',
        'msgid_plural',
    ]

    def __init__(self, catalog: Catalog, ignore_obsolete: bool = False, abort_invalid: bool = False) -> None:
        self.catalog = catalog
        self.ignore_obsolete = ignore_obsolete
        self.counter = 0
        self.offset = 0
        self.abort_invalid = abort_invalid
        self._reset_message_state()

    def _reset_message_state(self) -> None:
        self.messages = []
        self.translations = []
        self.locations = []
        self.flags = []
        self.user_comments = []
        self.auto_comments = []
        self.context = None
        self.obsolete = False
        self.in_msgid = False
        self.in_msgstr = False
        self.in_msgctxt = False

    def _add_message(self) -> None:
        """
        Add a message to the catalog based on the current parser state and
        clear the state ready to process the next message.
        """
        self.translations.sort()
        if len(self.messages) > 1:
            msgid = tuple(m.denormalize() for m in self.messages)
        else:
            msgid = self.messages[0].denormalize()
        if isinstance(msgid, (list, tuple)):
            string = ['' for _ in range(self.catalog.num_plurals)]
            for idx, translation in self.translations:
                if idx >= self.catalog.num_plurals:
                    self._invalid_pofile("", self.offset, "msg has more translations than num_plurals of catalog")
                    continue
                string[idx] = translation.denormalize()
            string = tuple(string)
        else:
            string = self.translations[0][1].denormalize()
        msgctxt = self.context.denormalize() if self.context else None
        message = Message(msgid, string, list(self.locations), set(self.flags),
                          self.auto_comments, self.user_comments, lineno=self.offset + 1,
                          context=msgctxt)
        if self.obsolete:
            if not self.ignore_obsolete:
                self.catalog.obsolete[msgid] = message
        else:
```

```
            self.catalog[msgid] = message
        self.counter += 1
        self._reset_message_state()

    def _finish_current_message(self) -> None:
        if self.messages:
            self._add_message()

    def _process_message_line(self, lineno, line, obsolete=False) -> None:
        if line.startswith('""'):
            self._process_string_continuation_line(line, lineno)
        else:
            self._process_keyword_line(lineno, line, obsolete)

    def _process_keyword_line(self, lineno, line, obsolete=False) -> None:

        for keyword in self._keywords:
            try:
                if line.startswith(keyword) and line[len(keyword)] in [' ', '[']:
                    arg = line[len(keyword):]
                    break
            except IndexError:
                self._invalid_pofile(line, lineno, "Keyword must be followed by a string")
        else:
            self._invalid_pofile(line, lineno, "Start of line didn't match any expected keyword.")
            return

        if keyword in ['msgid', 'msgctxt']:
            self._finish_current_message()

        self.obsolete = obsolete

        # The line that has the msgid is stored as the offset of the msg
        # should this be the msgctxt if it has one?
        if keyword == 'msgid':
            self.offset = lineno

        if keyword in ['msgid', 'msgid_plural']:
            self.in_msgctxt = False
            self.in_msgid = True
            self.messages.append(_NormalizedString(arg))

        elif keyword == 'msgstr':
            self.in_msgid = False
            self.in_msgstr = True
            if arg.startswith('['):
                idx, msg = arg[1:].split(']', 1)
                self.translations.append([int(idx), _NormalizedString(msg)])
            else:
                self.translations.append([0, _NormalizedString(arg)])

        elif keyword == 'msgctxt':
            self.in_msgctxt = True
            self.context = _NormalizedString(arg)

    def _process_string_continuation_line(self, line, lineno) -> None:
        if self.in_msgid:
            s = self.messages[-1]
        elif self.in_msgstr:
            s = self.translations[-1][1]
        elif self.in_msgctxt:
            s = self.context
        else:
            self._invalid_pofile(line, lineno, "Got line starting with \" but not in msgid, msgstr or msgctxt")
            return
        s.append(line)

    def _process_comment(self, line) -> None:

        self._finish_current_message()

        if line[1:].startswith(':'):
            for location in line[2:].lstrip().split():
                pos = location.rfind(':')
                if pos >= 0:
                    try:
                        lineno = int(location[pos + 1:])
                    except ValueError:
                        continue
                    self.locations.append((location[:pos], lineno))
                else:
```

```
                    self.locations.append((location, None))
        elif line[1:].startswith(','):
            for flag in line[2:].lstrip().split(','):
                self.flags.append(flag.strip())
        elif line[1:].startswith('.'):
            # These are called auto-comments
            comment = line[2:].strip()
            if comment:  # Just check that we're not adding empty comments
                self.auto_comments.append(comment)
        else:
            # These are called user comments
            self.user_comments.append(line[1:].strip())

    def parse(self, fileobj: IO[AnyStr]) -> None:
        """
        Reads from the file-like object `fileobj` and adds any po file
        units found in it to the `Catalog` supplied to the constructor.
        """

        for lineno, line in enumerate(fileobj):
            line = line.strip()
            if not isinstance(line, str):
                line = line.decode(self.catalog.charset)
            if not line:
                continue
            if line.startswith('#'):
                if line[1:].startswith('~'):
                    self._process_message_line(lineno, line[2:].lstrip(), obsolete=True)
                else:
                    self._process_comment(line)
            else:
                self._process_message_line(lineno, line)

        self._finish_current_message()

        # No actual messages found, but there was some info in comments, from which
        # we'll construct an empty header message
        if not self.counter and (self.flags or self.user_comments or self.auto_comments):
            self.messages.append(_NormalizedString('"'))
            self.translations.append([0, _NormalizedString('"')])
            self._add_message()

    def _invalid_pofile(self, line, lineno, msg) -> None:
        assert isinstance(line, str)
        if self.abort_invalid:
            raise PoFileError(msg, self.catalog, line, lineno)
        print("WARNING:", msg)
        print(f"WARNING: Problem on line {lineno + 1}: {line!r}")


def read_po(
    fileobj: IO[AnyStr],
    locale: str | Locale | None = None,
    domain: str | None = None,
    ignore_obsolete: bool = False,
    charset: str | None = None,
    abort_invalid: bool = False,
) -> Catalog:
    """Read messages from a ``gettext`` PO (portable object) file from the given
    file-like object and return a `Catalog`.

    >>> from datetime import datetime
    >>> from io import StringIO
    >>> buf = StringIO('''
    ... #: main.py:1
    ... #, fuzzy, python-format
    ... msgid "foo %(name)s"
    ... msgstr "quux %(name)s"
    ...
    ... # A user comment
    ... #. An auto comment
    ... #: main.py:3
    ... msgid "bar"
    ... msgid_plural "baz"
    ... msgstr[0] "bar"
    ... msgstr[1] "baaz"
    ... ''')
    >>> catalog = read_po(buf)
    >>> catalog.revision_date = datetime(2007, 4, 1)

    >>> for message in catalog:
```

```
    ...     if message.id:
    ...         print((message.id, message.string))
    ...         print(' ', (message.locations, sorted(list(message.flags))))
    ...         print(' ', (message.user_comments, message.auto_comments))
    (u'foo %(name)s', u'quux %(name)s')
      ([(u'main.py', 1)], [u'fuzzy', u'python-format'])
      ([], [])
    ((u'bar', u'baz'), (u'bar', u'baaz'))
      ([(u'main.py', 3)], [])
      ([u'A user comment'], [u'An auto comment'])

    .. versionadded:: 1.0
       Added support for explicit charset argument.

    :param fileobj: the file-like object to read the PO file from
    :param locale: the locale identifier or `Locale` object, or `None`
                   if the catalog is not bound to a locale (which basically
                   means it's a template)
    :param domain: the message domain
    :param ignore_obsolete: whether to ignore obsolete messages in the input
    :param charset: the character set of the catalog.
    :param abort_invalid: abort read if po file is invalid
    """
    catalog = Catalog(locale=locale, domain=domain, charset=charset)
    parser = PoFileParser(catalog, ignore_obsolete, abort_invalid=abort_invalid)
    parser.parse(fileobj)
    return catalog


WORD_SEP = re.compile('('
                      r'\s+|'                                 # any whitespace
                      r'[^\s\w]*\w+[a-zA-Z]-(?=\w+[a-zA-Z])|'  # hyphenated words
                      r'(?<=[\w\!\"\'\&\.\,\?])-{2,}(?=\w)'   # em-dash
                      ')')


def escape(string: str) -> str:
    r"""Escape the given string so that it can be included in double-quoted
    strings in ``PO`` files.

    >>> escape('''Say:
    ...   "hello, world!"
    ... ''')
    '"Say:\\n  \\"hello, world!\\"\\n"'

    :param string: the string to escape
    """
    return '"%s"' % string.replace('\\', '\\\\') \
                          .replace('\t', '\\t') \
                          .replace('\r', '\\r') \
                          .replace('\n', '\\n') \
                          .replace('\"', '\\"')


def normalize(string: str, prefix: str = '', width: int = 76) -> str:
    r"""Convert a string into a format that is appropriate for .po files.

    >>> print(normalize('''Say:
    ...   "hello, world!"
    ... ''', width=None))
    ""
    "Say:\n"
    "  \"hello, world!\"\n"

    >>> print(normalize('''Say:
    ...   "Lorem ipsum dolor sit amet, consectetur adipisicing elit, "
    ... ''', width=32))
    ""
    "Say:\n"
    "  \"Lorem ipsum dolor sit "
    "amet, consectetur adipisicing"
    " elit, \"\n"

    :param string: the string to normalize
    :param prefix: a string that should be prepended to every line
    :param width: the maximum line width; use `None`, 0, or a negative number
                  to completely disable line wrapping
    """
    if width and width > 0:
        prefixlen = len(prefix)
        lines = []
```

```
        for line in string.splitlines(True):
            if len(escape(line)) + prefixlen > width:
                chunks = WORD_SEP.split(line)
                chunks.reverse()
                while chunks:
                    buf = []
                    size = 2
                    while chunks:
                        length = len(escape(chunks[-1])) - 2 + prefixlen
                        if size + length < width:
                            buf.append(chunks.pop())
                            size += length
                        else:
                            if not buf:
                                # handle long chunks by putting them on a
                                # separate line
                                buf.append(chunks.pop())
                            break
                    lines.append(''.join(buf))
            else:
                lines.append(line)
    else:
        lines = string.splitlines(True)

    if len(lines) <= 1:
        return escape(string)

    # Remove empty trailing line
    if lines and not lines[-1]:
        del lines[-1]
        lines[-1] += '\n'
    return '""\n' + '\n'.join([(prefix + escape(line)) for line in lines])


def write_po(
    fileobj: SupportsWrite[bytes],
    catalog: Catalog,
    width: int = 76,
    no_location: bool = False,
    omit_header: bool = False,
    sort_output: bool = False,
    sort_by_file: bool = False,
    ignore_obsolete: bool = False,
    include_previous: bool = False,
    include_lineno: bool = True,
) -> None:
    r"""Write a ``gettext`` PO (portable object) template file for a given
    message catalog to the provided file-like object.

    >>> catalog = Catalog()
    >>> catalog.add(u'foo %(name)s', locations=[('main.py', 1)],
    ...             flags=('fuzzy',))
    <Message...>
    >>> catalog.add((u'bar', u'baz'), locations=[('main.py', 3)])
    <Message...>
    >>> from io import BytesIO
    >>> buf = BytesIO()
    >>> write_po(buf, catalog, omit_header=True)
    >>> print(buf.getvalue().decode("utf8"))
    #: main.py:1
    #, fuzzy, python-format
    msgid "foo %(name)s"
    msgstr ""
    <BLANKLINE>
    #: main.py:3
    msgid "bar"
    msgid_plural "baz"
    msgstr[0] ""
    msgstr[1] ""
    <BLANKLINE>
    <BLANKLINE>

    :param fileobj: the file-like object to write to
    :param catalog: the `Catalog` instance
    :param width: the maximum line width for the generated output; use `None`,
                  0, or a negative number to completely disable line wrapping
    :param no_location: do not emit a location comment for every message
    :param omit_header: do not include the ``msgid ""`` entry at the top of the
                        output
    :param sort_output: whether to sort the messages in the output by msgid
    :param sort_by_file: whether to sort the messages in the output by their
```

```
                        locations
    :param ignore_obsolete: whether to ignore obsolete messages and not include
                            them in the output; by default they are included as
                            comments
    :param include_previous: include the old msgid as a comment when
                             updating the catalog
    :param include_lineno: include line number in the location comment
    """
    def _normalize(key, prefix=''):
        return normalize(key, prefix=prefix, width=width)

    def _write(text):
        if isinstance(text, str):
            text = text.encode(catalog.charset, 'backslashreplace')
        fileobj.write(text)

    def _write_comment(comment, prefix=''):
        # xgettext always wraps comments even if --no-wrap is passed;
        # provide the same behaviour
        _width = width if width and width > 0 else 76
        for line in wraptext(comment, _width):
            _write(f"#{prefix} {line.strip()}\n")

    def _write_message(message, prefix=''):
        if isinstance(message.id, (list, tuple)):
            if message.context:
                _write(f"{prefix}msgctxt {_normalize(message.context, prefix)}\n")
            _write(f"{prefix}msgid {_normalize(message.id[0], prefix)}\n")
            _write(f"{prefix}msgid_plural {_normalize(message.id[1], prefix)}\n")

            for idx in range(catalog.num_plurals):
                try:
                    string = message.string[idx]
                except IndexError:
                    string = ''
                _write(f"{prefix}msgstr[{idx:d}] {_normalize(string, prefix)}\n")
        else:
            if message.context:
                _write(f"{prefix}msgctxt {_normalize(message.context, prefix)}\n")
            _write(f"{prefix}msgid {_normalize(message.id, prefix)}\n")
            _write(f"{prefix}msgstr {_normalize(message.string or '', prefix)}\n")

    sort_by = None
    if sort_output:
        sort_by = "message"
    elif sort_by_file:
        sort_by = "location"

    for message in _sort_messages(catalog, sort_by=sort_by):
        if not message.id:  # This is the header "message"
            if omit_header:
                continue
            comment_header = catalog.header_comment
            if width and width > 0:
                lines = []
                for line in comment_header.splitlines():
                    lines += wraptext(line, width=width,
                                      subsequent_indent='# ')
                comment_header = '\n'.join(lines)
            _write(f"{comment_header}\n")

        for comment in message.user_comments:
            _write_comment(comment)
        for comment in message.auto_commen
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\babel\messages\setuptools_frontend.py

```
from __future__ import annotations

from babel.messages import frontend

try:
    # See: https://setuptools.pypa.io/en/latest/deprecated/distutils-legacy.html
    from setuptools import Command

    try:
        from setuptools.errors import BaseError, OptionError, SetupError
    except ImportError:  # Error aliases only added in setuptools 59 (2021-11).
```

```
        OptionError = SetupError = BaseError = Exception

except ImportError:
    from distutils.cmd import Command
    from distutils.errors import DistutilsSetupError as SetupError


def check_message_extractors(dist, name, value):
    """Validate the ``message_extractors`` keyword argument to ``setup()``.

    :param dist: the distutils/setuptools ``Distribution`` object
    :param name: the name of the keyword argument (should always be
                 "message_extractors")
    :param value: the value of the keyword argument
    :raise `DistutilsSetupError`: if the value is not valid
    """
    assert name == "message_extractors"
    if not isinstance(value, dict):
        raise SetupError(
            'the value of the "message_extractors" parameter must be a dictionary',
        )


class compile_catalog(frontend.CompileCatalog, Command):
    """Catalog compilation command for use in ``setup.py`` scripts.

    If correctly installed, this command is available to Setuptools-using
    setup scripts automatically. For projects using plain old ``distutils``,
    the command needs to be registered explicitly in ``setup.py``::

        from babel.messages.setuptools_frontend import compile_catalog

        setup(
            ...
            cmdclass = {'compile_catalog': compile_catalog}
        )

    .. versionadded:: 0.9
    """


class extract_messages(frontend.ExtractMessages, Command):
    """Message extraction command for use in ``setup.py`` scripts.

    If correctly installed, this command is available to Setuptools-using
    setup scripts automatically. For projects using plain old ``distutils``,
    the command needs to be registered explicitly in ``setup.py``::

        from babel.messages.setuptools_frontend import extract_messages

        setup(
            ...
            cmdclass = {'extract_messages': extract_messages}
        )
    """


class init_catalog(frontend.InitCatalog, Command):
    """New catalog initialization command for use in ``setup.py`` scripts.

    If correctly installed, this command is available to Setuptools-using
    setup scripts automatically. For projects using plain old ``distutils``,
    the command needs to be registered explicitly in ``setup.py``::

        from babel.messages.setuptools_frontend import init_catalog

        setup(
            ...
            cmdclass = {'init_catalog': init_catalog}
        )
    """


class update_catalog(frontend.UpdateCatalog, Command):
    """Catalog merging command for use in ``setup.py`` scripts.

    If correctly installed, this command is available to Setuptools-using
    setup scripts automatically. For projects using plain old ``distutils``,
    the command needs to be registered explicitly in ``setup.py``::

        from babel.messages.setuptools_frontend import update_catalog
```

```
        setup(
            ...
            cmdclass = {'update_catalog': update_catalog}
        )

    .. versionadded:: 0.9
    """


COMMANDS = {
    "compile_catalog": compile_catalog,
    "extract_messages": extract_messages,
    "init_catalog": init_catalog,
    "update_catalog": update_catalog,
}
```

File: .venv\Lib\site-packages\babel\messages__init__.py

```
"""
    babel.messages
    ~~~~~~~~~~~~~~

    Support for ``gettext`` message catalogs.

    :copyright: (c) 2013-2023 by the Babel Team.
    :license: BSD, see LICENSE for more details.
"""

from babel.messages.catalog import (
    Catalog,
    Message,
    TranslationError,
)

__all__ = [
    "Catalog",
    "Message",
    "TranslationError",
]
```

File: .venv\Lib\site-packages\Babel-2.14.0.dist-info\entry_points.txt

```
[babel.checkers]
num_plurals = babel.messages.checkers:num_plurals
python_format = babel.messages.checkers:python_format

[babel.extractors]
ignore = babel.messages.extract:extract_nothing
javascript = babel.messages.extract:extract_javascript
python = babel.messages.extract:extract_python

[console_scripts]
pybabel = babel.messages.frontend:main

[distutils.commands]
compile_catalog = babel.messages.setuptools_frontend:compile_catalog
extract_messages = babel.messages.setuptools_frontend:extract_messages
init_catalog = babel.messages.setuptools_frontend:init_catalog
update_catalog = babel.messages.setuptools_frontend:update_catalog

[distutils.setup_keywords]
message_extractors = babel.messages.setuptools_frontend:check_message_extractors
```

File: .venv\Lib\site-packages\Babel-2.14.0.dist-info\top_level.txt

```
babel
```

File: .venv\Lib\site-packages\certifi\core.py

```
"""
certifi.py
~~~~~~~~~~

This module returns the installation location of cacert.pem or its contents.
"""
import sys
```

```
import atexit

def exit_cacert_ctx() -> None:
    _CACERT_CTX.__exit__(None, None, None)  # type: ignore[union-attr]


if sys.version_info >= (3, 11):

    from importlib.resources import as_file, files

    _CACERT_CTX = None
    _CACERT_PATH = None

    def where() -> str:
        # This is slightly terrible, but we want to delay extracting the file
        # in cases where we're inside of a zipimport situation until someone
        # actually calls where(), but we don't want to re-extract the file
        # on every call of where(), so we'll do it once then store it in a
        # global variable.
        global _CACERT_CTX
        global _CACERT_PATH
        if _CACERT_PATH is None:
            # This is slightly janky, the importlib.resources API wants you to
            # manage the cleanup of this file, so it doesn't actually return a
            # path, it returns a context manager that will give you the path
            # when you enter it and will do any cleanup when you leave it. In
            # the common case of not needing a temporary file, it will just
            # return the file system location and the __exit__() is a no-op.
            #
            # We also have to hold onto the actual context manager, because
            # it will do the cleanup whenever it gets garbage collected, so
            # we will also store that at the global level as well.
            _CACERT_CTX = as_file(files("certifi").joinpath("cacert.pem"))
            _CACERT_PATH = str(_CACERT_CTX.__enter__())
            atexit.register(exit_cacert_ctx)

        return _CACERT_PATH

    def contents() -> str:
        return files("certifi").joinpath("cacert.pem").read_text(encoding="ascii")

elif sys.version_info >= (3, 7):

    from importlib.resources import path as get_path, read_text

    _CACERT_CTX = None
    _CACERT_PATH = None

    def where() -> str:
        # This is slightly terrible, but we want to delay extracting the
        # file in cases where we're inside of a zipimport situation until
        # someone actually calls where(), but we don't want to re-extract
        # the file on every call of where(), so we'll do it once then store
        # it in a global variable.
        global _CACERT_CTX
        global _CACERT_PATH
        if _CACERT_PATH is None:
            # This is slightly janky, the importlib.resources API wants you
            # to manage the cleanup of this file, so it doesn't actually
            # return a path, it returns a context manager that will give
            # you the path when you enter it and will do any cleanup when
            # you leave it. In the common case of not needing a temporary
            # file, it will just return the file system location and the
            # __exit__() is a no-op.
            #
            # We also have to hold onto the actual context manager, because
            # it will do the cleanup whenever it gets garbage collected, so
            # we will also store that at the global level as well.
            _CACERT_CTX = get_path("certifi", "cacert.pem")
            _CACERT_PATH = str(_CACERT_CTX.__enter__())
            atexit.register(exit_cacert_ctx)

        return _CACERT_PATH

    def contents() -> str:
        return read_text("certifi", "cacert.pem", encoding="ascii")

else:
    import os
    import types
    from typing import Union
```

```
    Package = Union[types.ModuleType, str]
    Resource = Union[str, "os.PathLike"]

    # This fallback will work for Python versions prior to 3.7 that lack the
    # importlib.resources module but relies on the existing `where` function
    # so won't address issues with environments like PyOxidizer that don't set
    # __file__ on modules.
    def read_text(
        package: Package,
        resource: Resource,
        encoding: str = 'utf-8',
        errors: str = 'strict'
    ) -> str:
        with open(where(), encoding=encoding) as data:
            return data.read()

    # If we don't have importlib.resources, then we will just do the old logic
    # of assuming we're on the filesystem and munge the path directly.
    def where() -> str:
        f = os.path.dirname(__file__)

        return os.path.join(f, "cacert.pem")

    def contents() -> str:
        return read_text("certifi", "cacert.pem", encoding="ascii")
```

File: .venv\Lib\site-packages\certifi__init__.py

```
from .core import contents, where

__all__ = ["contents", "where"]
__version__ = "2024.02.02"
```

File: .venv\Lib\site-packages\certifi__main__.py

```
import argparse

from certifi import contents, where

parser = argparse.ArgumentParser()
parser.add_argument("-c", "--contents", action="store_true")
args = parser.parse_args()

if args.contents:
    print(contents())
else:
    print(where())
```

File: .venv\Lib\site-packages\certifi-2024.2.2.dist-info\top_level.txt

```
certifi
```

File: .venv\Lib\site-packages\charset_normalizer\api.py

```
import logging
from os import PathLike
from typing import BinaryIO, List, Optional, Set, Union

from .cd import (
    coherence_ratio,
    encoding_languages,
    mb_encoding_languages,
    merge_coherence_ratios,
)
from .constant import IANA_SUPPORTED, TOO_BIG_SEQUENCE, TOO_SMALL_SEQUENCE, TRACE
from .md import mess_ratio
from .models import CharsetMatch, CharsetMatches
from .utils import (
    any_specified_encoding,
    cut_sequence_chunks,
    iana_name,
    identify_sig_or_bom,
    is_cp_similar,
    is_multi_byte_encoding,
    should_strip_sig_or_bom,
)
```

```
# Will most likely be controversial
# logging.addLevelName(TRACE, "TRACE")
logger = logging.getLogger("charset_normalizer")
explain_handler = logging.StreamHandler()
explain_handler.setFormatter(
    logging.Formatter("%(asctime)s | %(levelname)s | %(message)s")
)


def from_bytes(
    sequences: Union[bytes, bytearray],
    steps: int = 5,
    chunk_size: int = 512,
    threshold: float = 0.2,
    cp_isolation: Optional[List[str]] = None,
    cp_exclusion: Optional[List[str]] = None,
    preemptive_behaviour: bool = True,
    explain: bool = False,
    language_threshold: float = 0.1,
    enable_fallback: bool = True,
) -> CharsetMatches:
    """
    Given a raw bytes sequence, return the best possibles charset usable to render str objects.
    If there is no results, it is a strong indicator that the source is binary/not text.
    By default, the process will extract 5 blocks of 512o each to assess the mess and coherence of a given sequence.
    And will give up a particular code page after 20% of measured mess. Those criteria are customizable at will.

    The preemptive behavior DOES NOT replace the traditional detection workflow, it prioritize a particular code page
    but never take it for granted. Can improve the performance.

    You may want to focus your attention to some code page or/and not others, use cp_isolation and cp_exclusion for that
    purpose.

    This function will strip the SIG in the payload/sequence every time except on UTF-16, UTF-32.
    By default the library does not setup any handler other than the NullHandler, if you choose to set the 'explain'
    toggle to True it will alter the logger configuration to add a StreamHandler that is suitable for debugging.
    Custom logging format and handler can be set manually.
    """

    if not isinstance(sequences, (bytearray, bytes)):
        raise TypeError(
            "Expected object of type bytes or bytearray, got: {0}".format(
                type(sequences)
            )
        )

    if explain:
        previous_logger_level: int = logger.level
        logger.addHandler(explain_handler)
        logger.setLevel(TRACE)

    length: int = len(sequences)

    if length == 0:
        logger.debug("Encoding detection on empty bytes, assuming utf_8 intention.")
        if explain:
            logger.removeHandler(explain_handler)
            logger.setLevel(previous_logger_level or logging.WARNING)
        return CharsetMatches([CharsetMatch(sequences, "utf_8", 0.0, False, [], "")])

    if cp_isolation is not None:
        logger.log(
            TRACE,
            "cp_isolation is set. use this flag for debugging purpose. "
            "limited list of encoding allowed : %s.",
            ", ".join(cp_isolation),
        )
        cp_isolation = [iana_name(cp, False) for cp in cp_isolation]
    else:
        cp_isolation = []

    if cp_exclusion is not None:
        logger.log(
            TRACE,
            "cp_exclusion is set. use this flag for debugging purpose. "
            "limited list of encoding excluded : %s.",
            ", ".join(cp_exclusion),
        )
        cp_exclusion = [iana_name(cp, False) for cp in cp_exclusion]
    else:
        cp_exclusion = []
```

```
if length <= (chunk_size * steps):
    logger.log(
        TRACE,
        "override steps (%i) and chunk_size (%i) as content does not fit (%i byte(s) given) parameters.",
        steps,
        chunk_size,
        length,
    )
    steps = 1
    chunk_size = length

if steps > 1 and length / steps < chunk_size:
    chunk_size = int(length / steps)

is_too_small_sequence: bool = len(sequences) < TOO_SMALL_SEQUENCE
is_too_large_sequence: bool = len(sequences) >= TOO_BIG_SEQUENCE

if is_too_small_sequence:
    logger.log(
        TRACE,
        "Trying to detect encoding from a tiny portion of ({}) byte(s).".format(
            length
        ),
    )
elif is_too_large_sequence:
    logger.log(
        TRACE,
        "Using lazy str decoding because the payload is quite large, ({}) byte(s).".format(
            length
        ),
    )

prioritized_encodings: List[str] = []

specified_encoding: Optional[str] = (
    any_specified_encoding(sequences) if preemptive_behaviour else None
)

if specified_encoding is not None:
    prioritized_encodings.append(specified_encoding)
    logger.log(
        TRACE,
        "Detected declarative mark in sequence. Priority +1 given for %s.",
        specified_encoding,
    )

tested: Set[str] = set()
tested_but_hard_failure: List[str] = []
tested_but_soft_failure: List[str] = []

fallback_ascii: Optional[CharsetMatch] = None
fallback_u8: Optional[CharsetMatch] = None
fallback_specified: Optional[CharsetMatch] = None

results: CharsetMatches = CharsetMatches()

sig_encoding, sig_payload = identify_sig_or_bom(sequences)

if sig_encoding is not None:
    prioritized_encodings.append(sig_encoding)
    logger.log(
        TRACE,
        "Detected a SIG or BOM mark on first %i byte(s). Priority +1 given for %s.",
        len(sig_payload),
        sig_encoding,
    )

prioritized_encodings.append("ascii")

if "utf_8" not in prioritized_encodings:
    prioritized_encodings.append("utf_8")

for encoding_iana in prioritized_encodings + IANA_SUPPORTED:
    if cp_isolation and encoding_iana not in cp_isolation:
        continue

    if cp_exclusion and encoding_iana in cp_exclusion:
        continue

    if encoding_iana in tested:
        continue
```

```
tested.add(encoding_iana)

decoded_payload: Optional[str] = None
bom_or_sig_available: bool = sig_encoding == encoding_iana
strip_sig_or_bom: bool = bom_or_sig_available and should_strip_sig_or_bom(
    encoding_iana
)

if encoding_iana in {"utf_16", "utf_32"} and not bom_or_sig_available:
    logger.log(
        TRACE,
        "Encoding %s won't be tested as-is because it require a BOM. Will try some sub-encoder LE/BE.",
        encoding_iana,
    )
    continue
if encoding_iana in {"utf_7"} and not bom_or_sig_available:
    logger.log(
        TRACE,
        "Encoding %s won't be tested as-is because detection is unreliable without BOM/SIG.",
        encoding_iana,
    )
    continue

try:
    is_multi_byte_decoder: bool = is_multi_byte_encoding(encoding_iana)
except (ModuleNotFoundError, ImportError):
    logger.log(
        TRACE,
        "Encoding %s does not provide an IncrementalDecoder",
        encoding_iana,
    )
    continue

try:
    if is_too_large_sequence and is_multi_byte_decoder is False:
        str(
            sequences[: int(50e4)]
            if strip_sig_or_bom is False
            else sequences[len(sig_payload) : int(50e4)],
            encoding=encoding_iana,
        )
    else:
        decoded_payload = str(
            sequences
            if strip_sig_or_bom is False
            else sequences[len(sig_payload) :],
            encoding=encoding_iana,
        )
except (UnicodeDecodeError, LookupError) as e:
    if not isinstance(e, LookupError):
        logger.log(
            TRACE,
            "Code page %s does not fit given bytes sequence at ALL. %s",
            encoding_iana,
            str(e),
        )
    tested_but_hard_failure.append(encoding_iana)
    continue

similar_soft_failure_test: bool = False

for encoding_soft_failed in tested_but_soft_failure:
    if is_cp_similar(encoding_iana, encoding_soft_failed):
        similar_soft_failure_test = True
        break

if similar_soft_failure_test:
    logger.log(
        TRACE,
        "%s is deemed too similar to code page %s and was consider unsuited already. Continuing!",
        encoding_iana,
        encoding_soft_failed,
    )
    continue

r_ = range(
    0 if not bom_or_sig_available else len(sig_payload),
    length,
    int(length / steps),
)
multi_byte_bonus: bool = (
```

```
    is_multi_byte_decoder
    and decoded_payload is not None
    and len(decoded_payload) < length
)

if multi_byte_bonus:
    logger.log(
        TRACE,
        "Code page %s is a multi byte encoding table and it appear that at least one character "
        "was encoded using n-bytes.",
        encoding_iana,
    )

max_chunk_gave_up: int = int(len(r_) / 4)

max_chunk_gave_up = max(max_chunk_gave_up, 2)
early_stop_count: int = 0
lazy_str_hard_failure = False

md_chunks: List[str] = []
md_ratios = []

try:
    for chunk in cut_sequence_chunks(
        sequences,
        encoding_iana,
        r_,
        chunk_size,
        bom_or_sig_available,
        strip_sig_or_bom,
        sig_payload,
        is_multi_byte_decoder,
        decoded_payload,
    ):
        md_chunks.append(chunk)

        md_ratios.append(
            mess_ratio(
                chunk,
                threshold,
                explain is True and 1 <= len(cp_isolation) <= 2,
            )
        )

        if md_ratios[-1] >= threshold:
            early_stop_count += 1

        if (early_stop_count >= max_chunk_gave_up) or (
            bom_or_sig_available and strip_sig_or_bom is False
        ):
            break
except (
    UnicodeDecodeError
) as e:  # Lazy str loading may have missed something there
    logger.log(
        TRACE,
        "LazyStr Loading: After MD chunk decode, code page %s does not fit given bytes sequence at ALL. %s",
        encoding_iana,
        str(e),
    )
    early_stop_count = max_chunk_gave_up
    lazy_str_hard_failure = True

# We might want to check the sequence again with the whole content
# Only if initial MD tests passes
if (
    not lazy_str_hard_failure
    and is_too_large_sequence
    and not is_multi_byte_decoder
):
    try:
        sequences[int(50e3) :].decode(encoding_iana, errors="strict")
    except UnicodeDecodeError as e:
        logger.log(
            TRACE,
            "LazyStr Loading: After final lookup, code page %s does not fit given bytes sequence at ALL. %s",
            encoding_iana,
            str(e),
        )
        tested_but_hard_failure.append(encoding_iana)
        continue
```

```
mean_mess_ratio: float = sum(md_ratios) / len(md_ratios) if md_ratios else 0.0
if mean_mess_ratio >= threshold or early_stop_count >= max_chunk_gave_up:
    tested_but_soft_failure.append(encoding_iana)
    logger.log(
        TRACE,
        "%s was excluded because of initial chaos probing. Gave up %i time(s). "
        "Computed mean chaos is %f %%.",
        encoding_iana,
        early_stop_count,
        round(mean_mess_ratio * 100, ndigits=3),
    )
    # Preparing those fallbacks in case we got nothing.
    if (
        enable_fallback
        and encoding_iana in ["ascii", "utf_8", specified_encoding]
        and not lazy_str_hard_failure
    ):
        fallback_entry = CharsetMatch(
            sequences, encoding_iana, threshold, False, [], decoded_payload
        )
        if encoding_iana == specified_encoding:
            fallback_specified = fallback_entry
        elif encoding_iana == "ascii":
            fallback_ascii = fallback_entry
        else:
            fallback_u8 = fallback_entry
    continue

logger.log(
    TRACE,
    "%s passed initial chaos probing. Mean measured chaos is %f %%",
    encoding_iana,
    round(mean_mess_ratio * 100, ndigits=3),
)

if not is_multi_byte_decoder:
    target_languages: List[str] = encoding_languages(encoding_iana)
else:
    target_languages = mb_encoding_languages(encoding_iana)

if target_languages:
    logger.log(
        TRACE,
        "{} should target any language(s) of {}".format(
            encoding_iana, str(target_languages)
        ),
    )

cd_ratios = []

# We shall skip the CD when its about ASCII
# Most of the time its not relevant to run "language-detection" on it.
if encoding_iana != "ascii":
    for chunk in md_chunks:
        chunk_languages = coherence_ratio(
            chunk,
            language_threshold,
            ",".join(target_languages) if target_languages else None,
        )

        cd_ratios.append(chunk_languages)

cd_ratios_merged = merge_coherence_ratios(cd_ratios)

if cd_ratios_merged:
    logger.log(
        TRACE,
        "We detected language {} using {}".format(
            cd_ratios_merged, encoding_iana
        ),
    )

results.append(
    CharsetMatch(
        sequences,
        encoding_iana,
        mean_mess_ratio,
        bom_or_sig_available,
        cd_ratios_merged,
        decoded_payload,
    )
```

```
        )

        if (
            encoding_iana in [specified_encoding, "ascii", "utf_8"]
            and mean_mess_ratio < 0.1
        ):
            logger.debug(
                "Encoding detection: %s is most likely the one.", encoding_iana
            )
            if explain:
                logger.removeHandler(explain_handler)
                logger.setLevel(previous_logger_level)
            return CharsetMatches([results[encoding_iana]])

        if encoding_iana == sig_encoding:
            logger.debug(
                "Encoding detection: %s is most likely the one as we detected a BOM or SIG within "
                "the beginning of the sequence.",
                encoding_iana,
            )
            if explain:
                logger.removeHandler(explain_handler)
                logger.setLevel(previous_logger_level)
            return CharsetMatches([results[encoding_iana]])

    if len(results) == 0:
        if fallback_u8 or fallback_ascii or fallback_specified:
            logger.log(
                TRACE,
                "Nothing got out of the detection process. Using ASCII/UTF-8/Specified fallback.",
            )

        if fallback_specified:
            logger.debug(
                "Encoding detection: %s will be used as a fallback match",
                fallback_specified.encoding,
            )
            results.append(fallback_specified)
        elif (
            (fallback_u8 and fallback_ascii is None)
            or (
                fallback_u8
                and fallback_ascii
                and fallback_u8.fingerprint != fallback_ascii.fingerprint
            )
            or (fallback_u8 is not None)
        ):
            logger.debug("Encoding detection: utf_8 will be used as a fallback match")
            results.append(fallback_u8)
        elif fallback_ascii:
            logger.debug("Encoding detection: ascii will be used as a fallback match")
            results.append(fallback_ascii)

    if results:
        logger.debug(
            "Encoding detection: Found %s as plausible (best-candidate) for content. With %i alternatives.",
            results.best().encoding,  # type: ignore
            len(results) - 1,
        )
    else:
        logger.debug("Encoding detection: Unable to determine any suitable charset.")

    if explain:
        logger.removeHandler(explain_handler)
        logger.setLevel(previous_logger_level)

    return results


def from_fp(
    fp: BinaryIO,
    steps: int = 5,
    chunk_size: int = 512,
    threshold: float = 0.20,
    cp_isolation: Optional[List[str]] = None,
    cp_exclusion: Optional[List[str]] = None,
    preemptive_behaviour: bool = True,
    explain: bool = False,
    language_threshold: float = 0.1,
    enable_fallback: bool = True,
) -> CharsetMatches:
```

```
    """
    Same thing than the function from_bytes but using a file pointer that is already ready.
    Will not close the file pointer.
    """
    return from_bytes(
        fp.read(),
        steps,
        chunk_size,
        threshold,
        cp_isolation,
        cp_exclusion,
        preemptive_behaviour,
        explain,
        language_threshold,
        enable_fallback,
    )


def from_path(
    path: Union[str, bytes, PathLike],  # type: ignore[type-arg]
    steps: int = 5,
    chunk_size: int = 512,
    threshold: float = 0.20,
    cp_isolation: Optional[List[str]] = None,
    cp_exclusion: Optional[List[str]] = None,
    preemptive_behaviour: bool = True,
    explain: bool = False,
    language_threshold: float = 0.1,
    enable_fallback: bool = True,
) -> CharsetMatches:
    """
    Same thing than the function from_bytes but with one extra step. Opening and reading given file path in binary mode.
    Can raise IOError.
    """
    with open(path, "rb") as fp:
        return from_fp(
            fp,
            steps,
            chunk_size,
            threshold,
            cp_isolation,
            cp_exclusion,
            preemptive_behaviour,
            explain,
            language_threshold,
            enable_fallback,
        )


def is_binary(
    fp_or_path_or_payload: Union[PathLike, str, BinaryIO, bytes],  # type: ignore[type-arg]
    steps: int = 5,
    chunk_size: int = 512,
    threshold: float = 0.20,
    cp_isolation: Optional[List[str]] = None,
    cp_exclusion: Optional[List[str]] = None,
    preemptive_behaviour: bool = True,
    explain: bool = False,
    language_threshold: float = 0.1,
    enable_fallback: bool = False,
) -> bool:
    """
    Detect if the given input (file, bytes, or path) points to a binary file. aka. not a string.
    Based on the same main heuristic algorithms and default kwargs at the sole exception that fallbacks match
    are disabled to be stricter around ASCII-compatible but unlikely to be a string.
    """
    if isinstance(fp_or_path_or_payload, (str, PathLike)):
        guesses = from_path(
            fp_or_path_or_payload,
            steps=steps,
            chunk_size=chunk_size,
            threshold=threshold,
            cp_isolation=cp_isolation,
            cp_exclusion=cp_exclusion,
            preemptive_behaviour=preemptive_behaviour,
            explain=explain,
            la
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\charset_normalizer\cd.py

```
import importlib
from codecs import IncrementalDecoder
from collections import Counter
from functools import lru_cache
from typing import Counter as TypeCounter, Dict, List, Optional, Tuple

from .constant import (
    FREQUENCIES,
    KO_NAMES,
    LANGUAGE_SUPPORTED_COUNT,
    TOO_SMALL_SEQUENCE,
    ZH_NAMES,
)
from .md import is_suspiciously_successive_range
from .models import CoherenceMatches
from .utils import (
    is_accentuated,
    is_latin,
    is_multi_byte_encoding,
    is_unicode_range_secondary,
    unicode_range,
)


def encoding_unicode_range(iana_name: str) -> List[str]:
    """
    Return associated unicode ranges in a single byte code page.
    """
    if is_multi_byte_encoding(iana_name):
        raise IOError("Function not supported on multi-byte code page")

    decoder = importlib.import_module(
        "encodings.{}".format(iana_name)
    ).IncrementalDecoder

    p: IncrementalDecoder = decoder(errors="ignore")
    seen_ranges: Dict[str, int] = {}
    character_count: int = 0

    for i in range(0x40, 0xFF):
        chunk: str = p.decode(bytes([i]))

        if chunk:
            character_range: Optional[str] = unicode_range(chunk)

            if character_range is None:
                continue

            if is_unicode_range_secondary(character_range) is False:
                if character_range not in seen_ranges:
                    seen_ranges[character_range] = 0
                seen_ranges[character_range] += 1
                character_count += 1

    return sorted(
        [
            character_range
            for character_range in seen_ranges
            if seen_ranges[character_range] / character_count >= 0.15
        ]
    )


def unicode_range_languages(primary_range: str) -> List[str]:
    """
    Return inferred languages used with a unicode range.
    """
    languages: List[str] = []

    for language, characters in FREQUENCIES.items():
        for character in characters:
            if unicode_range(character) == primary_range:
                languages.append(language)
                break

    return languages


@lru_cache()
```

```
def encoding_languages(iana_name: str) -> List[str]:
    """
    Single-byte encoding language association. Some code page are heavily linked to particular language(s).
    This function does the correspondence.
    """
    unicode_ranges: List[str] = encoding_unicode_range(iana_name)
    primary_range: Optional[str] = None

    for specified_range in unicode_ranges:
        if "Latin" not in specified_range:
            primary_range = specified_range
            break

    if primary_range is None:
        return ["Latin Based"]

    return unicode_range_languages(primary_range)


@lru_cache()
def mb_encoding_languages(iana_name: str) -> List[str]:
    """
    Multi-byte encoding language association. Some code page are heavily linked to particular language(s).
    This function does the correspondence.
    """
    if (
        iana_name.startswith("shift_")
        or iana_name.startswith("iso2022_jp")
        or iana_name.startswith("euc_j")
        or iana_name == "cp932"
    ):
        return ["Japanese"]
    if iana_name.startswith("gb") or iana_name in ZH_NAMES:
        return ["Chinese"]
    if iana_name.startswith("iso2022_kr") or iana_name in KO_NAMES:
        return ["Korean"]

    return []


@lru_cache(maxsize=LANGUAGE_SUPPORTED_COUNT)
def get_target_features(language: str) -> Tuple[bool, bool]:
    """
    Determine main aspects from a supported language if it contains accents and if is pure Latin.
    """
    target_have_accents: bool = False
    target_pure_latin: bool = True

    for character in FREQUENCIES[language]:
        if not target_have_accents and is_accentuated(character):
            target_have_accents = True
        if target_pure_latin and is_latin(character) is False:
            target_pure_latin = False

    return target_have_accents, target_pure_latin


def alphabet_languages(
    characters: List[str], ignore_non_latin: bool = False
) -> List[str]:
    """
    Return associated languages associated to given characters.
    """
    languages: List[Tuple[str, float]] = []

    source_have_accents = any(is_accentuated(character) for character in characters)

    for language, language_characters in FREQUENCIES.items():
        target_have_accents, target_pure_latin = get_target_features(language)

        if ignore_non_latin and target_pure_latin is False:
            continue

        if target_have_accents is False and source_have_accents:
            continue

        character_count: int = len(language_characters)

        character_match_count: int = len(
            [c for c in language_characters if c in characters]
        )
```

```
        ratio: float = character_match_count / character_count

        if ratio >= 0.2:
            languages.append((language, ratio))

    languages = sorted(languages, key=lambda x: x[1], reverse=True)

    return [compatible_language[0] for compatible_language in languages]


def characters_popularity_compare(
    language: str, ordered_characters: List[str]
) -> float:
    """
    Determine if a ordered characters list (by occurrence from most appearance to rarest) match a particular language.
    The result is a ratio between 0. (absolutely no correspondence) and 1. (near perfect fit).
    Beware that is function is not strict on the match in order to ease the detection. (Meaning close match is 1.)
    """
    if language not in FREQUENCIES:
        raise ValueError("{} not available".format(language))

    character_approved_count: int = 0
    FREQUENCIES_language_set = set(FREQUENCIES[language])

    ordered_characters_count: int = len(ordered_characters)
    target_language_characters_count: int = len(FREQUENCIES[language])

    large_alphabet: bool = target_language_characters_count > 26

    for character, character_rank in zip(
        ordered_characters, range(0, ordered_characters_count)
    ):
        if character not in FREQUENCIES_language_set:
            continue

        character_rank_in_language: int = FREQUENCIES[language].index(character)
        expected_projection_ratio: float = (
            target_language_characters_count / ordered_characters_count
        )
        character_rank_projection: int = int(character_rank * expected_projection_ratio)

        if (
            large_alphabet is False
            and abs(character_rank_projection - character_rank_in_language) > 4
        ):
            continue

        if (
            large_alphabet is True
            and abs(character_rank_projection - character_rank_in_language)
            < target_language_characters_count / 3
        ):
            character_approved_count += 1
            continue

        characters_before_source: List[str] = FREQUENCIES[language][
            0:character_rank_in_language
        ]
        characters_after_source: List[str] = FREQUENCIES[language][
            character_rank_in_language:
        ]
        characters_before: List[str] = ordered_characters[0:character_rank]
        characters_after: List[str] = ordered_characters[character_rank:]

        before_match_count: int = len(
            set(characters_before) & set(characters_before_source)
        )

        after_match_count: int = len(
            set(characters_after) & set(characters_after_source)
        )

        if len(characters_before_source) == 0 and before_match_count <= 4:
            character_approved_count += 1
            continue

        if len(characters_after_source) == 0 and after_match_count <= 4:
            character_approved_count += 1
            continue

        if (
```

```
            before_match_count / len(characters_before_source) >= 0.4
            or after_match_count / len(characters_after_source) >= 0.4
        ):
            character_approved_count += 1
            continue

    return character_approved_count / len(ordered_characters)


def alpha_unicode_split(decoded_sequence: str) -> List[str]:
    """
    Given a decoded text sequence, return a list of str. Unicode range / alphabet separation.
    Ex. a text containing English/Latin with a bit a Hebrew will return two items in the resulting list;
    One containing the latin letters and the other hebrew.
    """
    layers: Dict[str, str] = {}

    for character in decoded_sequence:
        if character.isalpha() is False:
            continue

        character_range: Optional[str] = unicode_range(character)

        if character_range is None:
            continue

        layer_target_range: Optional[str] = None

        for discovered_range in layers:
            if (
                is_suspiciously_successive_range(discovered_range, character_range)
                is False
            ):
                layer_target_range = discovered_range
                break

        if layer_target_range is None:
            layer_target_range = character_range

        if layer_target_range not in layers:
            layers[layer_target_range] = character.lower()
            continue

        layers[layer_target_range] += character.lower()

    return list(layers.values())


def merge_coherence_ratios(results: List[CoherenceMatches]) -> CoherenceMatches:
    """
    This function merge results previously given by the function coherence_ratio.
    The return type is the same as coherence_ratio.
    """
    per_language_ratios: Dict[str, List[float]] = {}
    for result in results:
        for sub_result in result:
            language, ratio = sub_result
            if language not in per_language_ratios:
                per_language_ratios[language] = [ratio]
                continue
            per_language_ratios[language].append(ratio)

    merge = [
        (
            language,
            round(
                sum(per_language_ratios[language]) / len(per_language_ratios[language]),
                4,
            ),
        )
        for language in per_language_ratios
    ]

    return sorted(merge, key=lambda x: x[1], reverse=True)


def filter_alt_coherence_matches(results: CoherenceMatches) -> CoherenceMatches:
    """
    We shall NOT return "English—" in CoherenceMatches because it is an alternative
    of "English". This function only keeps the best match and remove the em-dash in it.
    """
```

```
    index_results: Dict[str, List[float]] = dict()

    for result in results:
        language, ratio = result
        no_em_name: str = language.replace("–", "")

        if no_em_name not in index_results:
            index_results[no_em_name] = []

        index_results[no_em_name].append(ratio)

    if any(len(index_results[e]) > 1 for e in index_results):
        filtered_results: CoherenceMatches = []

        for language in index_results:
            filtered_results.append((language, max(index_results[language])))

        return filtered_results

    return results


@lru_cache(maxsize=2048)
def coherence_ratio(
    decoded_sequence: str, threshold: float = 0.1, lg_inclusion: Optional[str] = None
) -> CoherenceMatches:
    """
    Detect ANY language that can be identified in given sequence. The sequence will be analysed by layers.
    A layer = Character extraction by alphabets/ranges.
    """

    results: List[Tuple[str, float]] = []
    ignore_non_latin: bool = False

    sufficient_match_count: int = 0

    lg_inclusion_list = lg_inclusion.split(",") if lg_inclusion is not None else []
    if "Latin Based" in lg_inclusion_list:
        ignore_non_latin = True
        lg_inclusion_list.remove("Latin Based")

    for layer in alpha_unicode_split(decoded_sequence):
        sequence_frequencies: TypeCounter[str] = Counter(layer)
        most_common = sequence_frequencies.most_common()

        character_count: int = sum(o for c, o in most_common)

        if character_count <= TOO_SMALL_SEQUENCE:
            continue

        popular_character_ordered: List[str] = [c for c, o in most_common]

        for language in lg_inclusion_list or alphabet_languages(
            popular_character_ordered, ignore_non_latin
        ):
            ratio: float = characters_popularity_compare(
                language, popular_character_ordered
            )

            if ratio < threshold:
                continue
            elif ratio >= 0.8:
                sufficient_match_count += 1

            results.append((language, round(ratio, 4)))

            if sufficient_match_count >= 3:
                break

    return sorted(
        filter_alt_coherence_matches(results), key=lambda x: x[1], reverse=True
    )
```

File: .venv\Lib\site-packages\charset_normalizer\constant.py

```
# -*- coding: utf-8 -*-
from codecs import BOM_UTF8, BOM_UTF16_BE, BOM_UTF16_LE, BOM_UTF32_BE, BOM_UTF32_LE
from encodings.aliases import aliases
from re import IGNORECASE, compile as re_compile
```

```
from typing import Dict, List, Set, Union

# Contain for each eligible encoding a list of/item bytes SIG/BOM
ENCODING_MARKS: Dict[str, Union[bytes, List[bytes]]] = {
    "utf_8": BOM_UTF8,
    "utf_7": [
        b"\x2b\x2f\x76\x38",
        b"\x2b\x2f\x76\x39",
        b"\x2b\x2f\x76\x2b",
        b"\x2b\x2f\x76\x2f",
        b"\x2b\x2f\x76\x38\x2d",
    ],
    "gb18030": b"\x84\x31\x95\x33",
    "utf_32": [BOM_UTF32_BE, BOM_UTF32_LE],
    "utf_16": [BOM_UTF16_BE, BOM_UTF16_LE],
}

TOO_SMALL_SEQUENCE: int = 32
TOO_BIG_SEQUENCE: int = int(10e6)

UTF8_MAXIMAL_ALLOCATION: int = 1_112_064

# Up-to-date Unicode ucd/15.0.0
UNICODE_RANGES_COMBINED: Dict[str, range] = {
    "Control character": range(32),
    "Basic Latin": range(32, 128),
    "Latin-1 Supplement": range(128, 256),
    "Latin Extended-A": range(256, 384),
    "Latin Extended-B": range(384, 592),
    "IPA Extensions": range(592, 688),
    "Spacing Modifier Letters": range(688, 768),
    "Combining Diacritical Marks": range(768, 880),
    "Greek and Coptic": range(880, 1024),
    "Cyrillic": range(1024, 1280),
    "Cyrillic Supplement": range(1280, 1328),
    "Armenian": range(1328, 1424),
    "Hebrew": range(1424, 1536),
    "Arabic": range(1536, 1792),
    "Syriac": range(1792, 1872),
    "Arabic Supplement": range(1872, 1920),
    "Thaana": range(1920, 1984),
    "NKo": range(1984, 2048),
    "Samaritan": range(2048, 2112),
    "Mandaic": range(2112, 2144),
    "Syriac Supplement": range(2144, 2160),
    "Arabic Extended-B": range(2160, 2208),
    "Arabic Extended-A": range(2208, 2304),
    "Devanagari": range(2304, 2432),
    "Bengali": range(2432, 2560),
    "Gurmukhi": range(2560, 2688),
    "Gujarati": range(2688, 2816),
    "Oriya": range(2816, 2944),
    "Tamil": range(2944, 3072),
    "Telugu": range(3072, 3200),
    "Kannada": range(3200, 3328),
    "Malayalam": range(3328, 3456),
    "Sinhala": range(3456, 3584),
    "Thai": range(3584, 3712),
    "Lao": range(3712, 3840),
    "Tibetan": range(3840, 4096),
    "Myanmar": range(4096, 4256),
    "Georgian": range(4256, 4352),
    "Hangul Jamo": range(4352, 4608),
    "Ethiopic": range(4608, 4992),
    "Ethiopic Supplement": range(4992, 5024),
    "Cherokee": range(5024, 5120),
    "Unified Canadian Aboriginal Syllabics": range(5120, 5760),
    "Ogham": range(5760, 5792),
    "Runic": range(5792, 5888),
    "Tagalog": range(5888, 5920),
    "Hanunoo": range(5920, 5952),
    "Buhid": range(5952, 5984),
    "Tagbanwa": range(5984, 6016),
    "Khmer": range(6016, 6144),
    "Mongolian": range(6144, 6320),
    "Unified Canadian Aboriginal Syllabics Extended": range(6320, 6400),
    "Limbu": range(6400, 6480),
    "Tai Le": range(6480, 6528),
    "New Tai Lue": range(6528, 6624),
    "Khmer Symbols": range(6624, 6656),
    "Buginese": range(6656, 6688),
```

```
"Tai Tham": range(6688, 6832),
"Combining Diacritical Marks Extended": range(6832, 6912),
"Balinese": range(6912, 7040),
"Sundanese": range(7040, 7104),
"Batak": range(7104, 7168),
"Lepcha": range(7168, 7248),
"Ol Chiki": range(7248, 7296),
"Cyrillic Extended-C": range(7296, 7312),
"Georgian Extended": range(7312, 7360),
"Sundanese Supplement": range(7360, 7376),
"Vedic Extensions": range(7376, 7424),
"Phonetic Extensions": range(7424, 7552),
"Phonetic Extensions Supplement": range(7552, 7616),
"Combining Diacritical Marks Supplement": range(7616, 7680),
"Latin Extended Additional": range(7680, 7936),
"Greek Extended": range(7936, 8192),
"General Punctuation": range(8192, 8304),
"Superscripts and Subscripts": range(8304, 8352),
"Currency Symbols": range(8352, 8400),
"Combining Diacritical Marks for Symbols": range(8400, 8448),
"Letterlike Symbols": range(8448, 8528),
"Number Forms": range(8528, 8592),
"Arrows": range(8592, 8704),
"Mathematical Operators": range(8704, 8960),
"Miscellaneous Technical": range(8960, 9216),
"Control Pictures": range(9216, 9280),
"Optical Character Recognition": range(9280, 9312),
"Enclosed Alphanumerics": range(9312, 9472),
"Box Drawing": range(9472, 9600),
"Block Elements": range(9600, 9632),
"Geometric Shapes": range(9632, 9728),
"Miscellaneous Symbols": range(9728, 9984),
"Dingbats": range(9984, 10176),
"Miscellaneous Mathematical Symbols-A": range(10176, 10224),
"Supplemental Arrows-A": range(10224, 10240),
"Braille Patterns": range(10240, 10496),
"Supplemental Arrows-B": range(10496, 10624),
"Miscellaneous Mathematical Symbols-B": range(10624, 10752),
"Supplemental Mathematical Operators": range(10752, 11008),
"Miscellaneous Symbols and Arrows": range(11008, 11264),
"Glagolitic": range(11264, 11360),
"Latin Extended-C": range(11360, 11392),
"Coptic": range(11392, 11520),
"Georgian Supplement": range(11520, 11568),
"Tifinagh": range(11568, 11648),
"Ethiopic Extended": range(11648, 11744),
"Cyrillic Extended-A": range(11744, 11776),
"Supplemental Punctuation": range(11776, 11904),
"CJK Radicals Supplement": range(11904, 12032),
"Kangxi Radicals": range(12032, 12256),
"Ideographic Description Characters": range(12272, 12288),
"CJK Symbols and Punctuation": range(12288, 12352),
"Hiragana": range(12352, 12448),
"Katakana": range(12448, 12544),
"Bopomofo": range(12544, 12592),
"Hangul Compatibility Jamo": range(12592, 12688),
"Kanbun": range(12688, 12704),
"Bopomofo Extended": range(12704, 12736),
"CJK Strokes": range(12736, 12784),
"Katakana Phonetic Extensions": range(12784, 12800),
"Enclosed CJK Letters and Months": range(12800, 13056),
"CJK Compatibility": range(13056, 13312),
"CJK Unified Ideographs Extension A": range(13312, 19904),
"Yijing Hexagram Symbols": range(19904, 19968),
"CJK Unified Ideographs": range(19968, 40960),
"Yi Syllables": range(40960, 42128),
"Yi Radicals": range(42128, 42192),
"Lisu": range(42192, 42240),
"Vai": range(42240, 42560),
"Cyrillic Extended-B": range(42560, 42656),
"Bamum": range(42656, 42752),
"Modifier Tone Letters": range(42752, 42784),
"Latin Extended-D": range(42784, 43008),
"Syloti Nagri": range(43008, 43056),
"Common Indic Number Forms": range(43056, 43072),
"Phags-pa": range(43072, 43136),
"Saurashtra": range(43136, 43232),
"Devanagari Extended": range(43232, 43264),
"Kayah Li": range(43264, 43312),
"Rejang": range(43312, 43360),
"Hangul Jamo Extended-A": range(43360, 43392),
```

```
"Javanese": range(43392, 43488),
"Myanmar Extended-B": range(43488, 43520),
"Cham": range(43520, 43616),
"Myanmar Extended-A": range(43616, 43648),
"Tai Viet": range(43648, 43744),
"Meetei Mayek Extensions": range(43744, 43776),
"Ethiopic Extended-A": range(43776, 43824),
"Latin Extended-E": range(43824, 43888),
"Cherokee Supplement": range(43888, 43968),
"Meetei Mayek": range(43968, 44032),
"Hangul Syllables": range(44032, 55216),
"Hangul Jamo Extended-B": range(55216, 55296),
"High Surrogates": range(55296, 56192),
"High Private Use Surrogates": range(56192, 56320),
"Low Surrogates": range(56320, 57344),
"Private Use Area": range(57344, 63744),
"CJK Compatibility Ideographs": range(63744, 64256),
"Alphabetic Presentation Forms": range(64256, 64336),
"Arabic Presentation Forms-A": range(64336, 65024),
"Variation Selectors": range(65024, 65040),
"Vertical Forms": range(65040, 65056),
"Combining Half Marks": range(65056, 65072),
"CJK Compatibility Forms": range(65072, 65104),
"Small Form Variants": range(65104, 65136),
"Arabic Presentation Forms-B": range(65136, 65280),
"Halfwidth and Fullwidth Forms": range(65280, 65520),
"Specials": range(65520, 65536),
"Linear B Syllabary": range(65536, 65664),
"Linear B Ideograms": range(65664, 65792),
"Aegean Numbers": range(65792, 65856),
"Ancient Greek Numbers": range(65856, 65936),
"Ancient Symbols": range(65936, 66000),
"Phaistos Disc": range(66000, 66048),
"Lycian": range(66176, 66208),
"Carian": range(66208, 66272),
"Coptic Epact Numbers": range(66272, 66304),
"Old Italic": range(66304, 66352),
"Gothic": range(66352, 66384),
"Old Permic": range(66384, 66432),
"Ugaritic": range(66432, 66464),
"Old Persian": range(66464, 66528),
"Deseret": range(66560, 66640),
"Shavian": range(66640, 66688),
"Osmanya": range(66688, 66736),
"Osage": range(66736, 66816),
"Elbasan": range(66816, 66864),
"Caucasian Albanian": range(66864, 66928),
"Vithkuqi": range(66928, 67008),
"Linear A": range(67072, 67456),
"Latin Extended-F": range(67456, 67520),
"Cypriot Syllabary": range(67584, 67648),
"Imperial Aramaic": range(67648, 67680),
"Palmyrene": range(67680, 67712),
"Nabataean": range(67712, 67760),
"Hatran": range(67808, 67840),
"Phoenician": range(67840, 67872),
"Lydian": range(67872, 67904),
"Meroitic Hieroglyphs": range(67968, 68000),
"Meroitic Cursive": range(68000, 68096),
"Kharoshthi": range(68096, 68192),
"Old South Arabian": range(68192, 68224),
"Old North Arabian": range(68224, 68256),
"Manichaean": range(68288, 68352),
"Avestan": range(68352, 68416),
"Inscriptional Parthian": range(68416, 68448),
"Inscriptional Pahlavi": range(68448, 68480),
"Psalter Pahlavi": range(68480, 68528),
"Old Turkic": range(68608, 68688),
"Old Hungarian": range(68736, 68864),
"Hanifi Rohingya": range(68864, 68928),
"Rumi Numeral Symbols": range(69216, 69248),
"Yezidi": range(69248, 69312),
"Arabic Extended-C": range(69312, 69376),
"Old Sogdian": range(69376, 69424),
"Sogdian": range(69424, 69488),
"Old Uyghur": range(69488, 69552),
"Chorasmian": range(69552, 69600),
"Elymaic": range(69600, 69632),
"Brahmi": range(69632, 69760),
"Kaithi": range(69760, 69840),
"Sora Sompeng": range(69840, 69888),
```

```
"Chakma": range(69888, 69968),
"Mahajani": range(69968, 70016),
"Sharada": range(70016, 70112),
"Sinhala Archaic Numbers": range(70112, 70144),
"Khojki": range(70144, 70224),
"Multani": range(70272, 70320),
"Khudawadi": range(70320, 70400),
"Grantha": range(70400, 70528),
"Newa": range(70656, 70784),
"Tirhuta": range(70784, 70880),
"Siddham": range(71040, 71168),
"Modi": range(71168, 71264),
"Mongolian Supplement": range(71264, 71296),
"Takri": range(71296, 71376),
"Ahom": range(71424, 71504),
"Dogra": range(71680, 71760),
"Warang Citi": range(71840, 71936),
"Dives Akuru": range(71936, 72032),
"Nandinagari": range(72096, 72192),
"Zanabazar Square": range(72192, 72272),
"Soyombo": range(72272, 72368),
"Unified Canadian Aboriginal Syllabics Extended-A": range(72368, 72384),
"Pau Cin Hau": range(72384, 72448),
"Devanagari Extended-A": range(72448, 72544),
"Bhaiksuki": range(72704, 72816),
"Marchen": range(72816, 72896),
"Masaram Gondi": range(72960, 73056),
"Gunjala Gondi": range(73056, 73136),
"Makasar": range(73440, 73472),
"Kawi": range(73472, 73568),
"Lisu Supplement": range(73648, 73664),
"Tamil Supplement": range(73664, 73728),
"Cuneiform": range(73728, 74752),
"Cuneiform Numbers and Punctuation": range(74752, 74880),
"Early Dynastic Cuneiform": range(74880, 75088),
"Cypro-Minoan": range(77712, 77824),
"Egyptian Hieroglyphs": range(77824, 78896),
"Egyptian Hieroglyph Format Controls": range(78896, 78944),
"Anatolian Hieroglyphs": range(82944, 83584),
"Bamum Supplement": range(92160, 92736),
"Mro": range(92736, 92784),
"Tangsa": range(92784, 92880),
"Bassa Vah": range(92880, 92928),
"Pahawh Hmong": range(92928, 93072),
"Medefaidrin": range(93760, 93856),
"Miao": range(93952, 94112),
"Ideographic Symbols and Punctuation": range(94176, 94208),
"Tangut": range(94208, 100352),
"Tangut Components": range(100352, 101120),
"Khitan Small Script": range(101120, 101632),
"Tangut Supplement": range(101632, 101760),
"Kana Extended-B": range(110576, 110592),
"Kana Supplement": range(110592, 110848),
"Kana Extended-A": range(110848, 110896),
"Small Kana Extension": range(110896, 110960),
"Nushu": range(110960, 111360),
"Duployan": range(113664, 113824),
"Shorthand Format Controls": range(113824, 113840),
"Znamenny Musical Notation": range(118528, 118736),
"Byzantine Musical Symbols": range(118784, 119040),
"Musical Symbols": range(119040, 119296),
"Ancient Greek Musical Notation": range(119296, 119376),
"Kaktovik Numerals": range(119488, 119520),
"Mayan Numerals": range(119520, 119552),
"Tai Xuan Jing Symbols": range(119552, 119648),
"Counting Rod Numerals": range(119648, 119680),
"Mathematical Alphanumeric Symbols": range(119808, 120832),
"Sutton SignWriting": range(120832, 121520),
"Latin Extended-G": range(122624, 122880),
"Glagolitic Supplement": range(122880, 122928),
"Cyrillic Extended-D": range(122928, 123024),
"Nyiakeng Puachue Hmong": range(123136, 123216),
"Toto": range(123536, 123584),
"Wancho": range(123584, 123648),
"Nag Mundari": range(124112, 124160),
"Ethiopic Extended-B": range(124896, 124928),
"Mende Kikakui": range(124928, 125152),
"Adlam": range(125184, 125280),
"Indic Siyaq Numbers": range(126064, 126144),
"Ottoman Siyaq Numbers": range(126208, 126288),
"Arabic Mathematical Alphabetic Symbols": range(126464, 126720),
```

```
    "Mahjong Tiles": range(126976, 127024),
    "Domino Tiles": range(127024, 127136),
    "Playing Cards": range(127136, 127232),
    "Enclosed Alphanumeric Supplement": range(127232, 127488),
    "Enclosed Ideographic Supplement": range(127488, 127744),
    "Miscellaneous Symbols and Pictographs": range(127744, 128512),
    "Emoticons range(Emoji)": range(128512, 128592),
    "Ornamental Dingbats": range(128592, 128640),
    "Transport and Map Symbols": range(128640, 128768),
    "Alchemical Symbols": range(128768, 128896),
    "Geometric Shapes Extended": range(128896, 129024),
    "Supplemental Arrows-C": range(129024, 129280),
    "Supplemental Symbols and Pictographs": range(129280, 129536),
    "Chess Symbols": range(129536, 129648),
    "Symbols and Pictographs Extended-A": range(129648, 129792),
    "Symbols for Legacy Computing": range(129792, 130048),
    "CJK Unified Ideographs Extension B": range(131072, 173792),
    "CJK Unified Ideographs Extension C": range(173824, 177984),
    "CJK Unified Ideographs Extension D": range(177984, 178208),
    "CJK Unified Ideographs Extension E": range(178208, 183984),
    "CJK Unified Ideographs Extension F": range(183984, 191472),
    "CJK Compatibility Ideographs Supplement": range(194560, 195104),
    "CJK Unified Ideographs Extension G": range(196608, 201552),
    "CJK Unified Ideographs Extension H": range(201552, 205744),
    "Tags": range(917504, 917632),
    "Variation Selectors Supplement": range(917760, 918000),
    "Supplementary Private Use Area-A": range(983040, 1048576),
    "Supplementary Private Use Area-B": range(1048576, 1114112),
}


UNICODE_SECONDARY_RANGE_KEYWORD: List[str] = [
    "Supplement",
    "Extended",
    "Extensions",
    "Modifier",
    "Marks",
    "Punctuation",
    "Symbols",
    "Forms",
    "Operators",
    "Miscellaneous",
    "Drawing",
    "Block",
    "Shapes",
    "Supplemental",
    "Tags",
]

RE_POSSIBLE_ENCODING_INDICATION = re_compile(
    r"(?:(?:encoding)|(?:charset)|(?:coding))(?:[\:= ]{1,10})(?:[\"\']?)([a-zA-Z0-9\-_]+)(?:[\"\']?)",
    IGNORECASE,
)

IANA_NO_ALIASES = [
    "cp720",
    "cp737",
    "cp856",
    "cp874",
    "cp875",
    "cp1006",
    "koi8_r",
    "koi8_t",
    "koi8_u",
]

IANA_SUPPORTED: List[str] = sorted(
    filter(
        lambda x: x.endswith("_codec") is False
        and x not in {"rot_13", "tactis", "mbcs"},
        list(set(aliases.values())) + IANA_NO_ALIASES,
    )
)

IANA_SUPPORTED_COUNT: int = len(IANA_SUPPORTED)

# pre-computed code page that are similar using the function cp_similarity.
IANA_SUPPORTED_SIMILAR: Dict[str, List[str]] = {
    "cp037": ["cp1026", "cp1140", "cp273", "cp500"],
    "cp1026": ["cp037", "cp1140", "cp273", "cp500"],
    "cp1125": ["cp866"],
```

```
"cp1140": ["cp037", "cp1026", "cp273", "cp500"],
"cp1250": ["iso8859_2"],
"cp1251": ["kz1048", "ptcp154"],
"cp1252": ["iso8859_15", "iso8859_9", "latin_1"],
"cp1253": ["iso8859_7"],
"cp1254": ["iso8859_15", "iso8859_9", "latin_1"],
"cp1257": ["iso8859_13"],
"cp273": ["cp037", "cp1026", "cp1140", "cp500"],
"cp437": ["cp850", "cp858", "cp860", "cp861", "cp862", "cp863", "cp865"],
"cp500": ["cp037", "cp1026", "cp1140", "cp273"],
"cp850": ["cp437", "cp857", "cp858", "cp865"],
"cp857": ["cp850", "cp858", "cp865"],
"cp858": ["cp437", "cp850", "cp857", "cp865"],
"cp860": ["cp437", "cp861", "cp862", "cp863", "cp865"],
"cp861": ["cp437", "cp860", "cp862", "cp863", "cp865"],
"cp862": ["cp437", "cp860", "cp861", "cp863", "cp865"],
"cp863": ["cp437", "cp860", "cp861", "cp862", "cp865"],
"cp865": ["cp437", "cp850", "cp857", "cp858", "cp860", "cp861", "cp862", "cp863"],
"cp866": ["cp1125"],
"iso8859_10": ["iso8859_14", "iso8859_15", "iso8859_4", "iso8859_9", "latin_1"],
"iso8859_11": ["tis_620"],
"iso8859_13": ["cp1257"],
"iso8859_14": [
    "iso8859_10",
    "iso8859_15",
    "iso8859_16",
    "iso8859_3",
    "iso8859_9",
    "latin_1",
],
"iso8859_15": [
    "cp1252",
    "cp1254",
    "iso8859_10",
    "iso8859_14",
    "iso8859_16",
    "iso8859_3",
    "iso8859_9",
    "latin_1",
],
"iso8859_16": [
    "iso8859_14",
    "iso8859_15",
    "iso8859_2",
    "iso8859_3",
    "iso8859_9",
    "latin_1",
],
"iso8859_2": ["cp1250", "iso8859_16", "iso8859_4"],
"iso8859_3": ["iso8859_14", "iso8859_15", "iso8859_16", "iso8859_9", "latin_1"],
"iso8859_4": ["iso8859_10", "iso8859_2", "iso8859_9", "latin_1"],
"iso8859_7": ["cp1253"],
"iso8859_9": [
    "cp1252",
    "cp1254",
    "cp1258",
    "iso8859_10",
    "iso8859_14",
    "iso8859_15",
    "iso8859_16",
    "iso8859_3",
    "iso8859_4",
    "latin_1",
],
"kz1048": ["cp1251", "ptcp154"],
"latin_1": [
    "cp1252",
    "cp1254",
    "cp1258",
    "iso8859_10",
    "iso8859_14",
    "iso8859_15",
    "iso8859_16",
    "iso8859_3",
    "iso8859_4",
    "iso8859_9",
],
"mac_iceland": ["mac_roman", "mac_turkish"],
"mac_roman": ["mac_iceland", "mac_turkish"],
"mac_turkish": ["mac_iceland", "mac_roman"],
"ptcp154": ["cp1251", "kz1048"],
```

```
    "tis_620": ["iso8859_11"],
}


CHARDET_CORRESPONDENCE: Dict[str, str] = {
    "iso2022_kr": "ISO-2022-KR",
    "iso2022_jp": "ISO-2022-JP",
    "euc_kr": "EUC-KR",
    "tis_620": "TIS-620",
    "utf_32": "UTF-32",
    "euc_jp": "EUC-JP",
    "koi8_r": "KOI8-R",
    "iso8859_1": "ISO-8859-1",
    "iso8859_2": "ISO-8859-2",
    "iso8859_5": "ISO-8859-5",
    "iso8859_6": "ISO-8859-6",
    "iso8859_7": "ISO-8859-7",
    "iso8859_8": "ISO-8859-8",
    "utf_16": "UTF-16",
    "cp855": "IBM855",
    "mac_cyrillic": "MacCyrillic",
    "gb2312": "GB2312",
    "gb18030": "GB18030",
    "cp932": "CP932",
    "cp866": "IBM866",
    "utf_8": "utf-8",
    "utf_8_sig": "UTF-8-SIG",
    "shift_jis": "SHIFT_JIS",
    "big5": "Big5",
    "cp1250": "windows-1250",
    "cp1251": "windows-1251",
    "cp1252": "Windows-1252",
    "cp1253": "windows-1253",
    "cp1255": "windows-1255",
    "
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\charset_normalizer\legacy.py

```
from typing import Any, Dict, Optional, Union
from warnings import warn

from .api import from_bytes
from .constant import CHARDET_CORRESPONDENCE


def detect(
    byte_str: bytes, should_rename_legacy: bool = False, **kwargs: Any
) -> Dict[str, Optional[Union[str, float]]]:
    """
    chardet legacy method
    Detect the encoding of the given byte string. It should be mostly backward-compatible.
    Encoding name will match Chardet own writing whenever possible. (Not on encoding name unsupported by it)
    This function is deprecated and should be used to migrate your project easily, consult the documentation for
    further information. Not planned for removal.

    :param byte_str:     The byte sequence to examine.
    :param should_rename_legacy:  Should we rename legacy encodings
                                  to their more modern equivalents?
    """
    if len(kwargs):
        warn(
            f"charset-normalizer disregard arguments '{','.join(list(kwargs.keys()))}' in legacy function detect()"
        )

    if not isinstance(byte_str, (bytearray, bytes)):
        raise TypeError(  # pragma: nocover
            "Expected object of type bytes or bytearray, got: "
            "{0}".format(type(byte_str))
        )

    if isinstance(byte_str, bytearray):
        byte_str = bytes(byte_str)

    r = from_bytes(byte_str).best()

    encoding = r.encoding if r is not None else None
    language = r.language if r is not None and r.language != "Unknown" else ""
    confidence = 1.0 - r.chaos if r is not None else None
```

```
    # Note: CharsetNormalizer does not return 'UTF-8-SIG' as the sig get stripped in the detection/normalization process
    # but chardet does return 'utf-8-sig' and it is a valid codec name.
    if r is not None and encoding == "utf_8" and r.bom:
        encoding += "_sig"

    if should_rename_legacy is False and encoding in CHARDET_CORRESPONDENCE:
        encoding = CHARDET_CORRESPONDENCE[encoding]

    return {
        "encoding": encoding,
        "language": language,
        "confidence": confidence,
    }
```

File: .venv\Lib\site-packages\charset_normalizer\md.py

```
from functools import lru_cache
from logging import getLogger
from typing import List, Optional

from .constant import (
    COMMON_SAFE_ASCII_CHARACTERS,
    TRACE,
    UNICODE_SECONDARY_RANGE_KEYWORD,
)
from .utils import (
    is_accentuated,
    is_arabic,
    is_arabic_isolated_form,
    is_case_variable,
    is_cjk,
    is_emoticon,
    is_hangul,
    is_hiragana,
    is_katakana,
    is_latin,
    is_punctuation,
    is_separator,
    is_symbol,
    is_thai,
    is_unprintable,
    remove_accent,
    unicode_range,
)


class MessDetectorPlugin:
    """
    Base abstract class used for mess detection plugins.
    All detectors MUST extend and implement given methods.
    """

    def eligible(self, character: str) -> bool:
        """
        Determine if given character should be fed in.
        """
        raise NotImplementedError  # pragma: nocover

    def feed(self, character: str) -> None:
        """
        The main routine to be executed upon character.
        Insert the logic in witch the text would be considered chaotic.
        """
        raise NotImplementedError  # pragma: nocover

    def reset(self) -> None:  # pragma: no cover
        """
        Permit to reset the plugin to the initial state.
        """
        raise NotImplementedError

    @property
    def ratio(self) -> float:
        """
        Compute the chaos ratio based on what your feed() has seen.
        Must NOT be lower than 0.; No restriction gt 0.
        """
        raise NotImplementedError  # pragma: nocover
class TooManySymbolOrPunctuationPlugin(MessDetectorPlugin):
```

```
    def __init__(self) -> None:
        self._punctuation_count: int = 0
        self._symbol_count: int = 0
        self._character_count: int = 0

        self._last_printable_char: Optional[str] = None
        self._frenzy_symbol_in_word: bool = False

    def eligible(self, character: str) -> bool:
        return character.isprintable()

    def feed(self, character: str) -> None:
        self._character_count += 1

        if (
            character != self._last_printable_char
            and character not in COMMON_SAFE_ASCII_CHARACTERS
        ):
            if is_punctuation(character):
                self._punctuation_count += 1
            elif (
                character.isdigit() is False
                and is_symbol(character)
                and is_emoticon(character) is False
            ):
                self._symbol_count += 2

        self._last_printable_char = character

    def reset(self) -> None:  # pragma: no cover
        self._punctuation_count = 0
        self._character_count = 0
        self._symbol_count = 0

    @property
    def ratio(self) -> float:
        if self._character_count == 0:
            return 0.0

        ratio_of_punctuation: float = (
            self._punctuation_count + self._symbol_count
        ) / self._character_count

        return ratio_of_punctuation if ratio_of_punctuation >= 0.3 else 0.0


class TooManyAccentuatedPlugin(MessDetectorPlugin):
    def __init__(self) -> None:
        self._character_count: int = 0
        self._accentuated_count: int = 0

    def eligible(self, character: str) -> bool:
        return character.isalpha()

    def feed(self, character: str) -> None:
        self._character_count += 1

        if is_accentuated(character):
            self._accentuated_count += 1

    def reset(self) -> None:  # pragma: no cover
        self._character_count = 0
        self._accentuated_count = 0

    @property
    def ratio(self) -> float:
        if self._character_count < 8:
            return 0.0

        ratio_of_accentuation: float = self._accentuated_count / self._character_count
        return ratio_of_accentuation if ratio_of_accentuation >= 0.35 else 0.0


class UnprintablePlugin(MessDetectorPlugin):
    def __init__(self) -> None:
        self._unprintable_count: int = 0
        self._character_count: int = 0

    def eligible(self, character: str) -> bool:
        return True
    def feed(self, character: str) -> None:
```

```
        if is_unprintable(character):
            self._unprintable_count += 1
        self._character_count += 1

    def reset(self) -> None:  # pragma: no cover
        self._unprintable_count = 0

    @property
    def ratio(self) -> float:
        if self._character_count == 0:
            return 0.0

        return (self._unprintable_count * 8) / self._character_count


class SuspiciousDuplicateAccentPlugin(MessDetectorPlugin):
    def __init__(self) -> None:
        self._successive_count: int = 0
        self._character_count: int = 0

        self._last_latin_character: Optional[str] = None

    def eligible(self, character: str) -> bool:
        return character.isalpha() and is_latin(character)

    def feed(self, character: str) -> None:
        self._character_count += 1
        if (
            self._last_latin_character is not None
            and is_accentuated(character)
            and is_accentuated(self._last_latin_character)
        ):
            if character.isupper() and self._last_latin_character.isupper():
                self._successive_count += 1
            # Worse if its the same char duplicated with different accent.
            if remove_accent(character) == remove_accent(self._last_latin_character):
                self._successive_count += 1
        self._last_latin_character = character

    def reset(self) -> None:  # pragma: no cover
        self._successive_count = 0
        self._character_count = 0
        self._last_latin_character = None

    @property
    def ratio(self) -> float:
        if self._character_count == 0:
            return 0.0

        return (self._successive_count * 2) / self._character_count


class SuspiciousRange(MessDetectorPlugin):
    def __init__(self) -> None:
        self._suspicious_successive_range_count: int = 0
        self._character_count: int = 0
        self._last_printable_seen: Optional[str] = None

    def eligible(self, character: str) -> bool:
        return character.isprintable()

    def feed(self, character: str) -> None:
        self._character_count += 1

        if (
            character.isspace()
            or is_punctuation(character)
            or character in COMMON_SAFE_ASCII_CHARACTERS
        ):
            self._last_printable_seen = None
            return

        if self._last_printable_seen is None:
            self._last_printable_seen = character
            return

        unicode_range_a: Optional[str] = unicode_range(self._last_printable_seen)
        unicode_range_b: Optional[str] = unicode_range(character)

        if is_suspiciously_successive_range(unicode_range_a, unicode_range_b):
            self._suspicious_successive_range_count += 1
```

```
        self._last_printable_seen = character

    def reset(self) -> None:  # pragma: no cover
        self._character_count = 0
        self._suspicious_successive_range_count = 0
        self._last_printable_seen = None

    @property
    def ratio(self) -> float:
        if self._character_count <= 24:
            return 0.0

        ratio_of_suspicious_range_usage: float = (
            self._suspicious_successive_range_count * 2
        ) / self._character_count

        return ratio_of_suspicious_range_usage


class SuperWeirdWordPlugin(MessDetectorPlugin):
    def __init__(self) -> None:
        self._word_count: int = 0
        self._bad_word_count: int = 0
        self._foreign_long_count: int = 0

        self._is_current_word_bad: bool = False
        self._foreign_long_watch: bool = False

        self._character_count: int = 0
        self._bad_character_count: int = 0

        self._buffer: str = ""
        self._buffer_accent_count: int = 0

    def eligible(self, character: str) -> bool:
        return True

    def feed(self, character: str) -> None:
        if character.isalpha():
            self._buffer += character
            if is_accentuated(character):
                self._buffer_accent_count += 1
            if (
                self._foreign_long_watch is False
                and (is_latin(character) is False or is_accentuated(character))
                and is_cjk(character) is False
                and is_hangul(character) is False
                and is_katakana(character) is False
                and is_hiragana(character) is False
                and is_thai(character) is False
            ):
                self._foreign_long_watch = True
            return
        if not self._buffer:
            return
        if (
            character.isspace() or is_punctuation(character) or is_separator(character)
        ) and self._buffer:
            self._word_count += 1
            buffer_length: int = len(self._buffer)

            self._character_count += buffer_length

            if buffer_length >= 4:
                if self._buffer_accent_count / buffer_length > 0.34:
                    self._is_current_word_bad = True
                # Word/Buffer ending with an upper case accentuated letter are so rare,
                # that we will consider them all as suspicious. Same weight as foreign_long suspicious.
                if (
                    is_accentuated(self._buffer[-1])
                    and self._buffer[-1].isupper()
                    and all(_.isupper() for _ in self._buffer) is False
                ):
                    self._foreign_long_count += 1
                    self._is_current_word_bad = True
            if buffer_length >= 24 and self._foreign_long_watch:
                camel_case_dst = [
                    i
                    for c, i in zip(self._buffer, range(0, buffer_length))
                    if c.isupper()
                ]
```

```
                probable_camel_cased: bool = False

                if camel_case_dst and (len(camel_case_dst) / buffer_length <= 0.3):
                    probable_camel_cased = True

                if not probable_camel_cased:
                    self._foreign_long_count += 1
                    self._is_current_word_bad = True

            if self._is_current_word_bad:
                self._bad_word_count += 1
                self._bad_character_count += len(self._buffer)
                self._is_current_word_bad = False

            self._foreign_long_watch = False
            self._buffer = ""
            self._buffer_accent_count = 0
        elif (
            character not in {"<", ">", "-", "=", "~", "|", "_"}
            and character.isdigit() is False
            and is_symbol(character)
        ):
            self._is_current_word_bad = True
            self._buffer += character

    def reset(self) -> None:  # pragma: no cover
        self._buffer = ""
        self._is_current_word_bad = False
        self._foreign_long_watch = False
        self._bad_word_count = 0
        self._word_count = 0
        self._character_count = 0
        self._bad_character_count = 0
        self._foreign_long_count = 0

    @property
    def ratio(self) -> float:
        if self._word_count <= 10 and self._foreign_long_count == 0:
            return 0.0

        return self._bad_character_count / self._character_count


class CjkInvalidStopPlugin(MessDetectorPlugin):
    """
    GB(Chinese) based encoding often render the stop incorrectly when the content does not fit and
    can be easily detected. Searching for the overuse of '■' and '■'.
    """

    def __init__(self) -> None:
        self._wrong_stop_count: int = 0
        self._cjk_character_count: int = 0

    def eligible(self, character: str) -> bool:
        return True

    def feed(self, character: str) -> None:
        if character in {"■", "■"}:
            self._wrong_stop_count += 1
            return
        if is_cjk(character):
            self._cjk_character_count += 1

    def reset(self) -> None:  # pragma: no cover
        self._wrong_stop_count = 0
        self._cjk_character_count = 0

    @property
    def ratio(self) -> float:
        if self._cjk_character_count < 16:
            return 0.0
        return self._wrong_stop_count / self._cjk_character_count


class ArchaicUpperLowerPlugin(MessDetectorPlugin):
    def __init__(self) -> None:
        self._buf: bool = False

        self._character_count_since_last_sep: int = 0

        self._successive_upper_lower_count: int = 0
```

```
        self._successive_upper_lower_count_final: int = 0

        self._character_count: int = 0

        self._last_alpha_seen: Optional[str] = None
        self._current_ascii_only: bool = True

    def eligible(self, character: str) -> bool:
        return True

    def feed(self, character: str) -> None:
        is_concerned = character.isalpha() and is_case_variable(character)
        chunk_sep = is_concerned is False

        if chunk_sep and self._character_count_since_last_sep > 0:
            if (
                self._character_count_since_last_sep <= 64
                and character.isdigit() is False
                and self._current_ascii_only is False
            ):
                self._successive_upper_lower_count_final += (
                    self._successive_upper_lower_count
                )

            self._successive_upper_lower_count = 0
            self._character_count_since_last_sep = 0
            self._last_alpha_seen = None
            self._buf = False
            self._character_count += 1
            self._current_ascii_only = True

            return

        if self._current_ascii_only is True and character.isascii() is False:
            self._current_ascii_only = False

        if self._last_alpha_seen is not None:
            if (character.isupper() and self._last_alpha_seen.islower()) or (
                character.islower() and self._last_alpha_seen.isupper()
            ):
                if self._buf is True:
                    self._successive_upper_lower_count += 2
                    self._buf = False
                else:
                    self._buf = True
            else:
                self._buf = False

        self._character_count += 1
        self._character_count_since_last_sep += 1
        self._last_alpha_seen = character

    def reset(self) -> None:  # pragma: no cover
        self._character_count = 0
        self._character_count_since_last_sep = 0
        self._successive_upper_lower_count = 0
        self._successive_upper_lower_count_final = 0
        self._last_alpha_seen = None
        self._buf = False
        self._current_ascii_only = True

    @property
    def ratio(self) -> float:
        if self._character_count == 0:
            return 0.0

        return self._successive_upper_lower_count_final / self._character_count


class ArabicIsolatedFormPlugin(MessDetectorPlugin):
    def __init__(self) -> None:
        self._character_count: int = 0
        self._isolated_form_count: int = 0

    def reset(self) -> None:  # pragma: no cover
        self._character_count = 0
        self._isolated_form_count = 0

    def eligible(self, character: str) -> bool:
        return is_arabic(character)
    def feed(self, character: str) -> None:
```

```
        self._character_count += 1

        if is_arabic_isolated_form(character):
            self._isolated_form_count += 1

    @property
    def ratio(self) -> float:
        if self._character_count < 8:
            return 0.0

        isolated_form_usage: float = self._isolated_form_count / self._character_count

        return isolated_form_usage


@lru_cache(maxsize=1024)
def is_suspiciously_successive_range(
    unicode_range_a: Optional[str], unicode_range_b: Optional[str]
) -> bool:
    """
    Determine if two Unicode range seen next to each other can be considered as suspicious.
    """
    if unicode_range_a is None or unicode_range_b is None:
        return True

    if unicode_range_a == unicode_range_b:
        return False

    if "Latin" in unicode_range_a and "Latin" in unicode_range_b:
        return False

    if "Emoticons" in unicode_range_a or "Emoticons" in unicode_range_b:
        return False

    # Latin characters can be accompanied with a combining diacritical mark
    # eg. Vietnamese.
    if ("Latin" in unicode_range_a or "Latin" in unicode_range_b) and (
        "Combining" in unicode_range_a or "Combining" in unicode_range_b
    ):
        return False

    keywords_range_a, keywords_range_b = unicode_range_a.split(
        " "
    ), unicode_range_b.split(" ")

    for el in keywords_range_a:
        if el in UNICODE_SECONDARY_RANGE_KEYWORD:
            continue
        if el in keywords_range_b:
            return False

    # Japanese Exception
    range_a_jp_chars, range_b_jp_chars = (
        unicode_range_a
        in (
            "Hiragana",
            "Katakana",
        ),
        unicode_range_b in ("Hiragana", "Katakana"),
    )
    if (range_a_jp_chars or range_b_jp_chars) and (
        "CJK" in unicode_range_a or "CJK" in unicode_range_b
    ):
        return False
    if range_a_jp_chars and range_b_jp_chars:
        return False

    if "Hangul" in unicode_range_a or "Hangul" in unicode_range_b:
        if "CJK" in unicode_range_a or "CJK" in unicode_range_b:
            return False
        if unicode_range_a == "Basic Latin" or unicode_range_b == "Basic Latin":
            return False

    # Chinese/Japanese use dedicated range for punctuation and/or separators.
    if ("CJK" in unicode_range_a or "CJK" in unicode_range_b) or (
        unicode_range_a in ["Katakana", "Hiragana"]
        and unicode_range_b in ["Katakana", "Hiragana"]
    ):
        if "Punctuation" in unicode_range_a or "Punctuation" in unicode_range_b:
            return False
        if "Forms" in unicode_range_a or "Forms" in unicode_range_b:
```

```
            return False
        if unicode_range_a == "Basic Latin" or unicode_range_b == "Basic Latin":
            return False

    return True


@lru_cache(maxsize=2048)
def mess_ratio(
    decoded_sequence: str, maximum_threshold: float = 0.2, debug: bool = False
) -> float:
    """
    Compute a mess ratio given a decoded bytes sequence. The maximum threshold does stop the computation earlier.
    """

    detectors: List[MessDetectorPlugin] = [
        md_class() for md_class in MessDetectorPlugin.__subclasses__()
    ]

    length: int = len(decoded_sequence) + 1

    mean_mess_ratio: float = 0.0

    if length < 512:
        intermediary_mean_mess_ratio_calc: int = 32
    elif length <= 1024:
        intermediary_mean_mess_ratio_calc = 64
    else:
        intermediary_mean_mess_ratio_calc = 128

    for character, index in zip(decoded_sequence + "\n", range(length)):
        for detector in detectors:
            if detector.eligible(character):
                detector.feed(character)

        if (
            index > 0 and index % intermediary_mean_mess_ratio_calc == 0
        ) or index == length - 1:
            mean_mess_ratio = sum(dt.ratio for dt in detectors)

            if mean_mess_ratio >= maximum_threshold:
                break

    if debug:
        logger = getLogger("charset_normalizer")

        logger.log(
            TRACE,
            "Mess-detector extended-analysis start. "
            f"intermediary_mean_mess_ratio_calc={intermediary_mean_mess_ratio_calc} mean_mess_ratio={mean_mess_ratio} "
            f"maximum_threshold={maximum_threshold}",
        )

        if len(decoded_sequence) > 16:
            logger.log(TRACE, f"Starting with: {decoded_sequence[:16]}")
            logger.log(TRACE, f"Ending with: {decoded_sequence[-16::]}")

        for dt in detectors:  # pragma: nocover
            logger.log(TRACE, f"{dt.__class__}: {dt.ratio}")

    return round(mean_mess_ratio, 3)
```

File: .venv\Lib\site-packages\charset_normalizer\models.py

```
from encodings.aliases import aliases
from hashlib import sha256
from json import dumps
from typing import Any, Dict, Iterator, List, Optional, Tuple, Union

from .constant import TOO_BIG_SEQUENCE
from .utils import iana_name, is_multi_byte_encoding, unicode_range


class CharsetMatch:
    def __init__(
        self,
        payload: bytes,
        guessed_encoding: str,
        mean_mess_ratio: float,
```

```
    has_sig_or_bom: bool,
    languages: "CoherenceMatches",
    decoded_payload: Optional[str] = None,
):
    self._payload: bytes = payload

    self._encoding: str = guessed_encoding
    self._mean_mess_ratio: float = mean_mess_ratio
    self._languages: CoherenceMatches = languages
    self._has_sig_or_bom: bool = has_sig_or_bom
    self._unicode_ranges: Optional[List[str]] = None

    self._leaves: List[CharsetMatch] = []
    self._mean_coherence_ratio: float = 0.0

    self._output_payload: Optional[bytes] = None
    self._output_encoding: Optional[str] = None

    self._string: Optional[str] = decoded_payload

def __eq__(self, other: object) -> bool:
    if not isinstance(other, CharsetMatch):
        raise TypeError(
            "__eq__ cannot be invoked on {} and {}.".format(
                str(other.__class__), str(self.__class__)
            )
        )
    return self.encoding == other.encoding and self.fingerprint == other.fingerprint

def __lt__(self, other: object) -> bool:
    """
    Implemented to make sorted available upon CharsetMatches items.
    """
    if not isinstance(other, CharsetMatch):
        raise ValueError

    chaos_difference: float = abs(self.chaos - other.chaos)
    coherence_difference: float = abs(self.coherence - other.coherence)

    # Below 1% difference --> Use Coherence
    if chaos_difference < 0.01 and coherence_difference > 0.02:
        return self.coherence > other.coherence
    elif chaos_difference < 0.01 and coherence_difference <= 0.02:
        # When having a difficult decision, use the result that decoded as many multi-byte as possible.
        # preserve RAM usage!
        if len(self._payload) >= TOO_BIG_SEQUENCE:
            return self.chaos < other.chaos
        return self.multi_byte_usage > other.multi_byte_usage

    return self.chaos < other.chaos

@property
def multi_byte_usage(self) -> float:
    return 1.0 - (len(str(self)) / len(self.raw))

def __str__(self) -> str:
    # Lazy Str Loading
    if self._string is None:
        self._string = str(self._payload, self._encoding, "strict")
    return self._string

def __repr__(self) -> str:
    return "<CharsetMatch '{}' bytes({})>".format(self.encoding, self.fingerprint)

def add_submatch(self, other: "CharsetMatch") -> None:
    if not isinstance(other, CharsetMatch) or other == self:
        raise ValueError(
            "Unable to add instance <{}> as a submatch of a CharsetMatch".format(
                other.__class__
            )
        )

    other._string = None  # Unload RAM usage; dirty trick.
    self._leaves.append(other)

@property
def encoding(self) -> str:
    return self._encoding

@property
def encoding_aliases(self) -> List[str]:
```

```
        """
        Encoding name are known by many name, using this could help when searching for IBM855 when it's listed as CP855.
        """
        also_known_as: List[str] = []
        for u, p in aliases.items():
            if self.encoding == u:
                also_known_as.append(p)
            elif self.encoding == p:
                also_known_as.append(u)
        return also_known_as

    @property
    def bom(self) -> bool:
        return self._has_sig_or_bom

    @property
    def byte_order_mark(self) -> bool:
        return self._has_sig_or_bom

    @property
    def languages(self) -> List[str]:
        """
        Return the complete list of possible languages found in decoded sequence.
        Usually not really useful. Returned list may be empty even if 'language' property return something != 'Unknown'.
        """
        return [e[0] for e in self._languages]

    @property
    def language(self) -> str:
        """
        Most probable language found in decoded sequence. If none were detected or inferred, the property will return
        "Unknown".
        """
        if not self._languages:
            # Trying to infer the language based on the given encoding
            # Its either English or we should not pronounce ourselves in certain cases.
            if "ascii" in self.could_be_from_charset:
                return "English"

            # doing it there to avoid circular import
            from charset_normalizer.cd import encoding_languages, mb_encoding_languages

            languages = (
                mb_encoding_languages(self.encoding)
                if is_multi_byte_encoding(self.encoding)
                else encoding_languages(self.encoding)
            )

            if len(languages) == 0 or "Latin Based" in languages:
                return "Unknown"

            return languages[0]

        return self._languages[0][0]

    @property
    def chaos(self) -> float:
        return self._mean_mess_ratio

    @property
    def coherence(self) -> float:
        if not self._languages:
            return 0.0
        return self._languages[0][1]

    @property
    def percent_chaos(self) -> float:
        return round(self.chaos * 100, ndigits=3)

    @property
    def percent_coherence(self) -> float:
        return round(self.coherence * 100, ndigits=3)

    @property
    def raw(self) -> bytes:
        """
        Original untouched bytes.
        """
        return self._payload

    @property
```

```
    def submatch(self) -> List["CharsetMatch"]:
        return self._leaves

    @property
    def has_submatch(self) -> bool:
        return len(self._leaves) > 0

    @property
    def alphabets(self) -> List[str]:
        if self._unicode_ranges is not None:
            return self._unicode_ranges
        # list detected ranges
        detected_ranges: List[Optional[str]] = [
            unicode_range(char) for char in str(self)
        ]
        # filter and sort
        self._unicode_ranges = sorted(list({r for r in detected_ranges if r}))
        return self._unicode_ranges

    @property
    def could_be_from_charset(self) -> List[str]:
        """
        The complete list of encoding that output the exact SAME str result and therefore could be the originating
        encoding.
        This list does include the encoding available in property 'encoding'.
        """
        return [self._encoding] + [m.encoding for m in self._leaves]

    def output(self, encoding: str = "utf_8") -> bytes:
        """
        Method to get re-encoded bytes payload using given target encoding. Default to UTF-8.
        Any errors will be simply ignored by the encoder NOT replaced.
        """
        if self._output_encoding is None or self._output_encoding != encoding:
            self._output_encoding = encoding
            self._output_payload = str(self).encode(encoding, "replace")

        return self._output_payload  # type: ignore

    @property
    def fingerprint(self) -> str:
        """
        Retrieve the unique SHA256 computed using the transformed (re-encoded) payload. Not the original one.
        """
        return sha256(self.output()).hexdigest()


class CharsetMatches:
    """
    Container with every CharsetMatch items ordered by default from most probable to the less one.
    Act like a list(iterable) but does not implements all related methods.
    """

    def __init__(self, results: Optional[List[CharsetMatch]] = None):
        self._results: List[CharsetMatch] = sorted(results) if results else []

    def __iter__(self) -> Iterator[CharsetMatch]:
        yield from self._results

    def __getitem__(self, item: Union[int, str]) -> CharsetMatch:
        """
        Retrieve a single item either by its position or encoding name (alias may be used here).
        Raise KeyError upon invalid index or encoding not present in results.
        """
        if isinstance(item, int):
            return self._results[item]
        if isinstance(item, str):
            item = iana_name(item, False)
            for result in self._results:
                if item in result.could_be_from_charset:
                    return result
        raise KeyError

    def __len__(self) -> int:
        return len(self._results)

    def __bool__(self) -> bool:
        return len(self._results) > 0

    def append(self, item: CharsetMatch) -> None:
        """
```

```
        Insert a single match. Will be inserted accordingly to preserve sort.
        Can be inserted as a submatch.
        """
        if not isinstance(item, CharsetMatch):
            raise ValueError(
                "Cannot append instance '{}' to CharsetMatches".format(
                    str(item.__class__)
                )
            )
        # We should disable the submatch factoring when the input file is too heavy (conserve RAM usage)
        if len(item.raw) <= TOO_BIG_SEQUENCE:
            for match in self._results:
                if match.fingerprint == item.fingerprint and match.chaos == item.chaos:
                    match.add_submatch(item)
                    return
        self._results.append(item)
        self._results = sorted(self._results)

    def best(self) -> Optional["CharsetMatch"]:
        """
        Simply return the first match. Strict equivalent to matches[0].
        """
        if not self._results:
            return None
        return self._results[0]

    def first(self) -> Optional["CharsetMatch"]:
        """
        Redundant method, call the method best(). Kept for BC reasons.
        """
        return self.best()


CoherenceMatch = Tuple[str, float]
CoherenceMatches = List[CoherenceMatch]


class CliDetectionResult:
    def __init__(
        self,
        path: str,
        encoding: Optional[str],
        encoding_aliases: List[str],
        alternative_encodings: List[str],
        language: str,
        alphabets: List[str],
        has_sig_or_bom: bool,
        chaos: float,
        coherence: float,
        unicode_path: Optional[str],
        is_preferred: bool,
    ):
        self.path: str = path
        self.unicode_path: Optional[str] = unicode_path
        self.encoding: Optional[str] = encoding
        self.encoding_aliases: List[str] = encoding_aliases
        self.alternative_encodings: List[str] = alternative_encodings
        self.language: str = language
        self.alphabets: List[str] = alphabets
        self.has_sig_or_bom: bool = has_sig_or_bom
        self.chaos: float = chaos
        self.coherence: float = coherence
        self.is_preferred: bool = is_preferred

    @property
    def __dict__(self) -> Dict[str, Any]:  # type: ignore
        return {
            "path": self.path,
            "encoding": self.encoding,
            "encoding_aliases": self.encoding_aliases,
            "alternative_encodings": self.alternative_encodings,
            "language": self.language,
            "alphabets": self.alphabets,
            "has_sig_or_bom": self.has_sig_or_bom,
            "chaos": self.chaos,
            "coherence": self.coherence,
            "unicode_path": self.unicode_path,
            "is_preferred": self.is_preferred,
        }

    def to_json(self) -> str:
```

```
        return dumps(self.__dict__, ensure_ascii=True, indent=4)
```

File: .venv\Lib\site-packages\charset_normalizer\utils.py

```
import importlib
import logging
import unicodedata
from codecs import IncrementalDecoder
from encodings.aliases import aliases
from functools import lru_cache
from re import findall
from typing import Generator, List, Optional, Set, Tuple, Union

from _multibytecodec import MultibyteIncrementalDecoder

from .constant import (
    ENCODING_MARKS,
    IANA_SUPPORTED_SIMILAR,
    RE_POSSIBLE_ENCODING_INDICATION,
    UNICODE_RANGES_COMBINED,
    UNICODE_SECONDARY_RANGE_KEYWORD,
    UTF8_MAXIMAL_ALLOCATION,
)


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_accentuated(character: str) -> bool:
    try:
        description: str = unicodedata.name(character)
    except ValueError:
        return False
    return (
        "WITH GRAVE" in description
        or "WITH ACUTE" in description
        or "WITH CEDILLA" in description
        or "WITH DIAERESIS" in description
        or "WITH CIRCUMFLEX" in description
        or "WITH TILDE" in description
        or "WITH MACRON" in description
        or "WITH RING ABOVE" in description
    )


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def remove_accent(character: str) -> str:
    decomposed: str = unicodedata.decomposition(character)
    if not decomposed:
        return character

    codes: List[str] = decomposed.split(" ")

    return chr(int(codes[0], 16))


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def unicode_range(character: str) -> Optional[str]:
    """
    Retrieve the Unicode range official name from a single character.
    """
    character_ord: int = ord(character)

    for range_name, ord_range in UNICODE_RANGES_COMBINED.items():
        if character_ord in ord_range:
            return range_name

    return None


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_latin(character: str) -> bool:
    try:
        description: str = unicodedata.name(character)
    except ValueError:
        return False
    return "LATIN" in description


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_punctuation(character: str) -> bool:
```

```
    character_category: str = unicodedata.category(character)

    if "P" in character_category:
        return True

    character_range: Optional[str] = unicode_range(character)

    if character_range is None:
        return False

    return "Punctuation" in character_range


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_symbol(character: str) -> bool:
    character_category: str = unicodedata.category(character)

    if "S" in character_category or "N" in character_category:
        return True

    character_range: Optional[str] = unicode_range(character)

    if character_range is None:
        return False

    return "Forms" in character_range and character_category != "Lo"


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_emoticon(character: str) -> bool:
    character_range: Optional[str] = unicode_range(character)

    if character_range is None:
        return False

    return "Emoticons" in character_range or "Pictographs" in character_range


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_separator(character: str) -> bool:
    if character.isspace() or character in {"■", "+", "<", ">"}:
        return True

    character_category: str = unicodedata.category(character)

    return "Z" in character_category or character_category in {"Po", "Pd", "Pc"}


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_case_variable(character: str) -> bool:
    return character.islower() != character.isupper()


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_cjk(character: str) -> bool:
    try:
        character_name = unicodedata.name(character)
    except ValueError:
        return False

    return "CJK" in character_name


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_hiragana(character: str) -> bool:
    try:
        character_name = unicodedata.name(character)
    except ValueError:
        return False

    return "HIRAGANA" in character_name


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_katakana(character: str) -> bool:
    try:
        character_name = unicodedata.name(character)
    except ValueError:
        return False

    return "KATAKANA" in character_name
```

```
@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_hangul(character: str) -> bool:
    try:
        character_name = unicodedata.name(character)
    except ValueError:
        return False

    return "HANGUL" in character_name


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_thai(character: str) -> bool:
    try:
        character_name = unicodedata.name(character)
    except ValueError:
        return False

    return "THAI" in character_name


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_arabic(character: str) -> bool:
    try:
        character_name = unicodedata.name(character)
    except ValueError:
        return False

    return "ARABIC" in character_name


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_arabic_isolated_form(character: str) -> bool:
    try:
        character_name = unicodedata.name(character)
    except ValueError:
        return False

    return "ARABIC" in character_name and "ISOLATED FORM" in character_name


@lru_cache(maxsize=len(UNICODE_RANGES_COMBINED))
def is_unicode_range_secondary(range_name: str) -> bool:
    return any(keyword in range_name for keyword in UNICODE_SECONDARY_RANGE_KEYWORD)


@lru_cache(maxsize=UTF8_MAXIMAL_ALLOCATION)
def is_unprintable(character: str) -> bool:
    return (
        character.isspace() is False  # includes \n \t \r \v
        and character.isprintable() is False
        and character != "\x1A"  # Why? Its the ASCII substitute character.
        and character != "\ufeff"  # bug discovered in Python,
        # Zero Width No-Break Space located in ■Arabic Presentation Forms-B, Unicode 1.1 not acknowledged as space.
    )


def any_specified_encoding(sequence: bytes, search_zone: int = 8192) -> Optional[str]:
    """
    Extract using ASCII-only decoder any specified encoding in the first n-bytes.
    """
    if not isinstance(sequence, bytes):
        raise TypeError

    seq_len: int = len(sequence)

    results: List[str] = findall(
        RE_POSSIBLE_ENCODING_INDICATION,
        sequence[: min(seq_len, search_zone)].decode("ascii", errors="ignore"),
    )

    if len(results) == 0:
        return None

    for specified_encoding in results:
        specified_encoding = specified_encoding.lower().replace("-", "_")

        encoding_alias: str
        encoding_iana: str

        for encoding_alias, encoding_iana in aliases.items():
            if encoding_alias == specified_encoding:
```

```
                return encoding_iana
            if encoding_iana == specified_encoding:
                return encoding_iana

    return None


@lru_cache(maxsize=128)
def is_multi_byte_encoding(name: str) -> bool:
    """
    Verify is a specific encoding is a multi byte one based on it IANA name
    """
    return name in {
        "utf_8",
        "utf_8_sig",
        "utf_16",
        "utf_16_be",
        "utf_16_le",
        "utf_32",
        "utf_32_le",
        "utf_32_be",
        "utf_7",
    } or issubclass(
        importlib.import_module("encodings.{}".format(name)).IncrementalDecoder,
        MultibyteIncrementalDecoder,
    )


def identify_sig_or_bom(sequence: bytes) -> Tuple[Optional[str], bytes]:
    """
    Identify and extract SIG/BOM in given sequence.
    """

    for iana_encoding in ENCODING_MARKS:
        marks: Union[bytes, List[bytes]] = ENCODING_MARKS[iana_encoding]

        if isinstance(marks, bytes):
            marks = [marks]

        for mark in marks:
            if sequence.startswith(mark):
                return iana_encoding, mark

    return None, b""


def should_strip_sig_or_bom(iana_encoding: str) -> bool:
    return iana_encoding not in {"utf_16", "utf_32"}


def iana_name(cp_name: str, strict: bool = True) -> str:
    cp_name = cp_name.lower().replace("-", "_")

    encoding_alias: str
    encoding_iana: str

    for encoding_alias, encoding_iana in aliases.items():
        if cp_name in [encoding_alias, encoding_iana]:
            return encoding_iana

    if strict:
        raise ValueError("Unable to retrieve IANA for '{}'".format(cp_name))

    return cp_name


def range_scan(decoded_sequence: str) -> List[str]:
    ranges: Set[str] = set()

    for character in decoded_sequence:
        character_range: Optional[str] = unicode_range(character)

        if character_range is None:
            continue

        ranges.add(character_range)

    return list(ranges)


def cp_similarity(iana_name_a: str, iana_name_b: str) -> float:
```

```
    if is_multi_byte_encoding(iana_name_a) or is_multi_byte_encoding(iana_name_b):
        return 0.0

    decoder_a = importlib.import_module(
        "encodings.{}".format(iana_name_a)
    ).IncrementalDecoder
    decoder_b = importlib.import_module(
        "encodings.{}".format(iana_name_b)
    ).IncrementalDecoder

    id_a: IncrementalDecoder = decoder_a(errors="ignore")
    id_b: IncrementalDecoder = decoder_b(errors="ignore")

    character_match_count: int = 0

    for i in range(255):
        to_be_decoded: bytes = bytes([i])
        if id_a.decode(to_be_decoded) == id_b.decode(to_be_decoded):
            character_match_count += 1

    return character_match_count / 254


def is_cp_similar(iana_name_a: str, iana_name_b: str) -> bool:
    """
    Determine if two code page are at least 80% similar. IANA_SUPPORTED_SIMILAR dict was generated using
    the function cp_similarity.
    """
    return (
        iana_name_a in IANA_SUPPORTED_SIMILAR
        and iana_name_b in IANA_SUPPORTED_SIMILAR[iana_name_a]
    )


def set_logging_handler(
    name: str = "charset_normalizer",
    level: int = logging.INFO,
    format_string: str = "%(asctime)s | %(levelname)s | %(message)s",
) -> None:
    logger = logging.getLogger(name)
    logger.setLevel(level)

    handler = logging.StreamHandler()
    handler.setFormatter(logging.Formatter(format_string))
    logger.addHandler(handler)


def cut_sequence_chunks(
    sequences: bytes,
    encoding_iana: str,
    offsets: range,
    chunk_size: int,
    bom_or_sig_available: bool,
    strip_sig_or_bom: bool,
    sig_payload: bytes,
    is_multi_byte_decoder: bool,
    decoded_payload: Optional[str] = None,
) -> Generator[str, None, None]:
    if decoded_payload and is_multi_byte_decoder is False:
        for i in offsets:
            chunk = decoded_payload[i : i + chunk_size]
            if not chunk:
                break
            yield chunk
    else:
        for i in offsets:
            chunk_end = i + chunk_size
            if chunk_end > len(sequences) + 8:
                continue

            cut_sequence = sequences[i : i + chunk_size]

            if bom_or_sig_available and strip_sig_or_bom is False:
                cut_sequence = sig_payload + cut_sequence

            chunk = cut_sequence.decode(
                encoding_iana,
                errors="ignore" if is_multi_byte_decoder else "strict",
            )

            # multi-byte bad cutting detector and adjustment
```

```
            # not the cleanest way to perform that fix but clever enough for now.
            if is_multi_byte_decoder and i > 0:
                chunk_partial_size_chk: int = min(chunk_size, 16)

                if (
                    decoded_payload
                    and chunk[:chunk_partial_size_chk] not in decoded_payload
                ):
                    for j in range(i, i - 4, -1):
                        cut_sequence = sequences[j:chunk_end]

                        if bom_or_sig_available and strip_sig_or_bom is False:
                            cut_sequence = sig_payload + cut_sequence

                        chunk = cut_sequence.decode(encoding_iana, errors="ignore")

                        if chunk[:chunk_partial_size_chk] in decoded_payload:
                            break

            yield chunk
```

File: .venv\Lib\site-packages\charset_normalizer\version.py

```
"""
Expose version
"""

__version__ = "3.3.2"
VERSION = __version__.split(".")
```

File: .venv\Lib\site-packages\charset_normalizer__init__.py

```
# -*- coding: utf-8 -*-
"""
Charset-Normalizer
~~~~~~~~~~~~~~
The Real First Universal Charset Detector.
A library that helps you read text from an unknown charset encoding.
Motivated by chardet, This package is trying to resolve the issue by taking a new approach.
All IANA character set names for which the Python core library provides codecs are supported.

Basic usage:
   >>> from charset_normalizer import from_bytes
   >>> results = from_bytes('B[redacted]. O[redacted]!'.encode('utf_8'))
   >>> best_guess = results.best()
   >>> str(best_guess)
   'B[redacted]. O[redacted]!'

Others methods and usages are available - see the full documentation
at <https://github.com/Ousret/charset_normalizer>.
:copyright: (c) 2021 by Ahmed TAHRI
:license: MIT, see LICENSE for more details.
"""
import logging

from .api import from_bytes, from_fp, from_path, is_binary
from .legacy import detect
from .models import CharsetMatch, CharsetMatches
from .utils import set_logging_handler
from .version import VERSION, __version__

__all__ = (
    "from_fp",
    "from_path",
    "from_bytes",
    "is_binary",
    "detect",
    "CharsetMatch",
    "CharsetMatches",
    "__version__",
    "VERSION",
    "set_logging_handler",
)

# Attach a NullHandler to the top level logger by default
# https://docs.python.org/3.3/howto/logging.html#configuring-logging-for-a-library

logging.getLogger("charset_normalizer").addHandler(logging.NullHandler())
```

File: .venv\Lib\site-packages\charset_normalizer__main__.py

```
from .cli import cli_detect

if __name__ == "__main__":
    cli_detect()
```

File: .venv\Lib\site-packages\charset_normalizer\cli__init__.py

```
from .__main__ import cli_detect, query_yes_no

__all__ = (
    "cli_detect",
    "query_yes_no",
)
```

File: .venv\Lib\site-packages\charset_normalizer\cli__main__.py

```
import argparse
import sys
from json import dumps
from os.path import abspath, basename, dirname, join, realpath
from platform import python_version
from typing import List, Optional
from unicodedata import unidata_version

import charset_normalizer.md as md_module
from charset_normalizer import from_fp
from charset_normalizer.models import CliDetectionResult
from charset_normalizer.version import __version__


def query_yes_no(question: str, default: str = "yes") -> bool:
    """Ask a yes/no question via input() and return their answer.

    "question" is a string that is presented to the user.
    "default" is the presumed answer if the user just hits <Enter>.
        It must be "yes" (the default), "no" or None (meaning
        an answer is required of the user).

    The "answer" return value is True for "yes" or False for "no".

    Credit goes to (c) https://stackoverflow.com/questions/3041986/apt-command-line-interface-like-yes-no-input
    """
    valid = {"yes": True, "y": True, "ye": True, "no": False, "n": False}
    if default is None:
        prompt = " [y/n] "
    elif default == "yes":
        prompt = " [Y/n] "
    elif default == "no":
        prompt = " [y/N] "
    else:
        raise ValueError("invalid default answer: '%s'" % default)

    while True:
        sys.stdout.write(question + prompt)
        choice = input().lower()
        if default is not None and choice == "":
            return valid[default]
        elif choice in valid:
            return valid[choice]
        else:
            sys.stdout.write("Please respond with 'yes' or 'no' " "(or 'y' or 'n').\n")


def cli_detect(argv: Optional[List[str]] = None) -> int:
    """
    CLI assistant using ARGV and ArgumentParser
    :param argv:
    :return: 0 if everything is fine, anything else equal trouble
    """
    parser = argparse.ArgumentParser(
        description="The Real First Universal Charset Detector. "
        "Discover originating encoding used on text file. "
        "Normalize text to unicode."
    )
```

```
    parser.add_argument(
        "files", type=argparse.FileType("rb"), nargs="+", help="File(s) to be analysed"
    )
    parser.add_argument(
        "-v",
        "--verbose",
        action="store_true",
        default=False,
        dest="verbose",
        help="Display complementary information about file if any. "
        "Stdout will contain logs about the detection process.",
    )
    parser.add_argument(
        "-a",
        "--with-alternative",
        action="store_true",
        default=False,
        dest="alternatives",
        help="Output complementary possibilities if any. Top-level JSON WILL be a list.",
    )
    parser.add_argument(
        "-n",
        "--normalize",
        action="store_true",
        default=False,
        dest="normalize",
        help="Permit to normalize input file. If not set, program does not write anything.",
    )
    parser.add_argument(
        "-m",
        "--minimal",
        action="store_true",
        default=False,
        dest="minimal",
        help="Only output the charset detected to STDOUT. Disabling JSON output.",
    )
    parser.add_argument(
        "-r",
        "--replace",
        action="store_true",
        default=False,
        dest="replace",
        help="Replace file when trying to normalize it instead of creating a new one.",
    )
    parser.add_argument(
        "-f",
        "--force",
        action="store_true",
        default=False,
        dest="force",
        help="Replace file without asking if you are sure, use this flag with caution.",
    )
    parser.add_argument(
        "-t",
        "--threshold",
        action="store",
        default=0.2,
        type=float,
        dest="threshold",
        help="Define a custom maximum amount of chaos allowed in decoded content. 0. <= chaos <= 1.",
    )
    parser.add_argument(
        "--version",
        action="version",
        version="Charset-Normalizer {} - Python {} - Unicode {} - SpeedUp {}".format(
            __version__,
            python_version(),
            unidata_version,
            "OFF" if md_module.__file__.lower().endswith(".py") else "ON",
        ),
        help="Show version information and exit.",
    )

    args = parser.parse_args(argv)

    if args.replace is True and args.normalize is False:
        print("Use --replace in addition of --normalize only.", file=sys.stderr)
        return 1

    if args.force is True and args.replace is False:
        print("Use --force in addition of --replace only.", file=sys.stderr)
```

```
    return 1

if args.threshold < 0.0 or args.threshold > 1.0:
    print("--threshold VALUE should be between 0. AND 1.", file=sys.stderr)
    return 1

x_ = []

for my_file in args.files:
    matches = from_fp(my_file, threshold=args.threshold, explain=args.verbose)

    best_guess = matches.best()

    if best_guess is None:
        print(
            'Unable to identify originating encoding for "{}". {}'.format(
                my_file.name,
                "Maybe try increasing maximum amount of chaos."
                if args.threshold < 1.0
                else "",
            ),
            file=sys.stderr,
        )
        x_.append(
            CliDetectionResult(
                abspath(my_file.name),
                None,
                [],
                [],
                "Unknown",
                [],
                False,
                1.0,
                0.0,
                None,
                True,
            )
        )
    else:
        x_.append(
            CliDetectionResult(
                abspath(my_file.name),
                best_guess.encoding,
                best_guess.encoding_aliases,
                [
                    cp
                    for cp in best_guess.could_be_from_charset
                    if cp != best_guess.encoding
                ],
                best_guess.language,
                best_guess.alphabets,
                best_guess.bom,
                best_guess.percent_chaos,
                best_guess.percent_coherence,
                None,
                True,
            )
        )

        if len(matches) > 1 and args.alternatives:
            for el in matches:
                if el != best_guess:
                    x_.append(
                        CliDetectionResult(
                            abspath(my_file.name),
                            el.encoding,
                            el.encoding_aliases,
                            [
                                cp
                                for cp in el.could_be_from_charset
                                if cp != el.encoding
                            ],
                            el.language,
                            el.alphabets,
                            el.bom,
                            el.percent_chaos,
                            el.percent_coherence,
                            None,
                            False,
                        )
                    )
```

```
            if args.normalize is True:
                if best_guess.encoding.startswith("utf") is True:
                    print(
                        '"{}" file does not need to be normalized, as it already came from unicode.'.format(
                            my_file.name
                        ),
                        file=sys.stderr,
                    )
                    if my_file.closed is False:
                        my_file.close()
                    continue

                dir_path = dirname(realpath(my_file.name))
                file_name = basename(realpath(my_file.name))

                o_: List[str] = file_name.split(".")

                if args.replace is False:
                    o_.insert(-1, best_guess.encoding)
                    if my_file.closed is False:
                        my_file.close()
                elif (
                    args.force is False
                    and query_yes_no(
                        'Are you sure to normalize "{}" by replacing it ?'.format(
                            my_file.name
                        ),
                        "no",
                    )
                    is False
                ):
                    if my_file.closed is False:
                        my_file.close()
                    continue

                try:
                    x_[0].unicode_path = join(dir_path, ".".join(o_))

                    with open(x_[0].unicode_path, "w", encoding="utf-8") as fp:
                        fp.write(str(best_guess))
                except IOError as e:
                    print(str(e), file=sys.stderr)
                    if my_file.closed is False:
                        my_file.close()
                    return 2

            if my_file.closed is False:
                my_file.close()

        if args.minimal is False:
            print(
                dumps(
                    [el.__dict__ for el in x_] if len(x_) > 1 else x_[0].__dict__,
                    ensure_ascii=True,
                    indent=4,
                )
            )
        else:
            for my_file in args.files:
                print(
                    ", ".join(
                        [
                            el.encoding or "undefined"
                            for el in x_
                            if el.path == abspath(my_file.name)
                        ]
                    )
                )

        return 0


    if __name__ == "__main__":
        cli_detect()
```

File: .venv\Lib\site-packages\charset_normalizer-3.3.2.dist-info\entry_points.txt

```
[console_scripts]
normalizer = charset_normalizer.cli:cli_detect
```

File: .venv\Lib\site-packages\charset_normalizer-3.3.2.dist-info\top_level.txt

```
charset_normalizer
```

File: .venv\Lib\site-packages\colorama\ansi.py

```
# Copyright Jonathan Hartley 2013. BSD 3-Clause license, see LICENSE file.
'''
This module generates ANSI character codes to printing colors to terminals.
See: http://en.wikipedia.org/wiki/ANSI_escape_code
'''

CSI = '\033['
OSC = '\033]'
BEL = '\a'


def code_to_chars(code):
    return CSI + str(code) + 'm'

def set_title(title):
    return OSC + '2;' + title + BEL

def clear_screen(mode=2):
    return CSI + str(mode) + 'J'

def clear_line(mode=2):
    return CSI + str(mode) + 'K'


class AnsiCodes(object):
    def __init__(self):
        # the subclasses declare class attributes which are numbers.
        # Upon instantiation we define instance attributes, which are the same
        # as the class attributes but wrapped with the ANSI escape sequence
        for name in dir(self):
            if not name.startswith('_'):
                value = getattr(self, name)
                setattr(self, name, code_to_chars(value))


class AnsiCursor(object):
    def UP(self, n=1):
        return CSI + str(n) + 'A'
    def DOWN(self, n=1):
        return CSI + str(n) + 'B'
    def FORWARD(self, n=1):
        return CSI + str(n) + 'C'
    def BACK(self, n=1):
        return CSI + str(n) + 'D'
    def POS(self, x=1, y=1):
        return CSI + str(y) + ';' + str(x) + 'H'


class AnsiFore(AnsiCodes):
    BLACK           = 30
    RED             = 31
    GREEN           = 32
    YELLOW          = 33
    BLUE            = 34
    MAGENTA         = 35
    CYAN            = 36
    WHITE           = 37
    RESET           = 39

    # These are fairly well supported, but not part of the standard.
    LIGHTBLACK_EX   = 90
    LIGHTRED_EX     = 91
    LIGHTGREEN_EX   = 92
    LIGHTYELLOW_EX  = 93
    LIGHTBLUE_EX    = 94
    LIGHTMAGENTA_EX = 95
    LIGHTCYAN_EX    = 96
    LIGHTWHITE_EX   = 97


class AnsiBack(AnsiCodes):
```

```
    BLACK           = 40
    RED             = 41
    GREEN           = 42
    YELLOW          = 43
    BLUE            = 44
    MAGENTA         = 45
    CYAN            = 46
    WHITE           = 47
    RESET           = 49

    # These are fairly well supported, but not part of the standard.
    LIGHTBLACK_EX   = 100
    LIGHTRED_EX     = 101
    LIGHTGREEN_EX   = 102
    LIGHTYELLOW_EX  = 103
    LIGHTBLUE_EX    = 104
    LIGHTMAGENTA_EX = 105
    LIGHTCYAN_EX    = 106
    LIGHTWHITE_EX   = 107


class AnsiStyle(AnsiCodes):
    BRIGHT    = 1
    DIM       = 2
    NORMAL    = 22
    RESET_ALL = 0

Fore   = AnsiFore()
Back   = AnsiBack()
Style  = AnsiStyle()
Cursor = AnsiCursor()
```

File: .venv\Lib\site-packages\colorama\ansitowin32.py

```
# Copyright Jonathan Hartley 2013. BSD 3-Clause license, see LICENSE file.
import re
import sys
import os

from .ansi import AnsiFore, AnsiBack, AnsiStyle, Style, BEL
from .winterm import enable_vt_processing, WinTerm, WinColor, WinStyle
from .win32 import windll, winapi_test


winterm = None
if windll is not None:
    winterm = WinTerm()


class StreamWrapper(object):
    '''
    Wraps a stream (such as stdout), acting as a transparent proxy for all
    attribute access apart from method 'write()', which is delegated to our
    Converter instance.
    '''
    def __init__(self, wrapped, converter):
        # double-underscore everything to prevent clashes with names of
        # attributes on the wrapped stream object.
        self.__wrapped = wrapped
        self.__convertor = converter

    def __getattr__(self, name):
        return getattr(self.__wrapped, name)

    def __enter__(self, *args, **kwargs):
        # special method lookup bypasses __getattr__/__getattribute__, see
        # https://stackoverflow.com/questions/12632894/why-doesnt-getattr-work-with-exit
        # thus, contextlib magic methods are not proxied via __getattr__
        return self.__wrapped.__enter__(*args, **kwargs)

    def __exit__(self, *args, **kwargs):
        return self.__wrapped.__exit__(*args, **kwargs)

    def __setstate__(self, state):
        self.__dict__ = state

    def __getstate__(self):
        return self.__dict__
    def write(self, text):
```

```
        self.__convertor.write(text)

    def isatty(self):
        stream = self.__wrapped
        if 'PYCHARM_HOSTED' in os.environ:
            if stream is not None and (stream is sys.__stdout__ or stream is sys.__stderr__):
                return True
        try:
            stream_isatty = stream.isatty
        except AttributeError:
            return False
        else:
            return stream_isatty()

    @property
    def closed(self):
        stream = self.__wrapped
        try:
            return stream.closed
        # AttributeError in the case that the stream doesn't support being closed
        # ValueError for the case that the stream has already been detached when atexit runs
        except (AttributeError, ValueError):
            return True


class AnsiToWin32(object):
    '''
    Implements a 'write()' method which, on Windows, will strip ANSI character
    sequences from the text, and if outputting to a tty, will convert them into
    win32 function calls.
    '''
    ANSI_CSI_RE = re.compile('\001?\033\\[((?:\\d|;)*)([a-zA-Z])\002?')   # Control Sequence Introducer
    ANSI_OSC_RE = re.compile('\001?\033\\]([^\a]*)(\a)\002?')             # Operating System Command

    def __init__(self, wrapped, convert=None, strip=None, autoreset=False):
        # The wrapped stream (normally sys.stdout or sys.stderr)
        self.wrapped = wrapped

        # should we reset colors to defaults after every .write()
        self.autoreset = autoreset

        # create the proxy wrapping our output stream
        self.stream = StreamWrapper(wrapped, self)

        on_windows = os.name == 'nt'
        # We test if the WinAPI works, because even if we are on Windows
        # we may be using a terminal that doesn't support the WinAPI
        # (e.g. Cygwin Terminal). In this case it's up to the terminal
        # to support the ANSI codes.
        conversion_supported = on_windows and winapi_test()
        try:
            fd = wrapped.fileno()
        except Exception:
            fd = -1
        system_has_native_ansi = not on_windows or enable_vt_processing(fd)
        have_tty = not self.stream.closed and self.stream.isatty()
        need_conversion = conversion_supported and not system_has_native_ansi

        # should we strip ANSI sequences from our output?
        if strip is None:
            strip = need_conversion or not have_tty
        self.strip = strip

        # should we should convert ANSI sequences into win32 calls?
        if convert is None:
            convert = need_conversion and have_tty
        self.convert = convert

        # dict of ansi codes to win32 functions and parameters
        self.win32_calls = self.get_win32_calls()

        # are we wrapping stderr?
        self.on_stderr = self.wrapped is sys.stderr

    def should_wrap(self):
        '''
        True if this class is actually needed. If false, then the output
        stream will not be affected, nor will win32 calls be issued, so
        wrapping stdout is not actually required. This will generally be
        False on non-Windows platforms, unless optional functionality like
        autoreset has been requested using kwargs to init()
```

```
        '''
        return self.convert or self.strip or self.autoreset

    def get_win32_calls(self):
        if self.convert and winterm:
            return {
                AnsiStyle.RESET_ALL: (winterm.reset_all, ),
                AnsiStyle.BRIGHT: (winterm.style, WinStyle.BRIGHT),
                AnsiStyle.DIM: (winterm.style, WinStyle.NORMAL),
                AnsiStyle.NORMAL: (winterm.style, WinStyle.NORMAL),
                AnsiFore.BLACK: (winterm.fore, WinColor.BLACK),
                AnsiFore.RED: (winterm.fore, WinColor.RED),
                AnsiFore.GREEN: (winterm.fore, WinColor.GREEN),
                AnsiFore.YELLOW: (winterm.fore, WinColor.YELLOW),
                AnsiFore.BLUE: (winterm.fore, WinColor.BLUE),
                AnsiFore.MAGENTA: (winterm.fore, WinColor.MAGENTA),
                AnsiFore.CYAN: (winterm.fore, WinColor.CYAN),
                AnsiFore.WHITE: (winterm.fore, WinColor.GREY),
                AnsiFore.RESET: (winterm.fore, ),
                AnsiFore.LIGHTBLACK_EX: (winterm.fore, WinColor.BLACK, True),
                AnsiFore.LIGHTRED_EX: (winterm.fore, WinColor.RED, True),
                AnsiFore.LIGHTGREEN_EX: (winterm.fore, WinColor.GREEN, True),
                AnsiFore.LIGHTYELLOW_EX: (winterm.fore, WinColor.YELLOW, True),
                AnsiFore.LIGHTBLUE_EX: (winterm.fore, WinColor.BLUE, True),
                AnsiFore.LIGHTMAGENTA_EX: (winterm.fore, WinColor.MAGENTA, True),
                AnsiFore.LIGHTCYAN_EX: (winterm.fore, WinColor.CYAN, True),
                AnsiFore.LIGHTWHITE_EX: (winterm.fore, WinColor.GREY, True),
                AnsiBack.BLACK: (winterm.back, WinColor.BLACK),
                AnsiBack.RED: (winterm.back, WinColor.RED),
                AnsiBack.GREEN: (winterm.back, WinColor.GREEN),
                AnsiBack.YELLOW: (winterm.back, WinColor.YELLOW),
                AnsiBack.BLUE: (winterm.back, WinColor.BLUE),
                AnsiBack.MAGENTA: (winterm.back, WinColor.MAGENTA),
                AnsiBack.CYAN: (winterm.back, WinColor.CYAN),
                AnsiBack.WHITE: (winterm.back, WinColor.GREY),
                AnsiBack.RESET: (winterm.back, ),
                AnsiBack.LIGHTBLACK_EX: (winterm.back, WinColor.BLACK, True),
                AnsiBack.LIGHTRED_EX: (winterm.back, WinColor.RED, True),
                AnsiBack.LIGHTGREEN_EX: (winterm.back, WinColor.GREEN, True),
                AnsiBack.LIGHTYELLOW_EX: (winterm.back, WinColor.YELLOW, True),
                AnsiBack.LIGHTBLUE_EX: (winterm.back, WinColor.BLUE, True),
                AnsiBack.LIGHTMAGENTA_EX: (winterm.back, WinColor.MAGENTA, True),
                AnsiBack.LIGHTCYAN_EX: (winterm.back, WinColor.CYAN, True),
                AnsiBack.LIGHTWHITE_EX: (winterm.back, WinColor.GREY, True),
            }
        return dict()

    def write(self, text):
        if self.strip or self.convert:
            self.write_and_convert(text)
        else:
            self.wrapped.write(text)
            self.wrapped.flush()
        if self.autoreset:
            self.reset_all()


    def reset_all(self):
        if self.convert:
            self.call_win32('m', (0,))
        elif not self.strip and not self.stream.closed:
            self.wrapped.write(Style.RESET_ALL)


    def write_and_convert(self, text):
        '''
        Write the given text to our wrapped stream, stripping any ANSI
        sequences from the text, and optionally converting them into win32
        calls.
        '''
        cursor = 0
        text = self.convert_osc(text)
        for match in self.ANSI_CSI_RE.finditer(text):
            start, end = match.span()
            self.write_plain_text(text, cursor, start)
            self.convert_ansi(*match.groups())
            cursor = end
        self.write_plain_text(text, cursor, len(text))


    def write_plain_text(self, text, start, end):
```

```
        if start < end:
            self.wrapped.write(text[start:end])
            self.wrapped.flush()


    def convert_ansi(self, paramstring, command):
        if self.convert:
            params = self.extract_params(command, paramstring)
            self.call_win32(command, params)


    def extract_params(self, command, paramstring):
        if command in 'Hf':
            params = tuple(int(p) if len(p) != 0 else 1 for p in paramstring.split(';'))
            while len(params) < 2:
                # defaults:
                params = params + (1,)
        else:
            params = tuple(int(p) for p in paramstring.split(';') if len(p) != 0)
            if len(params) == 0:
                # defaults:
                if command in 'JKm':
                    params = (0,)
                elif command in 'ABCD':
                    params = (1,)

        return params


    def call_win32(self, command, params):
        if command == 'm':
            for param in params:
                if param in self.win32_calls:
                    func_args = self.win32_calls[param]
                    func = func_args[0]
                    args = func_args[1:]
                    kwargs = dict(on_stderr=self.on_stderr)
                    func(*args, **kwargs)
        elif command in 'J':
            winterm.erase_screen(params[0], on_stderr=self.on_stderr)
        elif command in 'K':
            winterm.erase_line(params[0], on_stderr=self.on_stderr)
        elif command in 'Hf':     # cursor position - absolute
            winterm.set_cursor_position(params, on_stderr=self.on_stderr)
        elif command in 'ABCD':   # cursor position - relative
            n = params[0]
            # A - up, B - down, C - forward, D - back
            x, y = {'A': (0, -n), 'B': (0, n), 'C': (n, 0), 'D': (-n, 0)}[command]
            winterm.cursor_adjust(x, y, on_stderr=self.on_stderr)


    def convert_osc(self, text):
        for match in self.ANSI_OSC_RE.finditer(text):
            start, end = match.span()
            text = text[:start] + text[end:]
            paramstring, command = match.groups()
            if command == BEL:
                if paramstring.count(";") == 1:
                    params = paramstring.split(";")
                    # 0 - change title and icon (we will only change title)
                    # 1 - change icon (we don't support this)
                    # 2 - change title
                    if params[0] in '02':
                        winterm.set_title(params[1])
        return text


    def flush(self):
        self.wrapped.flush()
```

File: .venv\Lib\site-packages\coloramа\initialise.py

```
# Copyright Jonathan Hartley 2013. BSD 3-Clause license, see LICENSE file.
import atexit
import contextlib
import sys

from .ansitowin32 import AnsiToWin32
def _wipe_internal_state_for_tests():
```

```
    global orig_stdout, orig_stderr
    orig_stdout = None
    orig_stderr = None

    global wrapped_stdout, wrapped_stderr
    wrapped_stdout = None
    wrapped_stderr = None

    global atexit_done
    atexit_done = False

    global fixed_windows_console
    fixed_windows_console = False

    try:
        # no-op if it wasn't registered
        atexit.unregister(reset_all)
    except AttributeError:
        # python 2: no atexit.unregister. Oh well, we did our best.
        pass


def reset_all():
    if AnsiToWin32 is not None:    # Issue #74: objects might become None at exit
        AnsiToWin32(orig_stdout).reset_all()


def init(autoreset=False, convert=None, strip=None, wrap=True):

    if not wrap and any([autoreset, convert, strip]):
        raise ValueError('wrap=False conflicts with any other arg=True')

    global wrapped_stdout, wrapped_stderr
    global orig_stdout, orig_stderr

    orig_stdout = sys.stdout
    orig_stderr = sys.stderr

    if sys.stdout is None:
        wrapped_stdout = None
    else:
        sys.stdout = wrapped_stdout = \
            wrap_stream(orig_stdout, convert, strip, autoreset, wrap)
    if sys.stderr is None:
        wrapped_stderr = None
    else:
        sys.stderr = wrapped_stderr = \
            wrap_stream(orig_stderr, convert, strip, autoreset, wrap)

    global atexit_done
    if not atexit_done:
        atexit.register(reset_all)
        atexit_done = True


def deinit():
    if orig_stdout is not None:
        sys.stdout = orig_stdout
    if orig_stderr is not None:
        sys.stderr = orig_stderr


def just_fix_windows_console():
    global fixed_windows_console

    if sys.platform != "win32":
        return
    if fixed_windows_console:
        return
    if wrapped_stdout is not None or wrapped_stderr is not None:
        # Someone already ran init() and it did stuff, so we won't second-guess them
        return

    # On newer versions of Windows, AnsiToWin32.__init__ will implicitly enable the
    # native ANSI support in the console as a side-effect. We only need to actually
    # replace sys.stdout/stderr if we're in the old-style conversion mode.
    new_stdout = AnsiToWin32(sys.stdout, convert=None, strip=None, autoreset=False)
    if new_stdout.convert:
        sys.stdout = new_stdout
    new_stderr = AnsiToWin32(sys.stderr, convert=None, strip=None, autoreset=False)
    if new_stderr.convert:
```

```
        sys.stderr = new_stderr

    fixed_windows_console = True

@contextlib.contextmanager
def colorama_text(*args, **kwargs):
    init(*args, **kwargs)
    try:
        yield
    finally:
        deinit()


def reinit():
    if wrapped_stdout is not None:
        sys.stdout = wrapped_stdout
    if wrapped_stderr is not None:
        sys.stderr = wrapped_stderr


def wrap_stream(stream, convert, strip, autoreset, wrap):
    if wrap:
        wrapper = AnsiToWin32(stream,
            convert=convert, strip=strip, autoreset=autoreset)
        if wrapper.should_wrap():
            stream = wrapper.stream
    return stream


# Use this for initial setup as well, to reduce code duplication
_wipe_internal_state_for_tests()
```

File: .venv\Lib\site-packages\colorama\win32.py

```
# Copyright Jonathan Hartley 2013. BSD 3-Clause license, see LICENSE file.

# from winbase.h
STDOUT = -11
STDERR = -12

ENABLE_VIRTUAL_TERMINAL_PROCESSING = 0x0004

try:
    import ctypes
    from ctypes import LibraryLoader
    windll = LibraryLoader(ctypes.WinDLL)
    from ctypes import wintypes
except (AttributeError, ImportError):
    windll = None
    SetConsoleTextAttribute = lambda *_: None
    winapi_test = lambda *_: None
else:
    from ctypes import byref, Structure, c_char, POINTER

    COORD = wintypes._COORD

    class CONSOLE_SCREEN_BUFFER_INFO(Structure):
        """struct in wincon.h."""
        _fields_ = [
            ("dwSize", COORD),
            ("dwCursorPosition", COORD),
            ("wAttributes", wintypes.WORD),
            ("srWindow", wintypes.SMALL_RECT),
            ("dwMaximumWindowSize", COORD),
        ]
        def __str__(self):
            return '(%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d)' % (
                self.dwSize.Y, self.dwSize.X
                , self.dwCursorPosition.Y, self.dwCursorPosition.X
                , self.wAttributes
                , self.srWindow.Top, self.srWindow.Left, self.srWindow.Bottom, self.srWindow.Right
                , self.dwMaximumWindowSize.Y, self.dwMaximumWindowSize.X
            )

    _GetStdHandle = windll.kernel32.GetStdHandle
    _GetStdHandle.argtypes = [
        wintypes.DWORD,
    ]
    _GetStdHandle.restype = wintypes.HANDLE
```

```
_GetConsoleScreenBufferInfo = windll.kernel32.GetConsoleScreenBufferInfo
_GetConsoleScreenBufferInfo.argtypes = [
    wintypes.HANDLE,
    POINTER(CONSOLE_SCREEN_BUFFER_INFO),
]
_GetConsoleScreenBufferInfo.restype = wintypes.BOOL

_SetConsoleTextAttribute = windll.kernel32.SetConsoleTextAttribute
_SetConsoleTextAttribute.argtypes = [
    wintypes.HANDLE,
    wintypes.WORD,
]
_SetConsoleTextAttribute.restype = wintypes.BOOL

_SetConsoleCursorPosition = windll.kernel32.SetConsoleCursorPosition
_SetConsoleCursorPosition.argtypes = [
    wintypes.HANDLE,
    COORD,
]
_SetConsoleCursorPosition.restype = wintypes.BOOL

_FillConsoleOutputCharacterA = windll.kernel32.FillConsoleOutputCharacterA
_FillConsoleOutputCharacterA.argtypes = [
    wintypes.HANDLE,
    c_char,
    wintypes.DWORD,
    COORD,
    POINTER(wintypes.DWORD),
]
_FillConsoleOutputCharacterA.restype = wintypes.BOOL

_FillConsoleOutputAttribute = windll.kernel32.FillConsoleOutputAttribute
_FillConsoleOutputAttribute.argtypes = [
    wintypes.HANDLE,
    wintypes.WORD,
    wintypes.DWORD,
    COORD,
    POINTER(wintypes.DWORD),
]
_FillConsoleOutputAttribute.restype = wintypes.BOOL

_SetConsoleTitleW = windll.kernel32.SetConsoleTitleW
_SetConsoleTitleW.argtypes = [
    wintypes.LPCWSTR
]
_SetConsoleTitleW.restype = wintypes.BOOL

_GetConsoleMode = windll.kernel32.GetConsoleMode
_GetConsoleMode.argtypes = [
    wintypes.HANDLE,
    POINTER(wintypes.DWORD)
]
_GetConsoleMode.restype = wintypes.BOOL

_SetConsoleMode = windll.kernel32.SetConsoleMode
_SetConsoleMode.argtypes = [
    wintypes.HANDLE,
    wintypes.DWORD
]
_SetConsoleMode.restype = wintypes.BOOL

def _winapi_test(handle):
    csbi = CONSOLE_SCREEN_BUFFER_INFO()
    success = _GetConsoleScreenBufferInfo(
        handle, byref(csbi))
    return bool(success)

def winapi_test():
    return any(_winapi_test(h) for h in
               (_GetStdHandle(STDOUT), _GetStdHandle(STDERR)))

def GetConsoleScreenBufferInfo(stream_id=STDOUT):
    handle = _GetStdHandle(stream_id)
    csbi = CONSOLE_SCREEN_BUFFER_INFO()
    success = _GetConsoleScreenBufferInfo(
        handle, byref(csbi))
    return csbi

def SetConsoleTextAttribute(stream_id, attrs):
    handle = _GetStdHandle(stream_id)
    return _SetConsoleTextAttribute(handle, attrs)
```

```
    def SetConsoleCursorPosition(stream_id, position, adjust=True):
        position = COORD(*position)
        # If the position is out of range, do nothing.
        if position.Y <= 0 or position.X <= 0:
            return
        # Adjust for Windows' SetConsoleCursorPosition:
        #    1. being 0-based, while ANSI is 1-based.
        #    2. expecting (x,y), while ANSI uses (y,x).
        adjusted_position = COORD(position.Y - 1, position.X - 1)
        if adjust:
            # Adjust for viewport's scroll position
            sr = GetConsoleScreenBufferInfo(STDOUT).srWindow
            adjusted_position.Y += sr.Top
            adjusted_position.X += sr.Left
        # Resume normal processing
        handle = _GetStdHandle(stream_id)
        return _SetConsoleCursorPosition(handle, adjusted_position)

    def FillConsoleOutputCharacter(stream_id, char, length, start):
        handle = _GetStdHandle(stream_id)
        char = c_char(char.encode())
        length = wintypes.DWORD(length)
        num_written = wintypes.DWORD(0)
        # Note that this is hard-coded for ANSI (vs wide) bytes.
        success = _FillConsoleOutputCharacterA(
            handle, char, length, start, byref(num_written))
        return num_written.value

    def FillConsoleOutputAttribute(stream_id, attr, length, start):
        ''' FillConsoleOutputAttribute( hConsole, csbi.wAttributes, dwConSize, coordScreen, &cCharsWritten )'''
        handle = _GetStdHandle(stream_id)
        attribute = wintypes.WORD(attr)
        length = wintypes.DWORD(length)
        num_written = wintypes.DWORD(0)
        # Note that this is hard-coded for ANSI (vs wide) bytes.
        return _FillConsoleOutputAttribute(
            handle, attribute, length, start, byref(num_written))

    def SetConsoleTitle(title):
        return _SetConsoleTitleW(title)

    def GetConsoleMode(handle):
        mode = wintypes.DWORD()
        success = _GetConsoleMode(handle, byref(mode))
        if not success:
            raise ctypes.WinError()
        return mode.value

    def SetConsoleMode(handle, mode):
        success = _SetConsoleMode(handle, mode)
        if not success:
            raise ctypes.WinError()
```

File: .venv\Lib\site-packages\colorama\winterm.py

```
# Copyright Jonathan Hartley 2013. BSD 3-Clause license, see LICENSE file.
try:
    from msvcrt import get_osfhandle
except ImportError:
    def get_osfhandle(_):
        raise OSError("This isn't windows!")


from . import win32

# from wincon.h
class WinColor(object):
    BLACK   = 0
    BLUE    = 1
    GREEN   = 2
    CYAN    = 3
    RED     = 4
    MAGENTA = 5
    YELLOW  = 6
    GREY    = 7

# from wincon.h
class WinStyle(object):
    NORMAL              = 0x00 # dim text, dim background
```

```
    BRIGHT              = 0x08 # bright text, dim background
    BRIGHT_BACKGROUND   = 0x80 # dim text, bright background

class WinTerm(object):

    def __init__(self):
        self._default = win32.GetConsoleScreenBufferInfo(win32.STDOUT).wAttributes
        self.set_attrs(self._default)
        self._default_fore = self._fore
        self._default_back = self._back
        self._default_style = self._style
        # In order to emulate LIGHT_EX in windows, we borrow the BRIGHT style.
        # So that LIGHT_EX colors and BRIGHT style do not clobber each other,
        # we track them separately, since LIGHT_EX is overwritten by Fore/Back
        # and BRIGHT is overwritten by Style codes.
        self._light = 0

    def get_attrs(self):
        return self._fore + self._back * 16 + (self._style | self._light)

    def set_attrs(self, value):
        self._fore = value & 7
        self._back = (value >> 4) & 7
        self._style = value & (WinStyle.BRIGHT | WinStyle.BRIGHT_BACKGROUND)

    def reset_all(self, on_stderr=None):
        self.set_attrs(self._default)
        self.set_console(attrs=self._default)
        self._light = 0

    def fore(self, fore=None, light=False, on_stderr=False):
        if fore is None:
            fore = self._default_fore
        self._fore = fore
        # Emulate LIGHT_EX with BRIGHT Style
        if light:
            self._light |= WinStyle.BRIGHT
        else:
            self._light &= ~WinStyle.BRIGHT
        self.set_console(on_stderr=on_stderr)

    def back(self, back=None, light=False, on_stderr=False):
        if back is None:
            back = self._default_back
        self._back = back
        # Emulate LIGHT_EX with BRIGHT_BACKGROUND Style
        if light:
            self._light |= WinStyle.BRIGHT_BACKGROUND
        else:
            self._light &= ~WinStyle.BRIGHT_BACKGROUND
        self.set_console(on_stderr=on_stderr)

    def style(self, style=None, on_stderr=False):
        if style is None:
            style = self._default_style
        self._style = style
        self.set_console(on_stderr=on_stderr)

    def set_console(self, attrs=None, on_stderr=False):
        if attrs is None:
            attrs = self.get_attrs()
        handle = win32.STDOUT
        if on_stderr:
            handle = win32.STDERR
        win32.SetConsoleTextAttribute(handle, attrs)

    def get_position(self, handle):
        position = win32.GetConsoleScreenBufferInfo(handle).dwCursorPosition
        # Because Windows coordinates are 0-based,
        # and win32.SetConsoleCursorPosition expects 1-based.
        position.X += 1
        position.Y += 1
        return position

    def set_cursor_position(self, position=None, on_stderr=False):
        if position is None:
            # I'm not currently tracking the position, so there is no default.
            # position = self.get_position()
            return
        handle = win32.STDOUT
        if on_stderr:
```

```
            handle = win32.STDERR
        win32.SetConsoleCursorPosition(handle, position)

    def cursor_adjust(self, x, y, on_stderr=False):
        handle = win32.STDOUT
        if on_stderr:
            handle = win32.STDERR
        position = self.get_position(handle)
        adjusted_position = (position.Y + y, position.X + x)
        win32.SetConsoleCursorPosition(handle, adjusted_position, adjust=False)

    def erase_screen(self, mode=0, on_stderr=False):
        # 0 should clear from the cursor to the end of the screen.
        # 1 should clear from the cursor to the beginning of the screen.
        # 2 should clear the entire screen, and move cursor to (1,1)
        handle = win32.STDOUT
        if on_stderr:
            handle = win32.STDERR
        csbi = win32.GetConsoleScreenBufferInfo(handle)
        # get the number of character cells in the current buffer
        cells_in_screen = csbi.dwSize.X * csbi.dwSize.Y
        # get number of character cells before current cursor position
        cells_before_cursor = csbi.dwSize.X * csbi.dwCursorPosition.Y + csbi.dwCursorPosition.X
        if mode == 0:
            from_coord = csbi.dwCursorPosition
            cells_to_erase = cells_in_screen - cells_before_cursor
        elif mode == 1:
            from_coord = win32.COORD(0, 0)
            cells_to_erase = cells_before_cursor
        elif mode == 2:
            from_coord = win32.COORD(0, 0)
            cells_to_erase = cells_in_screen
        else:
            # invalid mode
            return
        # fill the entire screen with blanks
        win32.FillConsoleOutputCharacter(handle, ' ', cells_to_erase, from_coord)
        # now set the buffer's attributes accordingly
        win32.FillConsoleOutputAttribute(handle, self.get_attrs(), cells_to_erase, from_coord)
        if mode == 2:
            # put the cursor where needed
            win32.SetConsoleCursorPosition(handle, (1, 1))

    def erase_line(self, mode=0, on_stderr=False):
        # 0 should clear from the cursor to the end of the line.
        # 1 should clear from the cursor to the beginning of the line.
        # 2 should clear the entire line.
        handle = win32.STDOUT
        if on_stderr:
            handle = win32.STDERR
        csbi = win32.GetConsoleScreenBufferInfo(handle)
        if mode == 0:
            from_coord = csbi.dwCursorPosition
            cells_to_erase = csbi.dwSize.X - csbi.dwCursorPosition.X
        elif mode == 1:
            from_coord = win32.COORD(0, csbi.dwCursorPosition.Y)
            cells_to_erase = csbi.dwCursorPosition.X
        elif mode == 2:
            from_coord = win32.COORD(0, csbi.dwCursorPosition.Y)
            cells_to_erase = csbi.dwSize.X
        else:
            # invalid mode
            return
        # fill the entire screen with blanks
        win32.FillConsoleOutputCharacter(handle, ' ', cells_to_erase, from_coord)
        # now set the buffer's attributes accordingly
        win32.FillConsoleOutputAttribute(handle, self.get_attrs(), cells_to_erase, from_coord)

    def set_title(self, title):
        win32.SetConsoleTitle(title)


def enable_vt_processing(fd):
    if win32.windll is None or not win32.winapi_test():
        return False

    try:
        handle = get_osfhandle(fd)
        mode = win32.GetConsoleMode(handle)
        win32.SetConsoleMode(
            handle,
```

```
            mode | win32.ENABLE_VIRTUAL_TERMINAL_PROCESSING,
        )

        mode = win32.GetConsoleMode(handle)
        if mode & win32.ENABLE_VIRTUAL_TERMINAL_PROCESSING:
            return True
    # Can get TypeError in testsuite where 'fd' is a Mock()
    except (OSError, TypeError):
        return False
```

File: .venv\Lib\site-packages\colorama__init__.py

```
# Copyright Jonathan Hartley 2013. BSD 3-Clause license, see LICENSE file.
from .initialise import init, deinit, reinit, colorama_text, just_fix_windows_console
from .ansi import Fore, Back, Style, Cursor
from .ansitowin32 import AnsiToWin32

__version__ = '0.4.6'
```

File: .venv\Lib\site-packages\colorama-0.4.6.dist-info\licenses\LICENSE.txt

```
Copyright (c) 2010 Jonathan Hartley
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

* Redistributions of source code must retain the above copyright notice, this
  list of conditions and the following disclaimer.

* Redistributions in binary form must reproduce the above copyright notice,
  this list of conditions and the following disclaimer in the documentation
  and/or other materials provided with the distribution.

* Neither the name of the copyright holders, nor those of its contributors
  may be used to endorse or promote products derived from this software without
  specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY,
OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
```

File: .venv\Lib\site-packages\docutils\core.py

```
# $Id: core.py 9369 2023-05-02 23:04:27Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
Calling the ``publish_*`` convenience functions (or instantiating a
`Publisher` object) with component names will result in default
behavior.  For custom behavior (setting component options), create
custom component objects first, and pass *them* to
``publish_*``/`Publisher`.  See `The Docutils Publisher`_.

.. _The Docutils Publisher:
    https://docutils.sourceforge.io/docs/api/publisher.html
"""

__docformat__ = 'reStructuredText'

import locale
import pprint
import os
import sys
import warnings

from docutils import (__version__, __version_details__, SettingsSpec,
                      io, utils, readers, writers)
from docutils.frontend import OptionParser
```

```
from docutils.readers import doctree


class Publisher:

    """
    A facade encapsulating the high-level logic of a Docutils system.
    """

    def __init__(self, reader=None, parser=None, writer=None,
                 source=None, source_class=io.FileInput,
                 destination=None, destination_class=io.FileOutput,
                 settings=None):
        """
        Initial setup.  If any of `reader`, `parser`, or `writer` are not
        specified, ``set_components()`` or the corresponding ``set_...()``
        method should be called with component names
        (`set_reader` sets the parser as well).
        """

        self.document = None
        """The document tree (`docutils.nodes` objects)."""

        self.reader = reader
        """A `docutils.readers.Reader` instance."""

        self.parser = parser
        """A `docutils.parsers.Parser` instance."""

        self.writer = writer
        """A `docutils.writers.Writer` instance."""

        for component in 'reader', 'parser', 'writer':
            assert not isinstance(getattr(self, component), str), (
                'passed string "%s" as "%s" parameter; pass an instance, '
                'or use the "%s_name" parameter instead (in '
                'docutils.core.publish_* convenience functions).'
                % (getattr(self, component), component, component))

        self.source = source
        """The source of input data, a `docutils.io.Input` instance."""

        self.source_class = source_class
        """The class for dynamically created source objects."""

        self.destination = destination
        """The destination for docutils output, a `docutils.io.Output`
        instance."""

        self.destination_class = destination_class
        """The class for dynamically created destination objects."""

        self.settings = settings
        """An object containing Docutils settings as instance attributes.
        Set by `self.process_command_line()` or `self.get_settings()`."""

        self._stderr = io.ErrorOutput()

    def set_reader(self, reader_name, parser, parser_name):
        """Set `self.reader` by name."""
        reader_class = readers.get_reader_class(reader_name)
        self.reader = reader_class(parser, parser_name)
        self.parser = self.reader.parser

    def set_writer(self, writer_name):
        """Set `self.writer` by name."""
        writer_class = writers.get_writer_class(writer_name)
        self.writer = writer_class()

    def set_components(self, reader_name, parser_name, writer_name):
        if self.reader is None:
            self.set_reader(reader_name, self.parser, parser_name)
        if self.parser is None:
            if self.reader.parser is None:
                self.reader.set_parser(parser_name)
            self.parser = self.reader.parser
        if self.writer is None:
            self.set_writer(writer_name)

    def setup_option_parser(self, usage=None, description=None,
                            settings_spec=None, config_section=None,
```

```
                        **defaults):
    warnings.warn('Publisher.setup_option_parser is deprecated, '
                  'and will be removed in Docutils 0.21.',
                  DeprecationWarning, stacklevel=2)
    if config_section:
        if not settings_spec:
            settings_spec = SettingsSpec()
        settings_spec.config_section = config_section
        parts = config_section.split()
        if len(parts) > 1 and parts[-1] == 'application':
            settings_spec.config_section_dependencies = ['applications']
    # @@@ Add self.source & self.destination to components in future?
    return OptionParser(
        components=(self.parser, self.reader, self.writer, settings_spec),
        defaults=defaults, read_config_files=True,
        usage=usage, description=description)

def _setup_settings_parser(self, *args, **kwargs):
    # Provisional: will change (docutils.frontend.OptionParser will
    # be replaced by a parser based on arparse.ArgumentParser)
    # and may be removed later.
    with warnings.catch_warnings():
        warnings.filterwarnings('ignore', category=DeprecationWarning)
        return self.setup_option_parser(*args, **kwargs)

def get_settings(self, usage=None, description=None,
                 settings_spec=None, config_section=None, **defaults):
    """
    Return settings from components and config files.

    Please set components first (`self.set_reader` & `self.set_writer`).
    Use keyword arguments to override component defaults
    (before updating from configuration files).

    Calling this function also sets `self.settings` which makes
    `self.publish()` skip parsing command line options.
    """
    option_parser = self._setup_settings_parser(
        usage, description, settings_spec, config_section, **defaults)
    self.settings = option_parser.get_default_values()
    return self.settings

def process_programmatic_settings(self, settings_spec,
                                  settings_overrides,
                                  config_section):
    if self.settings is None:
        defaults = settings_overrides.copy() if settings_overrides else {}
        # Propagate exceptions by default when used programmatically:
        defaults.setdefault('traceback', True)
        self.get_settings(settings_spec=settings_spec,
                          config_section=config_section,
                          **defaults)

def process_command_line(self, argv=None, usage=None, description=None,
                         settings_spec=None, config_section=None,
                         **defaults):
    """
    Parse command line arguments and set ``self.settings``.

    Pass an empty sequence to `argv` to avoid reading `sys.argv`
    (the default behaviour).

    Set components first (`self.set_reader` & `self.set_writer`).
    """
    option_parser = self._setup_settings_parser(
        usage, description, settings_spec, config_section, **defaults)
    if argv is None:
        argv = sys.argv[1:]
    self.settings = option_parser.parse_args(argv)

def set_io(self, source_path=None, destination_path=None):
    if self.source is None:
        self.set_source(source_path=source_path)
    if self.destination is None:
        self.set_destination(destination_path=destination_path)

def set_source(self, source=None, source_path=None):
    if source_path is None:
        source_path = self.settings._source
    else:
        self.settings._source = source_path
```

```
        self.source = self.source_class(
            source=source, source_path=source_path,
            encoding=self.settings.input_encoding,
            error_handler=self.settings.input_encoding_error_handler)

    def set_destination(self, destination=None, destination_path=None):
        if destination_path is None:
            if (self.settings.output and self.settings._destination
                and self.settings.output != self.settings._destination):
                raise SystemExit('The positional argument <destination> is '
                                 'obsoleted by the --output option.  '
                                 'You cannot use them together.')
            if self.settings.output == '-':      # means stdout
                self.settings.output = None
            destination_path = (self.settings.output
                                or self.settings._destination)
        self.settings._destination = destination_path
        self.destination = self.destination_class(
            destination=destination,
            destination_path=destination_path,
            encoding=self.settings.output_encoding,
            error_handler=self.settings.output_encoding_error_handler)

    def apply_transforms(self):
        self.document.transformer.populate_from_components(
            (self.source, self.reader, self.reader.parser, self.writer,
             self.destination))
        self.document.transformer.apply_transforms()

    def publish(self, argv=None, usage=None, description=None,
                settings_spec=None, settings_overrides=None,
                config_section=None, enable_exit_status=False):
        """
        Process command line options and arguments (if `self.settings` not
        already set), run `self.reader` and then `self.writer`.  Return
        `self.writer`'s output.
        """
        exit = None
        try:
            if self.settings is None:
                self.process_command_line(
                    argv, usage, description, settings_spec, config_section,
                    **(settings_overrides or {}))
            self.set_io()
            self.prompt()
            self.document = self.reader.read(self.source, self.parser,
                                             self.settings)
            self.apply_transforms()
            output = self.writer.write(self.document, self.destination)
            self.writer.assemble_parts()
        except SystemExit as error:
            exit = True
            exit_status = error.code
        except Exception as error:
            if not self.settings:       # exception too early to report nicely
                raise
            if self.settings.traceback:  # Propagate exceptions?
                self.debugging_dumps()
                raise
            self.report_Exception(error)
            exit = True
            exit_status = 1
        self.debugging_dumps()
        if (enable_exit_status and self.document
            and (self.document.reporter.max_level
                 >= self.settings.exit_status_level)):
            sys.exit(self.document.reporter.max_level + 10)
        elif exit:
            sys.exit(exit_status)
        return output

    def debugging_dumps(self):
        if not self.document:
            return
        if self.settings.dump_settings:
            print('\n::: Runtime settings:', file=self._stderr)
            print(pprint.pformat(self.settings.__dict__), file=self._stderr)
        if self.settings.dump_internals:
            print('\n::: Document internals:', file=self._stderr)
            print(pprint.pformat(self.document.__dict__), file=self._stderr)
        if self.settings.dump_transforms:
```

```
            print('\n::: Transforms applied:', file=self._stderr)
            print(' (priority, transform class, pending node details, '
                  'keyword args)', file=self._stderr)
            print(pprint.pformat(
                [(priority, '%s.%s' % (xclass.__module__, xclass.__name__),
                  pending and pending.details, kwargs)
                 for priority, xclass, pending, kwargs
                 in self.document.transformer.applied]), file=self._stderr)
        if self.settings.dump_pseudo_xml:
            print('\n::: Pseudo-XML:', file=self._stderr)
            print(self.document.pformat().encode(
                'raw_unicode_escape'), file=self._stderr)

    def prompt(self):
        """Print info and prompt when waiting for input from a terminal."""
        try:
            if not (self.source.isatty() and self._stderr.isatty()):
                return
        except AttributeError:
            return
        eot_key = 'Ctrl+Z' if os.name == 'nt' else 'Ctrl+D'
        in_format = ''
        out_format = 'useful formats'
        try:
            in_format = self.parser.supported[0]
            out_format = self.writer.supported[0]
        except (AttributeError, IndexError):
            pass
        print(f'Docutils {__version__} <https://docutils.sourceforge.io>\n'
              f'converting "{in_format}" into "{out_format}".\n'
              f'Call with option "--help" for more info.\n'
              f'.. Waiting for source text (finish with {eot_key} '
              'on an empty line):',
              file=self._stderr)

    def report_Exception(self, error):
        if isinstance(error, utils.SystemMessage):
            self.report_SystemMessage(error)
        elif isinstance(error, UnicodeEncodeError):
            self.report_UnicodeError(error)
        elif isinstance(error, io.InputError):
            self._stderr.write('Unable to open source file for reading:\n'
                               '  %s\n' % io.error_string(error))
        elif isinstance(error, io.OutputError):
            self._stderr.write(
                'Unable to open destination file for writing:\n'
                '  %s\n' % io.error_string(error))
        else:
            print('%s' % io.error_string(error), file=self._stderr)
            print(f"""\
Exiting due to error.  Use "--traceback" to diagnose.
Please report errors to <docutils-users@lists.sourceforge.net>.
Include "--traceback" output, Docutils version ({__version__}\
{f' [{__version_details__}]' if __version_details__ else ''}),
Python version ({sys.version.split()[0]}), your OS type & version, \
and the command line used.""", file=self._stderr)

    def report_SystemMessage(self, error):
        print('Exiting due to level-%s (%s) system message.' % (
                  error.level, utils.Reporter.levels[error.level]),
              file=self._stderr)

    def report_UnicodeError(self, error):
        data = error.object[error.start:error.end]
        self._stderr.write(
            '%s\n'
            '\n'
            'The specified output encoding (%s) cannot\n'
            'handle all of the output.\n'
            'Try setting "--output-encoding-error-handler" to\n'
            '\n'
            '* "xmlcharrefreplace" (for HTML & XML output);\n'
            '  the output will contain "%s" and should be usable.\n'
            '* "backslashreplace" (for other output formats);\n'
            '  look for "%s" in the output.\n'
            '* "replace"; look for "?" in the output.\n'
            '\n'
            '"--output-encoding-error-handler" is currently set to "%s".\n'
            '\n'
            'Exiting due to error.  Use "--traceback" to diagnose.\n'
            'If the advice above doesn\'t eliminate the error,\n'
```

```
            'please report it to <docutils-users@lists.sourceforge.net>.\n'
            'Include "--traceback" output, Docutils version (%s),\n'
            'Python version (%s), your OS type & version, and the\n'
            'command line used.\n'
            % (io.error_string(error),
               self.settings.output_encoding,
               data.encode('ascii', 'xmlcharrefreplace'),
               data.encode('ascii', 'backslashreplace'),
               self.settings.output_encoding_error_handler,
               __version__, sys.version.split()[0]))


default_usage = '%prog [options] [<source> [<destination>]]'
default_description = (
    'Reads from <source> (default is stdin) '
    'and writes to <destination> (default is stdout).  '
    'See https://docutils.sourceforge.io/docs/user/config.html '
    'for a detailed settings reference.')


# TODO: or not to do?  cf. https://clig.dev/#help
#
# Display output on success, but keep it brief.
# Provide a -q option to suppress all non-essential output.
#
# Chain several args as input and use --output or redirection for output:
#   argparser.add_argument('source', nargs='+')
#
def publish_cmdline(reader=None, reader_name='standalone',
                    parser=None, parser_name='restructuredtext',
                    writer=None, writer_name='pseudoxml',
                    settings=None, settings_spec=None,
                    settings_overrides=None, config_section=None,
                    enable_exit_status=True, argv=None,
                    usage=default_usage, description=default_description):
    """
    Set up & run a `Publisher` for command-line-based file I/O (input and
    output file paths taken automatically from the command line).
    Also return the output as `str` or `bytes` (for binary output document
    formats).

    Parameters: see `publish_programmatically()` for the remainder.

    - `argv`: Command-line argument list to use instead of ``sys.argv[1:]``.
    - `usage`: Usage string, output if there's a problem parsing the command
      line.
    - `description`: Program description, output for the "--help" option
      (along with command-line option descriptions).
    """
    publisher = Publisher(reader, parser, writer, settings=settings)
    publisher.set_components(reader_name, parser_name, writer_name)
    output = publisher.publish(
        argv, usage, description, settings_spec, settings_overrides,
        config_section=config_section, enable_exit_status=enable_exit_status)
    return output


def publish_file(source=None, source_path=None,
                 destination=None, destination_path=None,
                 reader=None, reader_name='standalone',
                 parser=None, parser_name='restructuredtext',
                 writer=None, writer_name='pseudoxml',
                 settings=None, settings_spec=None, settings_overrides=None,
                 config_section=None, enable_exit_status=False):
    """
    Set up & run a `Publisher` for programmatic use with file-like I/O.
    Also return the output as `str` or `bytes` (for binary output document
    formats).

    Parameters: see `publish_programmatically()`.
    """
    output, publisher = publish_programmatically(
        source_class=io.FileInput, source=source, source_path=source_path,
        destination_class=io.FileOutput,
        destination=destination, destination_path=destination_path,
        reader=reader, reader_name=reader_name,
        parser=parser, parser_name=parser_name,
        writer=writer, writer_name=writer_name,
        settings=settings, settings_spec=settings_spec,
        settings_overrides=settings_overrides,
        config_section=config_section,
```

```
        enable_exit_status=enable_exit_status)
    return output


def publish_string(source, source_path=None, destination_path=None,
                   reader=None, reader_name='standalone',
                   parser=None, parser_name='restructuredtext',
                   writer=None, writer_name='pseudoxml',
                   settings=None, settings_spec=None,
                   settings_overrides=None, config_section=None,
                   enable_exit_status=False):
    """
    Set up & run a `Publisher` for programmatic use with string I/O.

    Accepts a `bytes` or `str` instance as `source`.

    The output is encoded according to the `output_encoding`_ setting;
    the return value is a `bytes` instance (unless `output_encoding`_ is
    "unicode", cf. `docutils.io.StringOutput.write()`).

    Parameters: see `publish_programmatically()`.

    This function is provisional because in Python 3 name and behaviour
    no longer match.

    .. _output_encoding:
        https://docutils.sourceforge.io/docs/user/config.html#output-encoding
    """
    output, publisher = publish_programmatically(
        source_class=io.StringInput, source=source, source_path=source_path,
        destination_class=io.StringOutput,
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\docutils\examples.py

```
# $Id: examples.py 9026 2022-03-04 15:57:13Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
This module contains practical examples of Docutils client code.

Importing this module from client code is not recommended; its contents are
subject to change in future Docutils releases.  Instead, it is recommended
that you copy and paste the parts you need into your own code, modifying as
necessary.
"""

from docutils import core, io


def html_parts(input_string, source_path=None, destination_path=None,
               input_encoding='unicode', doctitle=True,
               initial_header_level=1):
    """
    Given an input string, returns a dictionary of HTML document parts.

    Dictionary keys are the names of parts, and values are Unicode strings;
    encoding is up to the client.

    Parameters:

    - `input_string`: A multi-line text string; required.
    - `source_path`: Path to the source file or object.  Optional, but useful
      for diagnostic output (system messages).
    - `destination_path`: Path to the file or object which will receive the
      output; optional.  Used for determining relative paths (stylesheets,
      source links, etc.).
    - `input_encoding`: The encoding of `input_string`.  If it is an encoded
      8-bit string, provide the correct encoding.  If it is a Unicode string,
      use "unicode", the default.
    - `doctitle`: Disable the promotion of a lone top-level section title to
      document title (and subsequent section title to document subtitle
      promotion); enabled by default.
    - `initial_header_level`: The initial level for header elements (e.g. 1
      for "<h1>").
    """
    overrides = {'input_encoding': input_encoding,
                 'doctitle_xform': doctitle,
```

```
                 'initial_header_level': initial_header_level}
    parts = core.publish_parts(
        source=input_string, source_path=source_path,
        destination_path=destination_path,
        writer_name='html', settings_overrides=overrides)
    return parts


def html_body(input_string, source_path=None, destination_path=None,
              input_encoding='unicode', output_encoding='unicode',
              doctitle=True, initial_header_level=1):
    """
    Given an input string, returns an HTML fragment as a string.

    The return value is the contents of the <body> element.

    Parameters (see `html_parts()` for the remainder):

    - `output_encoding`: The desired encoding of the output.  If a Unicode
      string is desired, use the default value of "unicode" .
    """
    parts = html_parts(
        input_string=input_string, source_path=source_path,
        destination_path=destination_path,
        input_encoding=input_encoding, doctitle=doctitle,
        initial_header_level=initial_header_level)
    fragment = parts['html_body']
    if output_encoding != 'unicode':
        fragment = fragment.encode(output_encoding)
    return fragment


def internals(input_string, source_path=None, destination_path=None,
              input_encoding='unicode', settings_overrides=None):
    """
    Return the document tree and publisher, for exploring Docutils internals.

    Parameters: see `html_parts()`.
    """
    if settings_overrides:
        overrides = settings_overrides.copy()
    else:
        overrides = {}
    overrides['input_encoding'] = input_encoding
    output, pub = core.publish_programmatically(
        source_class=io.StringInput, source=input_string,
        source_path=source_path,
        destination_class=io.NullOutput, destination=None,
        destination_path=destination_path,
        reader=None, reader_name='standalone',
        parser=None, parser_name='restructuredtext',
        writer=None, writer_name='null',
        settings=None, settings_spec=None, settings_overrides=overrides,
        config_section=None, enable_exit_status=None)
    return pub.writer.document, pub
```

File: .venv\Lib\site-packages\docutils\frontend.py

```
# $Id: frontend.py 9328 2023-02-07 14:24:37Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
Command-line and common processing for Docutils front-end tools.

This module is provisional.
Major changes will happen with the switch from the deprecated
"optparse" module to "arparse".

Applications should use the high-level API provided by `docutils.core`.
See https://docutils.sourceforge.io/docs/api/runtime-settings.html.

Exports the following classes:

* `OptionParser`: Standard Docutils command-line processing.
  Deprecated. Will be replaced by an ArgumentParser.
* `Option`: Customized version of `optparse.Option`; validation support.
  Deprecated. Will be removed.
* `Values`: Runtime settings; objects are simple structs
```

```
  (``object.attribute``).  Supports cumulative list settings (attributes).
  Deprecated. Will be removed.
* `ConfigParser`: Standard Docutils config file processing.
  Provisional. Details will change.

Also exports the following functions:

Interface function:
   `get_default_settings()`.  New in 0.19.

Option callbacks:
   `store_multiple()`, `read_config_file()`. Deprecated.

Setting validators:
  `validate_encoding()`, `validate_encoding_error_handler()`,
  `validate_encoding_and_error_handler()`,
  `validate_boolean()`, `validate_ternary()`,
  `validate_nonnegative_int()`, `validate_threshold()`,
  `validate_colon_separated_string_list()`,
  `validate_comma_separated_list()`,
  `validate_url_trailing_slash()`,
  `validate_dependency_file()`,
  `validate_strip_class()`
  `validate_smartquotes_locales()`.

  Provisional.

Misc:
  `make_paths_absolute()`, `filter_settings_spec()`. Provisional.
"""

__docformat__ = 'reStructuredText'


import codecs
import configparser
import optparse
from optparse import SUPPRESS_HELP
import os
import os.path
import sys
import warnings

import docutils
from docutils import io, utils


def store_multiple(option, opt, value, parser, *args, **kwargs):
    """
    Store multiple values in `parser.values`.  (Option callback.)

    Store `None` for each attribute named in `args`, and store the value for
    each key (attribute name) in `kwargs`.
    """
    for attribute in args:
        setattr(parser.values, attribute, None)
    for key, value in kwargs.items():
        setattr(parser.values, key, value)


def read_config_file(option, opt, value, parser):
    """
    Read a configuration file during option processing.  (Option callback.)
    """
    try:
        new_settings = parser.get_config_file_settings(value)
    except ValueError as err:
        parser.error(err)
    parser.values.update(new_settings, parser)


def validate_encoding(setting, value, option_parser,
                      config_parser=None, config_section=None):
    if value == '':
        return None  # allow overwriting a config file value
    try:
        codecs.lookup(value)
    except LookupError:
        raise LookupError('setting "%s": unknown encoding: "%s"'
                          % (setting, value))
    return value
```

```
def validate_encoding_error_handler(setting, value, option_parser,
                                    config_parser=None, config_section=None):
    try:
        codecs.lookup_error(value)
    except LookupError:
        raise LookupError(
            'unknown encoding error handler: "%s" (choices: '
            '"strict", "ignore", "replace", "backslashreplace", '
            '"xmlcharrefreplace", and possibly others; see documentation for '
            'the Python ``codecs`` module)' % value)
    return value


def validate_encoding_and_error_handler(
    setting, value, option_parser, config_parser=None, config_section=None):
    """
    Side-effect: if an error handler is included in the value, it is inserted
    into the appropriate place as if it was a separate setting/option.
    """
    if ':' in value:
        encoding, handler = value.split(':')
        validate_encoding_error_handler(
            setting + '_error_handler', handler, option_parser,
            config_parser, config_section)
        if config_parser:
            config_parser.set(config_section, setting + '_error_handler',
                              handler)
        else:
            setattr(option_parser.values, setting + '_error_handler', handler)
    else:
        encoding = value
    validate_encoding(setting, encoding, option_parser,
                      config_parser, config_section)
    return encoding


def validate_boolean(setting, value, option_parser,
                     config_parser=None, config_section=None):
    """Check/normalize boolean settings:
         True:  '1', 'on', 'yes', 'true'
         False: '0', 'off', 'no','false', ''
    """
    if isinstance(value, bool):
        return value
    try:
        return option_parser.booleans[value.strip().lower()]
    except KeyError:
        raise LookupError('unknown boolean value: "%s"' % value)


def validate_ternary(setting, value, option_parser,
                     config_parser=None, config_section=None):
    """Check/normalize three-value settings:
         True:  '1', 'on', 'yes', 'true'
         False: '0', 'off', 'no','false', ''
         any other value: returned as-is.
    """
    if isinstance(value, bool) or value is None:
        return value
    try:
        return option_parser.booleans[value.strip().lower()]
    except KeyError:
        return value


def validate_nonnegative_int(setting, value, option_parser,
                             config_parser=None, config_section=None):
    value = int(value)
    if value < 0:
        raise ValueError('negative value; must be positive or zero')
    return value


def validate_threshold(setting, value, option_parser,
                       config_parser=None, config_section=None):
    try:
        return int(value)
    except ValueError:
        try:
            return option_parser.thresholds[value.lower()]
        except (KeyError, AttributeError):
```

```
            raise LookupError('unknown threshold: %r.' % value)


def validate_colon_separated_string_list(
    setting, value, option_parser, config_parser=None, config_section=None):
    if not isinstance(value, list):
        value = value.split(':')
    else:
        last = value.pop()
        value.extend(last.split(':'))
    return value


def validate_comma_separated_list(setting, value, option_parser,
                                  config_parser=None, config_section=None):
    """Check/normalize list arguments (split at "," and strip whitespace).
    """
    # `value` may be ``bytes``, ``str``, or a ``list`` (when  given as
    # command line option and "action" is "append").
    if not isinstance(value, list):
        value = [value]
    # this function is called for every option added to `value`
    # -> split the last item and append the result:
    last = value.pop()
    items = [i.strip(' \t\n') for i in last.split(',') if i.strip(' \t\n')]
    value.extend(items)
    return value


def validate_url_trailing_slash(
    setting, value, option_parser, config_parser=None, config_section=None):
    if not value:
        return './'
    elif value.endswith('/'):
        return value
    else:
        return value + '/'


def validate_dependency_file(setting, value, option_parser,
                             config_parser=None, config_section=None):
    try:
        return utils.DependencyList(value)
    except OSError:
        # TODO: warn/info?
        return utils.DependencyList(None)


def validate_strip_class(setting, value, option_parser,
                         config_parser=None, config_section=None):
    # value is a comma separated string list:
    value = validate_comma_separated_list(setting, value, option_parser,
                                          config_parser, config_section)
    # validate list elements:
    for cls in value:
        normalized = docutils.nodes.make_id(cls)
        if cls != normalized:
            raise ValueError('Invalid class value %r (perhaps %r?)'
                             % (cls, normalized))
    return value


def validate_smartquotes_locales(setting, value, option_parser,
                                 config_parser=None, config_section=None):
    """Check/normalize a comma separated list of smart quote definitions.

    Return a list of (language-tag, quotes) string tuples."""

    # value is a comma separated string list:
    value = validate_comma_separated_list(setting, value, option_parser,
                                          config_parser, config_section)
    # validate list elements
    lc_quotes = []
    for item in value:
        try:
            lang, quotes = item.split(':', 1)
        except AttributeError:
            # this function is called for every option added to `value`
            # -> ignore if already a tuple:
            lc_quotes.append(item)
            continue
```

```
        except ValueError:
            raise ValueError('Invalid value "%s".'
                             ' Format is "<language>:<quotes>".'
                             % item.encode('ascii', 'backslashreplace'))
        # parse colon separated string list:
        quotes = quotes.strip()
        multichar_quotes = quotes.split(':')
        if len(multichar_quotes) == 4:
            quotes = multichar_quotes
        elif len(quotes) != 4:
            raise ValueError('Invalid value "%s". Please specify 4 quotes\n'
                             '    (primary open/close; secondary open/close).'
                             % item.encode('ascii', 'backslashreplace'))
        lc_quotes.append((lang, quotes))
    return lc_quotes


def make_paths_absolute(pathdict, keys, base_path=None):
    """
    Interpret filesystem path settings relative to the `base_path` given.

    Paths are values in `pathdict` whose keys are in `keys`.  Get `keys` from
    `OptionParser.relative_path_settings`.
    """
    if base_path is None:
        base_path = os.getcwd()
    for key in keys:
        if key in pathdict:
            value = pathdict[key]
            if isinstance(value, list):
                value = [make_one_path_absolute(base_path, path)
                         for path in value]
            elif value:
                value = make_one_path_absolute(base_path, value)
            pathdict[key] = value


def make_one_path_absolute(base_path, path):
    return os.path.abspath(os.path.join(base_path, path))


def filter_settings_spec(settings_spec, *exclude, **replace):
    """Return a copy of `settings_spec` excluding/replacing some settings.

    `settings_spec` is a tuple of configuration settings
    (cf. `docutils.SettingsSpec.settings_spec`).

    Optional positional arguments are names of to-be-excluded settings.
    Keyword arguments are option specification replacements.
    (See the html4strict writer for an example.)
    """
    settings = list(settings_spec)
    # every third item is a sequence of option tuples
    for i in range(2, len(settings), 3):
        newopts = []
        for opt_spec in settings[i]:
            # opt_spec is ("<help>", [<option strings>], {<keyword args>})
            opt_name = [opt_string[2:].replace('-', '_')
                        for opt_string in opt_spec[1]
                        if opt_string.startswith('--')][0]
            if opt_name in exclude:
                continue
            if opt_name in replace.keys():
                newopts.append(replace[opt_name])
            else:
                newopts.append(opt_spec)
        settings[i] = tuple(newopts)
    return tuple(settings)


class Values(optparse.Values):
    """Storage for option values.

    Updates list attributes by extension rather than by replacement.
    Works in conjunction with the `OptionParser.lists` instance attribute.

    Deprecated. Will be removed.
    """

    def __init__(self, *args, **kwargs):
        warnings.warn('frontend.Values class will be removed '
```

```
                      'in Docutils 0.21 or later.',
                      DeprecationWarning, stacklevel=2)
        super().__init__(*args, **kwargs)
        if getattr(self, 'record_dependencies', None) is None:
            # Set up dummy dependency list.
            self.record_dependencies = utils.DependencyList()

    def update(self, other_dict, option_parser):
        if isinstance(other_dict, Values):
            other_dict = other_dict.__dict__
        other_dict = dict(other_dict)  # also works with ConfigParser sections
        for setting in option_parser.lists.keys():
            if hasattr(self, setting) and setting in other_dict:
                value = getattr(self, setting)
                if value:
                    value += other_dict[setting]
                    del other_dict[setting]
        self._update_loose(other_dict)

    def copy(self):
        """Return a shallow copy of `self`."""
        with warnings.catch_warnings():
            warnings.filterwarnings('ignore', category=DeprecationWarning)
            return self.__class__(defaults=self.__dict__)

    def setdefault(self, name, default):
        """Return ``self.name`` or ``default``.

        If ``self.name`` is unset, set ``self.name = default``.
        """
        if getattr(self, name, None) is None:
            setattr(self, name, default)
        return getattr(self, name)


class Option(optparse.Option):
    """Add validation and override support to `optparse.Option`.

    Deprecated. Will be removed.
    """

    ATTRS = optparse.Option.ATTRS + ['validator', 'overrides']

    def __init__(self, *args, **kwargs):
        warnings.warn('The frontend.Option class will be removed '
                      'in Docutils 0.21 or later.',
                      DeprecationWarning, stacklevel=2)
        super().__init__(*args, **kwargs)

    def process(self, opt, value, values, parser):
        """
        Call the validator function on applicable settings and
        evaluate the 'overrides' option.
        Extends `optparse.Option.process`.
        """
        result = super().process(opt, value, values, parser)
        setting = self.dest
        if setting:
            if self.validator:
                value = getattr(values, setting)
                try:
                    new_value = self.validator(setting, value, parser)
                except Exception as err:
                    raise optparse.OptionValueError(
                        'Error in option "%s":\n    %s'
                       % (opt, io.error_string(err)))
                setattr(values, setting, new_value)
            if self.overrides:
                setattr(values, self.overrides, None)
        return result


class OptionParser(optparse.OptionParser, docutils.SettingsSpec):
    """
    Settings parser for command-line and library use.

    The `settings_spec` specification here and in other Docutils components
    are merged to build the set of command-line options and runtime settings
    for this process.

    Common settings (defined below) and component-specific settings must not
```

```
conflict.  Short options are reserved for common settings, and components
are restricted to using long options.

Deprecated.
Will be replaced by a subclass of `argparse.ArgumentParser`.
"""


standard_config_files = [
    '/etc/docutils.conf',           # system-wide
    './docutils.conf',              # project-specific
    '~/.docutils']                  # user-specific
"""Docutils configuration files, using ConfigParser syntax.

Filenames will be tilde-expanded later. Later files override earlier ones.
"""


threshold_choices = 'info 1 warning 2 error 3 severe 4 none 5'.split()
"""Possible inputs for for --report and --halt threshold values."""

thresholds = {'info': 1, 'warning': 2, 'error': 3, 'severe': 4, 'none': 5}
"""Lookup table for --report and --halt threshold values."""

booleans = {'1': True, 'on': True, 'yes': True, 'true': True, '0': False,
            'off': False, 'no': False, 'false': False, '': False}
"""Lookup table for boolean configuration file settings."""

default_error_encoding = (getattr(sys.stderr, 'encoding', None)
                          or io._locale_encoding  # noqa
                          or 'ascii')

default_error_encoding_error_handler = 'backslashreplace'

settings_spec = (
    'General Docutils Options',
    None,
    (('Output destination name. Obsoletes the <destination> '
      'positional argument. Default: None (stdout).',
      ['--output'], {'metavar': '<destination>'}),
     ('Specify the document title as metadata.',
      ['--title'], {'metavar': '<title>'}),
     ('Include a "Generated by Docutils" credit and link.',
      ['--generator', '-g'], {'action': 'store_true',
                              'validator': validate_boolean}),
     ('Do not include a generator credit.',
      ['--no-generator'], {'action': 'store_false', 'dest': 'generator'}),
     ('Include the date at the end of the document (UTC).',
      ['--date', '-d'], {'action': 'store_const', 'const': '%Y-%m-%d',
                         'dest': 'datestamp'}),
     ('Include the time & date (UTC).',
      ['--time', '-t'], {'action': 'store_const',
                         'const': '%Y-%m-%d %H:%M UTC',
                         'dest': 'datestamp'}),
     ('Do not include a datestamp of any kind.',
      ['--no-datestamp'], {'action': 'store_const', 'const': None,
                           'dest': 'datestamp'}),
     ('Include a "View document source" link.',
      ['--source-link', '-s'], {'action': 'store_true',
                                'validator': validate_boolean}),
     ('Use <URL> for a source link; implies --source-link.',
      ['--source-url'], {'metavar': '<URL>'}),
     ('Do not include a "View document source" link.',
      ['--no-source-link'],
      {'action': 'callback', 'callback': store_multiple,
       'callback_args': ('source_link', 'source_url')}),
     ('Link from section headers to TOC entries.  (default)',
      ['--toc-entry-backlinks'],
      {'dest': 'toc_backlinks', 'action': 'store_const', 'const': 'entry',
       'default': 'entry'}),
     ('Link from section headers to the top of the TOC.',
      ['--toc-top-backlinks'],
      {'dest': 'toc_backlinks', 'action': 'store_const', 'const': 'top'}),
     ('Disable backlinks to the table of contents.',
      ['--no-toc-backlinks'],
      {'dest': 'toc_backlinks', 'action': 'store_false'}),
     ('Link from footnotes/citations to references. (default)',
      ['--footnote-backlinks'],
      {'action': 'store_true', 'default': 1,
       'validator': validate_boolean}),
     ('Disable backlinks from footnotes and citations.',
      ['--no-footnote-backlinks'],
      {'dest': 'footnote_backlinks', 'action': 'store_false'}),
```

```
          ('Enable section numbering by Docutils.  (default)',
           ['--section-numbering'],
           {'action': 'store_true', 'dest': 'sectnum_xform',
            'default': 1, 'validator': validate_boolean}),
          ('Disable section numbering by Docutils.',
           ['--no-section-numbering'],
           {'action': 'store_false', 'dest': 'sectnum_xform'}),
          ('Remove comment elements from the document tree.',
           ['--strip-comments'],
           {'action': 'store_true', 'validator': validate_boolean}),
          ('Leave comment elements in the document tree. (default)',
           ['--leave-comments'],
           {'action': 'store_false', 'dest': 'strip_comments'}),
          ('Remove all elements with classes="<class>" from the document tree. '
           'Warning: potentially dangerous; use with caution. '
           '(Multiple-use option.)',
           ['--strip-elements-with-class'],
           {'action': 'append', 'dest': 'strip_elements_with_classes',
            'metav
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\docutils\io.py

```
# $Id: io.py 9369 2023-05-02 23:04:27Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
I/O classes provide a uniform API for low-level input and output.  Subclasses
exist for a variety of input/output mechanisms.
"""

__docformat__ = 'reStructuredText'

import codecs
import locale
import os
import re
import sys
import warnings

from docutils import TransformSpec


# Guess the locale's preferred encoding.
# If no valid guess can be made, _locale_encoding is set to `None`:
#
# TODO: check whether this is set correctly with every OS and Python version
#       or whether front-end tools need to call `locale.setlocale()`
#       before importing this module
try:
    # Return locale encoding also in UTF-8 mode
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        _locale_encoding = (locale.getlocale()[1]
                            or locale.getdefaultlocale()[1])
        _locale_encoding = _locale_encoding.lower()
except ValueError as error:  # OS X may set UTF-8 without language code
    # See https://bugs.python.org/issue18378 fixed in 3.8
    # and https://sourceforge.net/p/docutils/bugs/298/.
    # Drop the special case after requiring Python >= 3.8
    if "unknown locale: UTF-8" in error.args:
        _locale_encoding = "utf-8"
    else:
        _locale_encoding = None
except:  # noqa  any other problems determining the locale -> use None
    _locale_encoding = None
try:
    codecs.lookup(_locale_encoding)
except (LookupError, TypeError):
    _locale_encoding = None


class InputError(OSError): pass
class OutputError(OSError): pass


def check_encoding(stream, encoding):
    """Test, whether the encoding of `stream` matches `encoding`.
```

```
    Returns

    :None:  if `encoding` or `stream.encoding` are not a valid encoding
            argument (e.g. ``None``) or `stream.encoding is missing.
    :True:  if the encoding argument resolves to the same value as `encoding`,
    :False: if the encodings differ.
    """
    try:
        return codecs.lookup(stream.encoding) == codecs.lookup(encoding)
    except (LookupError, AttributeError, TypeError):
        return None


def error_string(err):
    """Return string representation of Exception `err`.
    """
    return f'{err.__class__.__name__}: {err}'


class Input(TransformSpec):
    """
    Abstract base class for input wrappers.

    Docutils input objects must provide a `read()` method that
    returns the source, typically as `str` instance.

    Inheriting `TransformSpec` allows input objects to add
    "transforms" and "unknown_reference_resolvers" to the "Transformer".
    (Optional for custom input objects since Docutils 0.19.)
    """

    component_type = 'input'

    default_source_path = None

    def __init__(self, source=None, source_path=None, encoding=None,
                 error_handler='strict'):
        self.encoding = encoding
        """Text encoding for the input source."""

        self.error_handler = error_handler
        """Text decoding error handler."""

        self.source = source
        """The source of input data."""

        self.source_path = source_path
        """A text reference to the source."""

        if not source_path:
            self.source_path = self.default_source_path

        self.successful_encoding = None
        """The encoding that successfully decoded the source data."""

    def __repr__(self):
        return '%s: source=%r, source_path=%r' % (self.__class__, self.source,
                                                  self.source_path)

    def read(self):
        """Return input as `str`. Define in subclasses."""
        raise NotImplementedError

    def decode(self, data):
        """
        Decode `data` if required.

        Return Unicode `str` instances unchanged (nothing to decode).

        If `self.encoding` is None, determine encoding from data
        or try UTF-8, locale encoding, and (as last ressort) 'latin-1'.
        The client application should call ``locale.setlocale`` at the
        beginning of processing::

            locale.setlocale(locale.LC_ALL, '')

        Raise UnicodeError if unsuccessful.

        Provisional:
          - Raise UnicodeError (instead of falling back to the locale
            encoding) if decoding the source with the default encoding (UTF-8)
```

```
        fails and Python is started in `UTF-8 mode`.

        Raise UnicodeError (instead of falling back to "latin1") if both,
        default and locale encoding, fail.

      - Only remove BOM (U+FEFF ZWNBSP at start of data),
        no other ZWNBSPs.
    """
    if self.encoding and self.encoding.lower() == 'unicode':
        assert isinstance(data, str), ('input encoding is "unicode" '
                                       'but `data` is no `str` instance')
    if isinstance(data, str):
        # nothing to decode
        return data
    if self.encoding:
        # We believe the user/application when the encoding is
        # explicitly given.
        encoding_candidates = [self.encoding]
    else:
        data_encoding = self.determine_encoding_from_data(data)
        if data_encoding:
            # If the data declares its encoding (explicitly or via a BOM),
            # we believe it.
            encoding_candidates = [data_encoding]
        else:
            # Apply heuristics only if no encoding is explicitly given and
            # no BOM found.  Start with UTF-8, because that only matches
            # data that *IS* UTF-8:
            encoding_candidates = ['utf-8']
            # TODO: use `locale.getpreferredlocale(do_setlocale=True)`
            # to respect UTF-8 mode (API change).
            # (Check if it is a valid encoding and not UTF-8)
            if _locale_encoding and _locale_encoding != 'utf-8':
                encoding_candidates.append(_locale_encoding)
            # TODO: don't fall back to 'latin-1' (API change).
            encoding_candidates.append('latin-1')
    for enc in encoding_candidates:
        try:
            decoded = str(data, enc, self.error_handler)
            self.successful_encoding = enc
            # Return decoded, removing BOM and other ZWNBSPs.
            # TODO: only remove BOM (ZWNBSP at start of data)
            #       and only if 'self.encoding' is None. (API change)
            return decoded.replace('\ufeff', '')
        except (UnicodeError, LookupError) as err:
            # keep exception instance for use outside of the "for" loop.
            error = err
    raise UnicodeError(
        'Unable to decode input data.  Tried the following encodings: '
        f'{", ".join(repr(enc) for enc in encoding_candidates)}.\n'
        f'({error_string(error)})')

coding_slug = re.compile(br"coding[:=]\s*([-\w.]+)")
"""Encoding declaration pattern."""

byte_order_marks = ((codecs.BOM_UTF8, 'utf-8'),
                    (codecs.BOM_UTF16_BE, 'utf-16-be'),
                    (codecs.BOM_UTF16_LE, 'utf-16-le'),)
"""Sequence of (start_bytes, encoding) tuples for encoding detection.
The first bytes of input data are checked against the start_bytes strings.
A match indicates the given encoding."""

def determine_encoding_from_data(self, data):
    """
    Try to determine the encoding of `data` by looking *in* `data`.
    Check for a byte order mark (BOM) or an encoding declaration.
    """
    # check for a byte order mark:
    for start_bytes, encoding in self.byte_order_marks:
        if data.startswith(start_bytes):
            return encoding
    # check for an encoding declaration pattern in first 2 lines of file:
    for line in data.splitlines()[:2]:
        match = self.coding_slug.search(line)
        if match:
            return match.group(1).decode('ascii')
    return None

def isatty(self):
    """Return True, if the input source is connected to a TTY device."""
    try:
```

```
            return self.source.isatty()
        except AttributeError:
            return False


class Output(TransformSpec):
    """
    Abstract base class for output wrappers.

    Docutils output objects must provide a `write()` method that
    expects and handles one argument (the output).

    Inheriting `TransformSpec` allows output objects to add
    "transforms" and "unknown_reference_resolvers" to the "Transformer".
    (Optional for custom output objects since Docutils 0.19.)
    """

    component_type = 'output'

    default_destination_path = None

    def __init__(self, destination=None, destination_path=None,
                 encoding=None, error_handler='strict'):
        self.encoding = encoding
        """Text encoding for the output destination."""

        self.error_handler = error_handler or 'strict'
        """Text encoding error handler."""

        self.destination = destination
        """The destination for output data."""

        self.destination_path = destination_path
        """A text reference to the destination."""

        if not destination_path:
            self.destination_path = self.default_destination_path

    def __repr__(self):
        return ('%s: destination=%r, destination_path=%r'
                % (self.__class__, self.destination, self.destination_path))

    def write(self, data):
        """Write `data`. Define in subclasses."""
        raise NotImplementedError

    def encode(self, data):
        """
        Encode and return `data`.

        If `data` is a `bytes` instance, it is returned unchanged.
        Otherwise it is encoded with `self.encoding`.

        Provisional: If `self.encoding` is set to the pseudo encoding name
        "unicode", `data` must be a `str` instance and is returned unchanged.
        """
        if self.encoding and self.encoding.lower() == 'unicode':
            assert isinstance(data, str), ('output encoding is "unicode" '
                                           'but `data` is no `str` instance')
            return data
        if not isinstance(data, str):
            # Non-unicode (e.g. bytes) output.
            return data
        else:
            return data.encode(self.encoding, self.error_handler)


class ErrorOutput:
    """
    Wrapper class for file-like error streams with
    failsafe de- and encoding of `str`, `bytes`, `unicode` and
    `Exception` instances.
    """

    def __init__(self, destination=None, encoding=None,
                 encoding_errors='backslashreplace',
                 decoding_errors='replace'):
        """
        :Parameters:
            - `destination`: a file-like object,
                        a string (path to a file),
```

```
                        `None` (write to `sys.stderr`, default), or
                        evaluating to `False` (write() requests are ignored).
            - `encoding`: `destination` text encoding. Guessed if None.
            - `encoding_errors`: how to treat encoding errors.
        """
        if destination is None:
            destination = sys.stderr
        elif not destination:
            destination = False
        # if `destination` is a file name, open it
        elif isinstance(destination, str):
            destination = open(destination, 'w')

        self.destination = destination
        """Where warning output is sent."""

        self.encoding = (encoding or getattr(destination, 'encoding', None)
                         or _locale_encoding or 'ascii')
        """The output character encoding."""

        self.encoding_errors = encoding_errors
        """Encoding error handler."""

        self.decoding_errors = decoding_errors
        """Decoding error handler."""

    def write(self, data):
        """
        Write `data` to self.destination. Ignore, if self.destination is False.

        `data` can be a `bytes`, `str`, or `Exception` instance.
        """
        if not self.destination:
            return
        if isinstance(data, Exception):
            data = str(data)
        try:
            self.destination.write(data)
        except UnicodeEncodeError:
            self.destination.write(data.encode(self.encoding,
                                               self.encoding_errors))
        except TypeError:
            if isinstance(data, str):  # destination may expect bytes
                self.destination.write(data.encode(self.encoding,
                                                   self.encoding_errors))
            elif self.destination in (sys.stderr, sys.stdout):
                # write bytes to raw stream
                self.destination.buffer.write(data)
            else:
                self.destination.write(str(data, self.encoding,
                                           self.decoding_errors))

    def close(self):
        """
        Close the error-output stream.

        Ignored if the destination is` sys.stderr` or `sys.stdout` or has no
        close() method.
        """
        if self.destination in (sys.stdout, sys.stderr):
            return
        try:
            self.destination.close()
        except AttributeError:
            pass

    def isatty(self):
        """Return True, if the destination is connected to a TTY device."""
        try:
            return self.destination.isatty()
        except AttributeError:
            return False


class FileInput(Input):

    """
    Input for single, simple file-like objects.
    """
    def __init__(self, source=None, source_path=None,
                 encoding=None, error_handler='strict',
```

```
                 autoclose=True, mode='r'):
        """
        :Parameters:
            - `source`: either a file-like object (which is read directly), or
              `None` (which implies `sys.stdin` if no `source_path` given).
            - `source_path`: a path to a file, which is opened and then read.
            - `encoding`: the expected text encoding of the input file.
            - `error_handler`: the encoding error handler to use.
            - `autoclose`: close automatically after read (except when
              `sys.stdin` is the source).
            - `mode`: how the file is to be opened (see standard function
              `open`). The default is read only ('r').
        """
        Input.__init__(self, source, source_path, encoding, error_handler)
        self.autoclose = autoclose
        self._stderr = ErrorOutput()

        if source is None:
            if source_path:
                try:
                    self.source = open(source_path, mode,
                                       encoding=self.encoding or 'utf-8-sig',
                                       errors=self.error_handler)
                except OSError as error:
                    raise InputError(error.errno, error.strerror, source_path)
            else:
                self.source = sys.stdin
        elif check_encoding(self.source, self.encoding) is False:
            # TODO: re-open, warn or raise error?
            raise UnicodeError('Encoding clash: encoding given is "%s" '
                               'but source is opened with encoding "%s".' %
                               (self.encoding, self.source.encoding))
        if not source_path:
            try:
                self.source_path = self.source.name
            except AttributeError:
                pass

    def read(self):
        """
        Read and decode a single file and return the data (Unicode string).
        """
        try:
            if self.source is sys.stdin:
                # read as binary data to circumvent auto-decoding
                data = self.source.buffer.read()
            else:
                data = self.source.read()
        except (UnicodeError, LookupError):
            if not self.encoding and self.source_path:
                # re-read in binary mode and decode with heuristics
                b_source = open(self.source_path, 'rb')
                data = b_source.read()
                b_source.close()
            else:
                raise
        finally:
            if self.autoclose:
                self.close()
        data = self.decode(data)
        # normalise newlines
        return '\n'.join(data.splitlines()+['''])

    def readlines(self):
        """
        Return lines of a single file as list of Unicode strings.
        """
        return self.read().splitlines(True)

    def close(self):
        if self.source is not sys.stdin:
            self.source.close()


class FileOutput(Output):

    """Output for single, simple file-like objects."""

    default_destination_path = '<file>'

    mode = 'w'
```

```
    """The mode argument for `open()`."""
    # 'wb' for binary (e.g. OpenOffice) files (see also `BinaryFileOutput`).
    # (Do not use binary mode ('wb') for text files, as this prevents the
    # conversion of newlines to the system specific default.)

    def __init__(self, destination=None, destination_path=None,
                 encoding=None, error_handler='strict', autoclose=True,
                 handle_io_errors=None, mode=None):
        """
        :Parameters:
            - `destination`: either a file-like object (which is written
              directly) or `None` (which implies `sys.stdout` if no
              `destination_path` given).
            - `destination_path`: a path to a file, which is opened and then
              written.
            - `encoding`: the text encoding of the output file.
            - `error_handler`: the encoding error handler to use.
            - `autoclose`: close automatically after write (except when
              `sys.stdout` or `sys.stderr` is the destination).
            - `handle_io_errors`: ignored, deprecated, will be removed.
            - `mode`: how the file is to be opened (see standard function
              `open`). The default is 'w', providing universal newline
              support for text files.
        """
        Output.__init__(self, destination, destination_path,
                        encoding, error_handler)
        self.opened = True
        self.autoclose = autoclose
        if handle_io_errors is not None:
            warnings.warn('io.FileOutput: init argument "handle_io_errors" '
                          'is ignored and will be removed in '
                          'Docutils 2.0.', DeprecationWarning, stacklevel=2)
        if mode is not None:
            self.mode = mode
        self._stderr = ErrorOutput()
        if destination is None:
            if destination_path:
                self.opened = False
            else:
                self.destination = sys.stdout
        elif (  # destination is file-type object -> check mode:
              mode and hasattr(self.destination, 'mode')
              and mode != self.destination.mode):
            print('Warning: Destination mode "%s" differs from specified '
                  'mode "%s"' % (self.destination.mode, mode),
                  file=self._stderr)
        if not destination_path:
            try:
                self.destination_path = self.destination.name
            except AttributeError:
                pass

    def open(self):
        # Specify encoding
        if 'b' not in self.mode:
            kwargs = {'encoding': self.encoding,
                      'errors': self.error_handler}
        else:
            kwargs = {}
        try:
            self.destination = open(self.destination_path, self.mode, **kwargs)
        except OSError as error:
            raise OutputError(error.errno, error.strerror,
                              self.destination_path)
        self.opened = True

    def write(self, data):
        """Write `data` to a single file, also return it.

        `data` can be a `str` or `bytes` instance.
        If writing `bytes` fails, an attempt is made to write to
        the low-
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\docutils__init__.py

```
# $Id: __init__.py 9376 2023-05-09 18:31:42Z grubert $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.
```

```
"""
This is the Docutils (Python Documentation Utilities) package.

Package Structure
=================

Modules:

- __init__.py: Contains component base classes, exception classes, and
  Docutils version information.

- core.py: Contains the ``Publisher`` class and ``publish_*()`` convenience
  functions.

- frontend.py: Runtime settings (command-line interface, configuration files)
  processing, for Docutils front-ends.

- io.py: Provides a uniform API for low-level input and output.

- nodes.py: Docutils document tree (doctree) node class library.

- statemachine.py: A finite state machine specialized for
  regular-expression-based text filters.

Subpackages:

- languages: Language-specific mappings of terms.

- parsers: Syntax-specific input parser modules or packages.

- readers: Context-specific input handlers which understand the data
  source and manage a parser.

- transforms: Modules used by readers and writers to modify
  the Docutils document tree.

- utils: Contains the ``Reporter`` system warning class and miscellaneous
  utilities used by readers, writers, and transforms.

  utils/urischemes.py: Contains a complete mapping of known URI addressing
  scheme names to descriptions.

- utils/math: Contains functions for conversion of mathematical notation
  between different formats (LaTeX, MathML, text, ...).

- writers: Format-specific output translators.
"""

from collections import namedtuple

__docformat__ = 'reStructuredText'

__version__ = '0.20.1'
"""Docutils version identifier (complies with PEP 440)::

    major.minor[.micro][releaselevel[serial]][.dev]

For version comparison operations, use `__version_info__` (see, below)
rather than parsing the text of `__version__`.

https://docutils.sourceforge.io/docs/dev/policies.html#version-identification
"""

__version_details__ = ''
"""Optional extra version details (e.g. 'snapshot 2005-05-29, r3410').

For development and release status, use `__version__ and `__version_info__`.
"""


class VersionInfo(namedtuple('VersionInfo',
                             'major minor micro releaselevel serial release')):

    def __new__(cls, major=0, minor=0, micro=0,
                releaselevel='final', serial=0, release=True):
        releaselevels = ('alpha', 'beta', 'candidate', 'final')
        if releaselevel not in releaselevels:
            raise ValueError('releaselevel must be one of %r.'
                             % (releaselevels, ))
        if releaselevel == 'final':
            if not release:
```

```
                raise ValueError('releaselevel "final" must not be used '
                                 'with development versions (leads to wrong '
                                 'version ordering of the related __version__')
                # cf. https://peps.python.org/pep-0440/#summary-of-permitted-suffixes-and-relative-ordering  # noqa
            if serial != 0:
                raise ValueError('"serial" must be 0 for final releases')

        return super().__new__(cls, major, minor, micro,
                               releaselevel, serial, release)

    def __lt__(self, other):
        if isinstance(other, tuple):
            other = VersionInfo(*other)
        return tuple.__lt__(self, other)

    def __gt__(self, other):
        if isinstance(other, tuple):
            other = VersionInfo(*other)
        return tuple.__gt__(self, other)

    def __le__(self, other):
        if isinstance(other, tuple):
            other = VersionInfo(*other)
        return tuple.__le__(self, other)

    def __ge__(self, other):
        if isinstance(other, tuple):
            other = VersionInfo(*other)
        return tuple.__ge__(self, other)


__version_info__ = VersionInfo(
    major=0,
    minor=20,
    micro=1,
    releaselevel='final',  # one of 'alpha', 'beta', 'candidate', 'final'
    serial=0,  # pre-release number (0 for final releases and snapshots)
    release=True  # True for official releases and pre-releases
    )
"""Comprehensive version information tuple.

https://docutils.sourceforge.io/docs/dev/policies.html#version-identification
"""


class ApplicationError(Exception): pass
class DataError(ApplicationError): pass


class SettingsSpec:

    """
    Runtime setting specification base class.

    SettingsSpec subclass objects used by `docutils.frontend.OptionParser`.
    """

    # TODO: replace settings_specs with a new data structure
    # Backwards compatiblity:
    #   Drop-in components:
    #   Sphinx supplies settings_spec in the current format in some places
    #   Myst parser provides a settings_spec tuple
    #
    #   Sphinx reads a settings_spec in order to set a default value
    #   in writers/html.py:59
    #   https://github.com/sphinx-doc/sphinx/blob/4.x/sphinx/writers/html.py
    #   This should be changed (before retiring the old format)
    #   to use `settings_default_overrides` instead.
    settings_spec = ()
    """Runtime settings specification.  Override in subclasses.

    Defines runtime settings and associated command-line options, as used by
    `docutils.frontend.OptionParser`.  This is a tuple of:

    - Option group title (string or `None` which implies no group, just a list
      of single options).

    - Description (string or `None`).

    - A sequence of option tuples.  Each consists of:
      - Help text (string)
```

```
        - List of option strings (e.g. ``['-Q', '--quux']``).

        - Dictionary of keyword arguments sent to the OptionParser/OptionGroup
          ``add_option`` method.

          Runtime setting names are derived implicitly from long option names
          ('--a-setting' becomes ``settings.a_setting``) or explicitly from the
          'dest' keyword argument.

          Most settings will also have a 'validator' keyword & function.  The
          validator function validates setting values (from configuration files
          and command-line option arguments) and converts them to appropriate
          types.  For example, the ``docutils.frontend.validate_boolean``
          function, **required by all boolean settings**, converts true values
          ('1', 'on', 'yes', and 'true') to 1 and false values ('0', 'off',
          'no', 'false', and '') to 0.  Validators need only be set once per
          setting.  See the `docutils.frontend.validate_*` functions.

          See the optparse docs for more details.

      - More triples of group title, description, options, as many times as
        needed.  Thus, `settings_spec` tuples can be simply concatenated.
      """

      settings_defaults = None
      """A dictionary of defaults for settings not in `settings_spec` (internal
      settings, intended to be inaccessible by command-line and config file).
      Override in subclasses."""

      settings_default_overrides = None
      """A dictionary of auxiliary defaults, to override defaults for settings
      defined in other components' `setting_specs`.  Override in subclasses."""

      relative_path_settings = ()
      """Settings containing filesystem paths.  Override in subclasses.
      Settings listed here are to be interpreted relative to the current working
      directory."""

      config_section = None
      """The name of the config file section specific to this component
      (lowercase, no brackets).  Override in subclasses."""

      config_section_dependencies = None
      """A list of names of config file sections that are to be applied before
      `config_section`, in order (from general to specific).  In other words,
      the settings in `config_section` are to be overlaid on top of the settings
      from these sections.  The "general" section is assumed implicitly.
      Override in subclasses."""


  class TransformSpec:
      """
      Runtime transform specification base class.

      Provides the interface to register "transforms" and helper functions
      to resolve references with a `docutils.transforms.Transformer`.

      https://docutils.sourceforge.io/docs/ref/transforms.html
      """

      def get_transforms(self):
          """Transforms required by this class.  Override in subclasses."""
          if self.default_transforms != ():
              import warnings
              warnings.warn('TransformSpec: the "default_transforms" attribute '
                            'will be removed in Docutils 2.0.\n'
                            'Use get_transforms() method instead.',
                            DeprecationWarning)
              return list(self.default_transforms)
          return []

      # Deprecated; for compatibility.
      default_transforms = ()

      unknown_reference_resolvers = ()
      """List of functions to try to resolve unknown references.

      Unknown references have a 'refname' attribute which doesn't correspond
      to any target in the document.  Called when the transforms in
      `docutils.transforms.references` are unable to find a correct target.
      The list should contain functions which will try to resolve unknown
```

```
    references, with the following signature::

        def reference_resolver(node):
            '''Returns boolean: true if resolved, false if not.'''

    If the function is able to resolve the reference, it should also remove
    the 'refname' attribute and mark the node as resolved::

        del node['refname']
        node.resolved = 1

    Each function must have a "priority" attribute which will affect the order
    the unknown_reference_resolvers are run::

        reference_resolver.priority = 100

    This hook is provided for 3rd party extensions.
    Example use case: the `MoinMoin - ReStructured Text Parser`
    in ``sandbox/mmgilbe/rst.py``.
    """


class Component(SettingsSpec, TransformSpec):

    """Base class for Docutils components."""

    component_type = None
    """Name of the component type ('reader', 'parser', 'writer').  Override in
    subclasses."""

    supported = ()
    """Name and aliases for this component.  Override in subclasses."""

    def supports(self, format):
        """
        Is `format` supported by this component?

        To be used by transforms to ask the dependent component if it supports
        a certain input context or output format.
        """
        return format in self.supported
```

File: .venv\Lib\site-packages\docutils__main__.py

```
#!/usr/bin/env python3
# :Copyright: © 2020, 2022 Günter Milde.
# :License: Released under the terms of the `2-Clause BSD license`_, in short:
#
#    Copying and distribution of this file, with or without modification,
#    are permitted in any medium without royalty provided the copyright
#    notice and this notice are preserved.
#    This file is offered as-is, without any warranty.
#
# .. _2-Clause BSD license: https://opensource.org/licenses/BSD-2-Clause
#
# Revision: $Revision: 9107 $
# Date: $Date: 2022-07-06 15:59:57 +0200 (Mi, 06. Jul 2022) $

"""Generic command line interface for the `docutils` package.

See also
https://docs.python.org/3/library/__main__.html#main-py-in-python-packages
"""

import argparse
import locale
import sys

import docutils
from docutils.core import Publisher, publish_cmdline, default_description


class CliSettingsSpec(docutils.SettingsSpec):
    """Runtime settings & command-line options for the generic CLI.

    Configurable reader, parser, and writer components.

    The "--writer" default will change to 'html' in Docutils 2.0
    when 'html' becomes an alias for the current value 'html5'.
```

```
    """

    settings_spec = (
        'Docutils Application Options',
        'Reader, writer, and parser settings influence the available options. '
        '  Example: use `--help --writer=latex` to see LaTeX writer options. ',
        # options: ('help text', [<option strings>], {<keyword arguments>})
        (('Reader name (currently: "%default").',
          ['--reader'], {'default': 'standalone', 'metavar': '<reader>'}),
         ('Parser name (currently: "%default").',
          ['--parser'], {'default': 'rst', 'metavar': '<parser>'}),
         ('Writer name (currently: "%default").',
          ['--writer'], {'default': 'html5', 'metavar': '<writer>'}),
         )
    )
    config_section = 'docutils application'
    config_section_dependencies = ('docutils-cli application',  # back-compat
                                   'applications')


def main():
    """Generic command line interface for the Docutils Publisher.
    """
    locale.setlocale(locale.LC_ALL, '')

    description = ('Convert documents into useful formats.  '
                   + default_description)

    # Update component selection from config file(s)
    components = Publisher().get_settings(settings_spec=CliSettingsSpec)

    # Update component selection from command-line
    argparser = argparse.ArgumentParser(add_help=False, allow_abbrev=False)
    argparser.add_argument('--reader', default=components.reader)
    argparser.add_argument('--parser', default=components.parser)
    argparser.add_argument('--writer', default=components.writer)
    # other options are parsed in a second pass via `publish_cmdline()`
    (args, remainder) = argparser.parse_known_args()
    # Ensure the current component selections are shown in help:
    CliSettingsSpec.settings_default_overrides = args.__dict__

    try:
        publish_cmdline(reader_name=args.reader,
                        parser_name=args.parser,
                        writer_name=args.writer,
                        settings_spec=CliSettingsSpec,
                        description=description,
                        argv=remainder)
    except ImportError as error:
        print('%s.' % error, file=sys.stderr)
        if '--traceback' in remainder:
            raise
        else:
            print('Use "--traceback" to show details.')


if __name__ == '__main__':
    if sys.argv[0].endswith('__main__.py'):
        # fix "usage" message
        sys.argv[0] = '%s -m docutils' % sys.executable
    main()
```

File: .venv\Lib\site-packages\docutils\languages\af.py

```
# $Id: af.py 9030 2022-03-05 23:28:32Z milde $
# Author: Jannie Hofmeyr <jhsh@sun.ac.za>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Afrikaans-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'
labels = {
```

```
      'author': 'Auteur',
      'authors': 'Auteurs',
      'organization': 'Organisasie',
      'address': 'Adres',
      'contact': 'Kontak',
      'version': 'Weergawe',
      'revision': 'Revisie',
      'status': 'Status',
      'date': 'Datum',
      'copyright': 'Kopiereg',
      'dedication': 'Opdrag',
      'abstract': 'Opsomming',
      'attention': 'Aandag!',
      'caution': 'Wees versigtig!',
      'danger': '!GEVAAR!',
      'error': 'Fout',
      'hint': 'Wenk',
      'important': 'Belangrik',
      'note': 'Nota',
      'tip': 'Tip',  # hint and tip both have the same translation: wenk
      'warning': 'Waarskuwing',
      'contents': 'Inhoud'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      'auteur': 'author',
      'auteurs': 'authors',
      'organisasie': 'organization',
      'adres': 'address',
      'kontak': 'contact',
      'weergawe': 'version',
      'revisie': 'revision',
      'status': 'status',
      'datum': 'date',
      'kopiereg': 'copyright',
      'opdrag': 'dedication',
      'opsomming': 'abstract'}
"""Afrikaans (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\ar.py

```
# $Id: fa.py 4564 2016-08-10 11:48:42Z
# Author: Shahin <me@5hah.in>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Arabic-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': '[redacted]',
      'authors': '[redacted]',
      'organization': '[redacted]',
      'address': '[redacted]',
      'contact': '[redacted]',
      'version': '[redacted]',
      'revision': '[redacted]',
      'status': '[redacted]',
      'date': '[redacted]',
      'copyright': '[redacted]',
      'dedication': '[redacted]',
      'abstract': '[redacted]',
      'attention': '[redacted]',
      'caution': '[redacted]',
      'danger': '[redacted]',
      'error': '[redacted]',
      'hint': '[redacted]',
```

```
      'important': '■■■',
      'note': '■■■■■■',
      'tip': '■■■■■',
      'warning': '■■■■■',
      'contents': '■■■■■■■■'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      '■■■■': 'author',
      '■■■■■■■': 'authors',
      '■■■■■■■■': 'organization',
      '■■■■■■■■': 'address',
      '■■■■■': 'contact',
      '■■■■■': 'version',
      '■■■■■■■': 'revision',
      '■■■■■■■': 'status',
      '■■■■■■': 'date',
      '■■■■■■■': 'copyright',
      '■■■■■■': 'dedication',
      '■■■■■': 'abstract'}
"""Arabic (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = ['■', '■']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\ca.py

```
# $Id: ca.py 9030 2022-03-05 23:28:32Z milde $
# Author: Ivan Vilata i Balaguer <ivan@selidor.net>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Catalan-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Autor',
      'authors': 'Autors',
      'organization': 'Organitzaci\u00F3',
      'address': 'Adre\u00E7a',
      'contact': 'Contacte',
      'version': 'Versi\u00F3',
      'revision': 'Revisi\u00F3',
      'status': 'Estat',
      'date': 'Data',
      'copyright': 'Copyright',
      'dedication': 'Dedicat\u00F2ria',
      'abstract': 'Resum',
      'attention': 'Atenci\u00F3!',
      'caution': 'Compte!',
      'danger': 'PERILL!',
      'error': 'Error',
      'hint': 'Suggeriment',
      'important': 'Important',
      'note': 'Nota',
      'tip': 'Consell',
      'warning': 'Av\u00EDs',
      'contents': 'Contingut'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'autor': 'author',
      'autors': 'authors',
      'organitzaci\u00F3': 'organization',
      'adre\u00E7a': 'address',
      'contacte': 'contact',
      'versi\u00F3': 'version',
      'revisi\u00F3': 'revision',
```

```
      'estat': 'status',
      'data': 'date',
      'copyright': 'copyright',
      'dedicat\u00F2ria': 'dedication',
      'resum': 'abstract'}
"""Catalan (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\cs.py

```
# $Id: cs.py 9030 2022-03-05 23:28:32Z milde $
# Author: Marek Blaha <mb@dat.cz>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Czech-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Autor',
      'authors': 'Auto\u0159i',
      'organization': 'Organizace',
      'address': 'Adresa',
      'contact': 'Kontakt',
      'version': 'Verze',
      'revision': 'Revize',
      'status': 'Stav',
      'date': 'Datum',
      'copyright': 'Copyright',
      'dedication': 'V\u011Bnov\u00E1n\u00ED',
      'abstract': 'Abstrakt',
      'attention': 'Pozor!',
      'caution': 'Opatrn\u011B!',
      'danger': '!NEBEZPE\u010C\u00CD!',
      'error': 'Chyba',
      'hint': 'Rada',
      'important': 'D\u016Fle\u017Eit\u00E9',
      'note': 'Pozn\u00E1mka',
      'tip': 'Tip',
      'warning': 'Varov\u00E1n\u00ED',
      'contents': 'Obsah'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'autor': 'author',
      'auto\u0159i': 'authors',
      'organizace': 'organization',
      'adresa': 'address',
      'kontakt': 'contact',
      'verze': 'version',
      'revize': 'revision',
      'stav': 'status',
      'datum': 'date',
      'copyright': 'copyright',
      'v\u011Bnov\u00E1n\u00ED': 'dedication',
      'abstrakt': 'abstract'}
"""Czech (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\da.py

```
# $Id: da.py 9030 2022-03-05 23:28:32Z milde $
# Author: E D
```

```
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Danish-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Forfatter',
      'authors': 'Forfattere',
      'organization': 'Organisation',
      'address': 'Adresse',
      'contact': 'Kontakt',
      'version': 'Version',
      'revision': 'Revision',
      'status': 'Status',
      'date': 'Dato',
      'copyright': 'Copyright',
      'dedication': 'Dedikation',
      'abstract': 'Resumé',
      'attention': 'Giv agt!',
      'caution': 'Pas på!',
      'danger': '!FARE!',
      'error': 'Fejl',
      'hint': 'Vink',
      'important': 'Vigtigt',
      'note': 'Bemærk',
      'tip': 'Tips',
      'warning': 'Advarsel',
      'contents': 'Indhold'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'forfatter': 'author',
      'forfattere': 'authors',
      'organisation': 'organization',
      'adresse': 'address',
      'kontakt': 'contact',
      'version': 'version',
      'revision': 'revision',
      'status': 'status',
      'dato': 'date',
      'copyright': 'copyright',
      'dedikation': 'dedication',
      'resume': 'abstract',
      'resumé': 'abstract'}
"""Danish (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\de.py

```
# $Id: de.py 9030 2022-03-05 23:28:32Z milde $
# Author: Gunnar Schwant <g.schwant@gmx.de>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
German language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
    'author': 'Autor',
```

```
    'authors': 'Autoren',
    'organization': 'Organisation',
    'address': 'Adresse',
    'contact': 'Kontakt',
    'version': 'Version',
    'revision': 'Revision',
    'status': 'Status',
    'date': 'Datum',
    'dedication': 'Widmung',
    'copyright': 'Copyright',
    'abstract': 'Zusammenfassung',
    'attention': 'Achtung!',
    'caution': 'Vorsicht!',
    'danger': '!GEFAHR!',
    'error': 'Fehler',
    'hint': 'Hinweis',
    'important': 'Wichtig',
    'note': 'Bemerkung',
    'tip': 'Tipp',
    'warning': 'Warnung',
    'contents': 'Inhalt'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
    'autor': 'author',
    'autoren': 'authors',
    'organisation': 'organization',
    'adresse': 'address',
    'kontakt': 'contact',
    'version': 'version',
    'revision': 'revision',
    'status': 'status',
    'datum': 'date',
    'copyright': 'copyright',
    'widmung': 'dedication',
    'zusammenfassung': 'abstract'}
"""German (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\en.py

```
# $Id: en.py 9030 2022-03-05 23:28:32Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
English-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Author',
      'authors': 'Authors',
      'organization': 'Organization',
      'address': 'Address',
      'contact': 'Contact',
      'version': 'Version',
      'revision': 'Revision',
      'status': 'Status',
      'date': 'Date',
      'copyright': 'Copyright',
      'dedication': 'Dedication',
      'abstract': 'Abstract',
      'attention': 'Attention!',
      'caution': 'Caution!',
      'danger': '!DANGER!',
      'error': 'Error',
      'hint': 'Hint',
      'important': 'Important',
```

```
      'note': 'Note',
      'tip': 'Tip',
      'warning': 'Warning',
      'contents': 'Contents'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'author': 'author',
      'authors': 'authors',
      'organization': 'organization',
      'address': 'address',
      'contact': 'contact',
      'version': 'version',
      'revision': 'revision',
      'status': 'status',
      'date': 'date',
      'copyright': 'copyright',
      'dedication': 'dedication',
      'abstract': 'abstract'}
"""English (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\eo.py

```
# $Id: eo.py 9030 2022-03-05 23:28:32Z milde $
# Author: Marcelo Huerta San Martin <richieadler@users.sourceforge.net>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Esperanto-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'A\u016dtoro',
      'authors': 'A\u016dtoroj',
      'organization': 'Organizo',
      'address': 'Adreso',
      'contact': 'Kontakto',
      'version': 'Versio',
      'revision': 'Revido',
      'status': 'Stato',
      'date': 'Dato',
      # 'copyright': 'Kopirajto',
      'copyright': 'A\u016dtorrajto',
      'dedication': 'Dedi\u0109o',
      'abstract': 'Resumo',
      'attention': 'Atentu!',
      'caution': 'Zorgu!',
      'danger': 'DAN\u011cERO!',
      'error': 'Eraro',
      'hint': 'Spuro',
      'important': 'Grava',
      'note': 'Noto',
      'tip': 'Helpeto',
      'warning': 'Averto',
      'contents': 'Enhavo'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'a\u016dtoro': 'author',
      'a\u016dtoroj': 'authors',
      'organizo': 'organization',
      'adreso': 'address',
      'kontakto': 'contact',
      'versio': 'version',
      'revido': 'revision',
```

```
      'stato': 'status',
      'dato': 'date',
      'a\u016dtorrajto': 'copyright',
      'dedi\u0109o': 'dedication',
      'resumo': 'abstract'}
"""Esperanto (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\es.py

```
# $Id: es.py 9030 2022-03-05 23:28:32Z milde $
# Author: Marcelo Huerta San Martín <richieadler@users.sourceforge.net>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Spanish-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      'author': 'Autor',
      'authors': 'Autores',
      'organization': 'Organizaci\u00f3n',
      'address': 'Direcci\u00f3n',
      'contact': 'Contacto',
      'version': 'Versi\u00f3n',
      'revision': 'Revisi\u00f3n',
      'status': 'Estado',
      'date': 'Fecha',
      'copyright': 'Copyright',
      'dedication': 'Dedicatoria',
      'abstract': 'Resumen',
      'attention': '\u00a1Atenci\u00f3n!',
      'caution': '\u00a1Precauci\u00f3n!',
      'danger': '\u00a1PELIGRO!',
      'error': 'Error',
      'hint': 'Sugerencia',
      'important': 'Importante',
      'note': 'Nota',
      'tip': 'Consejo',
      'warning': 'Advertencia',
      'contents': 'Contenido'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      'autor': 'author',
      'autores': 'authors',
      'organizaci\u00f3n': 'organization',
      'direcci\u00f3n': 'address',
      'contacto': 'contact',
      'versi\u00f3n': 'version',
      'revisi\u00f3n': 'revision',
      'estado': 'status',
      'fecha': 'date',
      'copyright': 'copyright',
      'dedicatoria': 'dedication',
      'resumen': 'abstract'}
"""Spanish (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\fa.py

```
# $Id: fa.py 4564 2016-08-10 11:48:42Z
# Author: Shahin <me@5hah.in>
# Copyright: This module has been placed in the public domain.
# New language mappings are welcome.  Before doing a new translation, please
```

```
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Persian-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': '[redacted]',
      'authors': '[redacted]',
      'organization': '[redacted]',
      'address': '[redacted]',
      'contact': '[redacted]',
      'version': '[redacted]',
      'revision': '[redacted]',
      'status': '[redacted]',
      'date': '[redacted]',
      'copyright': '[redacted]',
      'dedication': '[redacted]',
      'abstract': '[redacted]',
      'attention': '[redacted]!',
      'caution': '[redacted]!',
      'danger': '[redacted]!',
      'error': '[redacted]',
      'hint': '[redacted]',
      'important': '[redacted]',
      'note': '[redacted]',
      'tip': '[redacted]',
      'warning': '[redacted]',
      'contents': '[redacted]'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      '[redacted]': 'author',
      '[redacted]': 'authors',
      '[redacted]': 'organization',
      '[redacted]': 'address',
      '[redacted]': 'contact',
      '[redacted]': 'version',
      '[redacted]': 'revision',
      '[redacted]': 'status',
      '[redacted]': 'date',
      '[redacted]': 'copyright',
      '[redacted]': 'dedication',
      '[redacted]': 'abstract'}
"""Persian (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = ['[redacted]', '[redacted]']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\fi.py

```
# $Id: fi.py 9030 2022-03-05 23:28:32Z milde $
# Author: Asko Soukka <asko.soukka@iki.fi>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Finnish-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Tekij\u00e4',
      'authors': 'Tekij\u00e4t',
      'organization': 'Yhteis\u00f6',
      'address': 'Osoite',
```

```
      'contact': 'Yhteystiedot',
      'version': 'Versio',
      'revision': 'Vedos',
      'status': 'Tila',
      'date': 'P\u00e4iv\u00e4ys',
      'copyright': 'Tekij\u00e4noikeudet',
      'dedication': 'Omistuskirjoitus',
      'abstract': 'Tiivistelm\u00e4',
      'attention': 'Huomio!',
      'caution': 'Varo!',
      'danger': '!VAARA!',
      'error': 'Virhe',
      'hint': 'Vihje',
      'important': 'T\u00e4rke\u00e4\u00e4',
      'note': 'Huomautus',
      'tip': 'Neuvo',
      'warning': 'Varoitus',
      'contents': 'Sis\u00e4llys'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'tekij\u00e4': 'author',
      'tekij\u00e4t': 'authors',
      'yhteis\u00f6': 'organization',
      'osoite': 'address',
      'yhteystiedot': 'contact',
      'versio': 'version',
      'vedos': 'revision',
      'tila': 'status',
      'p\u00e4iv\u00e4ys': 'date',
      'tekij\u00e4noikeudet': 'copyright',
      'omistuskirjoitus': 'dedication',
      'tiivistelm\u00e4': 'abstract'}
"""Finnish (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\fr.py

```
# $Id: fr.py 9030 2022-03-05 23:28:32Z milde $
# Author: Stefane Fermigier <sf@fermigier.com>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
French-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      'author': 'Auteur',
      'authors': 'Auteurs',
      'organization': 'Organisation',
      'address': 'Adresse',
      'contact': 'Contact',
      'version': 'Version',
      'revision': 'R\u00e9vision',
      'status': 'Statut',
      'date': 'Date',
      'copyright': 'Copyright',
      'dedication': 'D\u00e9dicace',
      'abstract': 'R\u00e9sum\u00e9',
      'attention': 'Attention!',
      'caution': 'Avertissement!',
      'danger': '!DANGER!',
      'error': 'Erreur',
      'hint': 'Indication',
      'important': 'Important',
      'note': 'Note',
      'tip': 'Astuce',
      'warning': 'Avis',
```

```
      'contents': 'Sommaire'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      'auteur': 'author',
      'auteurs': 'authors',
      'organisation': 'organization',
      'adresse': 'address',
      'contact': 'contact',
      'version': 'version',
      'r\u00e9vision': 'revision',
      'statut': 'status',
      'date': 'date',
      'copyright': 'copyright',
      'd\u00e9dicace': 'dedication',
      'r\u00e9sum\u00e9': 'abstract'}
"""French (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\gl.py

```
# Author: David Goodger
# Contact: goodger@users.sourceforge.net
# Revision: $Revision: 2224 $
# Date: $Date: 2004-06-05 21:40:46 +0200 (Sat, 05 Jun 2004) $
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Galician-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Autor',
      'authors': 'Autores',
      'organization': 'Organizaci\u00f3n',
      'address': 'Enderezo',
      'contact': 'Contacto',
      'version': 'Versi\u00f3n',
      'revision': 'Revisi\u00f3n',
      'status': 'Estado',
      'date': 'Data',
      'copyright': 'Dereitos de copia',
      'dedication': 'Dedicatoria',
      'abstract': 'Abstract',
      'attention': 'Atenci\u00f3n!',
      'caution': 'Advertencia!',
      'danger': 'PERIGO!',
      'error': 'Erro',
      'hint': 'Consello',
      'important': 'Importante',
      'note': 'Nota',
      'tip': 'Suxesti\u00f3n',
      'warning': 'Aviso',
      'contents': 'Contido'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'autor': 'author',
      'autores': 'authors',
      'organizaci\u00f3n': 'organization',
      'enderezo': 'address',
      'contacto': 'contact',
      'versi\u00f3n': 'version',
      'revisi\u00f3n': 'revision',
      'estado': 'status',
      'data': 'date',
      'dereitos de copia': 'copyright',
```

```
      'dedicatoria': 'dedication',
      'abstract': 'abstract'}
"""Galician (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\he.py

```
# Author: Meir Kriheli
# Id: $Id: he.py 9030 2022-03-05 23:28:32Z milde $
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Hebrew-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': '\u05de\u05d7\u05d1\u05e8',
      'authors': '\u05de\u05d7\u05d1\u05e8\u05d9',
      'organization': '\u05d0\u05e8\u05d2\u05d5\u05df',
      'address': '\u05db\u05ea\u05d5\u05d1\u05ea',
      'contact': '\u05d0\u05d9\u05e9 \u05e7\u05e9\u05e8',
      'version': '\u05d2\u05e8\u05e1\u05d4',
      'revision': '\u05de\u05d4\u05d3\u05d5\u05e8\u05d4',
      'status': '\u05e1\u05d8\u05d8\u05d5\u05e1',
      'date': '\u05ea\u05d0\u05e8\u05d9\u05da',
      'copyright': ('\u05d6\u05db\u05d5\u05d9\u05d5\u05ea '
                    '\u05e9\u05de\u05d5\u05e8\u05d5\u05ea'),
      'dedication': '\u05d4\u05e7\u05d3\u05e9\u05d4',
      'abstract': '\u05ea\u05e7\u05e6\u05d9\u05e8',
      'attention': '\u05ea\u05e9\u05d5\u05de\u05ea \u05dc\u05d1',
      'caution': '\u05d6\u05d4\u05d9\u05e8\u05d5\u05ea',
      'danger': '\u05e1\u05db\u05e0\u05d4',
      'error': '\u05e9\u05d2\u05d9\u05d0\u05d4',
      'hint': '\u05e8\u05de\u05d6',
      'important': '\u05d7\u05e9\u05d5\u05d1',
      'note': '\u05d4\u05e2\u05e8\u05d4',
      'tip': '\u05d8\u05d9\u05e4',
      'warning': '\u05d0\u05d6\u05d4\u05e8\u05d4',
      'contents': '\u05ea\u05d5\u05db\u05df'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      '\u05de\u05d7\u05d1\u05e8': 'author',
      '\u05de\u05d7\u05d1\u05e8\u05d9': 'authors',
      '\u05d0\u05e8\u05d2\u05d5\u05df': 'organization',
      '\u05db\u05ea\u05d5\u05d1\u05ea': 'address',
      '\u05d0\u05d9\u05e9 \u05e7\u05e9\u05e8': 'contact',
      '\u05d2\u05e8\u05e1\u05d4': 'version',
      '\u05de\u05d4\u05d3\u05d5\u05e8\u05d4': 'revision',
      '\u05e1\u05d8\u05d8\u05d5\u05e1': 'status',
      '\u05ea\u05d0\u05e8\u05d9\u05da': 'date',
      '\u05d6\u05db\u05d5\u05d9\u05d5\u05ea \u05e9\u05de\u05d5\u05e8\u05d5\u05ea': 'copyright',  # noqa:E501
      '\u05d4\u05e7\u05d3\u05e9\u05d4': 'dedication',
      '\u05ea\u05e7\u05e6\u05d9\u05e8': 'abstract'}
"""Hebrew to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\it.py

```
# $Id: it.py 9030 2022-03-05 23:28:32Z milde $
# Author: Nicola Larosa <docutils@tekNico.net>
# Copyright: This module has been placed in the public domain.
# New language mappings are welcome.  Before doing a new translation, please
```

```
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Italian-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      'author': 'Autore',
      'authors': 'Autori',
      'organization': 'Organizzazione',
      'address': 'Indirizzo',
      'contact': 'Contatti',
      'version': 'Versione',
      'revision': 'Revisione',
      'status': 'Status',
      'date': 'Data',
      'copyright': 'Copyright',
      'dedication': 'Dedica',
      'abstract': 'Riassunto',
      'attention': 'Attenzione!',
      'caution': 'Cautela!',
      'danger': '!PERICOLO!',
      'error': 'Errore',
      'hint': 'Suggerimento',
      'important': 'Importante',
      'note': 'Nota',
      'tip': 'Consiglio',
      'warning': 'Avvertenza',
      'contents': 'Indice'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      'autore': 'author',
      'autori': 'authors',
      'organizzazione': 'organization',
      'indirizzo': 'address',
      'contatto': 'contact',
      'versione': 'version',
      'revisione': 'revision',
      'status': 'status',
      'data': 'date',
      'copyright': 'copyright',
      'dedica': 'dedication',
      'riassunto': 'abstract'}
"""Italian (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\ja.py

```
# $Id: ja.py 9030 2022-03-05 23:28:32Z milde $
# Author: Hisashi Morita <hisashim@kt.rim.or.jp>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Japanese-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': '■■',
      'authors': '■■',
      'organization': '■■',
      'address': '■■',
      'contact': '■■■',
      'version': '■■■■■',
```

```
      'revision': '■■■■■',
      'status': '■■■■■',
      'date': '■■',
      'copyright': '■■■',
      'dedication': '■■',
      'abstract': '■■',
      'attention': '■■!',
      'caution': '■■!',
      'danger': '!■■!',
      'error': '■■■',
      'hint': '■■■',
      'important': '■■',
      'note': '■■',
      'tip': '■■',
      'warning': '■■',
      'contents': '■■'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      '■■': 'author',
      ' n/a': 'authors',
      '■■': 'organization',
      '■■': 'address',
      '■■■': 'contact',
      '■■■■■': 'version',
      '■■■■■': 'revision',
      '■■■■■': 'status',
      '■■': 'date',
      '■■■': 'copyright',
      '■■': 'dedication',
      '■■': 'abstract'}
"""Japanese (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\ko.py

```
# $Id: ko.py 9030 2022-03-05 23:28:32Z milde $
# Author: Thomas SJ Kang <thomas.kangsj@ujuc.kr>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Korean-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': '■■',
      'authors': '■■■',
      'organization': '■■',
      'address': '■■',
      'contact': '■■■',
      'version': '■■',
      'revision': '■■■',
      'status': '■■',
      'date': '■■',
      'copyright': '■■■',
      'dedication': '■■',
      'abstract': '■■',
      'attention': '■■!',
      'caution': '■■!',
      'danger': '!■■!',
      'error': '■■',
      'hint': '■■■',
      'important': '■■■',
      'note': '■■',
      'tip': '■',
      'warning': '■■',
      'contents': '■■'}
```

```
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      '■■': 'author',
      '■■■': 'authors',
      '■■': 'organization',
      '■■': 'address',
      '■■■': 'contact',
      '■■': 'version',
      '■■■': 'revision',
      '■■': 'status',
      '■■': 'date',
      '■■■': 'copyright',
      '■■': 'dedication',
      '■■': 'abstract'}
"""Korean to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\lt.py

```
# $Id: lt.py 9030 2022-03-05 23:28:32Z milde $
# Author: Dalius Dobravolskas <dalius.do...@gmail.com>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Lithuanian language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Autorius',
      'authors': 'Autoriai',
      'organization': 'Organizacija',
      'address': 'Adresas',
      'contact': 'Kontaktas',
      'version': 'Versija',
      'revision': 'Revizija',
      'status': 'B■sena',
      'date': 'Data',
      'copyright': 'Autoriaus teis■s',
      'dedication': 'Dedikacija',
      'abstract': 'Santrauka',
      'attention': 'D■mesio!',
      'caution': 'Atsargiai!',
      'danger': '!PAVOJINGA!',
      'error': 'Klaida',
      'hint': 'Užuomina',
      'important': 'Svarbu',
      'note': 'Pastaba',
      'tip': 'Patarimas',
      'warning': '■sp■jimas',
      'contents': 'Turinys'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'autorius': 'author',
      'autoriai': 'authors',
      'organizacija': 'organization',
      'adresas': 'address',
      'kontaktas': 'contact',
      'versija': 'version',
      'revizija': 'revision',
      'b■sena': 'status',
      'data': 'date',
      'autoriaus teis■s': 'copyright',
      'dedikacija': 'dedication',
      'santrauka': 'abstract'}
```

```
"""Lithuanian (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\lv.py

```
# $Id: lv.py 9030 2022-03-05 23:28:32Z milde $
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Latvian-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Autors',
      'authors': 'Autori',
      'organization': 'Organiz■cija',
      'address': 'Adrese',
      'contact': 'Kontakti',
      'version': 'Versija',
      'revision': 'Rev■zija',
      'status': 'Statuss',
      'date': 'Datums',
      'copyright': 'Copyright',
      'dedication': 'Velt■jums',
      'abstract': 'Atrefer■jums',
      'attention': 'Uzman■bu!',
      'caution': 'Piesardz■bu!',
      'danger': '!B■STAMI!',
      'error': 'K■■da',
      'hint': 'Ieteikums',
      'important': 'Svar■gi',
      'note': 'Piez■me',
      'tip': 'Padoms',
      'warning': 'Br■din■jums',
      'contents': 'Saturs'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'autors': 'author',
      'autori': 'authors',
      'organiz■cija': 'organization',
      'adrese': 'address',
      'kontakti': 'contact',
      'versija': 'version',
      'rev■zija': 'revision',
      'statuss': 'status',
      'datums': 'date',
      'copyright': 'copyright',
      'velt■jums': 'dedication',
      'atrefer■jums': 'abstract'}
"""English (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\nl.py

```
# $Id: nl.py 9030 2022-03-05 23:28:32Z milde $
# Author: Martijn Pieters <mjpieters@users.sourceforge.net>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.
```

```
"""
Dutch-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Auteur',
      'authors': 'Auteurs',
      'organization': 'Organisatie',
      'address': 'Adres',
      'contact': 'Contact',
      'version': 'Versie',
      'revision': 'Revisie',
      'status': 'Status',
      'date': 'Datum',
      'copyright': 'Copyright',
      'dedication': 'Toewijding',
      'abstract': 'Samenvatting',
      'attention': 'Attentie!',
      'caution': 'Let op!',
      'danger': '!GEVAAR!',
      'error': 'Fout',
      'hint': 'Hint',
      'important': 'Belangrijk',
      'note': 'Opmerking',
      'tip': 'Tip',
      'warning': 'Waarschuwing',
      'contents': 'Inhoud'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'auteur': 'author',
      'auteurs': 'authors',
      'organisatie': 'organization',
      'adres': 'address',
      'contact': 'contact',
      'versie': 'version',
      'revisie': 'revision',
      'status': 'status',
      'datum': 'date',
      'copyright': 'copyright',
      'toewijding': 'dedication',
      'samenvatting': 'abstract'}
"""Dutch (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\pl.py

```
# $Id$
# Author: Robert Wojciechowicz <rw@smsnet.pl>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Polish-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Autor',
      'authors': 'Autorzy',
      'organization': 'Organizacja',
      'address': 'Adres',
      'contact': 'Kontakt',
      'version': 'Wersja',
      'revision': 'Korekta',
      'status': 'Status',
```

```
      'date': 'Data',
      'copyright': 'Copyright',
      'dedication': 'Dedykacja',
      'abstract': 'Streszczenie',
      'attention': 'Uwaga!',
      'caution': 'Ostro\u017cnie!',
      'danger': '!Niebezpiecze\u0144stwo!',
      'error': 'B\u0142\u0105d',
      'hint': 'Wskaz\u00f3wka',
      'important': 'Wa\u017cne',
      'note': 'Przypis',
      'tip': 'Rada',
      'warning': 'Ostrze\u017cenie',
      'contents': 'Tre\u015b\u0107'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'autor': 'author',
      'autorzy': 'authors',
      'organizacja': 'organization',
      'adres': 'address',
      'kontakt': 'contact',
      'wersja': 'version',
      'korekta': 'revision',
      'status': 'status',
      'data': 'date',
      'copyright': 'copyright',
      'dedykacja': 'dedication',
      'streszczenie': 'abstract'}
"""Polish (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\pt_br.py

```
# $Id: pt_br.py 9030 2022-03-05 23:28:32Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Brazilian Portuguese-language mappings for language-dependent features.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': 'Autor',
      'authors': 'Autores',
      'organization': 'Organiza\u00E7\u00E3o',
      'address': 'Endere\u00E7o',
      'contact': 'Contato',
      'version': 'Vers\u00E3o',
      'revision': 'Revis\u00E3o',
      'status': 'Estado',
      'date': 'Data',
      'copyright': 'Copyright',
      'dedication': 'Dedicat\u00F3ria',
      'abstract': 'Resumo',
      'attention': 'Aten\u00E7\u00E3o!',
      'caution': 'Cuidado!',
      'danger': 'PERIGO!',
      'error': 'Erro',
      'hint': 'Sugest\u00E3o',
      'important': 'Importante',
      'note': 'Nota',
      'tip': 'Dica',
      'warning': 'Aviso',
      'contents': 'Sum\u00E1rio'}
"""Mapping of node class name to label text."""
bibliographic_fields = {
```

```
      # language-dependent: fixed
      'autor': 'author',
      'autores': 'authors',
      'organiza\u00E7\u00E3o': 'organization',
      'endere\u00E7o': 'address',
      'contato': 'contact',
      'vers\u00E3o': 'version',
      'revis\u00E3o': 'revision',
      'estado': 'status',
      'data': 'date',
      'copyright': 'copyright',
      'dedicat\u00F3ria': 'dedication',
      'resumo': 'abstract'}
"""Brazilian Portuguese (lowcased) name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\ru.py

```
# $Id: ru.py 9030 2022-03-05 23:28:32Z milde $
# Author: Roman Suzi <rnd@onego.ru>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Russian-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      'abstract': '[redacted]',
      'address': '[redacted]',
      'attention': '[redacted]!',
      'author': '[redacted]',
      'authors': '[redacted]',
      'caution': '[redacted]!',
      'contact': '[redacted]',
      'contents': '[redacted]',
      'copyright': '[redacted] [redacted]',
      'danger': '[redacted]!',
      'date': '[redacted]',
      'dedication': '[redacted]',
      'error': '[redacted]',
      'hint': '[redacted]',
      'important': '[redacted]',
      'note': '[redacted]',
      'organization': '[redacted]',
      'revision': '[redacted]',
      'status': '[redacted]',
      'tip': '[redacted]',
      'version': '[redacted]',
      'warning': '[redacted]'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      '[redacted]': 'abstract',
      '[redacted]': 'address',
      '[redacted]': 'author',
      '[redacted]': 'authors',
      '[redacted]': 'contact',
      '[redacted] [redacted]': 'copyright',
      '[redacted]': 'date',
      '[redacted]': 'dedication',
      '[redacted]': 'organization',
      '[redacted]': 'revision',
      '[redacted]': 'status',
      '[redacted]': 'version'}
"""Russian (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\sk.py

```
# $Id: sk.py 9030 2022-03-05 23:28:32Z milde $
# Author: Miroslav Vasko <zemiak@zoznam.sk>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Slovak-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      'author': 'Autor',
      'authors': 'Autori',
      'organization': 'Organiz\u00E1cia',
      'address': 'Adresa',
      'contact': 'Kontakt',
      'version': 'Verzia',
      'revision': 'Rev\u00EDzia',
      'status': 'Stav',
      'date': 'D\u00E1tum',
      'copyright': 'Copyright',
      'dedication': 'Venovanie',
      'abstract': 'Abstraktne',
      'attention': 'Pozor!',
      'caution': 'Opatrne!',
      'danger': '!NEBEZPE\u010cENSTVO!',
      'error': 'Chyba',
      'hint': 'Rada',
      'important': 'D\u00F4le\u017Eit\u00E9',
      'note': 'Pozn\u00E1mka',
      'tip': 'Tip',
      'warning': 'Varovanie',
      'contents': 'Obsah'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      'autor': 'author',
      'autori': 'authors',
      'organiz\u00E1cia': 'organization',
      'adresa': 'address',
      'kontakt': 'contact',
      'verzia': 'version',
      'rev\u00EDzia': 'revision',
      'stav': 'status',
      'd\u00E1tum': 'date',
      'copyright': 'copyright',
      'venovanie': 'dedication',
      'abstraktne': 'abstract'}
"""Slovak (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\sv.py

```
# $Id: sv.py 9030 2022-03-05 23:28:32Z milde $
# Author: Adam Chodorowski <chodorowski@users.sourceforge.net>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Swedish language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'
```

```
labels = {
    'author': 'Författare',
    'authors': 'Författare',
    'organization': 'Organisation',
    'address': 'Adress',
    'contact': 'Kontakt',
    'version': 'Version',
    'revision': 'Revision',
    'status': 'Status',
    'date': 'Datum',
    'copyright': 'Copyright',
    'dedication': 'Dedikation',
    'abstract': 'Sammanfattning',
    'attention': 'Observera!',
    'caution': 'Akta!',  # 'Varning' already used for 'warning'
    'danger': 'FARA!',
    'error': 'Fel',
    'hint': 'Vink',
    'important': 'Viktigt',
    'note': 'Notera',
    'tip': 'Tips',
    'warning': 'Varning',
    'contents': 'Innehåll'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
    # 'Author' and 'Authors' identical in Swedish; assume the plural:
    'författare': 'authors',
    ' n/a': 'author',
    'organisation': 'organization',
    'adress': 'address',
    'kontakt': 'contact',
    'version': 'version',
    'revision': 'revision',
    'status': 'status',
    'datum': 'date',
    'copyright': 'copyright',
    'dedikation': 'dedication',
    'sammanfattning': 'abstract'}
"""Swedish (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\uk.py

```
# $Id: uk.py 9114 2022-07-28 17:06:10Z milde $
# Author: Dmytro Kazanzhy <dkazanzhy@gmail.com>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <http://docutils.sf.net/docs/howto/i18n.html>.  Two files must be
# translated for each language: one in docutils/languages, the other in
# docutils/parsers/rst/languages.

"""
Ukrainian-language mappings for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      'abstract': '[redacted]',
      'address': '[redacted]',
      'attention': '[redacted]!',
      'author': '[redacted]',
      'authors': '[redacted]',
      'caution': '[redacted]!',
      'contact': '[redacted]',
      'contents': '[redacted]',
      'copyright': '[redacted] [redacted]',
      'danger': '[redacted]!',
      'date': '[redacted]',
      'dedication': '[redacted]',
      'error': '[redacted]',
      'hint': '[redacted]',
      'important': '[redacted]',
      'note': '[redacted]',
```

```
      'organization': '[redacted]',
      'revision': '[redacted]',
      'status': '[redacted]',
      'tip': '[redacted]',
      'version': '[redacted]',
      'warning': '[redacted]'}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      '[redacted]': 'abstract',
      '[redacted]': 'address',
      '[redacted]': 'author',
      '[redacted]': 'authors',
      '[redacted]': 'contact',
      '[redacted] [redacted]': 'copyright',
      '[redacted]': 'date',
      '[redacted]': 'dedication',
      '[redacted]': 'organization',
      '[redacted]': 'revision',
      '[redacted]': 'status',
      '[redacted]': 'version'}
"""Ukrainian (lowcased) to canonical name mapping for bibliographic fields."""

author_separators = [';', ',']
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\zh_cn.py

```
# $Id: zh_cn.py 9030 2022-03-05 23:28:32Z milde $
# Author: Pan Junyong <panjy@zopechina.com>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Simplified Chinese language mappings for language-dependent features
of Docutils.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': '[redacted]',
      'authors': '[redacted]',
      'organization': '[redacted]',
      'address': '[redacted]',
      'contact': '[redacted]',
      'version': '[redacted]',
      'revision': '[redacted]',
      'status': '[redacted]',
      'date': '[redacted]',
      'copyright': '[redacted]',
      'dedication': '[redacted]',
      'abstract': '[redacted]',
      'attention': '[redacted]',
      'caution': '[redacted]',
      'danger': '[redacted]',
      'error': '[redacted]',
      'hint': '[redacted]',
      'important': '[redacted]',
      'note': '[redacted]',
      'tip': '[redacted]',
      'warning': '[redacted]',
      'contents': '[redacted]',
}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      '[redacted]': 'author',
      '[redacted]': 'authors',
      '[redacted]': 'organization',
      '[redacted]': 'address',
      '[redacted]': 'contact',
```

```
      '■■': 'version',
      '■■': 'revision',
      '■■': 'status',
      '■■': 'date',
      '■■': 'copyright',
      '■■': 'dedication',
      '■■': 'abstract'}
"""Simplified Chinese to canonical name mapping for bibliographic fields."""

author_separators = [';', ',',
                     '\uff1b',  # '■'
                     '\uff0c',  # '■'
                     '\u3001',  # '■'
                     ]
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages\zh_tw.py

```
# $Id: zh_tw.py 9030 2022-03-05 23:28:32Z milde $
# Author: Joe YS Jaw <joeysj@users.sourceforge.net>
# Copyright: This module has been placed in the public domain.

# New language mappings are welcome.  Before doing a new translation, please
# read <https://docutils.sourceforge.io/docs/howto/i18n.html>.
# Two files must be translated for each language: one in docutils/languages,
# the other in docutils/parsers/rst/languages.

"""
Traditional Chinese language mappings for language-dependent features.
"""

__docformat__ = 'reStructuredText'

labels = {
      # fixed: language-dependent
      'author': '\u4f5c\u8005',  # '■■' <-- Chinese word
      'authors': '\u4f5c\u8005\u7fa4',  # '■■■',
      'organization': '\u7d44\u7e54',  # '■■',
      'address': '\u5730\u5740',  # '■■',
      'contact': '\u9023\u7d61',  # '■■',
      'version': '\u7248\u672c',  # '■■',
      'revision': '\u4fee\u8a02',  # '■■',
      'status': '\u72c0\u614b',  # '■■',
      'date': '\u65e5\u671f',  # '■■',
      'copyright': '\u7248\u6b0a',  # '■■',
      'dedication': '\u984c\u737b',  # '■■',
      'abstract': '\u6458\u8981',  # '■■',
      'attention': '\u6ce8\u610f\uff01',  # '■■■',
      'caution': '\u5c0f\u5fc3\uff01',  # '■■■',
      'danger': '\uff01\u5371\u96aa\uff01',  # '■■■■',
      'error': '\u932f\u8aa4',  # '■■',
      'hint': '\u63d0\u793a',  # '■■',
      'important': '\u91cd\u8981',  # '■■■',
      'note': '\u8a3b\u91cb',  # '■■',
      'tip': '\u79d8\u8a23',  # '■■',
      'warning': '\u8b66\u544a',  # '■■',
      'contents': '\u76ee\u9304',  # '■■'
}
"""Mapping of node class name to label text."""

bibliographic_fields = {
      # language-dependent: fixed
      'author (translation required)': 'author',
      'authors (translation required)': 'authors',
      'organization (translation required)': 'organization',
      'address (translation required)': 'address',
      'contact (translation required)': 'contact',
      'version (translation required)': 'version',
      'revision (translation required)': 'revision',
      'status (translation required)': 'status',
      'date (translation required)': 'date',
      'copyright (translation required)': 'copyright',
      'dedication (translation required)': 'dedication',
      'abstract (translation required)': 'abstract'}
"""Traditional Chinese to canonical name mapping for bibliographic fields."""

author_separators = [';', ',',
                     '\uff1b',  # '■'
```

```
                    '\uff0c',  # '，'
                    '\u3001',  # '、'
                    ]
"""List of separator strings for the 'Authors' bibliographic field. Tried in
order."""
```

File: .venv\Lib\site-packages\docutils\languages__init__.py

```
# $Id: __init__.py 9030 2022-03-05 23:28:32Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

# Internationalization details are documented in
# <https://docutils.sourceforge.io/docs/howto/i18n.html>.

"""
This package contains modules for language-dependent features of Docutils.
"""

__docformat__ = 'reStructuredText'

from importlib import import_module

from docutils.utils import normalize_language_tag


class LanguageImporter:
    """Import language modules.

    When called with a BCP 47 language tag, instances return a module
    with localisations from `docutils.languages` or the PYTHONPATH.

    If there is no matching module, warn (if a `reporter` is passed)
    and fall back to English.
    """
    packages = ('docutils.languages.', '')
    warn_msg = ('Language "%s" not supported: '
                'Docutils-generated text will be in English.')
    fallback = 'en'
    # TODO: use a dummy module returning empty strings?, configurable?

    def __init__(self):
        self.cache = {}

    def import_from_packages(self, name, reporter=None):
        """Try loading module `name` from `self.packages`."""
        module = None
        for package in self.packages:
            try:
                module = import_module(package+name)
                self.check_content(module)
            except (ImportError, AttributeError):
                if reporter and module:
                    reporter.info(f'{module} is no complete '
                                  'Docutils language module.')
                elif reporter:
                    reporter.info(f'Module "{package+name}" not found.')
                continue
            break
        return module

    def check_content(self, module):
        """Check if we got a Docutils language module."""
        if not (isinstance(module.labels, dict)
                and isinstance(module.bibliographic_fields, dict)
                and isinstance(module.author_separators, list)):
            raise ImportError

    def __call__(self, language_code, reporter=None):
        try:
            return self.cache[language_code]
        except KeyError:
            pass
        for tag in normalize_language_tag(language_code):
            tag = tag.replace('-', '_')  # '-' not valid in module names
            module = self.import_from_packages(tag, reporter)
            if module is not None:
                break
        else:
```

```
            if reporter:
                reporter.warning(self.warn_msg % language_code)
            if self.fallback:
                module = self.import_from_packages(self.fallback)
        if reporter and (language_code != 'en'):
            reporter.info('Using %s for language "%s".'
                          % (module, language_code))
        self.cache[language_code] = module
        return module


get_language = LanguageImporter()
```

File: .venv\Lib\site-packages\docutils\parsers\commonmark_wrapper.py

```
#! /usr/bin/env python3
# :Copyright: © 2022 Günter Milde.
# :License: Released under the terms of the `2-Clause BSD license`_, in short:
#
#    Copying and distribution of this file, with or without modification,
#    are permitted in any medium without royalty provided the copyright
#    notice and this notice are preserved.
#    This file is offered as-is, without any warranty.
#
# .. _2-Clause BSD license: https://opensource.org/licenses/BSD-2-Clause
#
# Revision: $Revision: 9048 $
# Date: $Date: 2022-03-29 23:50:15 +0200 (Di, 29. Mär 2022) $
"""
An interface for parsing CommonMark input.

Select a locally installed parser from the following 3rd-party
parser packages:

:pycmark:       https://pypi.org/project/pycmark/
:myst:          https://pypi.org/project/pycmark/
:recommonmark:  https://pypi.org/project/pycmark/ (unmaintained, deprecated)

The first parser class that can be successfully imported is mapped to
`commonmark_wrapper.Parser`.

This module is provisional:
the API is not settled and may change with any minor Docutils version.
"""

import docutils.parsers


commonmark_parser_names = ('pycmark', 'myst', 'recommonmark')
"""Names of compatible drop-in CommonMark parsers"""

Parser = None
parser_name = ''

for name in commonmark_parser_names:
    try:
        Parser = docutils.parsers.get_parser_class(name)
    except ImportError:
        continue
    parser_name = name
    break

if Parser is None:
    raise ImportError(
              'Parsing "CommonMark" requires one of the packages\n'
              f'{commonmark_parser_names} available at https://pypi.org')

if parser_name == 'myst':
    if not Parser.settings_defaults:
        Parser.settings_defaults = {}
    Parser.settings_defaults['myst_commonmark_only'] = True
```

File: .venv\Lib\site-packages\docutils\parsers\null.py

```
# $Id: null.py 4564 2006-05-21 20:44:42Z wiemann $
# Author: Martin Blais <blais@furius.ca>
# Copyright: This module has been placed in the public domain.
"""A do-nothing parser."""
```

```
from docutils import parsers


class Parser(parsers.Parser):

    """A do-nothing parser."""

    supported = ('null',)

    config_section = 'null parser'
    config_section_dependencies = ('parsers',)

    def parse(self, inputstring, document):
        pass
```

File: .venv\Lib\site-packages\docutils\parsers\recommonmark_wrapper.py

```
#!/usr/bin/env python3
# :Copyright: © 2020 Günter Milde.
# :License: Released under the terms of the `2-Clause BSD license`_, in short:
#
#    Copying and distribution of this file, with or without modification,
#    are permitted in any medium without royalty provided the copyright
#    notice and this notice are preserved.
#    This file is offered as-is, without any warranty.
#
# .. _2-Clause BSD license: https://opensource.org/licenses/BSD-2-Clause
#
# Revision: $Revision: 9302 $
# Date: $Date: 2022-12-02 18:14:05 +0100 (Fr, 02. Dez 2022) $
"""
A parser for CommonMark Markdown text using `recommonmark`__.

__ https://pypi.org/project/recommonmark/

.. important:: This module is provisional

   * The "recommonmark" package is unmaintained and deprecated.
     This wrapper module will be removed in a future Docutils version.

   * The API is not settled and may change with any minor Docutils version.
"""

from docutils import Component
from docutils import nodes

try:
    # If possible, import Sphinx's 'addnodes'
    from sphinx import addnodes
except ImportError:
    # stub to prevent errors if Sphinx isn't installed
    import sys
    import types

    class pending_xref(nodes.Inline, nodes.Element): ... # NoQA

    sys.modules['sphinx'] = sphinx = types.ModuleType('sphinx')
    sphinx.addnodes = addnodes = types.SimpleNamespace()
    addnodes.pending_xref = pending_xref
try:
    import recommonmark
    from recommonmark.parser import CommonMarkParser
except ImportError as err:
    raise ImportError(
        'Parsing "recommonmark" Markdown flavour requires the\n'
        '  package https://pypi.org/project/recommonmark.'
    ) from err
else:
    if recommonmark.__version__ < '0.6.0':
        raise ImportError('The installed version of "recommonmark" is too old.'
                          ' Update with "pip install -U recommonmark".')


# auxiliary function for `document.findall()`
def is_literal(node):
    return isinstance(node, (nodes.literal, nodes.literal_block))


class Parser(CommonMarkParser):
```

```
"""MarkDown parser based on recommonmark.

This parser is provisional:
the API is not settled and may change with any minor Docutils version.
"""
supported = ('recommonmark', 'commonmark', 'markdown', 'md')
"""Formats this parser supports."""

config_section = 'recommonmark parser'
config_section_dependencies = ('parsers',)

def get_transforms(self):
    return Component.get_transforms(self)  # + [AutoStructify]

def parse(self, inputstring, document):
    """Use the upstream parser and clean up afterwards.
    """
    # check for exorbitantly long lines
    for i, line in enumerate(inputstring.split('\n')):
        if len(line) > document.settings.line_length_limit:
            error = document.reporter.error(
                'Line %d exceeds the line-length-limit.'%(i+1))
            document.append(error)
            return

    # pass to upstream parser
    try:
        CommonMarkParser.parse(self, inputstring, document)
    except Exception as err:
        if document.settings.traceback:
            raise err
        error = document.reporter.error('Parsing with "recommonmark" '
                                        'returned the error:\n%s'%err)
        document.append(error)

    # Post-Processing
    # ---------------

    # merge adjoining Text nodes:
    for node in document.findall(nodes.TextElement):
        children = node.children
        i = 0
        while i+1 < len(children):
            if (isinstance(children[i], nodes.Text)
                and isinstance(children[i+1], nodes.Text)):
                children[i] = nodes.Text(children[i]+children.pop(i+1))
                children[i].parent = node
            else:
                i += 1

    # add "code" class argument to literal elements (inline and block)
    for node in document.findall(is_literal):
        if 'code' not in node['classes']:
            node['classes'].append('code')
    # move "language" argument to classes
    for node in document.findall(nodes.literal_block):
        if 'language' in node.attributes:
            node['classes'].append(node['language'])
            del node['language']

    # replace raw nodes if raw is not allowed
    if not document.settings.raw_enabled:
        for node in document.findall(nodes.raw):
            warning = document.reporter.warning('Raw content disabled.')
            node.parent.replace(node, warning)

    # drop pending_xref (Sphinx cross reference extension)
    for node in document.findall(addnodes.pending_xref):
        reference = node.children[0]
        if 'name' not in reference:
            reference['name'] = nodes.fully_normalize_name(
                                                reference.astext())
        node.parent.replace(node, reference)

def visit_document(self, node):
    """Dummy function to prevent spurious warnings.

    cf. https://github.com/readthedocs/recommonmark/issues/177
    """
    pass
# Overwrite parent method with version that
```

```
    # doesn't pass deprecated `rawsource` argument to nodes.Text:
    def visit_text(self, mdnode):
        self.current_node.append(nodes.Text(mdnode.literal))
```

File: .venv\Lib\site-packages\docutils\parsers__init__.py

```
# $Id: __init__.py 9048 2022-03-29 21:50:15Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
This package contains Docutils parser modules.
"""

__docformat__ = 'reStructuredText'

from importlib import import_module

from docutils import Component, frontend


class Parser(Component):
    settings_spec = (
        'Generic Parser Options',
        None,
        (('Disable directives that insert the contents of an external file; '
          'replaced with a "warning" system message.',
          ['--no-file-insertion'],
          {'action': 'store_false', 'default': 1,
           'dest': 'file_insertion_enabled',
           'validator': frontend.validate_boolean}),
         ('Enable directives that insert the contents '
          'of an external file. (default)',
          ['--file-insertion-enabled'],
          {'action': 'store_true'}),
         ('Disable the "raw" directive; '
          'replaced with a "warning" system message.',
          ['--no-raw'],
          {'action': 'store_false', 'default': 1, 'dest': 'raw_enabled',
           'validator': frontend.validate_boolean}),
         ('Enable the "raw" directive. (default)',
          ['--raw-enabled'],
          {'action': 'store_true'}),
         ('Maximal number of characters in an input line. Default 10 000.',
          ['--line-length-limit'],
          {'metavar': '<length>', 'type': 'int', 'default': 10000,
           'validator': frontend.validate_nonnegative_int}),
         )
        )
    component_type = 'parser'
    config_section = 'parsers'

    def parse(self, inputstring, document):
        """Override to parse `inputstring` into document tree `document`."""
        raise NotImplementedError('subclass must override this method')

    def setup_parse(self, inputstring, document):
        """Initial parse setup.  Call at start of `self.parse()`."""
        self.inputstring = inputstring
        # provide fallbacks in case the document has only generic settings
        document.settings.setdefault('file_insertion_enabled', False)
        document.settings.setdefault('raw_enabled', False)
        document.settings.setdefault('line_length_limit', 10000)
        self.document = document
        document.reporter.attach_observer(document.note_parse_message)

    def finish_parse(self):
        """Finalize parse details.  Call at end of `self.parse()`."""
        self.document.reporter.detach_observer(
            self.document.note_parse_message)


_parser_aliases = {  # short names for known parsers
                    'null': 'docutils.parsers.null',
                    # reStructuredText
                    'rst': 'docutils.parsers.rst',
                    'restructuredtext': 'docutils.parsers.rst',
                    'rest': 'docutils.parsers.rst',
                    'restx': 'docutils.parsers.rst',
```

```
                    'rtxt': 'docutils.parsers.rst',
                    # 3rd-party Markdown parsers
                    'recommonmark': 'docutils.parsers.recommonmark_wrapper',
                    'myst': 'myst_parser.docutils_',
                    # 'pycmark': works out of the box
                    # dispatcher for 3rd-party Markdown parsers
                    'commonmark': 'docutils.parsers.commonmark_wrapper',
                    'markdown': 'docutils.parsers.commonmark_wrapper',
                   }


def get_parser_class(parser_name):
    """Return the Parser class from the `parser_name` module."""
    name = parser_name.lower()
    try:
        module = import_module(_parser_aliases.get(name, name))
    except ImportError as err:
        raise ImportError(f'Parser "{parser_name}" not found. {err}')
    return module.Parser
```

File: .venv\Lib\site-packages\docutils\parsers\rst\roles.py

```
# $Id: roles.py 9037 2022-03-05 23:31:10Z milde $
# Author: Edward Loper <edloper@gradient.cis.upenn.edu>
# Copyright: This module has been placed in the public domain.

"""
This module defines standard interpreted text role functions, a registry for
interpreted text roles, and an API for adding to and retrieving from the
registry. See also `Creating reStructuredText Interpreted Text Roles`__.

__ https://docutils.sourceforge.io/docs/ref/rst/roles.html


The interface for interpreted role functions is as follows::

    def role_fn(name, rawtext, text, lineno, inliner,
                options=None, content=None):
        code...

    # Set function attributes for customization:
    role_fn.options = ...
    role_fn.content = ...

Parameters:

- ``name`` is the local name of the interpreted text role, the role name
  actually used in the document.

- ``rawtext`` is a string containing the entire interpreted text construct.
  Return it as a ``problematic`` node linked to a system message if there is a
  problem.

- ``text`` is the interpreted text content, with backslash escapes converted
  to nulls (``\x00``).

- ``lineno`` is the line number where the text block containing the
  interpreted text begins.

- ``inliner`` is the Inliner object that called the role function.
  It defines the following useful attributes: ``reporter``,
  ``problematic``, ``memo``, ``parent``, ``document``.

- ``options``: A dictionary of directive options for customization, to be
  interpreted by the role function.  Used for additional attributes for the
  generated elements and other functionality.

- ``content``: A list of strings, the directive content for customization
  ("role" directive).  To be interpreted by the role function.

Function attributes for customization, interpreted by the "role" directive:

- ``options``: A dictionary, mapping known option names to conversion
  functions such as `int` or `float`.  ``None`` or an empty dict implies no
  options to parse.  Several directive option conversion functions are defined
  in the `directives` module.

  All role functions implicitly support the "class" option, unless disabled
  with an explicit ``{'class': None}``.
```

```
- ``content``: A boolean; true if content is allowed.  Client code must handle
  the case where content is required but not supplied (an empty content list
  will be supplied).

Note that unlike directives, the "arguments" function attribute is not
supported for role customization.  Directive arguments are handled by the
"role" directive itself.

Interpreted role functions return a tuple of two values:

- A list of nodes which will be inserted into the document tree at the
  point where the interpreted role was encountered (can be an empty
  list).

- A list of system messages, which will be inserted into the document tree
  immediately after the end of the current inline block (can also be empty).
"""

__docformat__ = 'reStructuredText'


from docutils import nodes
from docutils.parsers.rst import directives
from docutils.parsers.rst.languages import en as _fallback_language_module
from docutils.utils.code_analyzer import Lexer, LexerError

DEFAULT_INTERPRETED_ROLE = 'title-reference'
"""The canonical name of the default interpreted role.

This role is used when no role is specified for a piece of interpreted text.
"""

_role_registry = {}
"""Mapping of canonical role names to role functions.

Language-dependent role names are defined in the ``language`` subpackage.
"""

_roles = {}
"""Mapping of local or language-dependent interpreted text role names to role
functions."""


def role(role_name, language_module, lineno, reporter):
    """
    Locate and return a role function from its language-dependent name, along
    with a list of system messages.

    If the role is not found in the current language, check English. Return a
    2-tuple: role function (``None`` if the named role cannot be found) and a
    list of system messages.
    """
    normname = role_name.lower()
    messages = []
    msg_text = []

    if normname in _roles:
        return _roles[normname], messages

    if role_name:
        canonicalname = None
        try:
            canonicalname = language_module.roles[normname]
        except AttributeError as error:
            msg_text.append('Problem retrieving role entry from language '
                            'module %r: %s.' % (language_module, error))
        except KeyError:
            msg_text.append('No role entry for "%s" in module "%s".'
                            % (role_name, language_module.__name__))
    else:
        canonicalname = DEFAULT_INTERPRETED_ROLE

    # If we didn't find it, try English as a fallback.
    if not canonicalname:
        try:
            canonicalname = _fallback_language_module.roles[normname]
            msg_text.append('Using English fallback for role "%s".'
                            % role_name)
        except KeyError:
            msg_text.append('Trying "%s" as canonical role name.'
                            % role_name)
```

```
            # The canonical name should be an English name, but just in case:
            canonicalname = normname

    # Collect any messages that we generated.
    if msg_text:
        message = reporter.info('\n'.join(msg_text), line=lineno)
        messages.append(message)

    # Look the role up in the registry, and return it.
    if canonicalname in _role_registry:
        role_fn = _role_registry[canonicalname]
        register_local_role(normname, role_fn)
        return role_fn, messages
    return None, messages  # Error message will be generated by caller.


def register_canonical_role(name, role_fn):
    """
    Register an interpreted text role by its canonical name.

    :Parameters:
      - `name`: The canonical name of the interpreted role.
      - `role_fn`: The role function.  See the module docstring.
    """
    set_implicit_options(role_fn)
    _role_registry[name.lower()] = role_fn


def register_local_role(name, role_fn):
    """
    Register an interpreted text role by its local or language-dependent name.

    :Parameters:
      - `name`: The local or language-dependent name of the interpreted role.
      - `role_fn`: The role function.  See the module docstring.
    """
    set_implicit_options(role_fn)
    _roles[name.lower()] = role_fn


def set_implicit_options(role_fn):
    """
    Add customization options to role functions, unless explicitly set or
    disabled.
    """
    if not hasattr(role_fn, 'options') or role_fn.options is None:
        role_fn.options = {'class': directives.class_option}
    elif 'class' not in role_fn.options:
        role_fn.options['class'] = directives.class_option


def register_generic_role(canonical_name, node_class):
    """For roles which simply wrap a given `node_class` around the text."""
    role = GenericRole(canonical_name, node_class)
    register_canonical_role(canonical_name, role)


class GenericRole:
    """
    Generic interpreted text role.

    The interpreted text is simply wrapped with the provided node class.
    """

    def __init__(self, role_name, node_class):
        self.name = role_name
        self.node_class = node_class

    def __call__(self, role, rawtext, text, lineno, inliner,
                 options=None, content=None):
        options = normalized_role_options(options)
        return [self.node_class(rawtext, text, **options)], []


class CustomRole:
    """Wrapper for custom interpreted text roles."""

    def __init__(self, role_name, base_role, options=None, content=None):
        self.name = role_name
        self.base_role = base_role
        self.options = getattr(base_role, 'options', None)
```

```
        self.content = getattr(base_role, 'content', None)
        self.supplied_options = options
        self.supplied_content = content

    def __call__(self, role, rawtext, text, lineno, inliner,
                 options=None, content=None):
        opts = normalized_role_options(self.supplied_options)
        try:
            opts.update(options)
        except TypeError:  # options may be ``None``
            pass
        # pass concatenation of content from instance and call argument:
        supplied_content = self.supplied_content or []
        content = content or []
        delimiter = ['\n'] if supplied_content and content else []
        return self.base_role(role, rawtext, text, lineno, inliner,
                              options=opts,
                              content=supplied_content+delimiter+content)


def generic_custom_role(role, rawtext, text, lineno, inliner,
                        options=None, content=None):
    """Base for custom roles if no other base role is specified."""
    # Once nested inline markup is implemented, this and other methods should
    # recursively call inliner.nested_parse().
    options = normalized_role_options(options)
    return [nodes.inline(rawtext, text, **options)], []


generic_custom_role.options = {'class': directives.class_option}


###########################################################################
# Define and register the standard roles:
###########################################################################

register_generic_role('abbreviation', nodes.abbreviation)
register_generic_role('acronym', nodes.acronym)
register_generic_role('emphasis', nodes.emphasis)
register_generic_role('literal', nodes.literal)
register_generic_role('strong', nodes.strong)
register_generic_role('subscript', nodes.subscript)
register_generic_role('superscript', nodes.superscript)
register_generic_role('title-reference', nodes.title_reference)


def pep_reference_role(role, rawtext, text, lineno, inliner,
                       options=None, content=None):
    options = normalized_role_options(options)
    try:
        pepnum = int(nodes.unescape(text))
        if pepnum < 0 or pepnum > 9999:
            raise ValueError
    except ValueError:
        msg = inliner.reporter.error(
            'PEP number must be a number from 0 to 9999; "%s" is invalid.'
            % text, line=lineno)
        prb = inliner.problematic(rawtext, rawtext, msg)
        return [prb], [msg]
    # Base URL mainly used by inliner.pep_reference; so this is correct:
    ref = (inliner.document.settings.pep_base_url
           + inliner.document.settings.pep_file_url_template % pepnum)
    return [nodes.reference(rawtext, 'PEP ' + text, refuri=ref, **options)], []


register_canonical_role('pep-reference', pep_reference_role)


def rfc_reference_role(role, rawtext, text, lineno, inliner,
                       options=None, content=None):
    options = normalized_role_options(options)
    if "#" in text:
        rfcnum, section = nodes.unescape(text).split("#", 1)
    else:
        rfcnum, section = nodes.unescape(text), None
    try:
        rfcnum = int(rfcnum)
        if rfcnum < 1:
            raise ValueError
    except ValueError:
        msg = inliner.reporter.error(
```

```
            'RFC number must be a number greater than or equal to 1; '
            '"%s" is invalid.' % text, line=lineno)
        prb = inliner.problematic(rawtext, rawtext, msg)
        return [prb], [msg]
    # Base URL mainly used by inliner.rfc_reference, so this is correct:
    ref = inliner.document.settings.rfc_base_url + inliner.rfc_url % rfcnum
    if section is not None:
        ref += "#" + section
    node = nodes.reference(rawtext, 'RFC '+str(rfcnum), refuri=ref, **options)
    return [node], []


register_canonical_role('rfc-reference', rfc_reference_role)


def raw_role(role, rawtext, text, lineno, inliner, options=None, content=None):
    options = normalized_role_options(options)
    if not inliner.document.settings.raw_enabled:
        msg = inliner.reporter.warning('raw (and derived) roles disabled')
        prb = inliner.problematic(rawtext, rawtext, msg)
        return [prb], [msg]
    if 'format' not in options:
        msg = inliner.reporter.error(
            'No format (Writer name) is associated with this role: "%s".\n'
            'The "raw" role cannot be used directly.\n'
            'Instead, use the "role" directive to create a new role with '
            'an associated format.' % role, line=lineno)
        prb = inliner.problematic(rawtext, rawtext, msg)
        return [prb], [msg]
    node = nodes.raw(rawtext, nodes.unescape(text, True), **options)
    node.source, node.line = inliner.reporter.get_source_and_line(lineno)
    return [node], []


raw_role.options = {'format': directives.unchanged}

register_canonical_role('raw', raw_role)


def code_role(role, rawtext, text, lineno, inliner,
              options=None, content=None):
    options = normalized_role_options(options)
    language = options.get('language', '')
    classes = ['code']
    if 'classes' in options:
        classes.extend(options['classes'])
    if language and language not in classes:
        classes.append(language)
    try:
        tokens = Lexer(nodes.unescape(text, True), language,
                       inliner.document.settings.syntax_highlight)
    except LexerError as error:
        msg = inliner.reporter.warning(error)
        prb = inliner.problematic(rawtext, rawtext, msg)
        return [prb], [msg]

    node = nodes.literal(rawtext, '', classes=classes)

    # analyse content and add nodes for every token
    for classes, value in tokens:
        if classes:
            node += nodes.inline(value, value, classes=classes)
        else:
            # insert as Text to decrease the verbosity of the output
            node += nodes.Text(value)

    return [node], []


code_role.options = {'language': directives.unchanged}

register_canonical_role('code', code_role)


def math_role(role, rawtext, text, lineno, inliner,
              options=None, content=None):
    options = normalized_role_options(options)
    text = nodes.unescape(text, True)  # raw text without inline role markup
    node = nodes.math(rawtext, text, **options)
    return [node], []
register_canonical_role('math', math_role)
```

```
######################################################################
# Register roles that are currently unimplemented.
######################################################################

def unimplemented_role(role, rawtext, text, lineno, inliner,
                       options=None, content=None):
    msg = inliner.reporter.error(
        'Interpreted text role "%s" not implemented.' % role, line=lineno)
    prb = inliner.problematic(rawtext, rawtext, msg)
    return [prb], [msg]


register_canonical_role('index', unimplemented_role)
register_canonical_role('named-reference', unimplemented_role)
register_canonical_role('anonymous-reference', unimplemented_role)
register_canonical_role('uri-reference', unimplemented_role)
register_canonical_role('footnote-reference', unimplemented_role)
register_canonical_role('citation-reference', unimplemented_role)
register_canonical_role('substitution-reference', unimplemented_role)
register_canonical_role('target', unimplemented_role)

# This should remain unimplemented, for testing purposes:
register_canonical_role('restructuredtext-unimplemented-role',
                        unimplemented_role)


def set_classes(options):
    """Deprecated. Obsoleted by ``normalized_role_options()``."""
    # TODO: Change use in directives.py and uncomment.
    # warnings.warn('The auxiliary function roles.set_classes() is obsoleted'
    #     ' by roles.normalized_role_options() and will be removed'
    #     ' in Docutils 0.21 or later', DeprecationWarning, stacklevel=2)
    if options and 'class' in options:
        assert 'classes' not in options
        options['classes'] = options['class']
        del options['class']


def normalized_role_options(options):
    """
    Return normalized dictionary of role options.

    * ``None`` is replaced by an empty dictionary.
    * The key 'class' is renamed to 'classes'.
    """
    if options is None:
        return {}
    result = options.copy()
    if 'class' in result:
        assert 'classes' not in result
        result['classes'] = result['class']
        del result['class']
    return result
```

File: .venv\Lib\site-packages\docutils\parsers\rst\tableparser.py

```
# $Id: tableparser.py 9038 2022-03-05 23:31:46Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
This module defines table parser classes,which parse plaintext-graphic tables
and produce a well-formed data structure suitable for building a CALS table.

:Classes:
    - `GridTableParser`: Parse fully-formed tables represented with a grid.
    - `SimpleTableParser`: Parse simple tables, delimited by top & bottom
      borders.

:Exception class: `TableMarkupError`

:Function:
    `update_dict_of_lists()`: Merge two dictionaries containing list values.
"""

__docformat__ = 'reStructuredText'


import re
```

```
import sys
from docutils import DataError
from docutils.utils import strip_combining_chars


class TableMarkupError(DataError):

    """
    Raise if there is any problem with table markup.

    The keyword argument `offset` denotes the offset of the problem
    from the table's start line.
    """

    def __init__(self, *args, **kwargs):
        self.offset = kwargs.pop('offset', 0)
        DataError.__init__(self, *args)


class TableParser:

    """
    Abstract superclass for the common parts of the syntax-specific parsers.
    """

    head_body_separator_pat = None
    """Matches the row separator between head rows and body rows."""

    double_width_pad_char = '\x00'
    """Padding character for East Asian double-width text."""

    def parse(self, block):
        """
        Analyze the text `block` and return a table data structure.

        Given a plaintext-graphic table in `block` (list of lines of text; no
        whitespace padding), parse the table, construct and return the data
        necessary to construct a CALS table or equivalent.

        Raise `TableMarkupError` if there is any problem with the markup.
        """
        self.setup(block)
        self.find_head_body_sep()
        self.parse_table()
        return self.structure_from_cells()

    def find_head_body_sep(self):
        """Look for a head/body row separator line; store the line index."""
        for i in range(len(self.block)):
            line = self.block[i]
            if self.head_body_separator_pat.match(line):
                if self.head_body_sep:
                    raise TableMarkupError(
                        'Multiple head/body row separators '
                        '(table lines %s and %s); only one allowed.'
                       % (self.head_body_sep+1, i+1), offset=i)
                else:
                    self.head_body_sep = i
                    self.block[i] = line.replace('=', '-')
        if self.head_body_sep == 0 or self.head_body_sep == (len(self.block)
                                                             - 1):
            raise TableMarkupError('The head/body row separator may not be '
                                   'the first or last line of the table.',
                                   offset=i)


class GridTableParser(TableParser):

    """
    Parse a grid table using `parse()`.

    Here's an example of a grid table::

        +------------------------+------------+----------+----------+
        | Header row, column 1   | Header 2   | Header 3 | Header 4 |
        +========================+============+==========+==========+
        | body row 1, column 1   | column 2   | column 3 | column 4 |
        +------------------------+------------+----------+----------+
        | body row 2             | Cells may span columns.          |
        +------------------------+------------+---------------------+
        | body row 3             | Cells may  | - Table cells       |
```

```
    +------------------------+ span rows. | - contain           |
    | body row 4             |            | - body elements.    |
    +------------------------+------------+---------------------+

Intersections use '+', row separators use '-' (except for one optional
head/body row separator, which uses '='), and column separators use '|'.

Passing the above table to the `parse()` method will result in the
following data structure::

    ([24, 12, 10, 10],
     [[(0, 0, 1, ['Header row, column 1']),
       (0, 0, 1, ['Header 2']),
       (0, 0, 1, ['Header 3']),
       (0, 0, 1, ['Header 4'])]],
     [[(0, 0, 3, ['body row 1, column 1']),
       (0, 0, 3, ['column 2']),
       (0, 0, 3, ['column 3']),
       (0, 0, 3, ['column 4'])],
      [(0, 0, 5, ['body row 2']),
       (0, 2, 5, ['Cells may span columns.']),
       None,
       None],
      [(0, 0, 7, ['body row 3']),
       (1, 0, 7, ['Cells may', 'span rows.', '']),
       (1, 1, 7, ['- Table cells', '- contain', '- body elements.']),
       None],
      [(0, 0, 9, ['body row 4']), None, None, None]])

The first item is a list containing column widths (colspecs). The second
item is a list of head rows, and the third is a list of body rows. Each
row contains a list of cells. Each cell is either None (for a cell unused
because of another cell's span), or a tuple. A cell tuple contains four
items: the number of extra rows used by the cell in a vertical span
(morerows); the number of extra columns used by the cell in a horizontal
span (morecols); the line offset of the first line of the cell contents;
and the cell contents, a list of lines of text.
"""

head_body_separator_pat = re.compile(r'\+=[=+]+=\+ *$')

def setup(self, block):
    self.block = block[:]           # make a copy; it may be modified
    self.block.disconnect()         # don't propagate changes to parent
    self.bottom = len(block) - 1
    self.right = len(block[0]) - 1
    self.head_body_sep = None
    self.done = [-1] * len(block[0])
    self.cells = []
    self.rowseps = {0: [0]}
    self.colseps = {0: [0]}

def parse_table(self):
    """
    Start with a queue of upper-left corners, containing the upper-left
    corner of the table itself. Trace out one rectangular cell, remember
    it, and add its upper-right and lower-left corners to the queue of
    potential upper-left corners of further cells. Process the queue in
    top-to-bottom order, keeping track of how much of each text column has
    been seen.

    We'll end up knowing all the row and column boundaries, cell positions
    and their dimensions.
    """
    corners = [(0, 0)]
    while corners:
        top, left = corners.pop(0)
        if (top == self.bottom
            or left == self.right
            or top <= self.done[left]):
            continue
        result = self.scan_cell(top, left)
        if not result:
            continue
        bottom, right, rowseps, colseps = result
        update_dict_of_lists(self.rowseps, rowseps)
        update_dict_of_lists(self.colseps, colseps)
        self.mark_done(top, left, bottom, right)
        cellblock = self.block.get_2D_block(top + 1, left + 1,
                                            bottom, right)
        cellblock.disconnect()      # lines in cell can't sync with parent
```

```
            cellblock.replace(self.double_width_pad_char, '')
            self.cells.append((top, left, bottom, right, cellblock))
            corners.extend([(top, right), (bottom, left)])
            corners.sort()
        if not self.check_parse_complete():
            raise TableMarkupError('Malformed table; parse incomplete.')

    def mark_done(self, top, left, bottom, right):
        """For keeping track of how much of each text column has been seen."""
        before = top - 1
        after = bottom - 1
        for col in range(left, right):
            assert self.done[col] == before
            self.done[col] = after

    def check_parse_complete(self):
        """Each text column should have been completely seen."""
        last = self.bottom - 1
        for col in range(self.right):
            if self.done[col] != last:
                return False
        return True

    def scan_cell(self, top, left):
        """Starting at the top-left corner, start tracing out a cell."""
        assert self.block[top][left] == '+'
        return self.scan_right(top, left)

    def scan_right(self, top, left):
        """
        Look for the top-right corner of the cell, and make note of all column
        boundaries ('+').
        """
        colseps = {}
        line = self.block[top]
        for i in range(left + 1, self.right + 1):
            if line[i] == '+':
                colseps[i] = [top]
                result = self.scan_down(top, left, i)
                if result:
                    bottom, rowseps, newcolseps = result
                    update_dict_of_lists(colseps, newcolseps)
                    return bottom, i, rowseps, colseps
            elif line[i] != '-':
                return None
        return None

    def scan_down(self, top, left, right):
        """
        Look for the bottom-right corner of the cell, making note of all row
        boundaries.
        """
        rowseps = {}
        for i in range(top + 1, self.bottom + 1):
            if self.block[i][right] == '+':
                rowseps[i] = [right]
                result = self.scan_left(top, left, i, right)
                if result:
                    newrowseps, colseps = result
                    update_dict_of_lists(rowseps, newrowseps)
                    return i, rowseps, colseps
            elif self.block[i][right] != '|':
                return None
        return None

    def scan_left(self, top, left, bottom, right):
        """
        Noting column boundaries, look for the bottom-left corner of the cell.
        It must line up with the starting point.
        """
        colseps = {}
        line = self.block[bottom]
        for i in range(right - 1, left, -1):
            if line[i] == '+':
                colseps[i] = [bottom]
            elif line[i] != '-':
                return None
        if line[left] != '+':
            return None
        result = self.scan_up(top, left, bottom, right)
        if result is not None:
```

```
            rowseps = result
            return rowseps, colseps
        return None

    def scan_up(self, top, left, bottom, right):
        """
        Noting row boundaries, see if we can return to the starting point.
        """
        rowseps = {}
        for i in range(bottom - 1, top, -1):
            if self.block[i][left] == '+':
                rowseps[i] = [left]
            elif self.block[i][left] != '|':
                return None
        return rowseps

    def structure_from_cells(self):
        """
        From the data collected by `scan_cell()`, convert to the final data
        structure.
        """
        rowseps = sorted(self.rowseps.keys())   # list of row boundaries
        rowindex = {}
        for i in range(len(rowseps)):
            rowindex[rowseps[i]] = i    # row boundary -> row number mapping
        colseps = sorted(self.colseps.keys())   # list of column boundaries
        colindex = {}
        for i in range(len(colseps)):
            colindex[colseps[i]] = i    # column boundary -> col number map
        colspecs = [(colseps[i] - colseps[i - 1] - 1)
                    for i in range(1, len(colseps))]  # list of column widths
        # prepare an empty table with the correct number of rows & columns
        onerow = [None for i in range(len(colseps) - 1)]
        rows = [onerow[:] for i in range(len(rowseps) - 1)]
        # keep track of # of cells remaining; should reduce to zero
        remaining = (len(rowseps) - 1) * (len(colseps) - 1)
        for top, left, bottom, right, block in self.cells:
            rownum = rowindex[top]
            colnum = colindex[left]
            assert rows[rownum][colnum] is None, (
                  'Cell (row %s, column %s) already used.'
                  % (rownum + 1, colnum + 1))
            morerows = rowindex[bottom] - rownum - 1
            morecols = colindex[right] - colnum - 1
            remaining -= (morerows + 1) * (morecols + 1)
            # write the cell into the table
            rows[rownum][colnum] = (morerows, morecols, top + 1, block)
        assert remaining == 0, 'Unused cells remaining.'
        if self.head_body_sep:          # separate head rows from body rows
            numheadrows = rowindex[self.head_body_sep]
            headrows = rows[:numheadrows]
            bodyrows = rows[numheadrows:]
        else:
            headrows = []
            bodyrows = rows
        return colspecs, headrows, bodyrows


class SimpleTableParser(TableParser):

    """
    Parse a simple table using `parse()`.

    Here's an example of a simple table::

        =====  =====
        col 1  col 2
        =====  =====
        1      Second column of row 1.
        2      Second column of row 2.
               Second line of paragraph.
        3      - Second column of row 3.

               - Second item in bullet
                 list (row 3, column 2).
        4 is a span
        ------------
        5
        =====  =====

    Top and bottom borders use '=', column span underlines use '-', column
```

```
separation is indicated with spaces.

Passing the above table to the `parse()` method will result in the
following data structure, whose interpretation is the same as for
`GridTableParser`::

    ([5, 25],
     [[(0, 0, 1, ['col 1']),
       (0, 0, 1, ['col 2'])]],
     [[(0, 0, 3, ['1']),
       (0, 0, 3, ['Second column of row 1.'])],
      [(0, 0, 4, ['2']),
       (0, 0, 4, ['Second column of row 2.',
                  'Second line of paragraph.'])],
      [(0, 0, 6, ['3']),
       (0, 0, 6, ['- Second column of row 3.',
                  '',
                  '- Second item in bullet',
                  '  list (row 3, column 2).'])],
      [(0, 1, 10, ['4 is a span'])],
      [(0, 0, 12, ['5']),
       (0, 0, 12, ['''])]])
"""

head_body_separator_pat = re.compile('=[ =]*$')
span_pat = re.compile('-[ -]*$')

def setup(self, block):
    self.block = block[:]           # make a copy; it will be modified
    self.block.disconnect()         # don't propagate changes to parent
    # Convert top & bottom borders to column span underlines:
    self.block[0] = self.block[0].replace('=', '-')
    self.block[-1] = self.block[-1].replace('=', '-')
    self.head_body_sep = None
    self.columns = []
    self.border_end = None
    self.table = []
    self.done = [-1] * len(block[0])
    self.rowseps = {0: [0]}
    self.colseps = {0: [0]}

def parse_table(self):
    """
    First determine the column boundaries from the top border, then
    process rows.  Each row may consist of multiple lines; accumulate
    lines until a row is complete.  Call `self.parse_row` to finish the
    job.
    """
    # Top border must fully describe all table columns.
    self.columns = self.parse_columns(self.block[0], 0)
    self.border_end = self.columns[-1][1]
    firststart, firstend = self.columns[0]
    offset = 1                      # skip top border
    start = 1
    text_found = None
    while offset < len(self.block):
        line = self.block[offset]
        if self.span_pat.match(line):
            # Column span underline or border; row is complete.
            self.parse_row(self.block[start:offset], start,
                           (line.rstrip(), offset))
            start = offset + 1
            text_found = None
        elif line[firststart:firstend].strip():
            # First column not blank, therefore it's a new row.
            if text_found and offset != start:
                self.parse_row(self.block[start:offset], start)
            start = offset
            text_found = 1
        elif not text_found:
            start = offset + 1
        offset += 1

def parse_columns(self, line, offset):
    """
    Given a column span underline, return a list of (begin, end) pairs.
    """
    cols = []
    end = 0
    while True:
        begin = line.find('-', end)
```

```
        end = line.find(' ', begin)
        if begin < 0:
            break
        if end < 0:
            end = len(line)
        cols.append((begin, end))
    if self.columns:
        if cols[-1][1] != self.border_end:
            raise TableMarkupError('Column span incomplete in table '
                                   'line %s.' % (offset+1),
                                   offset=offset)
        # Allow for an unbounded rightmost column:
        cols[-1] = (cols[-1][0], self.columns[-1][1])
    return cols

def init_row(self, colspec, offset):
    i = 0
    cells = []
    for start, end in colspec:
        morecols = 0
        try:
            assert start == self.columns[i][0]
            while end != self.columns[i][1]:
                i += 1
                morecols += 1
        except (AssertionError, IndexError):
            raise TableMarkupError('Column span alignment problem '
                                   'in table line %s.' % (offset+2),
                                   offset=offset+1)
        cells.append([0, morecols, offset, []])
        i += 1
    return cells

def parse_row(self, lines, start, spanline=None):
    """
    Given the text `lines` of a row, parse it and append to `self.table`.

    The row is parsed according to the current column spec (either
    `spanline` if provided or `self.columns`).  For each column, extract
    text from each line, and check for text in column margins.  Finally,
    adjust for insignificant whitespace.
    """
    if not (lines or spanline):
        # No new row, just blank lines.
        return
    if spanline:
        columns = self.parse_columns(*spanline)
    else:
        columns = self.columns[:]
    self.check_columns(lines, start, columns)
    row = self.init_row(columns, start)
    for i in range(len(columns)):
        start, end = columns[i]
        cellblock = lines.get_2D_block(0, start, len(lines), end)
        cellblock.disconnect()      # lines in cell can't sync with parent
        cellblock.replace(self.double_width_pad_char, '')
        row[i][3] = cellblock
    self.table.append(row)

def check_columns(self, lines, first_line, columns):
    """
    Check for text in column margins and text overflow in the last column.
    Raise TableMarkupError if anything but whitespace is in column margins.
    Adjust the end value for the last column if there is text overflow.
    """
    # "Infinite" value for a dummy last column's beginning, used to
    # check for text overflow:
    columns.append((sys.maxsize, None))
    lastcol = len(columns) - 2
    # combining characters do not contribute to the column width
    lines = [strip_combining_chars(line) for line in lines]

    for i in range(len(columns) - 1):
        start, end = columns[i]
        nextstart = columns[i+1][0]
        offset = 0
        for line in lines:
            if i == lastcol and line[end:].strip():
                text = line[start:].rstrip()
                new_end = start + len(text)
                main_start, main_end = self.columns[-1]
```

```
                        columns[i] = (start, max(main_end, new_end))
                        if new_end > main_end:
                            self.columns[-1] = (main_start, new_end)
                    elif line[end:nextstart].strip():
                        raise TableMarkupError('Text in column margin in table '
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\docutils\parsers\rst__init__.py

```
# $Id: __init__.py 9258 2022-11-21 14:51:43Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
This is ``docutils.parsers.rst`` package. It exports a single class, `Parser`,
the reStructuredText parser.


Usage
=====

1. Create a parser::

       parser = docutils.parsers.rst.Parser()

   Several optional arguments may be passed to modify the parser's behavior.
   Please see `Customizing the Parser`_ below for details.

2. Gather input (a multi-line string), by reading a file or the standard
   input::

       input = sys.stdin.read()

3. Create a new empty `docutils.nodes.document` tree::

       document = docutils.utils.new_document(source, settings)

   See `docutils.utils.new_document()` for parameter details.

4. Run the parser, populating the document tree::

       parser.parse(input, document)


Parser Overview
===============

The reStructuredText parser is implemented as a state machine, examining its
input one line at a time. To understand how the parser works, please first
become familiar with the `docutils.statemachine` module, then see the
`states` module.


Customizing the Parser
----------------------

Anything that isn't already customizable is that way simply because that type
of customizability hasn't been implemented yet.  Patches welcome!

When instantiating an object of the `Parser` class, two parameters may be
passed: ``rfc2822`` and ``inliner``.  Pass ``rfc2822=True`` to enable an
initial RFC-2822 style header block, parsed as a "field_list" element (with
"class" attribute set to "rfc2822").  Currently this is the only body-level
element which is customizable without subclassing.  (Tip: subclass `Parser`
and change its "state_classes" and "initial_state" attributes to refer to new
classes. Contact the author if you need more details.)

The ``inliner`` parameter takes an instance of `states.Inliner` or a subclass.
It handles inline markup recognition.  A common extension is the addition of
further implicit hyperlinks, like "RFC 2822".  This can be done by subclassing
`states.Inliner`, adding a new method for the implicit markup, and adding a
``(pattern, method)`` pair to the "implicit_dispatch" attribute of the
subclass.  See `states.Inliner.implicit_inline()` for details.  Explicit
inline markup can be customized in a `states.Inliner` subclass via the
``patterns.initial`` and ``dispatch`` attributes (and new methods as
appropriate).
"""

__docformat__ = 'reStructuredText'
```

```
import docutils.parsers
import docutils.statemachine
from docutils.parsers.rst import roles, states
from docutils import frontend, nodes
from docutils.transforms import universal


class Parser(docutils.parsers.Parser):

    """The reStructuredText parser."""

    supported = ('rst', 'restructuredtext', 'rest', 'restx', 'rtxt', 'rstx')
    """Aliases this parser supports."""

    settings_spec = docutils.parsers.Parser.settings_spec + (
        'reStructuredText Parser Options',
        None,
        (('Recognize and link to standalone PEP references (like "PEP 258").',
          ['--pep-references'],
          {'action': 'store_true', 'validator': frontend.validate_boolean}),
         ('Base URL for PEP references '
          '(default "https://peps.python.org/").',
          ['--pep-base-url'],
          {'metavar': '<URL>', 'default': 'https://peps.python.org/',
           'validator': frontend.validate_url_trailing_slash}),
         ('Template for PEP file part of URL. (default "pep-%04d")',
          ['--pep-file-url-template'],
          {'metavar': '<URL>', 'default': 'pep-%04d'}),
         ('Recognize and link to standalone RFC references (like "RFC 822").',
          ['--rfc-references'],
          {'action': 'store_true', 'validator': frontend.validate_boolean}),
         ('Base URL for RFC references '
          '(default "https://tools.ietf.org/html/").',
          ['--rfc-base-url'],
          {'metavar': '<URL>', 'default': 'https://tools.ietf.org/html/',
           'validator': frontend.validate_url_trailing_slash}),
         ('Set number of spaces for tab expansion (default 8).',
          ['--tab-width'],
          {'metavar': '<width>', 'type': 'int', 'default': 8,
           'validator': frontend.validate_nonnegative_int}),
         ('Remove spaces before footnote references.',
          ['--trim-footnote-reference-space'],
          {'action': 'store_true', 'validator': frontend.validate_boolean}),
         ('Leave spaces before footnote references.',
          ['--leave-footnote-reference-space'],
          {'action': 'store_false', 'dest': 'trim_footnote_reference_space'}),
         ('Token name set for parsing code with Pygments: one of '
          '"long", "short", or "none" (no parsing). Default is "long".',
          ['--syntax-highlight'],
          {'choices': ['long', 'short', 'none'],
           'default': 'long', 'metavar': '<format>'}),
         ('Change straight quotation marks to typographic form: '
          'one of "yes", "no", "alt[ernative]" (default "no").',
          ['--smart-quotes'],
          {'default': False, 'metavar': '<yes/no/alt>',
           'validator': frontend.validate_ternary}),
         ('Characters to use as "smart quotes" for <language>. ',
          ['--smartquotes-locales'],
          {'metavar': '<language:quotes[,language:quotes,...]>',
           'action': 'append',
           'validator': frontend.validate_smartquotes_locales}),
         ('Inline markup recognized at word boundaries only '
          '(adjacent to punctuation or whitespace). '
          'Force character-level inline markup recognition with '
          '"\\ " (backslash + space). Default.',
          ['--word-level-inline-markup'],
          {'action': 'store_false', 'dest': 'character_level_inline_markup'}),
         ('Inline markup recognized anywhere, regardless of surrounding '
          'characters. Backslash-escapes must be used to avoid unwanted '
          'markup recognition. Useful for East Asian languages. '
          'Experimental.',
          ['--character-level-inline-markup'],
          {'action': 'store_true', 'default': False,
           'dest': 'character_level_inline_markup'}),
         )
        )

    config_section = 'restructuredtext parser'
    config_section_dependencies = ('parsers',)

    def __init__(self, rfc2822=False, inliner=None):
```

```
        if rfc2822:
            self.initial_state = 'RFC2822Body'
        else:
            self.initial_state = 'Body'
        self.state_classes = states.state_classes
        self.inliner = inliner

    def get_transforms(self):
        return super().get_transforms() + [universal.SmartQuotes]

    def parse(self, inputstring, document):
        """Parse `inputstring` and populate `document`, a document tree."""
        self.setup_parse(inputstring, document)
        # provide fallbacks in case the document has only generic settings
        self.document.settings.setdefault('tab_width', 8)
        self.document.settings.setdefault('syntax_highlight', 'long')
        self.statemachine = states.RSTStateMachine(
              state_classes=self.state_classes,
              initial_state=self.initial_state,
              debug=document.reporter.debug_flag)
        inputlines = docutils.statemachine.string2lines(
              inputstring, tab_width=document.settings.tab_width,
              convert_whitespace=True)
        for i, line in enumerate(inputlines):
            if len(line) > self.document.settings.line_length_limit:
                error = self.document.reporter.error(
                            'Line %d exceeds the line-length-limit.'%(i+1))
                self.document.append(error)
                break
        else:
            self.statemachine.run(inputlines, document, inliner=self.inliner)
        # restore the "default" default role after parsing a document
        if '' in roles._roles:
            del roles._roles['']
        self.finish_parse()


class DirectiveError(Exception):

    """
    Store a message and a system message level.

    To be thrown from inside directive code.

    Do not instantiate directly -- use `Directive.directive_error()`
    instead!
    """

    def __init__(self, level, message):
        """Set error `message` and `level`"""
        Exception.__init__(self)
        self.level = level
        self.msg = message


class Directive:

    """
    Base class for reStructuredText directives.

    The following attributes may be set by subclasses.  They are
    interpreted by the directive parser (which runs the directive
    class):

    - `required_arguments`: The number of required arguments (default:
      0).

    - `optional_arguments`: The number of optional arguments (default:
      0).

    - `final_argument_whitespace`: A boolean, indicating if the final
      argument may contain whitespace (default: False).

    - `option_spec`: A dictionary, mapping known option names to
      conversion functions such as `int` or `float` (default: {}, no
      options).  Several conversion functions are defined in the
      directives/__init__.py module.

      Option conversion functions take a single parameter, the option
      argument (a string or ``None``), validate it and/or convert it
      to the appropriate form.  Conversion functions may raise
```

```
  `ValueError` and `TypeError` exceptions.

- `has_content`: A boolean; True if content is allowed.  Client
  code must handle the case where content is required but not
  supplied (an empty content list will be supplied).

Arguments are normally single whitespace-separated words.  The
final argument may contain whitespace and/or newlines if
`final_argument_whitespace` is True.

If the form of the arguments is more complex, specify only one
argument (either required or optional) and set
`final_argument_whitespace` to True; the client code must do any
context-sensitive parsing.

When a directive implementation is being run, the directive class
is instantiated, and the `run()` method is executed.  During
instantiation, the following instance variables are set:

- ``name`` is the directive type or name (string).

- ``arguments`` is the list of positional arguments (strings).

- ``options`` is a dictionary mapping option names (strings) to
  values (type depends on option conversion functions; see
  `option_spec` above).

- ``content`` is a list of strings, the directive content line by line.

- ``lineno`` is the absolute line number of the first line
  of the directive.

- ``content_offset`` is the line offset of the first line
  of the content from the beginning of the current input.
  Used when initiating a nested parse.

- ``block_text`` is a string containing the entire directive.

- ``state`` is the state which called the directive function.

- ``state_machine`` is the state machine which controls the state
  which called the directive function.

- ``reporter`` is the state machine's `reporter` instance.

Directive functions return a list of nodes which will be inserted
into the document tree at the point where the directive was
encountered.  This can be an empty list if there is nothing to
insert.

For ordinary directives, the list must contain body elements or
structural elements.  Some directives are intended specifically
for substitution definitions, and must return a list of `Text`
nodes and/or inline elements (suitable for inline insertion, in
place of the substitution reference).  Such directives must verify
substitution definition context, typically using code like this::

    if not isinstance(state, states.SubstitutionDef):
        error = self.reporter.error(
            'Invalid context: the "%s" directive can only be used '
            'within a substitution definition.' % (name),
            nodes.literal_block(block_text, block_text), line=lineno)
        return [error]
"""

# There is a "Creating reStructuredText Directives" how-to at
# <https://docutils.sourceforge.io/docs/howto/rst-directives.html>.  If you
# update this docstring, please update the how-to as well.

required_arguments = 0
"""Number of required directive arguments."""

optional_arguments = 0
"""Number of optional arguments after the required arguments."""

final_argument_whitespace = False
"""May the final argument contain whitespace?"""

option_spec = None
"""Mapping of option names to validator functions."""
has_content = False
```

```
    """May the directive have content?"""

    def __init__(self, name, arguments, options, content, lineno,
                 content_offset, block_text, state, state_machine):
        self.name = name
        self.arguments = arguments
        self.options = options
        self.content = content
        self.lineno = lineno
        self.content_offset = content_offset
        self.block_text = block_text
        self.state = state
        self.state_machine = state_machine
        self.reporter = state_machine.reporter

    def run(self):
        raise NotImplementedError('Must override run() in subclass.')

    # Directive errors:

    def directive_error(self, level, message):
        """
        Return a DirectiveError suitable for being thrown as an exception.

        Call "raise self.directive_error(level, message)" from within
        a directive implementation to return one single system message
        at level `level`, which automatically gets the directive block
        and the line number added.

        Preferably use the `debug`, `info`, `warning`, `error`, or `severe`
        wrapper methods, e.g. ``self.error(message)`` to generate an
        ERROR-level directive error.
        """
        return DirectiveError(level, message)

    def debug(self, message):
        return self.directive_error(0, message)

    def info(self, message):
        return self.directive_error(1, message)

    def warning(self, message):
        return self.directive_error(2, message)

    def error(self, message):
        return self.directive_error(3, message)

    def severe(self, message):
        return self.directive_error(4, message)

    # Convenience methods:

    def assert_has_content(self):
        """
        Throw an ERROR-level DirectiveError if the directive doesn't
        have contents.
        """
        if not self.content:
            raise self.error('Content block expected for the "%s" directive; '
                             'none found.' % self.name)

    def add_name(self, node):
        """Append self.options['name'] to node['names'] if it exists.

        Also normalize the name string and register it as explicit target.
        """
        if 'name' in self.options:
            name = nodes.fully_normalize_name(self.options.pop('name'))
            if 'name' in node:
                del node['name']
            node['names'].append(name)
            self.state.document.note_explicit_target(node, node)


def convert_directive_function(directive_fn):
    """
    Define & return a directive class generated from `directive_fn`.

    `directive_fn` uses the old-style, functional interface.
    """
    class FunctionalDirective(Directive):
```

```
        option_spec = getattr(directive_fn, 'options', None)
        has_content = getattr(directive_fn, 'content', False)
        _argument_spec = getattr(directive_fn, 'arguments', (0, 0, False))
        required_arguments, optional_arguments, final_argument_whitespace \
            = _argument_spec

        def run(self):
            return directive_fn(
                self.name, self.arguments, self.options, self.content,
                self.lineno, self.content_offset, self.block_text,
                self.state, self.state_machine)

    # Return new-style directive.
    return FunctionalDirective
```

File: .venv\Lib\site-packages\docutils\parsers\rst\directives\admonitions.py

```
# $Id: admonitions.py 9037 2022-03-05 23:31:10Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
Admonition directives.
"""

__docformat__ = 'reStructuredText'


from docutils.parsers.rst import Directive
from docutils.parsers.rst import directives
from docutils.parsers.rst.roles import set_classes
from docutils import nodes


class BaseAdmonition(Directive):

    final_argument_whitespace = True
    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged}
    has_content = True

    node_class = None
    """Subclasses must set this to the appropriate admonition node class."""

    def run(self):
        set_classes(self.options)
        self.assert_has_content()
        text = '\n'.join(self.content)
        admonition_node = self.node_class(text, **self.options)
        self.add_name(admonition_node)
        if self.node_class is nodes.admonition:
            title_text = self.arguments[0]
            textnodes, messages = self.state.inline_text(title_text,
                                                         self.lineno)
            title = nodes.title(title_text, '', *textnodes)
            title.source, title.line = (
                    self.state_machine.get_source_and_line(self.lineno))
            admonition_node += title
            admonition_node += messages
            if 'classes' not in self.options:
                admonition_node['classes'] += ['admonition-'
                                               + nodes.make_id(title_text)]
        self.state.nested_parse(self.content, self.content_offset,
                                admonition_node)
        return [admonition_node]


class Admonition(BaseAdmonition):

    required_arguments = 1
    node_class = nodes.admonition


class Attention(BaseAdmonition):

    node_class = nodes.attention


class Caution(BaseAdmonition):
```

```
    node_class = nodes.caution


class Danger(BaseAdmonition):

    node_class = nodes.danger


class Error(BaseAdmonition):

    node_class = nodes.error


class Hint(BaseAdmonition):

    node_class = nodes.hint


class Important(BaseAdmonition):

    node_class = nodes.important


class Note(BaseAdmonition):

    node_class = nodes.note


class Tip(BaseAdmonition):

    node_class = nodes.tip


class Warning(BaseAdmonition):

    node_class = nodes.warning
```

File: .venv\Lib\site-packages\docutils\parsers\rst\directives\body.py

```
# $Id: body.py 9030 2022-03-05 23:28:32Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
Directives for additional body elements.

See `docutils.parsers.rst.directives` for API details.
"""

__docformat__ = 'reStructuredText'


from docutils import nodes
from docutils.parsers.rst import Directive
from docutils.parsers.rst import directives
from docutils.parsers.rst.roles import set_classes
from docutils.utils.code_analyzer import Lexer, LexerError, NumberLines


class BasePseudoSection(Directive):

    required_arguments = 1
    optional_arguments = 0
    final_argument_whitespace = True
    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged}
    has_content = True

    node_class = None
    """Node class to be used (must be set in subclasses)."""

    def run(self):
        if not (self.state_machine.match_titles
                or isinstance(self.state_machine.node, nodes.sidebar)):
            raise self.error('The "%s" directive may not be used within '
                             'topics or body elements.' % self.name)
        self.assert_has_content()
        if self.arguments:  # title (in sidebars optional)
            title_text = self.arguments[0]
```

```
            textnodes, messages = self.state.inline_text(
                                        title_text, self.lineno)
            titles = [nodes.title(title_text, '', *textnodes)]
            # Sidebar uses this code.
            if 'subtitle' in self.options:
                textnodes, more_messages = self.state.inline_text(
                    self.options['subtitle'], self.lineno)
                titles.append(nodes.subtitle(self.options['subtitle'], '',
                                             *textnodes))
                messages.extend(more_messages)
        else:
            titles = []
            messages = []
        text = '\n'.join(self.content)
        node = self.node_class(text, *(titles + messages))
        node['classes'] += self.options.get('class', [])
        self.add_name(node)
        if text:
            self.state.nested_parse(self.content, self.content_offset, node)
        return [node]


class Topic(BasePseudoSection):

    node_class = nodes.topic


class Sidebar(BasePseudoSection):

    node_class = nodes.sidebar

    required_arguments = 0
    optional_arguments = 1
    option_spec = BasePseudoSection.option_spec.copy()
    option_spec['subtitle'] = directives.unchanged_required

    def run(self):
        if isinstance(self.state_machine.node, nodes.sidebar):
            raise self.error('The "%s" directive may not be used within a '
                             'sidebar element.' % self.name)
        if 'subtitle' in self.options and not self.arguments:
            raise self.error('The "subtitle" option may not be used '
                             'without a title.')

        return BasePseudoSection.run(self)


class LineBlock(Directive):

    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged}
    has_content = True

    def run(self):
        self.assert_has_content()
        block = nodes.line_block(classes=self.options.get('class', []))
        self.add_name(block)
        node_list = [block]
        for line_text in self.content:
            text_nodes, messages = self.state.inline_text(
                line_text.strip(), self.lineno + self.content_offset)
            line = nodes.line(line_text, '', *text_nodes)
            if line_text.strip():
                line.indent = len(line_text) - len(line_text.lstrip())
            block += line
            node_list.extend(messages)
            self.content_offset += 1
        self.state.nest_line_block_lines(block)
        return node_list


class ParsedLiteral(Directive):

    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged}
    has_content = True

    def run(self):
        set_classes(self.options)
        self.assert_has_content()
        text = '\n'.join(self.content)
```

```
        text_nodes, messages = self.state.inline_text(text, self.lineno)
        node = nodes.literal_block(text, '', *text_nodes, **self.options)
        node.line = self.content_offset + 1
        self.add_name(node)
        return [node] + messages


class CodeBlock(Directive):
    """Parse and mark up content of a code block.

    Configuration setting: syntax_highlight
       Highlight Code content with Pygments?
       Possible values: ('long', 'short', 'none')

    """
    optional_arguments = 1
    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged,
                   'number-lines': directives.unchanged  # integer or None
                   }
    has_content = True

    def run(self):
        self.assert_has_content()
        if self.arguments:
            language = self.arguments[0]
        else:
            language = ''
        set_classes(self.options)
        classes = ['code']
        if language:
            classes.append(language)
        if 'classes' in self.options:
            classes.extend(self.options['classes'])

        # set up lexical analyzer
        try:
            tokens = Lexer('\n'.join(self.content), language,
                           self.state.document.settings.syntax_highlight)
        except LexerError as error:
            if self.state.document.settings.report_level > 2:
                # don't report warnings -> insert without syntax highlight
                tokens = Lexer('\n'.join(self.content), language, 'none')
            else:
                raise self.warning(error)

        if 'number-lines' in self.options:
            # optional argument `startline`, defaults to 1
            try:
                startline = int(self.options['number-lines'] or 1)
            except ValueError:
                raise self.error(':number-lines: with non-integer start value')
            endline = startline + len(self.content)
            # add linenumber filter:
            tokens = NumberLines(tokens, startline, endline)

        node = nodes.literal_block('\n'.join(self.content), classes=classes)
        self.add_name(node)
        # if called from "include", set the source
        if 'source' in self.options:
            node.attributes['source'] = self.options['source']
        # analyze content and add nodes for every token
        for classes, value in tokens:
            if classes:
                node += nodes.inline(value, value, classes=classes)
            else:
                # insert as Text to decrease the verbosity of the output
                node += nodes.Text(value)

        return [node]


class MathBlock(Directive):

    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged,
                   # TODO: Add Sphinx' ``mathbase.py`` option 'nowrap'?
                   # 'nowrap': directives.flag,
                   }
    has_content = True
    def run(self):
```

```
        set_classes(self.options)
        self.assert_has_content()
        # join lines, separate blocks
        content = '\n'.join(self.content).split('\n\n')
        _nodes = []
        for block in content:
            if not block:
                continue
            node = nodes.math_block(self.block_text, block, **self.options)
            node.line = self.content_offset + 1
            self.add_name(node)
            _nodes.append(node)
        return _nodes


class Rubric(Directive):

    required_arguments = 1
    optional_arguments = 0
    final_argument_whitespace = True
    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged}

    def run(self):
        set_classes(self.options)
        rubric_text = self.arguments[0]
        textnodes, messages = self.state.inline_text(rubric_text, self.lineno)
        rubric = nodes.rubric(rubric_text, '', *textnodes, **self.options)
        self.add_name(rubric)
        return [rubric] + messages


class BlockQuote(Directive):

    has_content = True
    classes = []

    def run(self):
        self.assert_has_content()
        elements = self.state.block_quote(self.content, self.content_offset)
        for element in elements:
            if isinstance(element, nodes.block_quote):
                element['classes'] += self.classes
        return elements


class Epigraph(BlockQuote):

    classes = ['epigraph']


class Highlights(BlockQuote):

    classes = ['highlights']


class PullQuote(BlockQuote):

    classes = ['pull-quote']


class Compound(Directive):

    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged}
    has_content = True

    def run(self):
        self.assert_has_content()
        text = '\n'.join(self.content)
        node = nodes.compound(text)
        node['classes'] += self.options.get('class', [])
        self.add_name(node)
        self.state.nested_parse(self.content, self.content_offset, node)
        return [node]


class Container(Directive):

    optional_arguments = 1
    final_argument_whitespace = True
```

```
    option_spec = {'name': directives.unchanged}
    has_content = True

    def run(self):
        self.assert_has_content()
        text = '\n'.join(self.content)
        try:
            if self.arguments:
                classes = directives.class_option(self.arguments[0])
            else:
                classes = []
        except ValueError:
            raise self.error(
                'Invalid class attribute value for "%s" directive: "%s".'
                % (self.name, self.arguments[0]))
        node = nodes.container(text)
        node['classes'].extend(classes)
        self.add_name(node)
        self.state.nested_parse(self.content, self.content_offset, node)
        return [node]
```

File: .venv\Lib\site-packages\docutils\parsers\rst\directives\html.py

```
# $Id: html.py 9062 2022-05-30 21:09:09Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
Dummy module for backwards compatibility.

This module is provisional: it will be removed in Docutils 2.0.
"""

__docformat__ = 'reStructuredText'

import warnings

from docutils.parsers.rst.directives.misc import MetaBody, Meta  # noqa: F401

warnings.warn('The `docutils.parsers.rst.directive.html` module'
              ' will be removed in Docutils 2.0.'
              ' Since Docutils 0.18, the "Meta" node is defined in'
              ' `docutils.parsers.rst.directives.misc`.',
              DeprecationWarning, stacklevel=2)
```

File: .venv\Lib\site-packages\docutils\parsers\rst\directives\images.py

```
# $Id: images.py 9037 2022-03-05 23:31:10Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
Directives for figures and simple images.
"""

__docformat__ = 'reStructuredText'

from urllib.request import url2pathname

try:  # check for the Python Imaging Library
    import PIL.Image
except ImportError:
    try:  # sometimes PIL modules are put in PYTHONPATH's root
        import Image
        class PIL: pass  # noqa:E701  dummy wrapper
        PIL.Image = Image
    except ImportError:
        PIL = None

from docutils import nodes
from docutils.nodes import fully_normalize_name, whitespace_normalize_name
from docutils.parsers.rst import Directive
from docutils.parsers.rst import directives, states
from docutils.parsers.rst.roles import set_classes


class Image(Directive):
    align_h_values = ('left', 'center', 'right')
```

```
    align_v_values = ('top', 'middle', 'bottom')
    align_values = align_v_values + align_h_values

    def align(argument):
        # This is not callable as self.align.  We cannot make it a
        # staticmethod because we're saving an unbound method in
        # option_spec below.
        return directives.choice(argument, Image.align_values)

    required_arguments = 1
    optional_arguments = 0
    final_argument_whitespace = True
    option_spec = {'alt': directives.unchanged,
                   'height': directives.length_or_unitless,
                   'width': directives.length_or_percentage_or_unitless,
                   'scale': directives.percentage,
                   'align': align,
                   'target': directives.unchanged_required,
                   'class': directives.class_option,
                   'name': directives.unchanged}

    def run(self):
        if 'align' in self.options:
            if isinstance(self.state, states.SubstitutionDef):
                # Check for align_v_values.
                if self.options['align'] not in self.align_v_values:
                    raise self.error(
                        'Error in "%s" directive: "%s" is not a valid value '
                        'for the "align" option within a substitution '
                        'definition.  Valid values for "align" are: "%s".'
                        % (self.name, self.options['align'],
                           '", "'.join(self.align_v_values)))
            elif self.options['align'] not in self.align_h_values:
                raise self.error(
                    'Error in "%s" directive: "%s" is not a valid value for '
                    'the "align" option.  Valid values for "align" are: "%s".'
                    % (self.name, self.options['align'],
                       '", "'.join(self.align_h_values)))
        messages = []
        reference = directives.uri(self.arguments[0])
        self.options['uri'] = reference
        reference_node = None
        if 'target' in self.options:
            block = states.escape2null(
                self.options['target']).splitlines()
            block = [line for line in block]
            target_type, data = self.state.parse_target(
                block, self.block_text, self.lineno)
            if target_type == 'refuri':
                reference_node = nodes.reference(refuri=data)
            elif target_type == 'refname':
                reference_node = nodes.reference(
                    refname=fully_normalize_name(data),
                    name=whitespace_normalize_name(data))
                reference_node.indirect_reference_name = data
                self.state.document.note_refname(reference_node)
            else:                          # malformed target
                messages.append(data)      # data is a system message
            del self.options['target']
        set_classes(self.options)
        image_node = nodes.image(self.block_text, **self.options)
        self.add_name(image_node)
        if reference_node:
            reference_node += image_node
            return messages + [reference_node]
        else:
            return messages + [image_node]


class Figure(Image):

    def align(argument):
        return directives.choice(argument, Figure.align_h_values)

    def figwidth_value(argument):
        if argument.lower() == 'image':
            return 'image'
        else:
            return directives.length_or_percentage_or_unitless(argument, 'px')

    option_spec = Image.option_spec.copy()
```

```
    option_spec['figwidth'] = figwidth_value
    option_spec['figclass'] = directives.class_option
    option_spec['align'] = align
    has_content = True

    def run(self):
        figwidth = self.options.pop('figwidth', None)
        figclasses = self.options.pop('figclass', None)
        align = self.options.pop('align', None)
        (image_node,) = Image.run(self)
        if isinstance(image_node, nodes.system_message):
            return [image_node]
        figure_node = nodes.figure('', image_node)
        if figwidth == 'image':
            if PIL and self.state.document.settings.file_insertion_enabled:
                imagepath = url2pathname(image_node['uri'])
                try:
                    with PIL.Image.open(imagepath) as img:
                        figure_node['width'] = '%dpx' % img.size[0]
                except (OSError, UnicodeEncodeError):
                    pass  # TODO: warn/info?
                else:
                    self.state.document.settings.record_dependencies.add(
                        imagepath.replace('\\', '/'))
        elif figwidth is not None:
            figure_node['width'] = figwidth
        if figclasses:
            figure_node['classes'] += figclasses
        if align:
            figure_node['align'] = align
        if self.content:
            node = nodes.Element()          # anonymous container for parsing
            self.state.nested_parse(self.content, self.content_offset, node)
            first_node = node[0]
            if isinstance(first_node, nodes.paragraph):
                caption = nodes.caption(first_node.rawsource, '',
                                        *first_node.children)
                caption.source = first_node.source
                caption.line = first_node.line
                figure_node += caption
            elif not (isinstance(first_node, nodes.comment)
                      and len(first_node) == 0):
                error = self.reporter.error(
                      'Figure caption must be a paragraph or empty comment.',
                      nodes.literal_block(self.block_text, self.block_text),
                      line=self.lineno)
                return [figure_node, error]
            if len(node) > 1:
                figure_node += nodes.legend('', *node[1:])
        return [figure_node]
```

File: .venv\Lib\site-packages\docutils\parsers\rst\directives\misc.py

```
# $Id: misc.py 9358 2023-04-19 23:31:13Z milde $
# Authors: David Goodger <goodger@python.org>; Dethe Elza
# Copyright: This module has been placed in the public domain.

"""Miscellaneous directives."""

__docformat__ = 'reStructuredText'

from pathlib import Path
import re
import time
from urllib.request import urlopen
from urllib.error import URLError

from docutils import io, nodes, statemachine, utils
from docutils.parsers.rst import Directive, convert_directive_function
from docutils.parsers.rst import directives, roles, states
from docutils.parsers.rst.directives.body import CodeBlock, NumberLines
from docutils.transforms import misc


class Include(Directive):

    """
    Include content read from a separate source file.
    Content may be parsed by the parser, or included as a literal
```

```
block.  The encoding of the included file can be specified.  Only
a part of the given file argument may be included by specifying
start and end line or text to match before and/or after the text
to be used.

https://docutils.sourceforge.io/docs/ref/rst/directives.html#including-an-external-document-fragment
"""

required_arguments = 1
optional_arguments = 0
final_argument_whitespace = True
option_spec = {'literal': directives.flag,
               'code': directives.unchanged,
               'encoding': directives.encoding,
               'parser': directives.parser_name,
               'tab-width': int,
               'start-line': int,
               'end-line': int,
               'start-after': directives.unchanged_required,
               'end-before': directives.unchanged_required,
               # ignored except for 'literal' or 'code':
               'number-lines': directives.unchanged,  # integer or None
               'class': directives.class_option,
               'name': directives.unchanged}

standard_include_path = Path(states.__file__).parent / 'include'

def run(self):
    """Include a file as part of the content of this reST file.

    Depending on the options, the file (or a clipping) is
    converted to nodes and returned or inserted into the input stream.
    """
    if not self.state.document.settings.file_insertion_enabled:
        raise self.warning('"%s" directive disabled.' % self.name)
    current_source = self.state.document.current_source
    path = directives.path(self.arguments[0])
    if path.startswith('<') and path.endswith('>'):
        _base = self.standard_include_path
        path = path[1:-1]
    else:
        _base = Path(current_source).parent
    path = utils.relative_path(None, _base/path)
    encoding = self.options.get(
        'encoding', self.state.document.settings.input_encoding)
    e_handler = self.state.document.settings.input_encoding_error_handler
    tab_width = self.options.get(
        'tab-width', self.state.document.settings.tab_width)
    try:
        include_file = io.FileInput(source_path=path,
                                    encoding=encoding,
                                    error_handler=e_handler)
    except UnicodeEncodeError:
        raise self.severe(f'Problems with "{self.name}" directive path:\n'
                          f'Cannot encode input file path "{path}" '
                          '(wrong locale?).')
    except OSError as error:
        raise self.severe(f'Problems with "{self.name}" directive '
                          f'path:\n{io.error_string(error)}.')
    else:
        self.state.document.settings.record_dependencies.add(path)

    # Get to-be-included content
    startline = self.options.get('start-line', None)
    endline = self.options.get('end-line', None)
    try:
        if startline or (endline is not None):
            lines = include_file.readlines()
            rawtext = ''.join(lines[startline:endline])
        else:
            rawtext = include_file.read()
    except UnicodeError as error:
        raise self.severe(f'Problem with "{self.name}" directive:\n'
                          + io.error_string(error))
    # start-after/end-before: no restrictions on newlines in match-text,
    # and no restrictions on matching inside lines vs. line boundaries
    after_text = self.options.get('start-after', None)
    if after_text:
        # skip content in rawtext before *and incl.* a matching text
        after_index = rawtext.find(after_text)
        if after_index < 0:
```

```
        raise self.severe('Problem with "start-after" option of "%s" '
                          'directive:\nText not found.' % self.name)
    rawtext = rawtext[after_index + len(after_text):]
before_text = self.options.get('end-before', None)
if before_text:
    # skip content in rawtext after *and incl.* a matching text
    before_index = rawtext.find(before_text)
    if before_index < 0:
        raise self.severe('Problem with "end-before" option of "%s" '
                          'directive:\nText not found.' % self.name)
    rawtext = rawtext[:before_index]

include_lines = statemachine.string2lines(rawtext, tab_width,
                                          convert_whitespace=True)
for i, line in enumerate(include_lines):
    if len(line) > self.state.document.settings.line_length_limit:
        raise self.warning('"%s": line %d exceeds the'
                           ' line-length-limit.' % (path, i+1))

if 'literal' in self.options:
    # Don't convert tabs to spaces, if `tab_width` is negative.
    if tab_width >= 0:
        text = rawtext.expandtabs(tab_width)
    else:
        text = rawtext
    literal_block = nodes.literal_block(
                        rawtext, source=path,
                        classes=self.options.get('class', []))
    literal_block.line = 1
    self.add_name(literal_block)
    if 'number-lines' in self.options:
        try:
            startline = int(self.options['number-lines'] or 1)
        except ValueError:
            raise self.error(':number-lines: with non-integer '
                             'start value')
        endline = startline + len(include_lines)
        if text.endswith('\n'):
            text = text[:-1]
        tokens = NumberLines([([], text)], startline, endline)
        for classes, value in tokens:
            if classes:
                literal_block += nodes.inline(value, value,
                                              classes=classes)
            else:
                literal_block += nodes.Text(value)
    else:
        literal_block += nodes.Text(text)
    return [literal_block]

if 'code' in self.options:
    self.options['source'] = path
    # Don't convert tabs to spaces, if `tab_width` is negative:
    if tab_width < 0:
        include_lines = rawtext.splitlines()
    codeblock = CodeBlock(self.name,
                          [self.options.pop('code')],  # arguments
                          self.options,
                          include_lines,  # content
                          self.lineno,
                          self.content_offset,
                          self.block_text,
                          self.state,
                          self.state_machine)
    return codeblock.run()

# Prevent circular inclusion:
clip_options = (startline, endline, before_text, after_text)
include_log = self.state.document.include_log
# log entries are tuples (<source>, <clip-options>)
if not include_log:  # new document, initialize with document source
    include_log.append((utils.relative_path(None, current_source),
                        (None, None, None, None)))
if (path, clip_options) in include_log:
    master_paths = (pth for (pth, opt) in reversed(include_log))
    inclusion_chain = '\n> '.join((path, *master_paths))
    raise self.warning('circular inclusion in "%s" directive:\n%s'
                       % (self.name, inclusion_chain))

if 'parser' in self.options:
    # parse into a dummy document and return created nodes
```

```
            document = utils.new_document(path, self.state.document.settings)
            document.include_log = include_log + [(path, clip_options)]
            parser = self.options['parser']()
            parser.parse('\n'.join(include_lines), document)
            # clean up doctree and complete parsing
            document.transformer.populate_from_components((parser,))
            document.transformer.apply_transforms()
            return document.children

        # Include as rST source:
        #
        # mark end (cf. parsers.rst.states.Body.comment())
        include_lines += ['', '.. end of inclusion from "%s"' % path]
        self.state_machine.insert_input(include_lines, path)
        # update include-log
        include_log.append((path, clip_options))
        return []


class Raw(Directive):

    """
    Pass through content unchanged

    Content is included in output based on type argument

    Content may be included inline (content section of directive) or
    imported from a file or url.
    """

    required_arguments = 1
    optional_arguments = 0
    final_argument_whitespace = True
    option_spec = {'file': directives.path,
                   'url': directives.uri,
                   'encoding': directives.encoding,
                   'class': directives.class_option}
    has_content = True

    def run(self):
        if (not self.state.document.settings.raw_enabled
            or (not self.state.document.settings.file_insertion_enabled
                and ('file' in self.options
                     or 'url' in self.options))):
            raise self.warning('"%s" directive disabled.' % self.name)
        attributes = {'format': ' '.join(self.arguments[0].lower().split())}
        encoding = self.options.get(
            'encoding', self.state.document.settings.input_encoding)
        e_handler = self.state.document.settings.input_encoding_error_handler
        if self.content:
            if 'file' in self.options or 'url' in self.options:
                raise self.error(
                    '"%s" directive may not both specify an external file '
                    'and have content.' % self.name)
            text = '\n'.join(self.content)
        elif 'file' in self.options:
            if 'url' in self.options:
                raise self.error(
                    'The "file" and "url" options may not be simultaneously '
                    'specified for the "%s" directive.' % self.name)
            path = self.options['file']
            _base = Path(self.state.document.current_source).parent
            path = utils.relative_path(None, _base/path)
            try:
                raw_file = io.FileInput(source_path=path,
                                        encoding=encoding,
                                        error_handler=e_handler)
            except OSError as error:
                raise self.severe(f'Problems with "{self.name}" directive '
                                  f'path:\n{io.error_string(error)}.')
            else:
                # TODO: currently, raw input files are recorded as
                # dependencies even if not used for the chosen output format.
                self.state.document.settings.record_dependencies.add(path)
            try:
                text = raw_file.read()
            except UnicodeError as error:
                raise self.severe(f'Problem with "{self.name}" directive:\n'
                                  + io.error_string(error))
            attributes['source'] = path
        elif 'url' in self.options:
```

```
            source = self.options['url']
            try:
                raw_text = urlopen(source).read()
            except (URLError, OSError) as error:
                raise self.severe(f'Problems with "{self.name}" directive URL '
                                  f'"{self.options["url"]}":\n'
                                  f'{io.error_string(error)}.')
            raw_file = io.StringInput(source=raw_text, source_path=source,
                                      encoding=encoding,
                                      error_handler=e_handler)
            try:
                text = raw_file.read()
            except UnicodeError as error:
                raise self.severe(f'Problem with "{self.name}" directive:\n'
                                  + io.error_string(error))
            attributes['source'] = source
        else:
            # This will always fail because there is no content.
            self.assert_has_content()
        raw_node = nodes.raw('', text, classes=self.options.get('class', []),
                             **attributes)
        (raw_node.source,
         raw_node.line) = self.state_machine.get_source_and_line(self.lineno)
        return [raw_node]


class Replace(Directive):

    has_content = True

    def run(self):
        if not isinstance(self.state, states.SubstitutionDef):
            raise self.error(
                'Invalid context: the "%s" directive can only be used within '
                'a substitution definition.' % self.name)
        self.assert_has_content()
        text = '\n'.join(self.content)
        element = nodes.Element(text)
        self.state.nested_parse(self.content, self.content_offset,
                                element)
        # element might contain [paragraph] + system_message(s)
        node = None
        messages = []
        for elem in element:
            if not node and isinstance(elem, nodes.paragraph):
                node = elem
            elif isinstance(elem, nodes.system_message):
                elem['backrefs'] = []
                messages.append(elem)
            else:
                return [
                    self.reporter.error(
                        f'Error in "{self.name}" directive: may contain '
                        'a single paragraph only.', line=self.lineno)]
        if node:
            return messages + node.children
        return messages


class Unicode(Directive):

    r"""
    Convert Unicode character codes (numbers) to characters.  Codes may be
    decimal numbers, hexadecimal numbers (prefixed by ``0x``, ``x``, ``\x``,
    ``U+``, ``u``, or ``\u``; e.g. ``U+262E``), or XML-style numeric character
    entities (e.g. ``&#x262E;``).  Text following ".." is a comment and is
    ignored.  Spaces are ignored, and any other text remains as-is.
    """

    required_arguments = 1
    optional_arguments = 0
    final_argument_whitespace = True
    option_spec = {'trim': directives.flag,
                   'ltrim': directives.flag,
                   'rtrim': directives.flag}

    comment_pattern = re.compile(r'( |\n|^)\.\. ')

    def run(self):
        if not isinstance(self.state, states.SubstitutionDef):
            raise self.error(
```

```
                'Invalid context: the "%s" directive can only be used within '
                'a substitution definition.' % self.name)
        substitution_definition = self.state_machine.node
        if 'trim' in self.options:
            substitution_definition.attributes['ltrim'] = 1
            substitution_definition.attributes['rtrim'] = 1
        if 'ltrim' in self.options:
            substitution_definition.attributes['ltrim'] = 1
        if 'rtrim' in self.options:
            substitution_definition.attributes['rtrim'] = 1
        codes = self.comment_pattern.split(self.arguments[0])[0].split()
        element = nodes.Element()
        for code in codes:
            try:
                decoded = directives.unicode_code(code)
            except ValueError as error:
                raise self.error('Invalid character code: %s\n%s'
                                 % (code, io.error_string(error)))
            element += nodes.Text(decoded)
        return element.children


class Class(Directive):

    """
    Set a "class" attribute on the directive content or the next element.
    When applied to the next element, a "pending" element is inserted, and a
    transform does the work later.
    """

    required_arguments = 1
    optional_arguments = 0
    final_argument_whitespace = True
    has_content = True

    def run(self):
        try:
            class_value = directives.class_option(self.arguments[0])
        except ValueError:
            raise self.error(
                'Invalid class attribute value for "%s" directive: "%s".'
                % (self.name, self.arguments[0]))
        node_list = []
        if self.content:
            container = nodes.Element()
            self.state.nested_parse(self.content, self.content_offset,
                                    container)
            for node in container:
                node['classes'].extend(class_value)
            node_list.extend(container.children)
        else:
            pending = nodes.pending(
                misc.ClassAttribute,
                {'class': class_value, 'directive': self.name},
                self.block_text)
            self.state_machine.document.note_pending(pending)
            node_list.append(pending)
        return node_list


class Role(Directive):

    has_content = True

    argument_pattern = re.compile(r'(%s)\s*(\(\s*(%s)\s*\)\s*)?$'
                                  % ((states.Inliner.simplename,) * 2))

    def run(self):
        """Dynamically create and register a custom interpreted text role."""
        if self.content_offset > self.lineno or not self.content:
            raise self.error('"%s" directive requires arguments on the first '
                             'line.' % self.name)
        args = self.content[0]
        match = self.argument_pattern.match(args)
        if not match:
            raise self.error('"%s" directive arguments not valid role names: '
                             '"%s".' % (self.name, args))
        new_role_name = match.group(1)
        base_role_name = match.group(3)
        messages = []
        if base_role_name:
```

```
            base_role, messages = roles.role(
                base_role_name, self.state_machine.language, self.lineno,
                self.state.reporter)
            if base_role is None:
                error = self.state.reporter.error(
                    'Unknown interpreted text role "%s".' % base_role_name,
                    nodes.literal_block(self.block_text, self.block_text),
                    line=self.lineno)
                return messages + [error]
        else:
            base_role = roles.generic_custom_role
        assert not hasattr(base_role, 'arguments'), (
            'Supplemental directive arguments for "%s" directive not '
            'supported (specified by "%r" role).' % (self.name, base_role))
        try:
            converted_role = convert_directive_function(base_role)
            (arguments, options, content, content_offset
             ) = self.state.parse_directive_block(
                    self.content[1:], self.content_offset,
                    converted_role, option_presets={})
        except states.MarkupError as detail:
            error = self.reporter.error(
                'Error in "%s" directive:\n%s.' % (self.name, detail),
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            return messages + [error]
        if 'class' not in options:
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\docutils\parsers\rst\directives\parts.py

```
# $Id: parts.py 8993 2022-01-29 13:20:04Z milde $
# Authors: David Goodger <goodger@python.org>; Dmitry Jemerov
# Copyright: This module has been placed in the public domain.

"""
Directives for document parts.
"""

__docformat__ = 'reStructuredText'

from docutils import nodes, languages
from docutils.transforms import parts
from docutils.parsers.rst import Directive
from docutils.parsers.rst import directives


class Contents(Directive):

    """
    Table of contents.

    The table of contents is generated in two passes: initial parse and
    transform.  During the initial parse, a 'pending' element is generated
    which acts as a placeholder, storing the TOC title and any options
    internally.  At a later stage in the processing, the 'pending' element is
    replaced by a 'topic' element, a title and the table of contents proper.
    """

    backlinks_values = ('top', 'entry', 'none')

    def backlinks(arg):
        value = directives.choice(arg, Contents.backlinks_values)
        if value == 'none':
            return None
        else:
            return value

    optional_arguments = 1
    final_argument_whitespace = True
    option_spec = {'depth': directives.nonnegative_int,
                   'local': directives.flag,
                   'backlinks': backlinks,
                   'class': directives.class_option}

    def run(self):
        if not (self.state_machine.match_titles
                or isinstance(self.state_machine.node, nodes.sidebar)):
            raise self.error('The "%s" directive may not be used within '
```

```
                        'topics or body elements.' % self.name)
        document = self.state_machine.document
        language = languages.get_language(document.settings.language_code,
                                          document.reporter)
        if self.arguments:
            title_text = self.arguments[0]
            text_nodes, messages = self.state.inline_text(title_text,
                                                          self.lineno)
            title = nodes.title(title_text, '', *text_nodes)
        else:
            messages = []
            if 'local' in self.options:
                title = None
            else:
                title = nodes.title('', language.labels['contents'])
        topic = nodes.topic(classes=['contents'])
        topic['classes'] += self.options.get('class', [])
        # the latex2e writer needs source and line for a warning:
        topic.source, topic.line = self.state_machine.get_source_and_line()
        topic.line -= 1
        if 'local' in self.options:
            topic['classes'].append('local')
        if title:
            name = title.astext()
            topic += title
        else:
            name = language.labels['contents']
        name = nodes.fully_normalize_name(name)
        if not document.has_name(name):
            topic['names'].append(name)
        document.note_implicit_target(topic)
        pending = nodes.pending(parts.Contents, rawsource=self.block_text)
        pending.details.update(self.options)
        document.note_pending(pending)
        topic += pending
        return [topic] + messages


class Sectnum(Directive):

    """Automatic section numbering."""

    option_spec = {'depth': int,
                   'start': int,
                   'prefix': directives.unchanged_required,
                   'suffix': directives.unchanged_required}

    def run(self):
        pending = nodes.pending(parts.SectNum)
        pending.details.update(self.options)
        self.state_machine.document.note_pending(pending)
        return [pending]


class Header(Directive):

    """Contents of document header."""

    has_content = True

    def run(self):
        self.assert_has_content()
        header = self.state_machine.document.get_decoration().get_header()
        self.state.nested_parse(self.content, self.content_offset, header)
        return []


class Footer(Directive):

    """Contents of document footer."""

    has_content = True

    def run(self):
        self.assert_has_content()
        footer = self.state_machine.document.get_decoration().get_footer()
        self.state.nested_parse(self.content, self.content_offset, footer)
        return []
```

File: .venv\Lib\site-packages\docutils\parsers\rst\directives\references.py

```
# $Id: references.py 7062 2011-06-30 22:14:29Z milde $
# Authors: David Goodger <goodger@python.org>; Dmitry Jemerov
# Copyright: This module has been placed in the public domain.

"""
Directives for references and targets.
"""

__docformat__ = 'reStructuredText'

from docutils import nodes
from docutils.transforms import references
from docutils.parsers.rst import Directive
from docutils.parsers.rst import directives


class TargetNotes(Directive):

    """Target footnote generation."""

    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged}

    def run(self):
        pending = nodes.pending(references.TargetNotes)
        self.add_name(pending)
        pending.details.update(self.options)
        self.state_machine.document.note_pending(pending)
        return [pending]
```

File: .venv\Lib\site-packages\docutils\parsers\rst\directives\tables.py

```
# $Id: tables.py 9363 2023-04-24 16:37:37Z aa-turner $
# Authors: David Goodger <goodger@python.org>; David Priest
# Copyright: This module has been placed in the public domain.

"""
Directives for table elements.
"""

__docformat__ = 'reStructuredText'


import csv
from pathlib import Path
import warnings

from docutils import io, nodes, statemachine, utils
from docutils.utils import SystemMessagePropagation
from docutils.parsers.rst import Directive
from docutils.parsers.rst import directives
from urllib.request import urlopen
from urllib.error import URLError


def align(argument):
    return directives.choice(argument, ('left', 'center', 'right'))


class Table(Directive):

    """
    Generic table base class.
    """

    optional_arguments = 1
    final_argument_whitespace = True
    option_spec = {'class': directives.class_option,
                   'name': directives.unchanged,
                   'align': align,
                   'width': directives.length_or_percentage_or_unitless,
                   'widths': directives.value_or(('auto', 'grid'),
                                                 directives.positive_int_list)}
    has_content = True

    def make_title(self):
        if self.arguments:
```

```
        title_text = self.arguments[0]
        text_nodes, messages = self.state.inline_text(title_text,
                                                      self.lineno)
        title = nodes.title(title_text, '', *text_nodes)
        (title.source,
         title.line) = self.state_machine.get_source_and_line(self.lineno)
    else:
        title = None
        messages = []
    return title, messages

def process_header_option(self):
    # Provisional
    # * Will move to CSVTable in Docutils 0.21
    #   as it calls `self.HeaderDialect()` only defined in CSVTable.
    # * Will change to use the same CSV dialect as the body to get in line
    #   with the specification in ref/rst/directives.txt in Docutils 0.21.
    source = self.state_machine.get_source(self.lineno - 1)
    table_head = []
    max_header_cols = 0
    if 'header' in self.options:   # separate table header in option
        with warnings.catch_warnings():
            warnings.simplefilter('ignore')
            header_dialect = self.HeaderDialect()
        rows, max_header_cols = self.parse_csv_data_into_rows(
            self.options['header'].split('\n'), header_dialect,
            source)
        table_head.extend(rows)
    return table_head, max_header_cols

def check_table_dimensions(self, rows, header_rows, stub_columns):
    if len(rows) < header_rows:
        error = self.reporter.error('%s header row(s) specified but '
            'only %s row(s) of data supplied ("%s" directive).'
            % (header_rows, len(rows), self.name),
            nodes.literal_block(self.block_text, self.block_text),
            line=self.lineno)
        raise SystemMessagePropagation(error)
    if len(rows) == header_rows > 0:
        error = self.reporter.error(
            f'Insufficient data supplied ({len(rows)} row(s)); '
            'no data remaining for table body, '
            f'required by "{self.name}" directive.',
            nodes.literal_block(self.block_text, self.block_text),
            line=self.lineno)
        raise SystemMessagePropagation(error)
    for row in rows:
        if len(row) < stub_columns:
            error = self.reporter.error(
                f'{stub_columns} stub column(s) specified '
                f'but only {len(row)} columns(s) of data supplied '
                f'("{self.name}" directive).',
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            raise SystemMessagePropagation(error)
        if len(row) == stub_columns > 0:
            error = self.reporter.error(
                'Insufficient data supplied (%s columns(s)); '
                'no data remaining for table body, required '
                'by "%s" directive.' % (len(row), self.name),
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            raise SystemMessagePropagation(error)

def set_table_width(self, table_node):
    if 'width' in self.options:
        table_node['width'] = self.options.get('width')

@property
def widths(self):
    return self.options.get('widths', '')

def get_column_widths(self, n_cols):
    if isinstance(self.widths, list):
        if len(self.widths) != n_cols:
            # TODO: use last value for missing columns?
            error = self.reporter.error('"%s" widths do not match the '
                'number of columns in table (%s).' % (self.name, n_cols),
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            raise SystemMessagePropagation(error)
```

```
            col_widths = self.widths
        elif n_cols:
            col_widths = [100 // n_cols] * n_cols
        else:
            error = self.reporter.error('No table data detected in CSV file.',
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            raise SystemMessagePropagation(error)
        return col_widths

    def extend_short_rows_with_empty_cells(self, columns, parts):
        for part in parts:
            for row in part:
                if len(row) < columns:
                    row.extend([(0, 0, 0, [])] * (columns - len(row)))


class RSTTable(Table):
    """
    Class for the `"table" directive`__ for formal tables using rST syntax.

    __ https://docutils.sourceforge.io/docs/ref/rst/directives.html
    """

    def run(self):
        if not self.content:
            warning = self.reporter.warning('Content block expected '
                'for the "%s" directive; none found.' % self.name,
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            return [warning]
        title, messages = self.make_title()
        node = nodes.Element()          # anonymous container for parsing
        self.state.nested_parse(self.content, self.content_offset, node)
        if len(node) != 1 or not isinstance(node[0], nodes.table):
            error = self.reporter.error('Error parsing content block for the '
                '"%s" directive: exactly one table expected.' % self.name,
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            return [error]
        table_node = node[0]
        table_node['classes'] += self.options.get('class', [])
        self.set_table_width(table_node)
        if 'align' in self.options:
            table_node['align'] = self.options.get('align')
        if isinstance(self.widths, list):
            tgroup = table_node[0]
            try:
                col_widths = self.get_column_widths(tgroup["cols"])
            except SystemMessagePropagation as detail:
                return [detail.args[0]]
            colspecs = [child for child in tgroup.children
                        if child.tagname == 'colspec']
            for colspec, col_width in zip(colspecs, col_widths):
                colspec['colwidth'] = col_width
        if self.widths == 'auto':
            table_node['classes'] += ['colwidths-auto']
        elif self.widths:  # "grid" or list of integers
            table_node['classes'] += ['colwidths-given']
        self.add_name(table_node)
        if title:
            table_node.insert(0, title)
        return [table_node] + messages


class CSVTable(Table):

    option_spec = {'header-rows': directives.nonnegative_int,
                   'stub-columns': directives.nonnegative_int,
                   'header': directives.unchanged,
                   'width': directives.length_or_percentage_or_unitless,
                   'widths': directives.value_or(('auto', ),
                                                 directives.positive_int_list),
                   'file': directives.path,
                   'url': directives.uri,
                   'encoding': directives.encoding,
                   'class': directives.class_option,
                   'name': directives.unchanged,
                   'align': align,
                   # field delimiter char
                   'delim': directives.single_char_or_whitespace_or_unicode,
```

```
                # treat whitespace after delimiter as significant
                'keepspace': directives.flag,
                # text field quote/unquote char:
                'quote': directives.single_char_or_unicode,
                # char used to escape delim & quote as-needed:
                'escape': directives.single_char_or_unicode}

class DocutilsDialect(csv.Dialect):

    """CSV dialect for `csv_table` directive."""

    delimiter = ','
    quotechar = '"'
    doublequote = True
    skipinitialspace = True
    strict = True
    lineterminator = '\n'
    quoting = csv.QUOTE_MINIMAL

    def __init__(self, options):
        if 'delim' in options:
            self.delimiter = options['delim']
        if 'keepspace' in options:
            self.skipinitialspace = False
        if 'quote' in options:
            self.quotechar = options['quote']
        if 'escape' in options:
            self.doublequote = False
            self.escapechar = options['escape']
        super().__init__()

class HeaderDialect(csv.Dialect):
    """
    CSV dialect used for the "header" option data.

    Deprecated. Will be removed in Docutils 0.22.
    """
    # The separate HeaderDialect was introduced in revision 2294
    # (2004-06-17) in the sandbox before the "csv-table" directive moved
    # to the trunk in r2309. Discussion in docutils-devel around this time
    # did not mention a rationale (part of the discussion was in private
    # mail).
    # This is in conflict with the documentation, which always said:
    # ""
    # and did not change in this aspect.
    #
    # Maybe it was intended to have similar escape rules for rST and CSV,
    # however with the current implementation this means we need
    # `\\` for rST markup and ``\\\\`` for a literal backslash
    # in the "option" header but ``\`` and ``\\`` in the header-lines and
    # table cells of the main CSV data.
    delimiter = ','
    quotechar = '"'
    escapechar = '\\'
    doublequote = False
    skipinitialspace = True
    strict = True
    lineterminator = '\n'
    quoting = csv.QUOTE_MINIMAL

    def __init__(self):
        warnings.warn('CSVTable.HeaderDialect will be removed '
                      'in Docutils 0.22.',
                      PendingDeprecationWarning, stacklevel=2)
        super().__init__()

@staticmethod
def check_requirements():
    warnings.warn('CSVTable.check_requirements()'
                  ' is not required with Python 3'
                  ' and will be removed in Docutils 0.22.',
                  DeprecationWarning, stacklevel=2)

def run(self):
    try:
        if (not self.state.document.settings.file_insertion_enabled
            and ('file' in self.options
                 or 'url' in self.options)):
            warning = self.reporter.warning('File and URL access '
                'deactivated; ignoring "%s" directive.' % self.name,
                nodes.literal_block(self.block_text, self.block_text),
```

```
                line=self.lineno)
            return [warning]
        title, messages = self.make_title()
        csv_data, source = self.get_csv_data()
        table_head, max_header_cols = self.process_header_option()
        rows, max_cols = self.parse_csv_data_into_rows(
            csv_data, self.DocutilsDialect(self.options), source)
        max_cols = max(max_cols, max_header_cols)
        header_rows = self.options.get('header-rows', 0)
        stub_columns = self.options.get('stub-columns', 0)
        self.check_table_dimensions(rows, header_rows, stub_columns)
        table_head.extend(rows[:header_rows])
        table_body = rows[header_rows:]
        col_widths = self.get_column_widths(max_cols)
        self.extend_short_rows_with_empty_cells(max_cols,
                                                (table_head, table_body))
    except SystemMessagePropagation as detail:
        return [detail.args[0]]
    except csv.Error as detail:
        message = str(detail)
        error = self.reporter.error('Error with CSV data'
            ' in "%s" directive:\n%s' % (self.name, message),
            nodes.literal_block(self.block_text, self.block_text),
            line=self.lineno)
        return [error]
    table = (col_widths, table_head, table_body)
    table_node = self.state.build_table(table, self.content_offset,
                                        stub_columns, widths=self.widths)
    table_node['classes'] += self.options.get('class', [])
    if 'align' in self.options:
        table_node['align'] = self.options.get('align')
    self.set_table_width(table_node)
    self.add_name(table_node)
    if title:
        table_node.insert(0, title)
    return [table_node] + messages

def get_csv_data(self):
    """
    Get CSV data from the directive content, from an external
    file, or from a URL reference.
    """
    encoding = self.options.get(
        'encoding', self.state.document.settings.input_encoding)
    error_handler = self.state.document.settings.input_encoding_error_handler  # noqa:E501
    if self.content:
        # CSV data is from directive content.
        if 'file' in self.options or 'url' in self.options:
            error = self.reporter.error('"%s" directive may not both '
                'specify an external file and have content.' % self.name,
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            raise SystemMessagePropagation(error)
        source = self.content.source(0)
        csv_data = self.content
    elif 'file' in self.options:
        # CSV data is from an external file.
        if 'url' in self.options:
            error = self.reporter.error('The "file" and "url" options '
                'may not be simultaneously specified '
                'for the "%s" directive.' % self.name,
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            raise SystemMessagePropagation(error)
        source = self.options['file']
        # resolve path to external file
        _base = Path(self.state.document.current_source).parent
        source = utils.relative_path(None, _base/source)
        try:
            csv_file = io.FileInput(source_path=source,
                                    encoding=encoding,
                                    error_handler=error_handler)
            csv_data = csv_file.read().splitlines()
        except OSError as error:
            severe = self.reporter.severe(
                'Problems with "%s" directive path:\n%s.'
                % (self.name, error),
                nodes.literal_block(self.block_text, self.block_text),
                line=self.lineno)
            raise SystemMessagePropagation(severe)
        else:
```

```
                self.state.document.settings.record_dependencies.add(source)
            elif 'url' in self.options:
                source = self.options['url']
                try:
                    with urlopen(source) as response:
                        csv_text = response.read()
                except (URLError, OSError, ValueError) as error:
                    severe = self.reporter.severe(
                          'Problems with "%s" directive URL "%s":\n%s.'
                          % (self.name, self.options['url'], error),
                          nodes.literal_block(self.block_text, self.block_text),
                          line=self.lineno)
                    raise SystemMessagePropagation(severe)
                csv_file = io.StringInput(
                    source=csv_text, source_path=source, encoding=encoding,
                    error_handler=(self.state.document.settings.
                                   input_encoding_error_handler))
                csv_data = csv_file.read().splitlines()
            else:
                error = self.reporter.warning(
                    'The "%s" directive requires content; none supplied.'
                    % self.name,
                    nodes.literal_block(self.block_text, self.block_text),
                    line=self.lineno)
                raise SystemMessagePropagation(error)
            return csv_data, source

        @staticmethod
        def decode_from_csv(s):
            warnings.warn('CSVTable.decode_from_csv()'
                          ' is not required with Python 3'
                          ' and will be removed in Docutils 0.21 or later.',
                          DeprecationWarning, stacklevel=2)
            return s

        @staticmethod
        def encode_for_csv(s):
            warnings.warn('CSVTable.encode_from_csv()'
                          ' is not required with Python 3'
                          ' and will be removed in Docutils 0.21 or later.',
                          DeprecationWarning, stacklevel=2)
            return s

        def parse_csv_data_into_rows(self, csv_data, dialect, source):
            csv_reader = csv.reader((line + '\n' for line in csv_data),
                                    dialect=dialect)
            rows = []
            max_cols = 0
            for row in csv_reader:
                row_data = []
                for cell in row:
                    cell_data = (0, 0, 0, statemachine.StringList(
                        cell.splitlines(), source=source))
                    row_data.append(cell_data)
                rows.append(row_data)
                max_cols = max(max_cols, len(row))
            return rows, max_cols


class ListTable(Table):

    """
    Implement tables whose data is encoded as a uniform two-level bullet list.
    For further ideas, see
    https://docutils.sourceforge.io/docs/dev/rst/alternatives.html#list-driven-tables
    """

    option_spec = {'header-rows': directives.nonnegative_int,
                   'stub-columns': directives.nonnegative_int,
                   'width': directives.length_or_percentage_or_unitless,
                   'widths': directives.value_or(('auto', ),
                                                 directives.positive_int_list),
                   'class': directives.class_option,
                   'name': directives.unchanged,
                   'align': align}

    def run(self):
        if not self.content:
            error = self.reporter.error('The "%s" directive is empty; '
                'content required.' % self.name,
                nodes.literal_block(self.block_text, self.block_text),
```

```
                line=self.lineno)
            return [error]
        title, messages = self.make_title()
        node = nodes.Element()          # anonymous container for parsing
        self.state.nested_parse(self.content, self.content_offset, node)
        try:
            num_cols, col_widths = self.check_list_content(node)
            table_data = [[item.children for item in row_list[0]]
                          for row_list in node[0]]
            header_rows = self.options.get('header-rows', 0)
            stub_columns = self.options.get('stub-columns', 0)
            self.check_table_dimens
```

(Content truncated due to size - showing first 20KB)

File: .venv\Lib\site-packages\docutils\parsers\rst\directives__init__.py

```
# $Id: __init__.py 9356 2023-04-18 23:50:48Z milde $
# Author: David Goodger <goodger@python.org>
# Copyright: This module has been placed in the public domain.

"""
This package contains directive implementation modules.
"""

__docformat__ = 'reStructuredText'

import re
import codecs
from importlib import import_module

from docutils import nodes, parsers
from docutils.utils import split_escaped_whitespace, escape2null
from docutils.parsers.rst.languages import en as _fallback_language_module


_directive_registry = {
      'attention': ('admonitions', 'Attention'),
      'caution': ('admonitions', 'Caution'),
      'code': ('body', 'CodeBlock'),
      'danger': ('admonitions', 'Danger'),
      'error': ('admonitions', 'Error'),
      'important': ('admonitions', 'Important'),
      'note': ('admonitions', 'Note'),
      'tip': ('admonitions', 'Tip'),
      'hint': ('admonitions', 'Hint'),
      'warning': ('admonitions', 'Warning'),
      'admonition': ('admonitions', 'Admonition'),
      'sidebar': ('body', 'Sidebar'),
      'topic': ('body', 'Topic'),
      'line-block': ('body', 'LineBlock'),
      'parsed-literal': ('body', 'ParsedLiteral'),
      'math': ('body', 'MathBlock'),
      'rubric': ('body', 'Rubric'),
      'epigraph': ('body', 'Epigraph'),
      'highlights': ('body', 'Highlights'),
      'pull-quote': ('body', 'PullQuote'),
      'compound': ('body', 'Compound'),
      'container': ('body', 'Container'),
      # 'questions': ('body', 'question_list'),
      'table': ('tables', 'RSTTable'),
      'csv-table': ('tables', 'CSVTable'),
      'list-table': ('tables', 'ListTable'),
      'image': ('images', 'Image'),
      'figure': ('images', 'Figure'),
      'contents': ('parts', 'Contents'),
      'sectnum': ('parts', 'Sectnum'),
      'header': ('parts', 'Header'),
      'footer': ('parts', 'Footer'),
      # 'footnotes': ('parts', 'footnotes'),
      # 'citations': ('parts', 'citations'),
      'target-notes': ('references', 'TargetNotes'),
      'meta': ('misc', 'Meta'),
      # 'imagemap': ('html', 'imagemap'),
      'raw': ('misc', 'Raw'),
      'include': ('misc', 'Include'),
      'replace': ('misc', 'Replace'),
      'unicode': ('misc', 'Unicode'),
      'class': ('misc', 'Class'),
      'role': ('misc', 'Role'),
```

```
      'default-role': ('misc', 'DefaultRole'),
      'title': ('misc', 'Title'),
      'date': ('misc', 'Date'),
      'restructuredtext-test-directive': ('misc', 'TestDirective'),
      }
"""Mapping of directive name to (module name, class name).  The
directive name is canonical & must be lowercase.  Language-dependent
names are defined in the ``language`` subpackage."""

_directives = {}
"""Cache of imported directives."""


def directive(directive_name, language_module, document):
    """
    Locate and return a directive function from its language-dependent name.
    If not found in the current language, check English.  Return None if the
    named directive cannot be found.
    """
    normname = directive_name.lower()
    messages = []
    msg_text = []
    if normname in _directives:
        return _directives[normname], messages
    canonicalname = None
    try:
        canonicalname = language_module.directives[normname]
    except AttributeError as error:
        msg_text.append('Problem retrieving directive entry from language '
                        'module %r: %s.' % (language_module, error))
    except KeyError:
        msg_text.append('No directive entry for "%s" in module "%s".'
                        % (directive_name, language_module.__name__))
    if not canonicalname:
        try:
            canonicalname = _fallback_language_module.directives[normname]
            msg_text.append('Using English fallback for directive "%s".'
                            % directive_name)
        except KeyError:
            msg_text.append('Trying "%s" as canonical directive name.'
                            % directive_name)
            # The canonical name should be an English name, but just in case:
            canonicalname = normname
    if msg_text:
        message = document.reporter.info(
            '\n'.join(msg_text), line=document.current_line)
        messages.append(message)
    try:
        modulename, classname = _directive_registry[canonicalname]
    except KeyError:
        # Error handling done by caller.
        return None, messages
    try:
        module = import_module('docutils.parsers.rst.directives.'+modulename)
    except ImportError as detail:
        messages.append(document.reporter.error(
            'Error importing directive module "%s" (directive "%s"):\n%s'
            % (modulename, directive_name, detail),
            line=document.current_line))
        return None, messages
    try:
        directive = getattr(module, classname)
        _directives[normname] = directive
    except AttributeError:
        messages.append(document.reporter.error(
            'No directive class "%s" in module "%s" (directive "%s").'
            % (classname, modulename, directive_name),
            line=document.current_line))
        return None, messages
    return directive, messages


def register_directive(name, directive):
    """
    Register a nonstandard application-defined directive function.
    Language lookups are not needed for such functions.
    """
    _directives[name] = directive


def flag(argument):
```

```
    """
    Check for a valid flag option (no argument) and return ``None``.
    (Directive option conversion function.)

    Raise ``ValueError`` if an argument is found.
    """
    if argument and argument.strip():
        raise ValueError('no argument is allowed; "%s" supplied' % argument)
    else:
        return None


def unchanged_required(argument):
    """
    Return the argument text, unchanged.
    (Directive option conversion function.)

    Raise ``ValueError`` if no argument is found.
    """
    if argument is None:
        raise ValueError('argument required but none supplied')
    else:
        return argument  # unchanged!


def unchanged(argument):
    """
    Return the argument text, unchanged.
    (Directive option conversion function.)

    No argument implies empty string ("").
    """
    if argument is None:
        return ''
    else:
        return argument  # unchanged!


def path(argument):
    """
    Return the path argument unwrapped (with newlines removed).
    (Directive option conversion function.)

    Raise ``ValueError`` if no argument is found.
    """
    if argument is None:
        raise ValueError('argument required but none supplied')
    else:
        return ''.join(s.strip() for s in argument.splitlines())


def uri(argument):
    """
    Return the URI argument with unescaped whitespace removed.
    (Directive option conversion function.)

    Raise ``ValueError`` if no argument is found.
    """
    if argument is None:
        raise ValueError('argument required but none supplied')
    else:
        parts = split_escaped_whitespace(escape2null(argument))
        return ' '.join(''.join(nodes.unescape(part).split())
                        for part in parts)


def nonnegative_int(argument):
    """
    Check for a nonnegative integer argument; raise ``ValueError`` if not.
    (Directive option conversion function.)
    """
    value = int(argument)
    if value < 0:
        raise ValueError('negative value; must be positive or zero')
    return value


def percentage(argument):
    """
    Check for an integer percentage value with optional percent sign.
    (Directive option conversion function.)
```

```
    """
    try:
        argument = argument.rstrip(' %')
    except AttributeError:
        pass
    return nonnegative_int(argument)


length_units = ['em', 'ex', 'px', 'in', 'cm', 'mm', 'pt', 'pc']


def get_measure(argument, units):
    """
    Check for a positive argument of one of the units and return a
    normalized string of the form "<value><unit>" (without space in
    between).
    (Directive option conversion function.)

    To be called from directive option conversion functions.
    """
    match = re.match(r'^([0-9.]+) *(%s)$' % '|'.join(units), argument)
    try:
        float(match.group(1))
    except (AttributeError, ValueError):
        raise ValueError(
            'not a positive measure of one of the following units:\n%s'
            % ' '.join('"%s"' % i for i in units))
    return match.group(1) + match.group(2)


def length_or_unitless(argument):
    return get_measure(argument, length_units + [''])


def length_or_percentage_or_unitless(argument, default=''):
    """
    Return normalized string of a length or percentage unit.
    (Directive option conversion function.)

    Add <default> if there is no unit. Raise ValueError if the argument is not
    a positive measure of one of the valid CSS units (or without unit).

    >>> length_or_percentage_or_unitless('3 pt')
    '3pt'
    >>> length_or_percentage_or_unitless('3%', 'em')
    '3%'
    >>> length_or_percentage_or_unitless('3')
    '3'
    >>> length_or_percentage_or_unitless('3', 'px')
    '3px'
    """
    try:
        return get_measure(argument, length_units + ['%'])
    except ValueError:
        try:
            return get_measure(argument, ['']) + default
        except ValueError:
            # raise ValueError with list of valid units:
            return get_measure(argument, length_units + ['%'])


def class_option(argument):
    """
    Convert the argument into a list of ID-compatible strings and return it.
    (Directive option conversion function.)

    Raise ``ValueError`` if no argument is found.
    """
    if argument is None:
        raise ValueError('argument required but none supplied')
    names = argument.split()
    class_names = []
    for name in names:
        class_name = nodes.make_id(name)
        if not class_name:
            raise ValueError('cannot make "%s" into a class name' % name)
        class_names.append(class_name)
    return class_names


unicode_pattern = re.compile(
```

```
    r'(?:0x|x|\\x|U\+?|\\u)([0-9a-f]+)$|&#x([0-9a-f]+);$', re.IGNORECASE)


def unicode_code(code):
    r"""
    Convert a Unicode character code to a Unicode character.
    (Directive option conversion function.)

    Codes may be decimal numbers, hexadecimal numbers (prefixed by ``0x``,
    ``x``, ``\x``, ``U+``, ``u``, or ``\u``; e.g. ``U+262E``), or XML-style
    numeric character entities (e.g. ``&#x262E;``).  Other text remains as-is.

    Raise ValueError for illegal Unicode code values.
    """
    try:
        if code.isdigit():                  # decimal number
            return chr(int(code))
        else:
            match = unicode_pattern.match(code)
            if match:                       # hex number
                value = match.group(1) or match.group(2)
                return chr(int(value, 16))
            else:                           # other text
                return code
    except OverflowError as detail:
        raise ValueError('code too large (%s)' % detail)


def single_char_or_unicode(argument):
    """
    A single character is returned as-is.  Unicode character codes are
    converted as in `unicode_code`.  (Directive option conversion function.)
    """
    char = unicode_code(argument)
    if len(char) > 1:
        raise ValueError('%r invalid; must be a single character or '
                         'a Unicode code' % char)
    return char


def single_char_or_whitespace_or_unicode(argument):
    """
    As with `single_char_or_unicode`, but "tab" and "space" are also supported.
    (Directive option conversion function.)
    """
    if argument == 'tab':
        char = '\t'
    elif argument == 'space':
        char = ' '
    else:
        char = single_char_or_unicode(argument)
    return char


def positive_int(argument):
    """
    Converts the argument into an integer.  Raises ValueError for negative,
    zero, or non-integer values.  (Directive option conversion function.)
    """
    value = int(argument)
    if value < 1:
        raise ValueError('negative or zero value; must be positive')
    return value


def positive_int_list(argument):
    """
    Converts a space- or comma-separated list of values into a Python list
    of integers.
    (Directive option conversion function.)

    Raises ValueError for non-positive-integer values.
    """
    if ',' in argument:
        entries = argument.split(',')
    else:
        entries = argument.split()
    return [positive_int(entry) for entry in entries]


def encoding(argument):
```

```
    """
    Verifies the encoding argument by lookup.
    (Directive option conversion function.)

    Raises ValueError for unknown encodings.
    """
    try:
        codecs.lookup(argument)
    except LookupError:
        raise ValueError('unknown encoding: "%s"' % argument)
    return argument


def choice(argument, values):
    """
    Directive option utility function, supplied to enable options whose
    argument must be a member of a finite set of possible values (must be
    lower case).  A custom conversion function must be written to use it.  For
    example::

        from docutils.parsers.rst import directives

        def yesno(argument):
            return directives.choice(argument, ('yes', 'no'))

    Raise ``ValueError`` if no argument is found or if the argument's value is
    not valid (not an entry in the supplied list).
    """
    try:
        value = argument.lower().strip()
    except AttributeError:
        raise ValueError('must supply an argument; choose from %s'
                         % format_values(values))
    if value in values:
        return value
    else:
        raise ValueError('"%s" unknown; choose from %s'
                         % (argument, format_values(values)))


def format_values(values):
    return '%s, or "%s"' % (', '.join('"%s"' % s for s in values[:-1]),
                            values[-1])


def value_or(values, other):
    """
    Directive option conversion function.

    The argument can be any of `values` or `argument_type`.
    """
    def auto_or_other(argument):
        if argument in values:
            return argument
        else:
            return other(argument)
    return auto_or_other


def parser_name(argument):
    """
    Return a docutils parser whose name matches the argument.
    (Directive option conversion function.)

    Return `None`, if the argument evaluates to `False`.
    Raise `ValueError` if importing the parser module fails.
    """
    if not argument:
        return None
    try:
        return parsers.get_parser_class(argument)
    except ImportError as err:
        raise ValueError(str(err))
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isoamsa.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
```

```
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |angzarr|  unicode:: U+0237C .. RIGHT ANGLE WITH DOWNWARDS ZIGZAG ARROW
.. |cirmid|   unicode:: U+02AEF .. VERTICAL LINE WITH CIRCLE ABOVE
.. |cudarrl|  unicode:: U+02938 .. RIGHT-SIDE ARC CLOCKWISE ARROW
.. |cudarrr|  unicode:: U+02935 .. ARROW POINTING RIGHTWARDS THEN CURVING DOWNWARDS
.. |cularr|   unicode:: U+021B6 .. ANTICLOCKWISE TOP SEMICIRCLE ARROW
.. |cularrp|  unicode:: U+0293D .. TOP ARC ANTICLOCKWISE ARROW WITH PLUS
.. |curarr|   unicode:: U+021B7 .. CLOCKWISE TOP SEMICIRCLE ARROW
.. |curarrm|  unicode:: U+0293C .. TOP ARC CLOCKWISE ARROW WITH MINUS
.. |Darr|     unicode:: U+021A1 .. DOWNWARDS TWO HEADED ARROW
.. |dArr|     unicode:: U+021D3 .. DOWNWARDS DOUBLE ARROW
.. |darr2|    unicode:: U+021CA .. DOWNWARDS PAIRED ARROWS
.. |ddarr|    unicode:: U+021CA .. DOWNWARDS PAIRED ARROWS
.. |DDotrahd| unicode:: U+02911 .. RIGHTWARDS ARROW WITH DOTTED STEM
.. |dfisht|   unicode:: U+0297F .. DOWN FISH TAIL
.. |dHar|     unicode:: U+02965 .. DOWNWARDS HARPOON WITH BARB LEFT BESIDE DOWNWARDS HARPOON WITH BARB RIGHT
.. |dharl|    unicode:: U+021C3 .. DOWNWARDS HARPOON WITH BARB LEFTWARDS
.. |dharr|    unicode:: U+021C2 .. DOWNWARDS HARPOON WITH BARB RIGHTWARDS
.. |dlarr|    unicode:: U+02199 .. SOUTH WEST ARROW
.. |drarr|    unicode:: U+02198 .. SOUTH EAST ARROW
.. |duarr|    unicode:: U+021F5 .. DOWNWARDS ARROW LEFTWARDS OF UPWARDS ARROW
.. |duhar|    unicode:: U+0296F .. DOWNWARDS HARPOON WITH BARB LEFT BESIDE UPWARDS HARPOON WITH BARB RIGHT
.. |dzigrarr| unicode:: U+027FF .. LONG RIGHTWARDS SQUIGGLE ARROW
.. |erarr|    unicode:: U+02971 .. EQUALS SIGN ABOVE RIGHTWARDS ARROW
.. |hArr|     unicode:: U+021D4 .. LEFT RIGHT DOUBLE ARROW
.. |harr|     unicode:: U+02194 .. LEFT RIGHT ARROW
.. |harrcir|  unicode:: U+02948 .. LEFT RIGHT ARROW THROUGH SMALL CIRCLE
.. |harrw|    unicode:: U+021AD .. LEFT RIGHT WAVE ARROW
.. |hoarr|    unicode:: U+021FF .. LEFT RIGHT OPEN-HEADED ARROW
.. |imof|     unicode:: U+022B7 .. IMAGE OF
.. |lAarr|    unicode:: U+021DA .. LEFTWARDS TRIPLE ARROW
.. |Larr|     unicode:: U+0219E .. LEFTWARDS TWO HEADED ARROW
.. |larr2|    unicode:: U+021C7 .. LEFTWARDS PAIRED ARROWS
.. |larrbfs|  unicode:: U+0291F .. LEFTWARDS ARROW FROM BAR TO BLACK DIAMOND
.. |larrfs|   unicode:: U+0291D .. LEFTWARDS ARROW TO BLACK DIAMOND
.. |larrhk|   unicode:: U+021A9 .. LEFTWARDS ARROW WITH HOOK
.. |larrlp|   unicode:: U+021AB .. LEFTWARDS ARROW WITH LOOP
.. |larrpl|   unicode:: U+02939 .. LEFT-SIDE ARC ANTICLOCKWISE ARROW
.. |larrsim|  unicode:: U+02973 .. LEFTWARDS ARROW ABOVE TILDE OPERATOR
.. |larrtl|   unicode:: U+021A2 .. LEFTWARDS ARROW WITH TAIL
.. |lAtail|   unicode:: U+0291B .. LEFTWARDS DOUBLE ARROW-TAIL
.. |latail|   unicode:: U+02919 .. LEFTWARDS ARROW-TAIL
.. |lBarr|    unicode:: U+0290E .. LEFTWARDS TRIPLE DASH ARROW
.. |lbarr|    unicode:: U+0290C .. LEFTWARDS DOUBLE DASH ARROW
.. |ldca|     unicode:: U+02936 .. ARROW POINTING DOWNWARDS THEN CURVING LEFTWARDS
.. |ldrdhar|  unicode:: U+02967 .. LEFTWARDS HARPOON WITH BARB DOWN ABOVE RIGHTWARDS HARPOON WITH BARB DOWN
.. |ldrushar| unicode:: U+0294B .. LEFT BARB DOWN RIGHT BARB UP HARPOON
.. |ldsh|     unicode:: U+021B2 .. DOWNWARDS ARROW WITH TIP LEFTWARDS
.. |lfisht|   unicode:: U+0297C .. LEFT FISH TAIL
.. |lHar|     unicode:: U+02962 .. LEFTWARDS HARPOON WITH BARB UP ABOVE LEFTWARDS HARPOON WITH BARB DOWN
.. |lhard|    unicode:: U+021BD .. LEFTWARDS HARPOON WITH BARB DOWNWARDS
.. |lharu|    unicode:: U+021BC .. LEFTWARDS HARPOON WITH BARB UPWARDS
.. |lharul|   unicode:: U+0296A .. LEFTWARDS HARPOON WITH BARB UP ABOVE LONG DASH
.. |llarr|    unicode:: U+021C7 .. LEFTWARDS PAIRED ARROWS
.. |llhard|   unicode:: U+0296B .. LEFTWARDS HARPOON WITH BARB DOWN BELOW LONG DASH
.. |loarr|    unicode:: U+021FD .. LEFTWARDS OPEN-HEADED ARROW
.. |lrarr|    unicode:: U+021C6 .. LEFTWARDS ARROW OVER RIGHTWARDS ARROW
.. |lrarr2|   unicode:: U+021C6 .. LEFTWARDS ARROW OVER RIGHTWARDS ARROW
.. |lrhar|    unicode:: U+021CB .. LEFTWARDS HARPOON OVER RIGHTWARDS HARPOON
.. |lrhar2|   unicode:: U+021CB .. LEFTWARDS HARPOON OVER RIGHTWARDS HARPOON
.. |lrhard|   unicode:: U+0296D .. RIGHTWARDS HARPOON WITH BARB DOWN BELOW LONG DASH
.. |lsh|      unicode:: U+021B0 .. UPWARDS ARROW WITH TIP LEFTWARDS
.. |lurdshar| unicode:: U+0294A .. LEFT BARB UP RIGHT BARB DOWN HARPOON
.. |luruhar|  unicode:: U+02966 .. LEFTWARDS HARPOON WITH BARB UP ABOVE RIGHTWARDS HARPOON WITH BARB UP
.. |Map|      unicode:: U+02905 .. RIGHTWARDS TWO-HEADED ARROW FROM BAR
.. |map|      unicode:: U+021A6 .. RIGHTWARDS ARROW FROM BAR
.. |midcir|   unicode:: U+02AF0 .. VERTICAL LINE WITH CIRCLE BELOW
.. |mumap|    unicode:: U+022B8 .. MULTIMAP
.. |nearhk|   unicode:: U+02924 .. NORTH EAST ARROW WITH HOOK
.. |neArr|    unicode:: U+021D7 .. NORTH EAST DOUBLE ARROW
.. |nearr|    unicode:: U+02197 .. NORTH EAST ARROW
.. |nesear|   unicode:: U+02928 .. NORTH EAST ARROW AND SOUTH EAST ARROW
.. |nhArr|    unicode:: U+021CE .. LEFT RIGHT DOUBLE ARROW WITH STROKE
.. |nharr|    unicode:: U+021AE .. LEFT RIGHT ARROW WITH STROKE
.. |nlArr|    unicode:: U+021CD .. LEFTWARDS DOUBLE ARROW WITH STROKE
.. |nlarr|    unicode:: U+0219A .. LEFTWARDS ARROW WITH STROKE
.. |nrArr|    unicode:: U+021CF .. RIGHTWARDS DOUBLE ARROW WITH STROKE
.. |nrarr|    unicode:: U+0219B .. RIGHTWARDS ARROW WITH STROKE
.. |nrarrc|   unicode:: U+02933 U+00338 .. WAVE ARROW POINTING DIRECTLY RIGHT with slash
```

```
.. |nrarrw|   unicode:: U+0219D U+00338 .. RIGHTWARDS WAVE ARROW with slash
.. |nvHarr|   unicode:: U+02904 .. LEFT RIGHT DOUBLE ARROW WITH VERTICAL STROKE
.. |nvlArr|   unicode:: U+02902 .. LEFTWARDS DOUBLE ARROW WITH VERTICAL STROKE
.. |nvrArr|   unicode:: U+02903 .. RIGHTWARDS DOUBLE ARROW WITH VERTICAL STROKE
.. |nwarhk|   unicode:: U+02923 .. NORTH WEST ARROW WITH HOOK
.. |nwArr|    unicode:: U+021D6 .. NORTH WEST DOUBLE ARROW
.. |nwarr|    unicode:: U+02196 .. NORTH WEST ARROW
.. |nwnear|   unicode:: U+02927 .. NORTH WEST ARROW AND NORTH EAST ARROW
.. |olarr|    unicode:: U+021BA .. ANTICLOCKWISE OPEN CIRCLE ARROW
.. |orarr|    unicode:: U+021BB .. CLOCKWISE OPEN CIRCLE ARROW
.. |origof|   unicode:: U+022B6 .. ORIGINAL OF
.. |rAarr|    unicode:: U+021DB .. RIGHTWARDS TRIPLE ARROW
.. |Rarr|     unicode:: U+021A0 .. RIGHTWARDS TWO HEADED ARROW
.. |rarr2|    unicode:: U+021C9 .. RIGHTWARDS PAIRED ARROWS
.. |rarrap|   unicode:: U+02975 .. RIGHTWARDS ARROW ABOVE ALMOST EQUAL TO
.. |rarrbfs|  unicode:: U+02920 .. RIGHTWARDS ARROW FROM BAR TO BLACK DIAMOND
.. |rarrc|    unicode:: U+02933 .. WAVE ARROW POINTING DIRECTLY RIGHT
.. |rarrfs|   unicode:: U+0291E .. RIGHTWARDS ARROW TO BLACK DIAMOND
.. |rarrhk|   unicode:: U+021AA .. RIGHTWARDS ARROW WITH HOOK
.. |rarrlp|   unicode:: U+021AC .. RIGHTWARDS ARROW WITH LOOP
.. |rarrpl|   unicode:: U+02945 .. RIGHTWARDS ARROW WITH PLUS BELOW
.. |rarrsim|  unicode:: U+02974 .. RIGHTWARDS ARROW ABOVE TILDE OPERATOR
.. |Rarrtl|   unicode:: U+02916 .. RIGHTWARDS TWO-HEADED ARROW WITH TAIL
.. |rarrtl|   unicode:: U+021A3 .. RIGHTWARDS ARROW WITH TAIL
.. |rarrw|    unicode:: U+0219D .. RIGHTWARDS WAVE ARROW
.. |rAtail|   unicode:: U+0291C .. RIGHTWARDS DOUBLE ARROW-TAIL
.. |ratail|   unicode:: U+0291A .. RIGHTWARDS ARROW-TAIL
.. |RBarr|    unicode:: U+02910 .. RIGHTWARDS TWO-HEADED TRIPLE DASH ARROW
.. |rBarr|    unicode:: U+0290F .. RIGHTWARDS TRIPLE DASH ARROW
.. |rbarr|    unicode:: U+0290D .. RIGHTWARDS DOUBLE DASH ARROW
.. |rdca|     unicode:: U+02937 .. ARROW POINTING DOWNWARDS THEN CURVING RIGHTWARDS
.. |rdldhar|  unicode:: U+02969 .. RIGHTWARDS HARPOON WITH BARB DOWN ABOVE LEFTWARDS HARPOON WITH BARB DOWN
.. |rdsh|     unicode:: U+021B3 .. DOWNWARDS ARROW WITH TIP RIGHTWARDS
.. |rfisht|   unicode:: U+0297D .. RIGHT FISH TAIL
.. |rHar|     unicode:: U+02964 .. RIGHTWARDS HARPOON WITH BARB UP ABOVE RIGHTWARDS HARPOON WITH BARB DOWN
.. |rhard|    unicode:: U+021C1 .. RIGHTWARDS HARPOON WITH BARB DOWNWARDS
.. |rharu|    unicode:: U+021C0 .. RIGHTWARDS HARPOON WITH BARB UPWARDS
.. |rharul|   unicode:: U+0296C .. RIGHTWARDS HARPOON WITH BARB UP ABOVE LONG DASH
.. |rlarr|    unicode:: U+021C4 .. RIGHTWARDS ARROW OVER LEFTWARDS ARROW
.. |rlarr2|   unicode:: U+021C4 .. RIGHTWARDS ARROW OVER LEFTWARDS ARROW
.. |rlhar|    unicode:: U+021CC .. RIGHTWARDS HARPOON OVER LEFTWARDS HARPOON
.. |rlhar2|   unicode:: U+021CC .. RIGHTWARDS HARPOON OVER LEFTWARDS HARPOON
.. |roarr|    unicode:: U+021FE .. RIGHTWARDS OPEN-HEADED ARROW
.. |rrarr|    unicode:: U+021C9 .. RIGHTWARDS PAIRED ARROWS
.. |rsh|      unicode:: U+021B1 .. UPWARDS ARROW WITH TIP RIGHTWARDS
.. |ruluhar|  unicode:: U+02968 .. RIGHTWARDS HARPOON WITH BARB UP ABOVE LEFTWARDS HARPOON WITH BARB UP
.. |searhk|   unicode:: U+02925 .. SOUTH EAST ARROW WITH HOOK
.. |seArr|    unicode:: U+021D8 .. SOUTH EAST DOUBLE ARROW
.. |searr|    unicode:: U+02198 .. SOUTH EAST ARROW
.. |seswar|   unicode:: U+02929 .. SOUTH EAST ARROW AND SOUTH WEST ARROW
.. |simrarr|  unicode:: U+02972 .. TILDE OPERATOR ABOVE RIGHTWARDS ARROW
.. |slarr|    unicode:: U+02190 .. LEFTWARDS ARROW
.. |srarr|    unicode:: U+02192 .. RIGHTWARDS ARROW
.. |swarhk|   unicode:: U+02926 .. SOUTH WEST ARROW WITH HOOK
.. |swArr|    unicode:: U+021D9 .. SOUTH WEST DOUBLE ARROW
.. |swarr|    unicode:: U+02199 .. SOUTH WEST ARROW
.. |swnwar|   unicode:: U+0292A .. SOUTH WEST ARROW AND NORTH WEST ARROW
.. |Uarr|     unicode:: U+0219F .. UPWARDS TWO HEADED ARROW
.. |uArr|     unicode:: U+021D1 .. UPWARDS DOUBLE ARROW
.. |uarr2|    unicode:: U+021C8 .. UPWARDS PAIRED ARROWS
.. |Uarrocir| unicode:: U+02949 .. UPWARDS TWO-HEADED ARROW FROM SMALL CIRCLE
.. |udarr|    unicode:: U+021C5 .. UPWARDS ARROW LEFTWARDS OF DOWNWARDS ARROW
.. |udhar|    unicode:: U+0296E .. UPWARDS HARPOON WITH BARB LEFT BESIDE DOWNWARDS HARPOON WITH BARB RIGHT
.. |ufisht|   unicode:: U+0297E .. UP FISH TAIL
.. |uHar|     unicode:: U+02963 .. UPWARDS HARPOON WITH BARB LEFT BESIDE UPWARDS HARPOON WITH BARB RIGHT
.. |uharl|    unicode:: U+021BF .. UPWARDS HARPOON WITH BARB LEFTWARDS
.. |uharr|    unicode:: U+021BE .. UPWARDS HARPOON WITH BARB RIGHTWARDS
.. |uuarr|    unicode:: U+021C8 .. UPWARDS PAIRED ARROWS
.. |vArr|     unicode:: U+021D5 .. UP DOWN DOUBLE ARROW
.. |varr|     unicode:: U+02195 .. UP DOWN ARROW
.. |xhArr|    unicode:: U+027FA .. LONG LEFT RIGHT DOUBLE ARROW
.. |xharr|    unicode:: U+027F7 .. LONG LEFT RIGHT ARROW
.. |xlArr|    unicode:: U+027F8 .. LONG LEFTWARDS DOUBLE ARROW
.. |xlarr|    unicode:: U+027F5 .. LONG LEFTWARDS ARROW
.. |xmap|     unicode:: U+027FC .. LONG RIGHTWARDS ARROW FROM BAR
.. |xrArr|    unicode:: U+027F9 .. LONG RIGHTWARDS DOUBLE ARROW
.. |xrarr|    unicode:: U+027F6 .. LONG RIGHTWARDS ARROW
.. |zigrarr|  unicode:: U+021DD .. RIGHTWARDS SQUIGGLE ARROW
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isoamsb.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |ac|       unicode:: U+0223E .. INVERTED LAZY S
.. |acE|      unicode:: U+0223E U+00333 .. INVERTED LAZY S with double underline
.. |amalg|    unicode:: U+02A3F .. AMALGAMATION OR COPRODUCT
.. |barvee|   unicode:: U+022BD .. NOR
.. |Barwed|   unicode:: U+02306 .. PERSPECTIVE
.. |barwed|   unicode:: U+02305 .. PROJECTIVE
.. |bsolb|    unicode:: U+029C5 .. SQUARED FALLING DIAGONAL SLASH
.. |Cap|      unicode:: U+022D2 .. DOUBLE INTERSECTION
.. |capand|   unicode:: U+02A44 .. INTERSECTION WITH LOGICAL AND
.. |capbrcup| unicode:: U+02A49 .. INTERSECTION ABOVE BAR ABOVE UNION
.. |capcap|   unicode:: U+02A4B .. INTERSECTION BESIDE AND JOINED WITH INTERSECTION
.. |capcup|   unicode:: U+02A47 .. INTERSECTION ABOVE UNION
.. |capdot|   unicode:: U+02A40 .. INTERSECTION WITH DOT
.. |caps|     unicode:: U+02229 U+0FE00 .. INTERSECTION with serifs
.. |ccaps|    unicode:: U+02A4D .. CLOSED INTERSECTION WITH SERIFS
.. |ccups|    unicode:: U+02A4C .. CLOSED UNION WITH SERIFS
.. |ccupssm|  unicode:: U+02A50 .. CLOSED UNION WITH SERIFS AND SMASH PRODUCT
.. |coprod|   unicode:: U+02210 .. N-ARY COPRODUCT
.. |Cup|      unicode:: U+022D3 .. DOUBLE UNION
.. |cupbrcap| unicode:: U+02A48 .. UNION ABOVE BAR ABOVE INTERSECTION
.. |cupcap|   unicode:: U+02A46 .. UNION ABOVE INTERSECTION
.. |cupcup|   unicode:: U+02A4A .. UNION BESIDE AND JOINED WITH UNION
.. |cupdot|   unicode:: U+0228D .. MULTISET MULTIPLICATION
.. |cupor|    unicode:: U+02A45 .. UNION WITH LOGICAL OR
.. |cups|     unicode:: U+0222A U+0FE00 .. UNION with serifs
.. |cuvee|    unicode:: U+022CE .. CURLY LOGICAL OR
.. |cuwed|    unicode:: U+022CF .. CURLY LOGICAL AND
.. |Dagger|   unicode:: U+02021 .. DOUBLE DAGGER
.. |dagger|   unicode:: U+02020 .. DAGGER
.. |diam|     unicode:: U+022C4 .. DIAMOND OPERATOR
.. |divonx|   unicode:: U+022C7 .. DIVISION TIMES
.. |eplus|    unicode:: U+02A71 .. EQUALS SIGN ABOVE PLUS SIGN
.. |hercon|   unicode:: U+022B9 .. HERMITIAN CONJUGATE MATRIX
.. |intcal|   unicode:: U+022BA .. INTERCALATE
.. |iprod|    unicode:: U+02A3C .. INTERIOR PRODUCT
.. |loplus|   unicode:: U+02A2D .. PLUS SIGN IN LEFT HALF CIRCLE
.. |lotimes|  unicode:: U+02A34 .. MULTIPLICATION SIGN IN LEFT HALF CIRCLE
.. |lthree|   unicode:: U+022CB .. LEFT SEMIDIRECT PRODUCT
.. |ltimes|   unicode:: U+022C9 .. LEFT NORMAL FACTOR SEMIDIRECT PRODUCT
.. |midast|   unicode:: U+0002A .. ASTERISK
.. |minusb|   unicode:: U+0229F .. SQUARED MINUS
.. |minusd|   unicode:: U+02238 .. DOT MINUS
.. |minusdu|  unicode:: U+02A2A .. MINUS SIGN WITH DOT BELOW
.. |ncap|     unicode:: U+02A43 .. INTERSECTION WITH OVERBAR
.. |ncup|     unicode:: U+02A42 .. UNION WITH OVERBAR
.. |oast|     unicode:: U+0229B .. CIRCLED ASTERISK OPERATOR
.. |ocir|     unicode:: U+0229A .. CIRCLED RING OPERATOR
.. |odash|    unicode:: U+0229D .. CIRCLED DASH
.. |odiv|     unicode:: U+02A38 .. CIRCLED DIVISION SIGN
.. |odot|     unicode:: U+02299 .. CIRCLED DOT OPERATOR
.. |odsold|   unicode:: U+029BC .. CIRCLED ANTICLOCKWISE-ROTATED DIVISION SIGN
.. |ofcir|    unicode:: U+029BF .. CIRCLED BULLET
.. |ogt|      unicode:: U+029C1 .. CIRCLED GREATER-THAN
.. |ohbar|    unicode:: U+029B5 .. CIRCLE WITH HORIZONTAL BAR
.. |olcir|    unicode:: U+029BE .. CIRCLED WHITE BULLET
.. |olt|      unicode:: U+029C0 .. CIRCLED LESS-THAN
.. |omid|     unicode:: U+029B6 .. CIRCLED VERTICAL BAR
.. |ominus|   unicode:: U+02296 .. CIRCLED MINUS
.. |opar|     unicode:: U+029B7 .. CIRCLED PARALLEL
.. |operp|    unicode:: U+029B9 .. CIRCLED PERPENDICULAR
.. |oplus|    unicode:: U+02295 .. CIRCLED PLUS
.. |osol|     unicode:: U+02298 .. CIRCLED DIVISION SLASH
.. |Otimes|   unicode:: U+02A37 .. MULTIPLICATION SIGN IN DOUBLE CIRCLE
.. |otimes|   unicode:: U+02297 .. CIRCLED TIMES
.. |otimesas| unicode:: U+02A36 .. CIRCLED MULTIPLICATION SIGN WITH CIRCUMFLEX ACCENT
.. |ovbar|    unicode:: U+0233D .. APL FUNCTIONAL SYMBOL CIRCLE STILE
.. |plusacir| unicode:: U+02A23 .. PLUS SIGN WITH CIRCUMFLEX ACCENT ABOVE
.. |plusb|    unicode:: U+0229E .. SQUARED PLUS
.. |pluscir|  unicode:: U+02A22 .. PLUS SIGN WITH SMALL CIRCLE ABOVE
.. |plusdo|   unicode:: U+02214 .. DOT PLUS
.. |plusdu|   unicode:: U+02A25 .. PLUS SIGN WITH DOT BELOW
.. |pluse|    unicode:: U+02A72 .. PLUS SIGN ABOVE EQUALS SIGN
.. |plussim|  unicode:: U+02A26 .. PLUS SIGN WITH TILDE BELOW
```

```
.. |plustwo|  unicode:: U+02A27 .. PLUS SIGN WITH SUBSCRIPT TWO
.. |prod|     unicode:: U+0220F .. N-ARY PRODUCT
.. |race|     unicode:: U+029DA .. LEFT DOUBLE WIGGLY FENCE
.. |roplus|   unicode:: U+02A2E .. PLUS SIGN IN RIGHT HALF CIRCLE
.. |rotimes|  unicode:: U+02A35 .. MULTIPLICATION SIGN IN RIGHT HALF CIRCLE
.. |rthree|   unicode:: U+022CC .. RIGHT SEMIDIRECT PRODUCT
.. |rtimes|   unicode:: U+022CA .. RIGHT NORMAL FACTOR SEMIDIRECT PRODUCT
.. |sdot|     unicode:: U+022C5 .. DOT OPERATOR
.. |sdotb|    unicode:: U+022A1 .. SQUARED DOT OPERATOR
.. |setmn|    unicode:: U+02216 .. SET MINUS
.. |simplus|  unicode:: U+02A24 .. PLUS SIGN WITH TILDE ABOVE
.. |smashp|   unicode:: U+02A33 .. SMASH PRODUCT
.. |solb|     unicode:: U+029C4 .. SQUARED RISING DIAGONAL SLASH
.. |sqcap|    unicode:: U+02293 .. SQUARE CAP
.. |sqcaps|   unicode:: U+02293 U+0FE00 .. SQUARE CAP with serifs
.. |sqcup|    unicode:: U+02294 .. SQUARE CUP
.. |sqcups|   unicode:: U+02294 U+0FE00 .. SQUARE CUP with serifs
.. |ssetmn|   unicode:: U+02216 .. SET MINUS
.. |sstarf|   unicode:: U+022C6 .. STAR OPERATOR
.. |subdot|   unicode:: U+02ABD .. SUBSET WITH DOT
.. |sum|      unicode:: U+02211 .. N-ARY SUMMATION
.. |supdot|   unicode:: U+02ABE .. SUPERSET WITH DOT
.. |timesb|   unicode:: U+022A0 .. SQUARED TIMES
.. |timesbar| unicode:: U+02A31 .. MULTIPLICATION SIGN WITH UNDERBAR
.. |timesd|   unicode:: U+02A30 .. MULTIPLICATION SIGN WITH DOT ABOVE
.. |top|      unicode:: U+022A4 .. DOWN TACK
.. |tridot|   unicode:: U+025EC .. WHITE UP-POINTING TRIANGLE WITH DOT
.. |triminus| unicode:: U+02A3A .. MINUS SIGN IN TRIANGLE
.. |triplus|  unicode:: U+02A39 .. PLUS SIGN IN TRIANGLE
.. |trisb|    unicode:: U+029CD .. TRIANGLE WITH SERIFS AT BOTTOM
.. |tritime|  unicode:: U+02A3B .. MULTIPLICATION SIGN IN TRIANGLE
.. |uplus|    unicode:: U+0228E .. MULTISET UNION
.. |veebar|   unicode:: U+022BB .. XOR
.. |wedbar|   unicode:: U+02A5F .. LOGICAL AND WITH UNDERBAR
.. |wreath|   unicode:: U+02240 .. WREATH PRODUCT
.. |xcap|     unicode:: U+022C2 .. N-ARY INTERSECTION
.. |xcirc|    unicode:: U+025EF .. LARGE CIRCLE
.. |xcup|     unicode:: U+022C3 .. N-ARY UNION
.. |xdtri|    unicode:: U+025BD .. WHITE DOWN-POINTING TRIANGLE
.. |xodot|    unicode:: U+02A00 .. N-ARY CIRCLED DOT OPERATOR
.. |xoplus|   unicode:: U+02A01 .. N-ARY CIRCLED PLUS OPERATOR
.. |xotime|   unicode:: U+02A02 .. N-ARY CIRCLED TIMES OPERATOR
.. |xsqcup|   unicode:: U+02A06 .. N-ARY SQUARE UNION OPERATOR
.. |xuplus|   unicode:: U+02A04 .. N-ARY UNION OPERATOR WITH PLUS
.. |xutri|    unicode:: U+025B3 .. WHITE UP-POINTING TRIANGLE
.. |xvee|     unicode:: U+022C1 .. N-ARY LOGICAL OR
.. |xwedge|   unicode:: U+022C0 .. N-ARY LOGICAL AND
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isoamsc.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |dlcorn|  unicode:: U+0231E .. BOTTOM LEFT CORNER
.. |drcorn|  unicode:: U+0231F .. BOTTOM RIGHT CORNER
.. |gtlPar|  unicode:: U+02995 .. DOUBLE LEFT ARC GREATER-THAN BRACKET
.. |langd|   unicode:: U+02991 .. LEFT ANGLE BRACKET WITH DOT
.. |lbrke|   unicode:: U+0298B .. LEFT SQUARE BRACKET WITH UNDERBAR
.. |lbrksld| unicode:: U+0298F .. LEFT SQUARE BRACKET WITH TICK IN BOTTOM CORNER
.. |lbrkslu| unicode:: U+0298D .. LEFT SQUARE BRACKET WITH TICK IN TOP CORNER
.. |lceil|   unicode:: U+02308 .. LEFT CEILING
.. |lfloor|  unicode:: U+0230A .. LEFT FLOOR
.. |lmoust|  unicode:: U+023B0 .. UPPER LEFT OR LOWER RIGHT CURLY BRACKET SECTION
.. |lpargt|  unicode:: U+029A0 .. SPHERICAL ANGLE OPENING LEFT
.. |lparlt|  unicode:: U+02993 .. LEFT ARC LESS-THAN BRACKET
.. |ltrPar|  unicode:: U+02996 .. DOUBLE RIGHT ARC LESS-THAN BRACKET
.. |rangd|   unicode:: U+02992 .. RIGHT ANGLE BRACKET WITH DOT
.. |rbrke|   unicode:: U+0298C .. RIGHT SQUARE BRACKET WITH UNDERBAR
.. |rbrksld| unicode:: U+0298E .. RIGHT SQUARE BRACKET WITH TICK IN BOTTOM CORNER
.. |rbrkslu| unicode:: U+02990 .. RIGHT SQUARE BRACKET WITH TICK IN TOP CORNER
.. |rceil|   unicode:: U+02309 .. RIGHT CEILING
.. |rfloor|  unicode:: U+0230B .. RIGHT FLOOR
.. |rmoust|  unicode:: U+023B1 .. UPPER RIGHT OR LOWER LEFT CURLY BRACKET SECTION
.. |rpargt|  unicode:: U+02994 .. RIGHT ARC GREATER-THAN BRACKET
.. |ulcorn|  unicode:: U+0231C .. TOP LEFT CORNER
.. |urcorn|  unicode:: U+0231D .. TOP RIGHT CORNER
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isoamsn.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |gnap|     unicode:: U+02A8A .. GREATER-THAN AND NOT APPROXIMATE
.. |gnE|      unicode:: U+02269 .. GREATER-THAN BUT NOT EQUAL TO
.. |gne|      unicode:: U+02A88 .. GREATER-THAN AND SINGLE-LINE NOT EQUAL TO
.. |gnsim|    unicode:: U+022E7 .. GREATER-THAN BUT NOT EQUIVALENT TO
.. |gvnE|     unicode:: U+02269 U+0FE00 .. GREATER-THAN BUT NOT EQUAL TO - with vertical stroke
.. |lnap|     unicode:: U+02A89 .. LESS-THAN AND NOT APPROXIMATE
.. |lnE|      unicode:: U+02268 .. LESS-THAN BUT NOT EQUAL TO
.. |lne|      unicode:: U+02A87 .. LESS-THAN AND SINGLE-LINE NOT EQUAL TO
.. |lnsim|    unicode:: U+022E6 .. LESS-THAN BUT NOT EQUIVALENT TO
.. |lvnE|     unicode:: U+02268 U+0FE00 .. LESS-THAN BUT NOT EQUAL TO - with vertical stroke
.. |nap|      unicode:: U+02249 .. NOT ALMOST EQUAL TO
.. |napE|     unicode:: U+02A70 U+00338 .. APPROXIMATELY EQUAL OR EQUAL TO with slash
.. |napid|    unicode:: U+0224B U+00338 .. TRIPLE TILDE with slash
.. |ncong|    unicode:: U+02247 .. NEITHER APPROXIMATELY NOR ACTUALLY EQUAL TO
.. |ncongdot| unicode:: U+02A6D U+00338 .. CONGRUENT WITH DOT ABOVE with slash
.. |nequiv|   unicode:: U+02262 .. NOT IDENTICAL TO
.. |ngE|      unicode:: U+02267 U+00338 .. GREATER-THAN OVER EQUAL TO with slash
.. |nge|      unicode:: U+02271 .. NEITHER GREATER-THAN NOR EQUAL TO
.. |nges|     unicode:: U+02A7E U+00338 .. GREATER-THAN OR SLANTED EQUAL TO with slash
.. |nGg|      unicode:: U+022D9 U+00338 .. VERY MUCH GREATER-THAN with slash
.. |ngsim|    unicode:: U+02275 .. NEITHER GREATER-THAN NOR EQUIVALENT TO
.. |nGt|      unicode:: U+0226B U+020D2 .. MUCH GREATER THAN with vertical line
.. |ngt|      unicode:: U+0226F .. NOT GREATER-THAN
.. |nGtv|     unicode:: U+0226B U+00338 .. MUCH GREATER THAN with slash
.. |nlE|      unicode:: U+02266 U+00338 .. LESS-THAN OVER EQUAL TO with slash
.. |nle|      unicode:: U+02270 .. NEITHER LESS-THAN NOR EQUAL TO
.. |nles|     unicode:: U+02A7D U+00338 .. LESS-THAN OR SLANTED EQUAL TO with slash
.. |nLl|      unicode:: U+022D8 U+00338 .. VERY MUCH LESS-THAN with slash
.. |nlsim|    unicode:: U+02274 .. NEITHER LESS-THAN NOR EQUIVALENT TO
.. |nLt|      unicode:: U+0226A U+020D2 .. MUCH LESS THAN with vertical line
.. |nlt|      unicode:: U+0226E .. NOT LESS-THAN
.. |nltri|    unicode:: U+022EA .. NOT NORMAL SUBGROUP OF
.. |nltrie|   unicode:: U+022EC .. NOT NORMAL SUBGROUP OF OR EQUAL TO
.. |nLtv|     unicode:: U+0226A U+00338 .. MUCH LESS THAN with slash
.. |nmid|     unicode:: U+02224 .. DOES NOT DIVIDE
.. |npar|     unicode:: U+02226 .. NOT PARALLEL TO
.. |npr|      unicode:: U+02280 .. DOES NOT PRECEDE
.. |nprcue|   unicode:: U+022E0 .. DOES NOT PRECEDE OR EQUAL
.. |npre|     unicode:: U+02AAF U+00338 .. PRECEDES ABOVE SINGLE-LINE EQUALS SIGN with slash
.. |nrtri|    unicode:: U+022EB .. DOES NOT CONTAIN AS NORMAL SUBGROUP
.. |nrtrie|   unicode:: U+022ED .. DOES NOT CONTAIN AS NORMAL SUBGROUP OR EQUAL
.. |nsc|      unicode:: U+02281 .. DOES NOT SUCCEED
.. |nsccue|   unicode:: U+022E1 .. DOES NOT SUCCEED OR EQUAL
.. |nsce|     unicode:: U+02AB0 U+00338 .. SUCCEEDS ABOVE SINGLE-LINE EQUALS SIGN with slash
.. |nsim|     unicode:: U+02241 .. NOT TILDE
.. |nsime|    unicode:: U+02244 .. NOT ASYMPTOTICALLY EQUAL TO
.. |nsmid|    unicode:: U+02224 .. DOES NOT DIVIDE
.. |nspar|    unicode:: U+02226 .. NOT PARALLEL TO
.. |nsqsube|  unicode:: U+022E2 .. NOT SQUARE IMAGE OF OR EQUAL TO
.. |nsqsupe|  unicode:: U+022E3 .. NOT SQUARE ORIGINAL OF OR EQUAL TO
.. |nsub|     unicode:: U+02284 .. NOT A SUBSET OF
.. |nsubE|    unicode:: U+02AC5 U+00338 .. SUBSET OF ABOVE EQUALS SIGN with slash
.. |nsube|    unicode:: U+02288 .. NEITHER A SUBSET OF NOR EQUAL TO
.. |nsup|     unicode:: U+02285 .. NOT A SUPERSET OF
.. |nsupE|    unicode:: U+02AC6 U+00338 .. SUPERSET OF ABOVE EQUALS SIGN with slash
.. |nsupe|    unicode:: U+02289 .. NEITHER A SUPERSET OF NOR EQUAL TO
.. |ntgl|     unicode:: U+02279 .. NEITHER GREATER-THAN NOR LESS-THAN
.. |ntlg|     unicode:: U+02278 .. NEITHER LESS-THAN NOR GREATER-THAN
.. |nvap|     unicode:: U+0224D U+020D2 .. EQUIVALENT TO with vertical line
.. |nVDash|   unicode:: U+022AF .. NEGATED DOUBLE VERTICAL BAR DOUBLE RIGHT TURNSTILE
.. |nVdash|   unicode:: U+022AE .. DOES NOT FORCE
.. |nvDash|   unicode:: U+022AD .. NOT TRUE
.. |nvdash|   unicode:: U+022AC .. DOES NOT PROVE
.. |nvge|     unicode:: U+02265 U+020D2 .. GREATER-THAN OR EQUAL TO with vertical line
.. |nvgt|     unicode:: U+0003E U+020D2 .. GREATER-THAN SIGN with vertical line
.. |nvle|     unicode:: U+02264 U+020D2 .. LESS-THAN OR EQUAL TO with vertical line
.. |nvlt|     unicode:: U+0003C U+020D2 .. LESS-THAN SIGN with vertical line
.. |nvltrie|  unicode:: U+022B4 U+020D2 .. NORMAL SUBGROUP OF OR EQUAL TO with vertical line
.. |nvrtrie|  unicode:: U+022B5 U+020D2 .. CONTAINS AS NORMAL SUBGROUP OR EQUAL TO with vertical line
.. |nvsim|    unicode:: U+0223C U+020D2 .. TILDE OPERATOR with vertical line
```

```
.. |parsim|   unicode:: U+02AF3 .. PARALLEL WITH TILDE OPERATOR
.. |prnap|    unicode:: U+02AB9 .. PRECEDES ABOVE NOT ALMOST EQUAL TO
.. |prnE|     unicode:: U+02AB5 .. PRECEDES ABOVE NOT EQUAL TO
.. |prnsim|   unicode:: U+022E8 .. PRECEDES BUT NOT EQUIVALENT TO
.. |rnmid|    unicode:: U+02AEE .. DOES NOT DIVIDE WITH REVERSED NEGATION SLASH
.. |scnap|    unicode:: U+02ABA .. SUCCEEDS ABOVE NOT ALMOST EQUAL TO
.. |scnE|     unicode:: U+02AB6 .. SUCCEEDS ABOVE NOT EQUAL TO
.. |scnsim|   unicode:: U+022E9 .. SUCCEEDS BUT NOT EQUIVALENT TO
.. |simne|    unicode:: U+02246 .. APPROXIMATELY BUT NOT ACTUALLY EQUAL TO
.. |solbar|   unicode:: U+0233F .. APL FUNCTIONAL SYMBOL SLASH BAR
.. |subnE|    unicode:: U+02ACB .. SUBSET OF ABOVE NOT EQUAL TO
.. |subne|    unicode:: U+0228A .. SUBSET OF WITH NOT EQUAL TO
.. |supnE|    unicode:: U+02ACC .. SUPERSET OF ABOVE NOT EQUAL TO
.. |supne|    unicode:: U+0228B .. SUPERSET OF WITH NOT EQUAL TO
.. |vnsub|    unicode:: U+02282 U+020D2 .. SUBSET OF with vertical line
.. |vnsup|    unicode:: U+02283 U+020D2 .. SUPERSET OF with vertical line
.. |vsubnE|   unicode:: U+02ACB U+0FE00 .. SUBSET OF ABOVE NOT EQUAL TO - variant with stroke through bottom members
.. |vsubne|   unicode:: U+0228A U+0FE00 .. SUBSET OF WITH NOT EQUAL TO - variant with stroke through bottom members
.. |vsupnE|   unicode:: U+02ACC U+0FE00 .. SUPERSET OF ABOVE NOT EQUAL TO - variant with stroke through bottom members
.. |vsupne|   unicode:: U+0228B U+0FE00 .. SUPERSET OF WITH NOT EQUAL TO - variant with stroke through bottom members
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isoamso.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |ang|      unicode:: U+02220 .. ANGLE
.. |ange|     unicode:: U+029A4 .. ANGLE WITH UNDERBAR
.. |angmsd|   unicode:: U+02221 .. MEASURED ANGLE
.. |angmsdaa| unicode:: U+029A8 .. MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING UP AND RIGHT
.. |angmsdab| unicode:: U+029A9 .. MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING UP AND LEFT
.. |angmsdac| unicode:: U+029AA .. MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING DOWN AND RIGHT
.. |angmsdad| unicode:: U+029AB .. MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING DOWN AND LEFT
.. |angmsdae| unicode:: U+029AC .. MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING RIGHT AND UP
.. |angmsdaf| unicode:: U+029AD .. MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING LEFT AND UP
.. |angmsdag| unicode:: U+029AE .. MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING RIGHT AND DOWN
.. |angmsdah| unicode:: U+029AF .. MEASURED ANGLE WITH OPEN ARM ENDING IN ARROW POINTING LEFT AND DOWN
.. |angrtvb|  unicode:: U+022BE .. RIGHT ANGLE WITH ARC
.. |angrtvbd| unicode:: U+0299D .. MEASURED RIGHT ANGLE WITH DOT
.. |bbrk|     unicode:: U+023B5 .. BOTTOM SQUARE BRACKET
.. |bbrktbrk| unicode:: U+023B6 .. BOTTOM SQUARE BRACKET OVER TOP SQUARE BRACKET
.. |bemptyv|  unicode:: U+029B0 .. REVERSED EMPTY SET
.. |beth|     unicode:: U+02136 .. BET SYMBOL
.. |boxbox|   unicode:: U+029C9 .. TWO JOINED SQUARES
.. |bprime|   unicode:: U+02035 .. REVERSED PRIME
.. |bsemi|    unicode:: U+0204F .. REVERSED SEMICOLON
.. |cemptyv|  unicode:: U+029B2 .. EMPTY SET WITH SMALL CIRCLE ABOVE
.. |cirE|     unicode:: U+029C3 .. CIRCLE WITH TWO HORIZONTAL STROKES TO THE RIGHT
.. |cirscir|  unicode:: U+029C2 .. CIRCLE WITH SMALL CIRCLE TO THE RIGHT
.. |comp|     unicode:: U+02201 .. COMPLEMENT
.. |daleth|   unicode:: U+02138 .. DALET SYMBOL
.. |demptyv|  unicode:: U+029B1 .. EMPTY SET WITH OVERBAR
.. |ell|      unicode:: U+02113 .. SCRIPT SMALL L
.. |empty|    unicode:: U+02205 .. EMPTY SET
.. |emptyv|   unicode:: U+02205 .. EMPTY SET
.. |gimel|    unicode:: U+02137 .. GIMEL SYMBOL
.. |iiota|    unicode:: U+02129 .. TURNED GREEK SMALL LETTER IOTA
.. |image|    unicode:: U+02111 .. BLACK-LETTER CAPITAL I
.. |imath|    unicode:: U+00131 .. LATIN SMALL LETTER DOTLESS I
.. |inodot|   unicode:: U+00131 .. LATIN SMALL LETTER DOTLESS I
.. |jmath|    unicode:: U+0006A .. LATIN SMALL LETTER J
.. |jnodot|   unicode:: U+0006A .. LATIN SMALL LETTER J
.. |laemptyv| unicode:: U+029B4 .. EMPTY SET WITH LEFT ARROW ABOVE
.. |lltri|    unicode:: U+025FA .. LOWER LEFT TRIANGLE
.. |lrtri|    unicode:: U+022BF .. RIGHT TRIANGLE
.. |mho|      unicode:: U+02127 .. INVERTED OHM SIGN
.. |nang|     unicode:: U+02220 U+020D2 .. ANGLE with vertical line
.. |nexist|   unicode:: U+02204 .. THERE DOES NOT EXIST
.. |oS|       unicode:: U+024C8 .. CIRCLED LATIN CAPITAL LETTER S
.. |planck|   unicode:: U+0210F .. PLANCK CONSTANT OVER TWO PI
.. |plankv|   unicode:: U+0210F .. PLANCK CONSTANT OVER TWO PI
.. |raemptyv| unicode:: U+029B3 .. EMPTY SET WITH RIGHT ARROW ABOVE
.. |range|    unicode:: U+029A5 .. REVERSED ANGLE WITH UNDERBAR
.. |real|     unicode:: U+0211C .. BLACK-LETTER CAPITAL R
.. |sbsol|    unicode:: U+0FE68 .. SMALL REVERSE SOLIDUS
.. |tbrk|     unicode:: U+023B4 .. TOP SQUARE BRACKET
```

```
.. |trpezium| unicode:: U+0FFFD .. REPLACEMENT CHARACTER
.. |ultri|    unicode:: U+025F8 .. UPPER LEFT TRIANGLE
.. |urtri|    unicode:: U+025F9 .. UPPER RIGHT TRIANGLE
.. |vprime|   unicode:: U+02032 .. PRIME
.. |vzigzag|  unicode:: U+0299A .. VERTICAL ZIGZAG LINE
.. |weierp|   unicode:: U+02118 .. SCRIPT CAPITAL P
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isoamsr.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |apE|      unicode:: U+02A70 .. APPROXIMATELY EQUAL OR EQUAL TO
.. |ape|      unicode:: U+0224A .. ALMOST EQUAL OR EQUAL TO
.. |apid|     unicode:: U+0224B .. TRIPLE TILDE
.. |asymp|    unicode:: U+02248 .. ALMOST EQUAL TO
.. |Barv|     unicode:: U+02AE7 .. SHORT DOWN TACK WITH OVERBAR
.. |bcong|    unicode:: U+0224C .. ALL EQUAL TO
.. |bepsi|    unicode:: U+003F6 .. GREEK REVERSED LUNATE EPSILON SYMBOL
.. |bowtie|   unicode:: U+022C8 .. BOWTIE
.. |bsim|     unicode:: U+0223D .. REVERSED TILDE
.. |bsime|    unicode:: U+022CD .. REVERSED TILDE EQUALS
.. |bsolhsub| unicode:: U+0005C U+02282 .. REVERSE SOLIDUS, SUBSET OF
.. |bump|     unicode:: U+0224E .. GEOMETRICALLY EQUIVALENT TO
.. |bumpE|    unicode:: U+02AAE .. EQUALS SIGN WITH BUMPY ABOVE
.. |bumpe|    unicode:: U+0224F .. DIFFERENCE BETWEEN
.. |cire|     unicode:: U+02257 .. RING EQUAL TO
.. |Colon|    unicode:: U+02237 .. PROPORTION
.. |Colone|   unicode:: U+02A74 .. DOUBLE COLON EQUAL
.. |colone|   unicode:: U+02254 .. COLON EQUALS
.. |congdot|  unicode:: U+02A6D .. CONGRUENT WITH DOT ABOVE
.. |csub|     unicode:: U+02ACF .. CLOSED SUBSET
.. |csube|    unicode:: U+02AD1 .. CLOSED SUBSET OR EQUAL TO
.. |csup|     unicode:: U+02AD0 .. CLOSED SUPERSET
.. |csupe|    unicode:: U+02AD2 .. CLOSED SUPERSET OR EQUAL TO
.. |cuepr|    unicode:: U+022DE .. EQUAL TO OR PRECEDES
.. |cuesc|    unicode:: U+022DF .. EQUAL TO OR SUCCEEDS
.. |cupre|    unicode:: U+0227C .. PRECEDES OR EQUAL TO
.. |Dashv|    unicode:: U+02AE4 .. VERTICAL BAR DOUBLE LEFT TURNSTILE
.. |dashv|    unicode:: U+022A3 .. LEFT TACK
.. |easter|   unicode:: U+02A6E .. EQUALS WITH ASTERISK
.. |ecir|     unicode:: U+02256 .. RING IN EQUAL TO
.. |ecolon|   unicode:: U+02255 .. EQUALS COLON
.. |eDDot|    unicode:: U+02A77 .. EQUALS SIGN WITH TWO DOTS ABOVE AND TWO DOTS BELOW
.. |eDot|     unicode:: U+02251 .. GEOMETRICALLY EQUAL TO
.. |efDot|    unicode:: U+02252 .. APPROXIMATELY EQUAL TO OR THE IMAGE OF
.. |eg|       unicode:: U+02A9A .. DOUBLE-LINE EQUAL TO OR GREATER-THAN
.. |egs|      unicode:: U+02A96 .. SLANTED EQUAL TO OR GREATER-THAN
.. |egsdot|   unicode:: U+02A98 .. SLANTED EQUAL TO OR GREATER-THAN WITH DOT INSIDE
.. |el|       unicode:: U+02A99 .. DOUBLE-LINE EQUAL TO OR LESS-THAN
.. |els|      unicode:: U+02A95 .. SLANTED EQUAL TO OR LESS-THAN
.. |elsdot|   unicode:: U+02A97 .. SLANTED EQUAL TO OR LESS-THAN WITH DOT INSIDE
.. |equest|   unicode:: U+0225F .. QUESTIONED EQUAL TO
.. |equivDD|  unicode:: U+02A78 .. EQUIVALENT WITH FOUR DOTS ABOVE
.. |erDot|    unicode:: U+02253 .. IMAGE OF OR APPROXIMATELY EQUAL TO
.. |esdot|    unicode:: U+02250 .. APPROACHES THE LIMIT
.. |Esim|     unicode:: U+02A73 .. EQUALS SIGN ABOVE TILDE OPERATOR
.. |esim|     unicode:: U+02242 .. MINUS TILDE
.. |fork|     unicode:: U+022D4 .. PITCHFORK
.. |forkv|    unicode:: U+02AD9 .. ELEMENT OF OPENING DOWNWARDS
.. |frown|    unicode:: U+02322 .. FROWN
.. |gap|      unicode:: U+02A86 .. GREATER-THAN OR APPROXIMATE
.. |gE|       unicode:: U+02267 .. GREATER-THAN OVER EQUAL TO
.. |gEl|      unicode:: U+02A8C .. GREATER-THAN ABOVE DOUBLE-LINE EQUAL ABOVE LESS-THAN
.. |gel|      unicode:: U+022DB .. GREATER-THAN EQUAL TO OR LESS-THAN
.. |ges|      unicode:: U+02A7E .. GREATER-THAN OR SLANTED EQUAL TO
.. |gescc|    unicode:: U+02AA9 .. GREATER-THAN CLOSED BY CURVE ABOVE SLANTED EQUAL
.. |gesdot|   unicode:: U+02A80 .. GREATER-THAN OR SLANTED EQUAL TO WITH DOT INSIDE
.. |gesdoto|  unicode:: U+02A82 .. GREATER-THAN OR SLANTED EQUAL TO WITH DOT ABOVE
.. |gesdotol| unicode:: U+02A84 .. GREATER-THAN OR SLANTED EQUAL TO WITH DOT ABOVE LEFT
.. |gesl|     unicode:: U+022DB U+0FE00 .. GREATER-THAN slanted EQUAL TO OR LESS-THAN
.. |gesles|   unicode:: U+02A94 .. GREATER-THAN ABOVE SLANTED EQUAL ABOVE LESS-THAN ABOVE SLANTED EQUAL
.. |Gg|       unicode:: U+022D9 .. VERY MUCH GREATER-THAN
.. |gl|       unicode:: U+02277 .. GREATER-THAN OR LESS-THAN
.. |gla|      unicode:: U+02AA5 .. GREATER-THAN BESIDE LESS-THAN
.. |glE|      unicode:: U+02A92 .. GREATER-THAN ABOVE LESS-THAN ABOVE DOUBLE-LINE EQUAL
```

```
.. |glj|      unicode:: U+02AA4 .. GREATER-THAN OVERLAPPING LESS-THAN
.. |gsdot|    unicode:: U+022D7 .. GREATER-THAN WITH DOT
.. |gsim|     unicode:: U+02273 .. GREATER-THAN OR EQUIVALENT TO
.. |gsime|    unicode:: U+02A8E .. GREATER-THAN ABOVE SIMILAR OR EQUAL
.. |gsiml|    unicode:: U+02A90 .. GREATER-THAN ABOVE SIMILAR ABOVE LESS-THAN
.. |Gt|       unicode:: U+0226B .. MUCH GREATER-THAN
.. |gtcc|     unicode:: U+02AA7 .. GREATER-THAN CLOSED BY CURVE
.. |gtcir|    unicode:: U+02A7A .. GREATER-THAN WITH CIRCLE INSIDE
.. |gtdot|    unicode:: U+022D7 .. GREATER-THAN WITH DOT
.. |gtquest|  unicode:: U+02A7C .. GREATER-THAN WITH QUESTION MARK ABOVE
.. |gtrarr|   unicode:: U+02978 .. GREATER-THAN ABOVE RIGHTWARDS ARROW
.. |homtht|   unicode:: U+0223B .. HOMOTHETIC
.. |lap|      unicode:: U+02A85 .. LESS-THAN OR APPROXIMATE
.. |lat|      unicode:: U+02AAB .. LARGER THAN
.. |late|     unicode:: U+02AAD .. LARGER THAN OR EQUAL TO
.. |lates|    unicode:: U+02AAD U+0FE00 .. LARGER THAN OR slanted EQUAL
.. |ldot|     unicode:: U+022D6 .. LESS-THAN WITH DOT
.. |lE|       unicode:: U+02266 .. LESS-THAN OVER EQUAL TO
.. |lEg|      unicode:: U+02A8B .. LESS-THAN ABOVE DOUBLE-LINE EQUAL ABOVE GREATER-THAN
.. |leg|      unicode:: U+022DA .. LESS-THAN EQUAL TO OR GREATER-THAN
.. |les|      unicode:: U+02A7D .. LESS-THAN OR SLANTED EQUAL TO
.. |lescc|    unicode:: U+02AA8 .. LESS-THAN CLOSED BY CURVE ABOVE SLANTED EQUAL
.. |lesdot|   unicode:: U+02A7F .. LESS-THAN OR SLANTED EQUAL TO WITH DOT INSIDE
.. |lesdoto|  unicode:: U+02A81 .. LESS-THAN OR SLANTED EQUAL TO WITH DOT ABOVE
.. |lesdotor| unicode:: U+02A83 .. LESS-THAN OR SLANTED EQUAL TO WITH DOT ABOVE RIGHT
.. |lesg|     unicode:: U+022DA U+0FE00 .. LESS-THAN slanted EQUAL TO OR GREATER-THAN
.. |lesges|   unicode:: U+02A93 .. LESS-THAN ABOVE SLANTED EQUAL ABOVE GREATER-THAN ABOVE SLANTED EQUAL
.. |lg|       unicode:: U+02276 .. LESS-THAN OR GREATER-THAN
.. |lgE|      unicode:: U+02A91 .. LESS-THAN ABOVE GREATER-THAN ABOVE DOUBLE-LINE EQUAL
.. |Ll|       unicode:: U+022D8 .. VERY MUCH LESS-THAN
.. |lsim|     unicode:: U+02272 .. LESS-THAN OR EQUIVALENT TO
.. |lsime|    unicode:: U+02A8D .. LESS-THAN ABOVE SIMILAR OR EQUAL
.. |lsimg|    unicode:: U+02A8F .. LESS-THAN ABOVE SIMILAR ABOVE GREATER-THAN
.. |Lt|       unicode:: U+0226A .. MUCH LESS-THAN
.. |ltcc|     unicode:: U+02AA6 .. LESS-THAN CLOSED BY CURVE
.. |ltcir|    unicode:: U+02A79 .. LESS-THAN WITH CIRCLE INSIDE
.. |ltdot|    unicode:: U+022D6 .. LESS-THAN WITH DOT
.. |ltlarr|   unicode:: U+02976 .. LESS-THAN ABOVE LEFTWARDS ARROW
.. |ltquest|  unicode:: U+02A7B .. LESS-THAN WITH QUESTION MARK ABOVE
.. |ltrie|    unicode:: U+022B4 .. NORMAL SUBGROUP OF OR EQUAL TO
.. |mcomma|   unicode:: U+02A29 .. MINUS SIGN WITH COMMA ABOVE
.. |mDDot|    unicode:: U+0223A .. GEOMETRIC PROPORTION
.. |mid|      unicode:: U+02223 .. DIVIDES
.. |mlcp|     unicode:: U+02ADB .. TRANSVERSAL INTERSECTION
.. |models|   unicode:: U+022A7 .. MODELS
.. |mstpos|   unicode:: U+0223E .. INVERTED LAZY S
.. |Pr|       unicode:: U+02ABB .. DOUBLE PRECEDES
.. |pr|       unicode:: U+0227A .. PRECEDES
.. |prap|     unicode:: U+02AB7 .. PRECEDES ABOVE ALMOST EQUAL TO
.. |prcue|    unicode:: U+0227C .. PRECEDES OR EQUAL TO
.. |prE|      unicode:: U+02AB3 .. PRECEDES ABOVE EQUALS SIGN
.. |pre|      unicode:: U+02AAF .. PRECEDES ABOVE SINGLE-LINE EQUALS SIGN
.. |prsim|    unicode:: U+0227E .. PRECEDES OR EQUIVALENT TO
.. |prurel|   unicode:: U+022B0 .. PRECEDES UNDER RELATION
.. |ratio|    unicode:: U+02236 .. RATIO
.. |rtrie|    unicode:: U+022B5 .. CONTAINS AS NORMAL SUBGROUP OR EQUAL TO
.. |rtriltri| unicode:: U+029CE .. RIGHT TRIANGLE ABOVE LEFT TRIANGLE
.. |samalg|   unicode:: U+02210 .. N-ARY COPRODUCT
.. |Sc|       unicode:: U+02ABC .. DOUBLE SUCCEEDS
.. |sc|       unicode:: U+0227B .. SUCCEEDS
.. |scap|     unicode:: U+02AB8 .. SUCCEEDS ABOVE ALMOST EQUAL TO
.. |sccue|    unicode:: U+0227D .. SUCCEEDS OR EQUAL TO
.. |scE|      unicode:: U+02AB4 .. SUCCEEDS ABOVE EQUALS SIGN
.. |sce|      unicode:: U+02AB0 .. SUCCEEDS ABOVE SINGLE-LINE EQUALS SIGN
.. |scsim|    unicode:: U+0227F .. SUCCEEDS OR EQUIVALENT TO
.. |sdote|    unicode:: U+02A66 .. EQUALS SIGN WITH DOT BELOW
.. |sfrown|   unicode:: U+02322 .. FROWN
.. |simg|     unicode:: U+02A9E .. SIMILAR OR GREATER-THAN
.. |simgE|    unicode:: U+02AA0 .. SIMILAR ABOVE GREATER-THAN ABOVE EQUALS SIGN
.. |siml|     unicode:: U+02A9D .. SIMILAR OR LESS-THAN
.. |simlE|    unicode:: U+02A9F .. SIMILAR ABOVE LESS-THAN ABOVE EQUALS SIGN
.. |smid|     unicode:: U+02223 .. DIVIDES
.. |smile|    unicode:: U+02323 .. SMILE
.. |smt|      unicode:: U+02AAA .. SMALLER THAN
.. |smte|     unicode:: U+02AAC .. SMALLER THAN OR EQUAL TO
.. |smtes|    unicode:: U+02AAC U+0FE00 .. SMALLER THAN OR slanted EQUAL
.. |spar|     unicode:: U+02225 .. PARALLEL TO
.. |sqsub|    unicode:: U+0228F .. SQUARE IMAGE OF
.. |sqsube|   unicode:: U+02291 .. SQUARE IMAGE OF OR EQUAL TO
.. |sqsup|    unicode:: U+02290 .. SQUARE ORIGINAL OF
.. |sqsupe|   unicode:: U+02292 .. SQUARE ORIGINAL OF OR EQUAL TO
```

```
.. |ssmile|   unicode:: U+02323 .. SMILE
.. |Sub|      unicode:: U+022D0 .. DOUBLE SUBSET
.. |subE|     unicode:: U+02AC5 .. SUBSET OF ABOVE EQUALS SIGN
.. |subedot|  unicode:: U+02AC3 .. SUBSET OF OR EQUAL TO WITH DOT ABOVE
.. |submult|  unicode:: U+02AC1 .. SUBSET WITH MULTIPLICATION SIGN BELOW
.. |subplus|  unicode:: U+02ABF .. SUBSET WITH PLUS SIGN BELOW
.. |subrarr|  unicode:: U+02979 .. SUBSET ABOVE RIGHTWARDS ARROW
.. |subsim|   unicode:: U+02AC7 .. SUBSET OF ABOVE TILDE OPERATOR
.. |subsub|   unicode:: U+02AD5 .. SUBSET ABOVE SUBSET
.. |subsup|   unicode:: U+02AD3 .. SUBSET ABOVE SUPERSET
.. |Sup|      unicode:: U+022D1 .. DOUBLE SUPERSET
.. |supdsub|  unicode:: U+02AD8 .. SUPERSET BESIDE AND JOINED BY DASH WITH SUBSET
.. |supE|     unicode:: U+02AC6 .. SUPERSET OF ABOVE EQUALS SIGN
.. |supedot|  unicode:: U+02AC4 .. SUPERSET OF OR EQUAL TO WITH DOT ABOVE
.. |suphsol|  unicode:: U+02283 U+0002F .. SUPERSET OF, SOLIDUS
.. |suphsub|  unicode:: U+02AD7 .. SUPERSET BESIDE SUBSET
.. |suplarr|  unicode:: U+0297B .. SUPERSET ABOVE LEFTWARDS ARROW
.. |supmult|  unicode:: U+02AC2 .. SUPERSET WITH MULTIPLICATION SIGN BELOW
.. |supplus|  unicode:: U+02AC0 .. SUPERSET WITH PLUS SIGN BELOW
.. |supsim|   unicode:: U+02AC8 .. SUPERSET OF ABOVE TILDE OPERATOR
.. |supsub|   unicode:: U+02AD4 .. SUPERSET ABOVE SUBSET
.. |supsup|   unicode:: U+02AD6 .. SUPERSET ABOVE SUPERSET
.. |thkap|    unicode:: U+02248 .. ALMOST EQUAL TO
.. |thksim|   unicode:: U+0223C .. TILDE OPERATOR
.. |topfork|  unicode:: U+02ADA .. PITCHFORK WITH TEE TOP
.. |trie|     unicode:: U+0225C .. DELTA EQUAL TO
.. |twixt|    unicode:: U+0226C .. BETWEEN
.. |Vbar|     unicode:: U+02AEB .. DOUBLE UP TACK
.. |vBar|     unicode:: U+02AE8 .. SHORT UP TACK WITH UNDERBAR
.. |vBarv|    unicode:: U+02AE9 .. SHORT UP TACK ABOVE SHORT DOWN TACK
.. |VDash|    unicode:: U+022AB .. DOUBLE VERTICAL BAR DOUBLE RIGHT TURNSTILE
.. |Vdash|    unicode:: U+022A9 .. FORCES
.. |vDash|    unicode:: U+022A8 .. TRUE
.. |vdash|    unicode:: U+022A2 .. RIGHT TACK
.. |Vdashl|   unicode:: U+02AE6 .. LONG DASH FROM LEFT MEMBER OF DOUBLE VERTICAL
.. |veebar|   unicode:: U+022BB .. XOR
.. |vltri|    unicode:: U+022B2 .. NORMAL SUBGROUP OF
.. |vprop|    unicode:: U+0221D .. PROPORTIONAL TO
.. |vrtri|    unicode:: U+022B3 .. CONTAINS AS NORMAL SUBGROUP
.. |Vvdash|   unicode:: U+022AA .. TRIPLE VERTICAL BAR RIGHT TURNSTILE
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isobox.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |boxDL| unicode:: U+02557 .. BOX DRAWINGS DOUBLE DOWN AND LEFT
.. |boxDl| unicode:: U+02556 .. BOX DRAWINGS DOWN DOUBLE AND LEFT SINGLE
.. |boxdL| unicode:: U+02555 .. BOX DRAWINGS DOWN SINGLE AND LEFT DOUBLE
.. |boxdl| unicode:: U+02510 .. BOX DRAWINGS LIGHT DOWN AND LEFT
.. |boxDR| unicode:: U+02554 .. BOX DRAWINGS DOUBLE DOWN AND RIGHT
.. |boxDr| unicode:: U+02553 .. BOX DRAWINGS DOWN DOUBLE AND RIGHT SINGLE
.. |boxdR| unicode:: U+02552 .. BOX DRAWINGS DOWN SINGLE AND RIGHT DOUBLE
.. |boxdr| unicode:: U+0250C .. BOX DRAWINGS LIGHT DOWN AND RIGHT
.. |boxH|  unicode:: U+02550 .. BOX DRAWINGS DOUBLE HORIZONTAL
.. |boxh|  unicode:: U+02500 .. BOX DRAWINGS LIGHT HORIZONTAL
.. |boxHD| unicode:: U+02566 .. BOX DRAWINGS DOUBLE DOWN AND HORIZONTAL
.. |boxHd| unicode:: U+02564 .. BOX DRAWINGS DOWN SINGLE AND HORIZONTAL DOUBLE
.. |boxhD| unicode:: U+02565 .. BOX DRAWINGS DOWN DOUBLE AND HORIZONTAL SINGLE
.. |boxhd| unicode:: U+0252C .. BOX DRAWINGS LIGHT DOWN AND HORIZONTAL
.. |boxHU| unicode:: U+02569 .. BOX DRAWINGS DOUBLE UP AND HORIZONTAL
.. |boxHu| unicode:: U+02567 .. BOX DRAWINGS UP SINGLE AND HORIZONTAL DOUBLE
.. |boxhU| unicode:: U+02568 .. BOX DRAWINGS UP DOUBLE AND HORIZONTAL SINGLE
.. |boxhu| unicode:: U+02534 .. BOX DRAWINGS LIGHT UP AND HORIZONTAL
.. |boxUL| unicode:: U+0255D .. BOX DRAWINGS DOUBLE UP AND LEFT
.. |boxUl| unicode:: U+0255C .. BOX DRAWINGS UP DOUBLE AND LEFT SINGLE
.. |boxuL| unicode:: U+0255B .. BOX DRAWINGS UP SINGLE AND LEFT DOUBLE
.. |boxul| unicode:: U+02518 .. BOX DRAWINGS LIGHT UP AND LEFT
.. |boxUR| unicode:: U+0255A .. BOX DRAWINGS DOUBLE UP AND RIGHT
.. |boxUr| unicode:: U+02559 .. BOX DRAWINGS UP DOUBLE AND RIGHT SINGLE
.. |boxuR| unicode:: U+02558 .. BOX DRAWINGS UP SINGLE AND RIGHT DOUBLE
.. |boxur| unicode:: U+02514 .. BOX DRAWINGS LIGHT UP AND RIGHT
.. |boxV|  unicode:: U+02551 .. BOX DRAWINGS DOUBLE VERTICAL
.. |boxv|  unicode:: U+02502 .. BOX DRAWINGS LIGHT VERTICAL
.. |boxVH| unicode:: U+0256C .. BOX DRAWINGS DOUBLE VERTICAL AND HORIZONTAL
.. |boxVh| unicode:: U+0256B .. BOX DRAWINGS VERTICAL DOUBLE AND HORIZONTAL SINGLE
```

```
.. |boxvH| unicode:: U+0256A .. BOX DRAWINGS VERTICAL SINGLE AND HORIZONTAL DOUBLE
.. |boxvh| unicode:: U+0253C .. BOX DRAWINGS LIGHT VERTICAL AND HORIZONTAL
.. |boxVL| unicode:: U+02563 .. BOX DRAWINGS DOUBLE VERTICAL AND LEFT
.. |boxVl| unicode:: U+02562 .. BOX DRAWINGS VERTICAL DOUBLE AND LEFT SINGLE
.. |boxvL| unicode:: U+02561 .. BOX DRAWINGS VERTICAL SINGLE AND LEFT DOUBLE
.. |boxvl| unicode:: U+02524 .. BOX DRAWINGS LIGHT VERTICAL AND LEFT
.. |boxVR| unicode:: U+02560 .. BOX DRAWINGS DOUBLE VERTICAL AND RIGHT
.. |boxVr| unicode:: U+0255F .. BOX DRAWINGS VERTICAL DOUBLE AND RIGHT SINGLE
.. |boxvR| unicode:: U+0255E .. BOX DRAWINGS VERTICAL SINGLE AND RIGHT DOUBLE
.. |boxvr| unicode:: U+0251C .. BOX DRAWINGS LIGHT VERTICAL AND RIGHT
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isocyr1.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Acy|    unicode:: U+00410 .. CYRILLIC CAPITAL LETTER A
.. |acy|    unicode:: U+00430 .. CYRILLIC SMALL LETTER A
.. |Bcy|    unicode:: U+00411 .. CYRILLIC CAPITAL LETTER BE
.. |bcy|    unicode:: U+00431 .. CYRILLIC SMALL LETTER BE
.. |CHcy|   unicode:: U+00427 .. CYRILLIC CAPITAL LETTER CHE
.. |chcy|   unicode:: U+00447 .. CYRILLIC SMALL LETTER CHE
.. |Dcy|    unicode:: U+00414 .. CYRILLIC CAPITAL LETTER DE
.. |dcy|    unicode:: U+00434 .. CYRILLIC SMALL LETTER DE
.. |Ecy|    unicode:: U+0042D .. CYRILLIC CAPITAL LETTER E
.. |ecy|    unicode:: U+0044D .. CYRILLIC SMALL LETTER E
.. |Fcy|    unicode:: U+00424 .. CYRILLIC CAPITAL LETTER EF
.. |fcy|    unicode:: U+00444 .. CYRILLIC SMALL LETTER EF
.. |Gcy|    unicode:: U+00413 .. CYRILLIC CAPITAL LETTER GHE
.. |gcy|    unicode:: U+00433 .. CYRILLIC SMALL LETTER GHE
.. |HARDcy| unicode:: U+0042A .. CYRILLIC CAPITAL LETTER HARD SIGN
.. |hardcy| unicode:: U+0044A .. CYRILLIC SMALL LETTER HARD SIGN
.. |Icy|    unicode:: U+00418 .. CYRILLIC CAPITAL LETTER I
.. |icy|    unicode:: U+00438 .. CYRILLIC SMALL LETTER I
.. |IEcy|   unicode:: U+00415 .. CYRILLIC CAPITAL LETTER IE
.. |iecy|   unicode:: U+00435 .. CYRILLIC SMALL LETTER IE
.. |IOcy|   unicode:: U+00401 .. CYRILLIC CAPITAL LETTER IO
.. |iocy|   unicode:: U+00451 .. CYRILLIC SMALL LETTER IO
.. |Jcy|    unicode:: U+00419 .. CYRILLIC CAPITAL LETTER SHORT I
.. |jcy|    unicode:: U+00439 .. CYRILLIC SMALL LETTER SHORT I
.. |Kcy|    unicode:: U+0041A .. CYRILLIC CAPITAL LETTER KA
.. |kcy|    unicode:: U+0043A .. CYRILLIC SMALL LETTER KA
.. |KHcy|   unicode:: U+00425 .. CYRILLIC CAPITAL LETTER HA
.. |khcy|   unicode:: U+00445 .. CYRILLIC SMALL LETTER HA
.. |Lcy|    unicode:: U+0041B .. CYRILLIC CAPITAL LETTER EL
.. |lcy|    unicode:: U+0043B .. CYRILLIC SMALL LETTER EL
.. |Mcy|    unicode:: U+0041C .. CYRILLIC CAPITAL LETTER EM
.. |mcy|    unicode:: U+0043C .. CYRILLIC SMALL LETTER EM
.. |Ncy|    unicode:: U+0041D .. CYRILLIC CAPITAL LETTER EN
.. |ncy|    unicode:: U+0043D .. CYRILLIC SMALL LETTER EN
.. |numero| unicode:: U+02116 .. NUMERO SIGN
.. |Ocy|    unicode:: U+0041E .. CYRILLIC CAPITAL LETTER O
.. |ocy|    unicode:: U+0043E .. CYRILLIC SMALL LETTER O
.. |Pcy|    unicode:: U+0041F .. CYRILLIC CAPITAL LETTER PE
.. |pcy|    unicode:: U+0043F .. CYRILLIC SMALL LETTER PE
.. |Rcy|    unicode:: U+00420 .. CYRILLIC CAPITAL LETTER ER
.. |rcy|    unicode:: U+00440 .. CYRILLIC SMALL LETTER ER
.. |Scy|    unicode:: U+00421 .. CYRILLIC CAPITAL LETTER ES
.. |scy|    unicode:: U+00441 .. CYRILLIC SMALL LETTER ES
.. |SHCHcy| unicode:: U+00429 .. CYRILLIC CAPITAL LETTER SHCHA
.. |shchcy| unicode:: U+00449 .. CYRILLIC SMALL LETTER SHCHA
.. |SHcy|   unicode:: U+00428 .. CYRILLIC CAPITAL LETTER SHA
.. |shcy|   unicode:: U+00448 .. CYRILLIC SMALL LETTER SHA
.. |SOFTcy| unicode:: U+0042C .. CYRILLIC CAPITAL LETTER SOFT SIGN
.. |softcy| unicode:: U+0044C .. CYRILLIC SMALL LETTER SOFT SIGN
.. |Tcy|    unicode:: U+00422 .. CYRILLIC CAPITAL LETTER TE
.. |tcy|    unicode:: U+00442 .. CYRILLIC SMALL LETTER TE
.. |TScy|   unicode:: U+00426 .. CYRILLIC CAPITAL LETTER TSE
.. |tscy|   unicode:: U+00446 .. CYRILLIC SMALL LETTER TSE
.. |Ucy|    unicode:: U+00423 .. CYRILLIC CAPITAL LETTER U
.. |ucy|    unicode:: U+00443 .. CYRILLIC SMALL LETTER U
.. |Vcy|    unicode:: U+00412 .. CYRILLIC CAPITAL LETTER VE
.. |vcy|    unicode:: U+00432 .. CYRILLIC SMALL LETTER VE
.. |YAcy|   unicode:: U+0042F .. CYRILLIC CAPITAL LETTER YA
.. |yacy|   unicode:: U+0044F .. CYRILLIC SMALL LETTER YA
.. |Ycy|    unicode:: U+0042B .. CYRILLIC CAPITAL LETTER YERU
```

```
.. |ycy|    unicode:: U+0044B .. CYRILLIC SMALL LETTER YERU
.. |YUcy|   unicode:: U+0042E .. CYRILLIC CAPITAL LETTER YU
.. |yucy|   unicode:: U+0044E .. CYRILLIC SMALL LETTER YU
.. |Zcy|    unicode:: U+00417 .. CYRILLIC CAPITAL LETTER ZE
.. |zcy|    unicode:: U+00437 .. CYRILLIC SMALL LETTER ZE
.. |ZHcy|   unicode:: U+00416 .. CYRILLIC CAPITAL LETTER ZHE
.. |zhcy|   unicode:: U+00436 .. CYRILLIC SMALL LETTER ZHE
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isocyr2.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |DJcy|   unicode:: U+00402 .. CYRILLIC CAPITAL LETTER DJE
.. |djcy|   unicode:: U+00452 .. CYRILLIC SMALL LETTER DJE
.. |DScy|   unicode:: U+00405 .. CYRILLIC CAPITAL LETTER DZE
.. |dscy|   unicode:: U+00455 .. CYRILLIC SMALL LETTER DZE
.. |DZcy|   unicode:: U+0040F .. CYRILLIC CAPITAL LETTER DZHE
.. |dzcy|   unicode:: U+0045F .. CYRILLIC SMALL LETTER DZHE
.. |GJcy|   unicode:: U+00403 .. CYRILLIC CAPITAL LETTER GJE
.. |gjcy|   unicode:: U+00453 .. CYRILLIC SMALL LETTER GJE
.. |Iukcy|  unicode:: U+00406 .. CYRILLIC CAPITAL LETTER BYELORUSSIAN-UKRAINIAN I
.. |iukcy|  unicode:: U+00456 .. CYRILLIC SMALL LETTER BYELORUSSIAN-UKRAINIAN I
.. |Jsercy| unicode:: U+00408 .. CYRILLIC CAPITAL LETTER JE
.. |jsercy| unicode:: U+00458 .. CYRILLIC SMALL LETTER JE
.. |Jukcy|  unicode:: U+00404 .. CYRILLIC CAPITAL LETTER UKRAINIAN IE
.. |jukcy|  unicode:: U+00454 .. CYRILLIC SMALL LETTER UKRAINIAN IE
.. |KJcy|   unicode:: U+0040C .. CYRILLIC CAPITAL LETTER KJE
.. |kjcy|   unicode:: U+0045C .. CYRILLIC SMALL LETTER KJE
.. |LJcy|   unicode:: U+00409 .. CYRILLIC CAPITAL LETTER LJE
.. |ljcy|   unicode:: U+00459 .. CYRILLIC SMALL LETTER LJE
.. |NJcy|   unicode:: U+0040A .. CYRILLIC CAPITAL LETTER NJE
.. |njcy|   unicode:: U+0045A .. CYRILLIC SMALL LETTER NJE
.. |TSHcy|  unicode:: U+0040B .. CYRILLIC CAPITAL LETTER TSHE
.. |tshcy|  unicode:: U+0045B .. CYRILLIC SMALL LETTER TSHE
.. |Ubrcy|  unicode:: U+0040E .. CYRILLIC CAPITAL LETTER SHORT U
.. |ubrcy|  unicode:: U+0045E .. CYRILLIC SMALL LETTER SHORT U
.. |YIcy|   unicode:: U+00407 .. CYRILLIC CAPITAL LETTER YI
.. |yicy|   unicode:: U+00457 .. CYRILLIC SMALL LETTER YI
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isodia.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |acute| unicode:: U+000B4 .. ACUTE ACCENT
.. |breve| unicode:: U+002D8 .. BREVE
.. |caron| unicode:: U+002C7 .. CARON
.. |cedil| unicode:: U+000B8 .. CEDILLA
.. |circ|  unicode:: U+002C6 .. MODIFIER LETTER CIRCUMFLEX ACCENT
.. |dblac| unicode:: U+002DD .. DOUBLE ACUTE ACCENT
.. |die|   unicode:: U+000A8 .. DIAERESIS
.. |dot|   unicode:: U+002D9 .. DOT ABOVE
.. |grave| unicode:: U+00060 .. GRAVE ACCENT
.. |macr|  unicode:: U+000AF .. MACRON
.. |ogon|  unicode:: U+002DB .. OGONEK
.. |ring|  unicode:: U+002DA .. RING ABOVE
.. |tilde| unicode:: U+002DC .. SMALL TILDE
.. |uml|   unicode:: U+000A8 .. DIAERESIS
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isogrk1.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Agr|  unicode:: U+00391 .. GREEK CAPITAL LETTER ALPHA
.. |agr|  unicode:: U+003B1 .. GREEK SMALL LETTER ALPHA
.. |Bgr|  unicode:: U+00392 .. GREEK CAPITAL LETTER BETA
```

```
.. |bgr|  unicode:: U+003B2 .. GREEK SMALL LETTER BETA
.. |Dgr|  unicode:: U+00394 .. GREEK CAPITAL LETTER DELTA
.. |dgr|  unicode:: U+003B4 .. GREEK SMALL LETTER DELTA
.. |EEgr| unicode:: U+00397 .. GREEK CAPITAL LETTER ETA
.. |eegr| unicode:: U+003B7 .. GREEK SMALL LETTER ETA
.. |Egr|  unicode:: U+00395 .. GREEK CAPITAL LETTER EPSILON
.. |egr|  unicode:: U+003B5 .. GREEK SMALL LETTER EPSILON
.. |Ggr|  unicode:: U+00393 .. GREEK CAPITAL LETTER GAMMA
.. |ggr|  unicode:: U+003B3 .. GREEK SMALL LETTER GAMMA
.. |Igr|  unicode:: U+00399 .. GREEK CAPITAL LETTER IOTA
.. |igr|  unicode:: U+003B9 .. GREEK SMALL LETTER IOTA
.. |Kgr|  unicode:: U+0039A .. GREEK CAPITAL LETTER KAPPA
.. |kgr|  unicode:: U+003BA .. GREEK SMALL LETTER KAPPA
.. |KHgr| unicode:: U+003A7 .. GREEK CAPITAL LETTER CHI
.. |khgr| unicode:: U+003C7 .. GREEK SMALL LETTER CHI
.. |Lgr|  unicode:: U+0039B .. GREEK CAPITAL LETTER LAMDA
.. |lgr|  unicode:: U+003BB .. GREEK SMALL LETTER LAMDA
.. |Mgr|  unicode:: U+0039C .. GREEK CAPITAL LETTER MU
.. |mgr|  unicode:: U+003BC .. GREEK SMALL LETTER MU
.. |Ngr|  unicode:: U+0039D .. GREEK CAPITAL LETTER NU
.. |ngr|  unicode:: U+003BD .. GREEK SMALL LETTER NU
.. |Ogr|  unicode:: U+0039F .. GREEK CAPITAL LETTER OMICRON
.. |ogr|  unicode:: U+003BF .. GREEK SMALL LETTER OMICRON
.. |OHgr| unicode:: U+003A9 .. GREEK CAPITAL LETTER OMEGA
.. |ohgr| unicode:: U+003C9 .. GREEK SMALL LETTER OMEGA
.. |Pgr|  unicode:: U+003A0 .. GREEK CAPITAL LETTER PI
.. |pgr|  unicode:: U+003C0 .. GREEK SMALL LETTER PI
.. |PHgr| unicode:: U+003A6 .. GREEK CAPITAL LETTER PHI
.. |phgr| unicode:: U+003C6 .. GREEK SMALL LETTER PHI
.. |PSgr| unicode:: U+003A8 .. GREEK CAPITAL LETTER PSI
.. |psgr| unicode:: U+003C8 .. GREEK SMALL LETTER PSI
.. |Rgr|  unicode:: U+003A1 .. GREEK CAPITAL LETTER RHO
.. |rgr|  unicode:: U+003C1 .. GREEK SMALL LETTER RHO
.. |sfgr| unicode:: U+003C2 .. GREEK SMALL LETTER FINAL SIGMA
.. |Sgr|  unicode:: U+003A3 .. GREEK CAPITAL LETTER SIGMA
.. |sgr|  unicode:: U+003C3 .. GREEK SMALL LETTER SIGMA
.. |Tgr|  unicode:: U+003A4 .. GREEK CAPITAL LETTER TAU
.. |tgr|  unicode:: U+003C4 .. GREEK SMALL LETTER TAU
.. |THgr| unicode:: U+00398 .. GREEK CAPITAL LETTER THETA
.. |thgr| unicode:: U+003B8 .. GREEK SMALL LETTER THETA
.. |Ugr|  unicode:: U+003A5 .. GREEK CAPITAL LETTER UPSILON
.. |ugr|  unicode:: U+003C5 .. GREEK SMALL LETTER UPSILON
.. |Xgr|  unicode:: U+0039E .. GREEK CAPITAL LETTER XI
.. |xgr|  unicode:: U+003BE .. GREEK SMALL LETTER XI
.. |Zgr|  unicode:: U+00396 .. GREEK CAPITAL LETTER ZETA
.. |zgr|  unicode:: U+003B6 .. GREEK SMALL LETTER ZETA
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isogrk2.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Aacgr|  unicode:: U+00386 .. GREEK CAPITAL LETTER ALPHA WITH TONOS
.. |aacgr|  unicode:: U+003AC .. GREEK SMALL LETTER ALPHA WITH TONOS
.. |Eacgr|  unicode:: U+00388 .. GREEK CAPITAL LETTER EPSILON WITH TONOS
.. |eacgr|  unicode:: U+003AD .. GREEK SMALL LETTER EPSILON WITH TONOS
.. |EEacgr| unicode:: U+00389 .. GREEK CAPITAL LETTER ETA WITH TONOS
.. |eeacgr| unicode:: U+003AE .. GREEK SMALL LETTER ETA WITH TONOS
.. |Iacgr|  unicode:: U+0038A .. GREEK CAPITAL LETTER IOTA WITH TONOS
.. |iacgr|  unicode:: U+003AF .. GREEK SMALL LETTER IOTA WITH TONOS
.. |idiagr| unicode:: U+00390 .. GREEK SMALL LETTER IOTA WITH DIALYTIKA AND TONOS
.. |Idigr|  unicode:: U+003AA .. GREEK CAPITAL LETTER IOTA WITH DIALYTIKA
.. |idigr|  unicode:: U+003CA .. GREEK SMALL LETTER IOTA WITH DIALYTIKA
.. |Oacgr|  unicode:: U+0038C .. GREEK CAPITAL LETTER OMICRON WITH TONOS
.. |oacgr|  unicode:: U+003CC .. GREEK SMALL LETTER OMICRON WITH TONOS
.. |OHacgr| unicode:: U+0038F .. GREEK CAPITAL LETTER OMEGA WITH TONOS
.. |ohacgr| unicode:: U+003CE .. GREEK SMALL LETTER OMEGA WITH TONOS
.. |Uacgr|  unicode:: U+0038E .. GREEK CAPITAL LETTER UPSILON WITH TONOS
.. |uacgr|  unicode:: U+003CD .. GREEK SMALL LETTER UPSILON WITH TONOS
.. |udiagr| unicode:: U+003B0 .. GREEK SMALL LETTER UPSILON WITH DIALYTIKA AND TONOS
.. |Udigr|  unicode:: U+003AB .. GREEK CAPITAL LETTER UPSILON WITH DIALYTIKA
.. |udigr|  unicode:: U+003CB .. GREEK SMALL LETTER UPSILON WITH DIALYTIKA
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isogrk3.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |alpha|  unicode:: U+003B1 .. GREEK SMALL LETTER ALPHA
.. |beta|   unicode:: U+003B2 .. GREEK SMALL LETTER BETA
.. |chi|    unicode:: U+003C7 .. GREEK SMALL LETTER CHI
.. |Delta|  unicode:: U+00394 .. GREEK CAPITAL LETTER DELTA
.. |delta|  unicode:: U+003B4 .. GREEK SMALL LETTER DELTA
.. |epsi|   unicode:: U+003F5 .. GREEK LUNATE EPSILON SYMBOL
.. |epsis|  unicode:: U+003F5 .. GREEK LUNATE EPSILON SYMBOL
.. |epsiv|  unicode:: U+003B5 .. GREEK SMALL LETTER EPSILON
.. |eta|    unicode:: U+003B7 .. GREEK SMALL LETTER ETA
.. |Gamma|  unicode:: U+00393 .. GREEK CAPITAL LETTER GAMMA
.. |gamma|  unicode:: U+003B3 .. GREEK SMALL LETTER GAMMA
.. |Gammad| unicode:: U+003DC .. GREEK LETTER DIGAMMA
.. |gammad| unicode:: U+003DD .. GREEK SMALL LETTER DIGAMMA
.. |iota|   unicode:: U+003B9 .. GREEK SMALL LETTER IOTA
.. |kappa|  unicode:: U+003BA .. GREEK SMALL LETTER KAPPA
.. |kappav| unicode:: U+003F0 .. GREEK KAPPA SYMBOL
.. |Lambda| unicode:: U+0039B .. GREEK CAPITAL LETTER LAMDA
.. |lambda| unicode:: U+003BB .. GREEK SMALL LETTER LAMDA
.. |mu|     unicode:: U+003BC .. GREEK SMALL LETTER MU
.. |nu|     unicode:: U+003BD .. GREEK SMALL LETTER NU
.. |Omega|  unicode:: U+003A9 .. GREEK CAPITAL LETTER OMEGA
.. |omega|  unicode:: U+003C9 .. GREEK SMALL LETTER OMEGA
.. |Phi|    unicode:: U+003A6 .. GREEK CAPITAL LETTER PHI
.. |phi|    unicode:: U+003D5 .. GREEK PHI SYMBOL
.. |phis|   unicode:: U+003D5 .. GREEK PHI SYMBOL
.. |phiv|   unicode:: U+003C6 .. GREEK SMALL LETTER PHI
.. |Pi|     unicode:: U+003A0 .. GREEK CAPITAL LETTER PI
.. |pi|     unicode:: U+003C0 .. GREEK SMALL LETTER PI
.. |piv|    unicode:: U+003D6 .. GREEK PI SYMBOL
.. |Psi|    unicode:: U+003A8 .. GREEK CAPITAL LETTER PSI
.. |psi|    unicode:: U+003C8 .. GREEK SMALL LETTER PSI
.. |rho|    unicode:: U+003C1 .. GREEK SMALL LETTER RHO
.. |rhov|   unicode:: U+003F1 .. GREEK RHO SYMBOL
.. |Sigma|  unicode:: U+003A3 .. GREEK CAPITAL LETTER SIGMA
.. |sigma|  unicode:: U+003C3 .. GREEK SMALL LETTER SIGMA
.. |sigmav| unicode:: U+003C2 .. GREEK SMALL LETTER FINAL SIGMA
.. |tau|    unicode:: U+003C4 .. GREEK SMALL LETTER TAU
.. |Theta|  unicode:: U+00398 .. GREEK CAPITAL LETTER THETA
.. |theta|  unicode:: U+003B8 .. GREEK SMALL LETTER THETA
.. |thetas| unicode:: U+003B8 .. GREEK SMALL LETTER THETA
.. |thetav| unicode:: U+003D1 .. GREEK THETA SYMBOL
.. |Upsi|   unicode:: U+003D2 .. GREEK UPSILON WITH HOOK SYMBOL
.. |upsi|   unicode:: U+003C5 .. GREEK SMALL LETTER UPSILON
.. |Xi|     unicode:: U+0039E .. GREEK CAPITAL LETTER XI
.. |xi|     unicode:: U+003BE .. GREEK SMALL LETTER XI
.. |zeta|   unicode:: U+003B6 .. GREEK SMALL LETTER ZETA
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isogrk4-wide.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |b.alpha|  unicode:: U+1D6C2 .. MATHEMATICAL BOLD SMALL ALPHA
.. |b.beta|   unicode:: U+1D6C3 .. MATHEMATICAL BOLD SMALL BETA
.. |b.chi|    unicode:: U+1D6D8 .. MATHEMATICAL BOLD SMALL CHI
.. |b.Delta|  unicode:: U+1D6AB .. MATHEMATICAL BOLD CAPITAL DELTA
.. |b.delta|  unicode:: U+1D6C5 .. MATHEMATICAL BOLD SMALL DELTA
.. |b.epsi|   unicode:: U+1D6C6 .. MATHEMATICAL BOLD SMALL EPSILON
.. |b.epsiv|  unicode:: U+1D6DC .. MATHEMATICAL BOLD EPSILON SYMBOL
.. |b.eta|    unicode:: U+1D6C8 .. MATHEMATICAL BOLD SMALL ETA
.. |b.Gamma|  unicode:: U+1D6AA .. MATHEMATICAL BOLD CAPITAL GAMMA
.. |b.gamma|  unicode:: U+1D6C4 .. MATHEMATICAL BOLD SMALL GAMMA
.. |b.Gammad| unicode:: U+003DC .. GREEK LETTER DIGAMMA
.. |b.gammad| unicode:: U+003DD .. GREEK SMALL LETTER DIGAMMA
.. |b.iota|   unicode:: U+1D6CA .. MATHEMATICAL BOLD SMALL IOTA
.. |b.kappa|  unicode:: U+1D6CB .. MATHEMATICAL BOLD SMALL KAPPA
.. |b.kappav| unicode:: U+1D6DE .. MATHEMATICAL BOLD KAPPA SYMBOL
.. |b.Lambda| unicode:: U+1D6B2 .. MATHEMATICAL BOLD CAPITAL LAMDA
.. |b.lambda| unicode:: U+1D6CC .. MATHEMATICAL BOLD SMALL LAMDA
.. |b.mu|     unicode:: U+1D6CD .. MATHEMATICAL BOLD SMALL MU
```

```
.. |b.nu|     unicode:: U+1D6CE .. MATHEMATICAL BOLD SMALL NU
.. |b.Omega|  unicode:: U+1D6C0 .. MATHEMATICAL BOLD CAPITAL OMEGA
.. |b.omega|  unicode:: U+1D6DA .. MATHEMATICAL BOLD SMALL OMEGA
.. |b.Phi|    unicode:: U+1D6BD .. MATHEMATICAL BOLD CAPITAL PHI
.. |b.phi|    unicode:: U+1D6D7 .. MATHEMATICAL BOLD SMALL PHI
.. |b.phiv|   unicode:: U+1D6DF .. MATHEMATICAL BOLD PHI SYMBOL
.. |b.Pi|     unicode:: U+1D6B7 .. MATHEMATICAL BOLD CAPITAL PI
.. |b.pi|     unicode:: U+1D6D1 .. MATHEMATICAL BOLD SMALL PI
.. |b.piv|    unicode:: U+1D6E1 .. MATHEMATICAL BOLD PI SYMBOL
.. |b.Psi|    unicode:: U+1D6BF .. MATHEMATICAL BOLD CAPITAL PSI
.. |b.psi|    unicode:: U+1D6D9 .. MATHEMATICAL BOLD SMALL PSI
.. |b.rho|    unicode:: U+1D6D2 .. MATHEMATICAL BOLD SMALL RHO
.. |b.rhov|   unicode:: U+1D6E0 .. MATHEMATICAL BOLD RHO SYMBOL
.. |b.Sigma|  unicode:: U+1D6BA .. MATHEMATICAL BOLD CAPITAL SIGMA
.. |b.sigma|  unicode:: U+1D6D4 .. MATHEMATICAL BOLD SMALL SIGMA
.. |b.sigmav| unicode:: U+1D6D3 .. MATHEMATICAL BOLD SMALL FINAL SIGMA
.. |b.tau|    unicode:: U+1D6D5 .. MATHEMATICAL BOLD SMALL TAU
.. |b.Theta|  unicode:: U+1D6AF .. MATHEMATICAL BOLD CAPITAL THETA
.. |b.thetas| unicode:: U+1D6C9 .. MATHEMATICAL BOLD SMALL THETA
.. |b.thetav| unicode:: U+1D6DD .. MATHEMATICAL BOLD THETA SYMBOL
.. |b.Upsi|   unicode:: U+1D6BC .. MATHEMATICAL BOLD CAPITAL UPSILON
.. |b.upsi|   unicode:: U+1D6D6 .. MATHEMATICAL BOLD SMALL UPSILON
.. |b.Xi|     unicode:: U+1D6B5 .. MATHEMATICAL BOLD CAPITAL XI
.. |b.xi|     unicode:: U+1D6CF .. MATHEMATICAL BOLD SMALL XI
.. |b.zeta|   unicode:: U+1D6C7 .. MATHEMATICAL BOLD SMALL ZETA
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isogrk4.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |b.Gammad| unicode:: U+003DC .. GREEK LETTER DIGAMMA
.. |b.gammad| unicode:: U+003DD .. GREEK SMALL LETTER DIGAMMA
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isolat1.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of
   Docutils:
   <https://docutils.sourceforge.io>.

.. |Aacute| unicode:: U+000C1 .. LATIN CAPITAL LETTER A WITH ACUTE
.. |aacute| unicode:: U+000E1 .. LATIN SMALL LETTER A WITH ACUTE
.. |Acirc|  unicode:: U+000C2 .. LATIN CAPITAL LETTER A WITH CIRCUMFLEX
.. |acirc|  unicode:: U+000E2 .. LATIN SMALL LETTER A WITH CIRCUMFLEX
.. |AElig|  unicode:: U+000C6 .. LATIN CAPITAL LETTER AE
.. |aelig|  unicode:: U+000E6 .. LATIN SMALL LETTER AE
.. |Agrave| unicode:: U+000C0 .. LATIN CAPITAL LETTER A WITH GRAVE
.. |agrave| unicode:: U+000E0 .. LATIN SMALL LETTER A WITH GRAVE
.. |Aring|  unicode:: U+000C5 .. LATIN CAPITAL LETTER A WITH RING ABOVE
.. |aring|  unicode:: U+000E5 .. LATIN SMALL LETTER A WITH RING ABOVE
.. |Atilde| unicode:: U+000C3 .. LATIN CAPITAL LETTER A WITH TILDE
.. |atilde| unicode:: U+000E3 .. LATIN SMALL LETTER A WITH TILDE
.. |Auml|   unicode:: U+000C4 .. LATIN CAPITAL LETTER A WITH DIAERESIS
.. |auml|   unicode:: U+000E4 .. LATIN SMALL LETTER A WITH DIAERESIS
.. |Ccedil| unicode:: U+000C7 .. LATIN CAPITAL LETTER C WITH CEDILLA
.. |ccedil| unicode:: U+000E7 .. LATIN SMALL LETTER C WITH CEDILLA
.. |Eacute| unicode:: U+000C9 .. LATIN CAPITAL LETTER E WITH ACUTE
.. |eacute| unicode:: U+000E9 .. LATIN SMALL LETTER E WITH ACUTE
.. |Ecirc|  unicode:: U+000CA .. LATIN CAPITAL LETTER E WITH CIRCUMFLEX
.. |ecirc|  unicode:: U+000EA .. LATIN SMALL LETTER E WITH CIRCUMFLEX
.. |Egrave| unicode:: U+000C8 .. LATIN CAPITAL LETTER E WITH GRAVE
.. |egrave| unicode:: U+000E8 .. LATIN SMALL LETTER E WITH GRAVE
.. |ETH|    unicode:: U+000D0 .. LATIN CAPITAL LETTER ETH
.. |eth|    unicode:: U+000F0 .. LATIN SMALL LETTER ETH
.. |Euml|   unicode:: U+000CB .. LATIN CAPITAL LETTER E WITH DIAERESIS
.. |euml|   unicode:: U+000EB .. LATIN SMALL LETTER E WITH DIAERESIS
.. |Iacute| unicode:: U+000CD .. LATIN CAPITAL LETTER I WITH ACUTE
.. |iacute| unicode:: U+000ED .. LATIN SMALL LETTER I WITH ACUTE
.. |Icirc|  unicode:: U+000CE .. LATIN CAPITAL LETTER I WITH CIRCUMFLEX
.. |icirc|  unicode:: U+000EE .. LATIN SMALL LETTER I WITH CIRCUMFLEX
.. |Igrave| unicode:: U+000CC .. LATIN CAPITAL LETTER I WITH GRAVE
.. |igrave| unicode:: U+000EC .. LATIN SMALL LETTER I WITH GRAVE
.. |Iuml|   unicode:: U+000CF .. LATIN CAPITAL LETTER I WITH DIAERESIS
```

```
.. |iuml|   unicode:: U+000EF .. LATIN SMALL LETTER I WITH DIAERESIS
.. |Ntilde| unicode:: U+000D1 .. LATIN CAPITAL LETTER N WITH TILDE
.. |ntilde| unicode:: U+000F1 .. LATIN SMALL LETTER N WITH TILDE
.. |Oacute| unicode:: U+000D3 .. LATIN CAPITAL LETTER O WITH ACUTE
.. |oacute| unicode:: U+000F3 .. LATIN SMALL LETTER O WITH ACUTE
.. |Ocirc|  unicode:: U+000D4 .. LATIN CAPITAL LETTER O WITH CIRCUMFLEX
.. |ocirc|  unicode:: U+000F4 .. LATIN SMALL LETTER O WITH CIRCUMFLEX
.. |Ograve| unicode:: U+000D2 .. LATIN CAPITAL LETTER O WITH GRAVE
.. |ograve| unicode:: U+000F2 .. LATIN SMALL LETTER O WITH GRAVE
.. |Oslash| unicode:: U+000D8 .. LATIN CAPITAL LETTER O WITH STROKE
.. |oslash| unicode:: U+000F8 .. LATIN SMALL LETTER O WITH STROKE
.. |Otilde| unicode:: U+000D5 .. LATIN CAPITAL LETTER O WITH TILDE
.. |otilde| unicode:: U+000F5 .. LATIN SMALL LETTER O WITH TILDE
.. |Ouml|   unicode:: U+000D6 .. LATIN CAPITAL LETTER O WITH DIAERESIS
.. |ouml|   unicode:: U+000F6 .. LATIN SMALL LETTER O WITH DIAERESIS
.. |szlig|  unicode:: U+000DF .. LATIN SMALL LETTER SHARP S
.. |THORN|  unicode:: U+000DE .. LATIN CAPITAL LETTER THORN
.. |thorn|  unicode:: U+000FE .. LATIN SMALL LETTER THORN
.. |Uacute| unicode:: U+000DA .. LATIN CAPITAL LETTER U WITH ACUTE
.. |uacute| unicode:: U+000FA .. LATIN SMALL LETTER U WITH ACUTE
.. |Ucirc|  unicode:: U+000DB .. LATIN CAPITAL LETTER U WITH CIRCUMFLEX
.. |ucirc|  unicode:: U+000FB .. LATIN SMALL LETTER U WITH CIRCUMFLEX
.. |Ugrave| unicode:: U+000D9 .. LATIN CAPITAL LETTER U WITH GRAVE
.. |ugrave| unicode:: U+000F9 .. LATIN SMALL LETTER U WITH GRAVE
.. |Uuml|   unicode:: U+000DC .. LATIN CAPITAL LETTER U WITH DIAERESIS
.. |uuml|   unicode:: U+000FC .. LATIN SMALL LETTER U WITH DIAERESIS
.. |Yacute| unicode:: U+000DD .. LATIN CAPITAL LETTER Y WITH ACUTE
.. |yacute| unicode:: U+000FD .. LATIN SMALL LETTER Y WITH ACUTE
.. |yuml|   unicode:: U+000FF .. LATIN SMALL LETTER Y WITH DIAERESIS
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isolat2.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Abreve| unicode:: U+00102 .. LATIN CAPITAL LETTER A WITH BREVE
.. |abreve| unicode:: U+00103 .. LATIN SMALL LETTER A WITH BREVE
.. |Amacr|  unicode:: U+00100 .. LATIN CAPITAL LETTER A WITH MACRON
.. |amacr|  unicode:: U+00101 .. LATIN SMALL LETTER A WITH MACRON
.. |Aogon|  unicode:: U+00104 .. LATIN CAPITAL LETTER A WITH OGONEK
.. |aogon|  unicode:: U+00105 .. LATIN SMALL LETTER A WITH OGONEK
.. |Cacute| unicode:: U+00106 .. LATIN CAPITAL LETTER C WITH ACUTE
.. |cacute| unicode:: U+00107 .. LATIN SMALL LETTER C WITH ACUTE
.. |Ccaron| unicode:: U+0010C .. LATIN CAPITAL LETTER C WITH CARON
.. |ccaron| unicode:: U+0010D .. LATIN SMALL LETTER C WITH CARON
.. |Ccirc|  unicode:: U+00108 .. LATIN CAPITAL LETTER C WITH CIRCUMFLEX
.. |ccirc|  unicode:: U+00109 .. LATIN SMALL LETTER C WITH CIRCUMFLEX
.. |Cdot|   unicode:: U+0010A .. LATIN CAPITAL LETTER C WITH DOT ABOVE
.. |cdot|   unicode:: U+0010B .. LATIN SMALL LETTER C WITH DOT ABOVE
.. |Dcaron| unicode:: U+0010E .. LATIN CAPITAL LETTER D WITH CARON
.. |dcaron| unicode:: U+0010F .. LATIN SMALL LETTER D WITH CARON
.. |Dstrok| unicode:: U+00110 .. LATIN CAPITAL LETTER D WITH STROKE
.. |dstrok| unicode:: U+00111 .. LATIN SMALL LETTER D WITH STROKE
.. |Ecaron| unicode:: U+0011A .. LATIN CAPITAL LETTER E WITH CARON
.. |ecaron| unicode:: U+0011B .. LATIN SMALL LETTER E WITH CARON
.. |Edot|   unicode:: U+00116 .. LATIN CAPITAL LETTER E WITH DOT ABOVE
.. |edot|   unicode:: U+00117 .. LATIN SMALL LETTER E WITH DOT ABOVE
.. |Emacr|  unicode:: U+00112 .. LATIN CAPITAL LETTER E WITH MACRON
.. |emacr|  unicode:: U+00113 .. LATIN SMALL LETTER E WITH MACRON
.. |ENG|    unicode:: U+0014A .. LATIN CAPITAL LETTER ENG
.. |eng|    unicode:: U+0014B .. LATIN SMALL LETTER ENG
.. |Eogon|  unicode:: U+00118 .. LATIN CAPITAL LETTER E WITH OGONEK
.. |eogon|  unicode:: U+00119 .. LATIN SMALL LETTER E WITH OGONEK
.. |gacute| unicode:: U+001F5 .. LATIN SMALL LETTER G WITH ACUTE
.. |Gbreve| unicode:: U+0011E .. LATIN CAPITAL LETTER G WITH BREVE
.. |gbreve| unicode:: U+0011F .. LATIN SMALL LETTER G WITH BREVE
.. |Gcedil| unicode:: U+00122 .. LATIN CAPITAL LETTER G WITH CEDILLA
.. |gcedil| unicode:: U+00123 .. LATIN SMALL LETTER G WITH CEDILLA
.. |Gcirc|  unicode:: U+0011C .. LATIN CAPITAL LETTER G WITH CIRCUMFLEX
.. |gcirc|  unicode:: U+0011D .. LATIN SMALL LETTER G WITH CIRCUMFLEX
.. |Gdot|   unicode:: U+00120 .. LATIN CAPITAL LETTER G WITH DOT ABOVE
.. |gdot|   unicode:: U+00121 .. LATIN SMALL LETTER G WITH DOT ABOVE
.. |Hcirc|  unicode:: U+00124 .. LATIN CAPITAL LETTER H WITH CIRCUMFLEX
.. |hcirc|  unicode:: U+00125 .. LATIN SMALL LETTER H WITH CIRCUMFLEX
.. |Hstrok| unicode:: U+00126 .. LATIN CAPITAL LETTER H WITH STROKE
.. |hstrok| unicode:: U+00127 .. LATIN SMALL LETTER H WITH STROKE
```

```
.. |Idot|   unicode:: U+00130 .. LATIN CAPITAL LETTER I WITH DOT ABOVE
.. |IJlig|  unicode:: U+00132 .. LATIN CAPITAL LIGATURE IJ
.. |ijlig|  unicode:: U+00133 .. LATIN SMALL LIGATURE IJ
.. |Imacr|  unicode:: U+0012A .. LATIN CAPITAL LETTER I WITH MACRON
.. |imacr|  unicode:: U+0012B .. LATIN SMALL LETTER I WITH MACRON
.. |inodot| unicode:: U+00131 .. LATIN SMALL LETTER DOTLESS I
.. |Iogon|  unicode:: U+0012E .. LATIN CAPITAL LETTER I WITH OGONEK
.. |iogon|  unicode:: U+0012F .. LATIN SMALL LETTER I WITH OGONEK
.. |Itilde| unicode:: U+00128 .. LATIN CAPITAL LETTER I WITH TILDE
.. |itilde| unicode:: U+00129 .. LATIN SMALL LETTER I WITH TILDE
.. |Jcirc|  unicode:: U+00134 .. LATIN CAPITAL LETTER J WITH CIRCUMFLEX
.. |jcirc|  unicode:: U+00135 .. LATIN SMALL LETTER J WITH CIRCUMFLEX
.. |Kcedil| unicode:: U+00136 .. LATIN CAPITAL LETTER K WITH CEDILLA
.. |kcedil| unicode:: U+00137 .. LATIN SMALL LETTER K WITH CEDILLA
.. |kgreen| unicode:: U+00138 .. LATIN SMALL LETTER KRA
.. |Lacute| unicode:: U+00139 .. LATIN CAPITAL LETTER L WITH ACUTE
.. |lacute| unicode:: U+0013A .. LATIN SMALL LETTER L WITH ACUTE
.. |Lcaron| unicode:: U+0013D .. LATIN CAPITAL LETTER L WITH CARON
.. |lcaron| unicode:: U+0013E .. LATIN SMALL LETTER L WITH CARON
.. |Lcedil| unicode:: U+0013B .. LATIN CAPITAL LETTER L WITH CEDILLA
.. |lcedil| unicode:: U+0013C .. LATIN SMALL LETTER L WITH CEDILLA
.. |Lmidot| unicode:: U+0013F .. LATIN CAPITAL LETTER L WITH MIDDLE DOT
.. |lmidot| unicode:: U+00140 .. LATIN SMALL LETTER L WITH MIDDLE DOT
.. |Lstrok| unicode:: U+00141 .. LATIN CAPITAL LETTER L WITH STROKE
.. |lstrok| unicode:: U+00142 .. LATIN SMALL LETTER L WITH STROKE
.. |Nacute| unicode:: U+00143 .. LATIN CAPITAL LETTER N WITH ACUTE
.. |nacute| unicode:: U+00144 .. LATIN SMALL LETTER N WITH ACUTE
.. |napos|  unicode:: U+00149 .. LATIN SMALL LETTER N PRECEDED BY APOSTROPHE
.. |Ncaron| unicode:: U+00147 .. LATIN CAPITAL LETTER N WITH CARON
.. |ncaron| unicode:: U+00148 .. LATIN SMALL LETTER N WITH CARON
.. |Ncedil| unicode:: U+00145 .. LATIN CAPITAL LETTER N WITH CEDILLA
.. |ncedil| unicode:: U+00146 .. LATIN SMALL LETTER N WITH CEDILLA
.. |Odblac| unicode:: U+00150 .. LATIN CAPITAL LETTER O WITH DOUBLE ACUTE
.. |odblac| unicode:: U+00151 .. LATIN SMALL LETTER O WITH DOUBLE ACUTE
.. |OElig|  unicode:: U+00152 .. LATIN CAPITAL LIGATURE OE
.. |oelig|  unicode:: U+00153 .. LATIN SMALL LIGATURE OE
.. |Omacr|  unicode:: U+0014C .. LATIN CAPITAL LETTER O WITH MACRON
.. |omacr|  unicode:: U+0014D .. LATIN SMALL LETTER O WITH MACRON
.. |Racute| unicode:: U+00154 .. LATIN CAPITAL LETTER R WITH ACUTE
.. |racute| unicode:: U+00155 .. LATIN SMALL LETTER R WITH ACUTE
.. |Rcaron| unicode:: U+00158 .. LATIN CAPITAL LETTER R WITH CARON
.. |rcaron| unicode:: U+00159 .. LATIN SMALL LETTER R WITH CARON
.. |Rcedil| unicode:: U+00156 .. LATIN CAPITAL LETTER R WITH CEDILLA
.. |rcedil| unicode:: U+00157 .. LATIN SMALL LETTER R WITH CEDILLA
.. |Sacute| unicode:: U+0015A .. LATIN CAPITAL LETTER S WITH ACUTE
.. |sacute| unicode:: U+0015B .. LATIN SMALL LETTER S WITH ACUTE
.. |Scaron| unicode:: U+00160 .. LATIN CAPITAL LETTER S WITH CARON
.. |scaron| unicode:: U+00161 .. LATIN SMALL LETTER S WITH CARON
.. |Scedil| unicode:: U+0015E .. LATIN CAPITAL LETTER S WITH CEDILLA
.. |scedil| unicode:: U+0015F .. LATIN SMALL LETTER S WITH CEDILLA
.. |Scirc|  unicode:: U+0015C .. LATIN CAPITAL LETTER S WITH CIRCUMFLEX
.. |scirc|  unicode:: U+0015D .. LATIN SMALL LETTER S WITH CIRCUMFLEX
.. |Tcaron| unicode:: U+00164 .. LATIN CAPITAL LETTER T WITH CARON
.. |tcaron| unicode:: U+00165 .. LATIN SMALL LETTER T WITH CARON
.. |Tcedil| unicode:: U+00162 .. LATIN CAPITAL LETTER T WITH CEDILLA
.. |tcedil| unicode:: U+00163 .. LATIN SMALL LETTER T WITH CEDILLA
.. |Tstrok| unicode:: U+00166 .. LATIN CAPITAL LETTER T WITH STROKE
.. |tstrok| unicode:: U+00167 .. LATIN SMALL LETTER T WITH STROKE
.. |Ubreve| unicode:: U+0016C .. LATIN CAPITAL LETTER U WITH BREVE
.. |ubreve| unicode:: U+0016D .. LATIN SMALL LETTER U WITH BREVE
.. |Udblac| unicode:: U+00170 .. LATIN CAPITAL LETTER U WITH DOUBLE ACUTE
.. |udblac| unicode:: U+00171 .. LATIN SMALL LETTER U WITH DOUBLE ACUTE
.. |Umacr|  unicode:: U+0016A .. LATIN CAPITAL LETTER U WITH MACRON
.. |umacr|  unicode:: U+0016B .. LATIN SMALL LETTER U WITH MACRON
.. |Uogon|  unicode:: U+00172 .. LATIN CAPITAL LETTER U WITH OGONEK
.. |uogon|  unicode:: U+00173 .. LATIN SMALL LETTER U WITH OGONEK
.. |Uring|  unicode:: U+0016E .. LATIN CAPITAL LETTER U WITH RING ABOVE
.. |uring|  unicode:: U+0016F .. LATIN SMALL LETTER U WITH RING ABOVE
.. |Utilde| unicode:: U+00168 .. LATIN CAPITAL LETTER U WITH TILDE
.. |utilde| unicode:: U+00169 .. LATIN SMALL LETTER U WITH TILDE
.. |Wcirc|  unicode:: U+00174 .. LATIN CAPITAL LETTER W WITH CIRCUMFLEX
.. |wcirc|  unicode:: U+00175 .. LATIN SMALL LETTER W WITH CIRCUMFLEX
.. |Ycirc|  unicode:: U+00176 .. LATIN CAPITAL LETTER Y WITH CIRCUMFLEX
.. |ycirc|  unicode:: U+00177 .. LATIN SMALL LETTER Y WITH CIRCUMFLEX
.. |Yuml|   unicode:: U+00178 .. LATIN CAPITAL LETTER Y WITH DIAERESIS
.. |Zacute| unicode:: U+00179 .. LATIN CAPITAL LETTER Z WITH ACUTE
.. |zacute| unicode:: U+0017A .. LATIN SMALL LETTER Z WITH ACUTE
.. |Zcaron| unicode:: U+0017D .. LATIN CAPITAL LETTER Z WITH CARON
.. |zcaron| unicode:: U+0017E .. LATIN SMALL LETTER Z WITH CARON
.. |Zdot|   unicode:: U+0017B .. LATIN CAPITAL LETTER Z WITH DOT ABOVE
.. |zdot|   unicode:: U+0017C .. LATIN SMALL LETTER Z WITH DOT ABOVE
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isomfrk-wide.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Afr| unicode:: U+1D504 .. MATHEMATICAL FRAKTUR CAPITAL A
.. |afr| unicode:: U+1D51E .. MATHEMATICAL FRAKTUR SMALL A
.. |Bfr| unicode:: U+1D505 .. MATHEMATICAL FRAKTUR CAPITAL B
.. |bfr| unicode:: U+1D51F .. MATHEMATICAL FRAKTUR SMALL B
.. |Cfr| unicode:: U+0212D .. BLACK-LETTER CAPITAL C
.. |cfr| unicode:: U+1D520 .. MATHEMATICAL FRAKTUR SMALL C
.. |Dfr| unicode:: U+1D507 .. MATHEMATICAL FRAKTUR CAPITAL D
.. |dfr| unicode:: U+1D521 .. MATHEMATICAL FRAKTUR SMALL D
.. |Efr| unicode:: U+1D508 .. MATHEMATICAL FRAKTUR CAPITAL E
.. |efr| unicode:: U+1D522 .. MATHEMATICAL FRAKTUR SMALL E
.. |Ffr| unicode:: U+1D509 .. MATHEMATICAL FRAKTUR CAPITAL F
.. |ffr| unicode:: U+1D523 .. MATHEMATICAL FRAKTUR SMALL F
.. |Gfr| unicode:: U+1D50A .. MATHEMATICAL FRAKTUR CAPITAL G
.. |gfr| unicode:: U+1D524 .. MATHEMATICAL FRAKTUR SMALL G
.. |Hfr| unicode:: U+0210C .. BLACK-LETTER CAPITAL H
.. |hfr| unicode:: U+1D525 .. MATHEMATICAL FRAKTUR SMALL H
.. |Ifr| unicode:: U+02111 .. BLACK-LETTER CAPITAL I
.. |ifr| unicode:: U+1D526 .. MATHEMATICAL FRAKTUR SMALL I
.. |Jfr| unicode:: U+1D50D .. MATHEMATICAL FRAKTUR CAPITAL J
.. |jfr| unicode:: U+1D527 .. MATHEMATICAL FRAKTUR SMALL J
.. |Kfr| unicode:: U+1D50E .. MATHEMATICAL FRAKTUR CAPITAL K
.. |kfr| unicode:: U+1D528 .. MATHEMATICAL FRAKTUR SMALL K
.. |Lfr| unicode:: U+1D50F .. MATHEMATICAL FRAKTUR CAPITAL L
.. |lfr| unicode:: U+1D529 .. MATHEMATICAL FRAKTUR SMALL L
.. |Mfr| unicode:: U+1D510 .. MATHEMATICAL FRAKTUR CAPITAL M
.. |mfr| unicode:: U+1D52A .. MATHEMATICAL FRAKTUR SMALL M
.. |Nfr| unicode:: U+1D511 .. MATHEMATICAL FRAKTUR CAPITAL N
.. |nfr| unicode:: U+1D52B .. MATHEMATICAL FRAKTUR SMALL N
.. |Ofr| unicode:: U+1D512 .. MATHEMATICAL FRAKTUR CAPITAL O
.. |ofr| unicode:: U+1D52C .. MATHEMATICAL FRAKTUR SMALL O
.. |Pfr| unicode:: U+1D513 .. MATHEMATICAL FRAKTUR CAPITAL P
.. |pfr| unicode:: U+1D52D .. MATHEMATICAL FRAKTUR SMALL P
.. |Qfr| unicode:: U+1D514 .. MATHEMATICAL FRAKTUR CAPITAL Q
.. |qfr| unicode:: U+1D52E .. MATHEMATICAL FRAKTUR SMALL Q
.. |Rfr| unicode:: U+0211C .. BLACK-LETTER CAPITAL R
.. |rfr| unicode:: U+1D52F .. MATHEMATICAL FRAKTUR SMALL R
.. |Sfr| unicode:: U+1D516 .. MATHEMATICAL FRAKTUR CAPITAL S
.. |sfr| unicode:: U+1D530 .. MATHEMATICAL FRAKTUR SMALL S
.. |Tfr| unicode:: U+1D517 .. MATHEMATICAL FRAKTUR CAPITAL T
.. |tfr| unicode:: U+1D531 .. MATHEMATICAL FRAKTUR SMALL T
.. |Ufr| unicode:: U+1D518 .. MATHEMATICAL FRAKTUR CAPITAL U
.. |ufr| unicode:: U+1D532 .. MATHEMATICAL FRAKTUR SMALL U
.. |Vfr| unicode:: U+1D519 .. MATHEMATICAL FRAKTUR CAPITAL V
.. |vfr| unicode:: U+1D533 .. MATHEMATICAL FRAKTUR SMALL V
.. |Wfr| unicode:: U+1D51A .. MATHEMATICAL FRAKTUR CAPITAL W
.. |wfr| unicode:: U+1D534 .. MATHEMATICAL FRAKTUR SMALL W
.. |Xfr| unicode:: U+1D51B .. MATHEMATICAL FRAKTUR CAPITAL X
.. |xfr| unicode:: U+1D535 .. MATHEMATICAL FRAKTUR SMALL X
.. |Yfr| unicode:: U+1D51C .. MATHEMATICAL FRAKTUR CAPITAL Y
.. |yfr| unicode:: U+1D536 .. MATHEMATICAL FRAKTUR SMALL Y
.. |Zfr| unicode:: U+02128 .. BLACK-LETTER CAPITAL Z
.. |zfr| unicode:: U+1D537 .. MATHEMATICAL FRAKTUR SMALL Z
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isomfrk.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Cfr| unicode:: U+0212D .. BLACK-LETTER CAPITAL C
.. |Hfr| unicode:: U+0210C .. BLACK-LETTER CAPITAL H
.. |Ifr| unicode:: U+02111 .. BLACK-LETTER CAPITAL I
.. |Rfr| unicode:: U+0211C .. BLACK-LETTER CAPITAL R
.. |Zfr| unicode:: U+02128 .. BLACK-LETTER CAPITAL Z
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isomopf-wide.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Aopf| unicode:: U+1D538 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL A
.. |Bopf| unicode:: U+1D539 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL B
.. |Copf| unicode:: U+02102 .. DOUBLE-STRUCK CAPITAL C
.. |Dopf| unicode:: U+1D53B .. MATHEMATICAL DOUBLE-STRUCK CAPITAL D
.. |Eopf| unicode:: U+1D53C .. MATHEMATICAL DOUBLE-STRUCK CAPITAL E
.. |Fopf| unicode:: U+1D53D .. MATHEMATICAL DOUBLE-STRUCK CAPITAL F
.. |Gopf| unicode:: U+1D53E .. MATHEMATICAL DOUBLE-STRUCK CAPITAL G
.. |Hopf| unicode:: U+0210D .. DOUBLE-STRUCK CAPITAL H
.. |Iopf| unicode:: U+1D540 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL I
.. |Jopf| unicode:: U+1D541 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL J
.. |Kopf| unicode:: U+1D542 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL K
.. |Lopf| unicode:: U+1D543 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL L
.. |Mopf| unicode:: U+1D544 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL M
.. |Nopf| unicode:: U+02115 .. DOUBLE-STRUCK CAPITAL N
.. |Oopf| unicode:: U+1D546 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL O
.. |Popf| unicode:: U+02119 .. DOUBLE-STRUCK CAPITAL P
.. |Qopf| unicode:: U+0211A .. DOUBLE-STRUCK CAPITAL Q
.. |Ropf| unicode:: U+0211D .. DOUBLE-STRUCK CAPITAL R
.. |Sopf| unicode:: U+1D54A .. MATHEMATICAL DOUBLE-STRUCK CAPITAL S
.. |Topf| unicode:: U+1D54B .. MATHEMATICAL DOUBLE-STRUCK CAPITAL T
.. |Uopf| unicode:: U+1D54C .. MATHEMATICAL DOUBLE-STRUCK CAPITAL U
.. |Vopf| unicode:: U+1D54D .. MATHEMATICAL DOUBLE-STRUCK CAPITAL V
.. |Wopf| unicode:: U+1D54E .. MATHEMATICAL DOUBLE-STRUCK CAPITAL W
.. |Xopf| unicode:: U+1D54F .. MATHEMATICAL DOUBLE-STRUCK CAPITAL X
.. |Yopf| unicode:: U+1D550 .. MATHEMATICAL DOUBLE-STRUCK CAPITAL Y
.. |Zopf| unicode:: U+02124 .. DOUBLE-STRUCK CAPITAL Z
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isomopf.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Copf| unicode:: U+02102 .. DOUBLE-STRUCK CAPITAL C
.. |Hopf| unicode:: U+0210D .. DOUBLE-STRUCK CAPITAL H
.. |Nopf| unicode:: U+02115 .. DOUBLE-STRUCK CAPITAL N
.. |Popf| unicode:: U+02119 .. DOUBLE-STRUCK CAPITAL P
.. |Qopf| unicode:: U+0211A .. DOUBLE-STRUCK CAPITAL Q
.. |Ropf| unicode:: U+0211D .. DOUBLE-STRUCK CAPITAL R
.. |Zopf| unicode:: U+02124 .. DOUBLE-STRUCK CAPITAL Z
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isomscr-wide.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Ascr| unicode:: U+1D49C .. MATHEMATICAL SCRIPT CAPITAL A
.. |ascr| unicode:: U+1D4B6 .. MATHEMATICAL SCRIPT SMALL A
.. |Bscr| unicode:: U+0212C .. SCRIPT CAPITAL B
.. |bscr| unicode:: U+1D4B7 .. MATHEMATICAL SCRIPT SMALL B
.. |Cscr| unicode:: U+1D49E .. MATHEMATICAL SCRIPT CAPITAL C
.. |cscr| unicode:: U+1D4B8 .. MATHEMATICAL SCRIPT SMALL C
.. |Dscr| unicode:: U+1D49F .. MATHEMATICAL SCRIPT CAPITAL D
.. |dscr| unicode:: U+1D4B9 .. MATHEMATICAL SCRIPT SMALL D
.. |Escr| unicode:: U+02130 .. SCRIPT CAPITAL E
.. |escr| unicode:: U+0212F .. SCRIPT SMALL E
.. |Fscr| unicode:: U+02131 .. SCRIPT CAPITAL F
.. |fscr| unicode:: U+1D4BB .. MATHEMATICAL SCRIPT SMALL F
.. |Gscr| unicode:: U+1D4A2 .. MATHEMATICAL SCRIPT CAPITAL G
.. |gscr| unicode:: U+0210A .. SCRIPT SMALL G
.. |Hscr| unicode:: U+0210B .. SCRIPT CAPITAL H
.. |hscr| unicode:: U+1D4BD .. MATHEMATICAL SCRIPT SMALL H
.. |Iscr| unicode:: U+02110 .. SCRIPT CAPITAL I
.. |iscr| unicode:: U+1D4BE .. MATHEMATICAL SCRIPT SMALL I
.. |Jscr| unicode:: U+1D4A5 .. MATHEMATICAL SCRIPT CAPITAL J
```

```
.. |jscr| unicode:: U+1D4BF .. MATHEMATICAL SCRIPT SMALL J
.. |Kscr| unicode:: U+1D4A6 .. MATHEMATICAL SCRIPT CAPITAL K
.. |kscr| unicode:: U+1D4C0 .. MATHEMATICAL SCRIPT SMALL K
.. |Lscr| unicode:: U+02112 .. SCRIPT CAPITAL L
.. |lscr| unicode:: U+1D4C1 .. MATHEMATICAL SCRIPT SMALL L
.. |Mscr| unicode:: U+02133 .. SCRIPT CAPITAL M
.. |mscr| unicode:: U+1D4C2 .. MATHEMATICAL SCRIPT SMALL M
.. |Nscr| unicode:: U+1D4A9 .. MATHEMATICAL SCRIPT CAPITAL N
.. |nscr| unicode:: U+1D4C3 .. MATHEMATICAL SCRIPT SMALL N
.. |Oscr| unicode:: U+1D4AA .. MATHEMATICAL SCRIPT CAPITAL O
.. |oscr| unicode:: U+02134 .. SCRIPT SMALL O
.. |Pscr| unicode:: U+1D4AB .. MATHEMATICAL SCRIPT CAPITAL P
.. |pscr| unicode:: U+1D4C5 .. MATHEMATICAL SCRIPT SMALL P
.. |Qscr| unicode:: U+1D4AC .. MATHEMATICAL SCRIPT CAPITAL Q
.. |qscr| unicode:: U+1D4C6 .. MATHEMATICAL SCRIPT SMALL Q
.. |Rscr| unicode:: U+0211B .. SCRIPT CAPITAL R
.. |rscr| unicode:: U+1D4C7 .. MATHEMATICAL SCRIPT SMALL R
.. |Sscr| unicode:: U+1D4AE .. MATHEMATICAL SCRIPT CAPITAL S
.. |sscr| unicode:: U+1D4C8 .. MATHEMATICAL SCRIPT SMALL S
.. |Tscr| unicode:: U+1D4AF .. MATHEMATICAL SCRIPT CAPITAL T
.. |tscr| unicode:: U+1D4C9 .. MATHEMATICAL SCRIPT SMALL T
.. |Uscr| unicode:: U+1D4B0 .. MATHEMATICAL SCRIPT CAPITAL U
.. |uscr| unicode:: U+1D4CA .. MATHEMATICAL SCRIPT SMALL U
.. |Vscr| unicode:: U+1D4B1 .. MATHEMATICAL SCRIPT CAPITAL V
.. |vscr| unicode:: U+1D4CB .. MATHEMATICAL SCRIPT SMALL V
.. |Wscr| unicode:: U+1D4B2 .. MATHEMATICAL SCRIPT CAPITAL W
.. |wscr| unicode:: U+1D4CC .. MATHEMATICAL SCRIPT SMALL W
.. |Xscr| unicode:: U+1D4B3 .. MATHEMATICAL SCRIPT CAPITAL X
.. |xscr| unicode:: U+1D4CD .. MATHEMATICAL SCRIPT SMALL X
.. |Yscr| unicode:: U+1D4B4 .. MATHEMATICAL SCRIPT CAPITAL Y
.. |yscr| unicode:: U+1D4CE .. MATHEMATICAL SCRIPT SMALL Y
.. |Zscr| unicode:: U+1D4B5 .. MATHEMATICAL SCRIPT CAPITAL Z
.. |zscr| unicode:: U+1D4CF .. MATHEMATICAL SCRIPT SMALL Z
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isomscr.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |Bscr| unicode:: U+0212C .. SCRIPT CAPITAL B
.. |Escr| unicode:: U+02130 .. SCRIPT CAPITAL E
.. |escr| unicode:: U+0212F .. SCRIPT SMALL E
.. |Fscr| unicode:: U+02131 .. SCRIPT CAPITAL F
.. |gscr| unicode:: U+0210A .. SCRIPT SMALL G
.. |Hscr| unicode:: U+0210B .. SCRIPT CAPITAL H
.. |Iscr| unicode:: U+02110 .. SCRIPT CAPITAL I
.. |Lscr| unicode:: U+02112 .. SCRIPT CAPITAL L
.. |Mscr| unicode:: U+02133 .. SCRIPT CAPITAL M
.. |oscr| unicode:: U+02134 .. SCRIPT SMALL O
.. |Rscr| unicode:: U+0211B .. SCRIPT CAPITAL R
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isonum.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |amp|    unicode:: U+00026 .. AMPERSAND
.. |apos|   unicode:: U+00027 .. APOSTROPHE
.. |ast|    unicode:: U+0002A .. ASTERISK
.. |brvbar| unicode:: U+000A6 .. BROKEN BAR
.. |bsol|   unicode:: U+0005C .. REVERSE SOLIDUS
.. |cent|   unicode:: U+000A2 .. CENT SIGN
.. |colon|  unicode:: U+0003A .. COLON
.. |comma|  unicode:: U+0002C .. COMMA
.. |commat| unicode:: U+00040 .. COMMERCIAL AT
.. |copy|   unicode:: U+000A9 .. COPYRIGHT SIGN
.. |curren| unicode:: U+000A4 .. CURRENCY SIGN
.. |darr|   unicode:: U+02193 .. DOWNWARDS ARROW
.. |deg|    unicode:: U+000B0 .. DEGREE SIGN
.. |divide| unicode:: U+000F7 .. DIVISION SIGN
.. |dollar| unicode:: U+00024 .. DOLLAR SIGN
.. |equals| unicode:: U+0003D .. EQUALS SIGN
```

```
.. |excl|   unicode:: U+00021 .. EXCLAMATION MARK
.. |frac12| unicode:: U+000BD .. VULGAR FRACTION ONE HALF
.. |frac14| unicode:: U+000BC .. VULGAR FRACTION ONE QUARTER
.. |frac18| unicode:: U+0215B .. VULGAR FRACTION ONE EIGHTH
.. |frac34| unicode:: U+000BE .. VULGAR FRACTION THREE QUARTERS
.. |frac38| unicode:: U+0215C .. VULGAR FRACTION THREE EIGHTHS
.. |frac58| unicode:: U+0215D .. VULGAR FRACTION FIVE EIGHTHS
.. |frac78| unicode:: U+0215E .. VULGAR FRACTION SEVEN EIGHTHS
.. |gt|     unicode:: U+0003E .. GREATER-THAN SIGN
.. |half|   unicode:: U+000BD .. VULGAR FRACTION ONE HALF
.. |horbar| unicode:: U+02015 .. HORIZONTAL BAR
.. |hyphen| unicode:: U+02010 .. HYPHEN
.. |iexcl|  unicode:: U+000A1 .. INVERTED EXCLAMATION MARK
.. |iquest| unicode:: U+000BF .. INVERTED QUESTION MARK
.. |laquo|  unicode:: U+000AB .. LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
.. |larr|   unicode:: U+02190 .. LEFTWARDS ARROW
.. |lcub|   unicode:: U+0007B .. LEFT CURLY BRACKET
.. |ldquo|  unicode:: U+0201C .. LEFT DOUBLE QUOTATION MARK
.. |lowbar| unicode:: U+0005F .. LOW LINE
.. |lpar|   unicode:: U+00028 .. LEFT PARENTHESIS
.. |lsqb|   unicode:: U+0005B .. LEFT SQUARE BRACKET
.. |lsquo|  unicode:: U+02018 .. LEFT SINGLE QUOTATION MARK
.. |lt|     unicode:: U+0003C .. LESS-THAN SIGN
.. |micro|  unicode:: U+000B5 .. MICRO SIGN
.. |middot| unicode:: U+000B7 .. MIDDLE DOT
.. |nbsp|   unicode:: U+000A0 .. NO-BREAK SPACE
.. |not|    unicode:: U+000AC .. NOT SIGN
.. |num|    unicode:: U+00023 .. NUMBER SIGN
.. |ohm|    unicode:: U+02126 .. OHM SIGN
.. |ordf|   unicode:: U+000AA .. FEMININE ORDINAL INDICATOR
.. |ordm|   unicode:: U+000BA .. MASCULINE ORDINAL INDICATOR
.. |para|   unicode:: U+000B6 .. PILCROW SIGN
.. |percnt| unicode:: U+00025 .. PERCENT SIGN
.. |period| unicode:: U+0002E .. FULL STOP
.. |plus|   unicode:: U+0002B .. PLUS SIGN
.. |plusmn| unicode:: U+000B1 .. PLUS-MINUS SIGN
.. |pound|  unicode:: U+000A3 .. POUND SIGN
.. |quest|  unicode:: U+0003F .. QUESTION MARK
.. |quot|   unicode:: U+00022 .. QUOTATION MARK
.. |raquo|  unicode:: U+000BB .. RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
.. |rarr|   unicode:: U+02192 .. RIGHTWARDS ARROW
.. |rcub|   unicode:: U+0007D .. RIGHT CURLY BRACKET
.. |rdquo|  unicode:: U+0201D .. RIGHT DOUBLE QUOTATION MARK
.. |reg|    unicode:: U+000AE .. REGISTERED SIGN
.. |rpar|   unicode:: U+00029 .. RIGHT PARENTHESIS
.. |rsqb|   unicode:: U+0005D .. RIGHT SQUARE BRACKET
.. |rsquo|  unicode:: U+02019 .. RIGHT SINGLE QUOTATION MARK
.. |sect|   unicode:: U+000A7 .. SECTION SIGN
.. |semi|   unicode:: U+0003B .. SEMICOLON
.. |shy|    unicode:: U+000AD .. SOFT HYPHEN
.. |sol|    unicode:: U+0002F .. SOLIDUS
.. |sung|   unicode:: U+0266A .. EIGHTH NOTE
.. |sup1|   unicode:: U+000B9 .. SUPERSCRIPT ONE
.. |sup2|   unicode:: U+000B2 .. SUPERSCRIPT TWO
.. |sup3|   unicode:: U+000B3 .. SUPERSCRIPT THREE
.. |times|  unicode:: U+000D7 .. MULTIPLICATION SIGN
.. |trade|  unicode:: U+02122 .. TRADE MARK SIGN
.. |uarr|   unicode:: U+02191 .. UPWARDS ARROW
.. |verbar| unicode:: U+0007C .. VERTICAL LINE
.. |yen|    unicode:: U+000A5 .. YEN SIGN
```

File: .venv\Lib\site-packages\docutils\parsers\rst\include\isopub.txt

```
.. This data file has been placed in the public domain.
.. Derived from the Unicode character mappings available from
   <http://www.w3.org/2003/entities/xml/>.
   Processed by unicode2rstsubs.py, part of Docutils:
   <https://docutils.sourceforge.io>.

.. |blank|  unicode:: U+02423 .. OPEN BOX
.. |blk12|  unicode:: U+02592 .. MEDIUM SHADE
.. |blk14|  unicode:: U+02591 .. LIGHT SHADE
.. |blk34|  unicode:: U+02593 .. DARK SHADE
.. |block|  unicode:: U+02588 .. FULL BLOCK
.. |bull|   unicode:: U+02022 .. BULLET
.. |caret|  unicode:: U+02041 .. CARET INSERTION POINT
.. |check|  unicode:: U+02713 .. CHECK MARK
.. |cir|    unicode:: U+025CB .. WHITE CIRCLE
.. |clubs|  unicode:: U+02663 .. BLACK CLUB SUIT
```

```
.. |copysr| unicode:: U+02117 .. SOUND RECORDING COPYRIGHT
.. |cross|  unicode:: U+02717 .. BALLOT X
.. |Dagger| unicode:: U+02021 .. DOUBLE DAGGER
.. |dagger| unicode:: U+02020 .. DAGGER
.. |dash|   unicode:: U+02010 .. HYPHEN
.. |diams|  unicode:: U+02666 .. BLACK DIAMOND SUIT
.. |dlcrop| unicode:: U+0230D .. BOTTOM LEFT CROP
.. |drcrop| unicode:: U+0230C .. BOTTOM RIGHT CROP
.. |dtri|   unicode:: U+025BF .. WHITE DOWN-POINTING SMALL TRIANGLE
.. |dtrif|  unicode:: U+025BE .. BLACK DOWN-POINTING SMALL TRIANGLE
.. |emsp|   unicode:: U+02003 .. EM SPACE
.. |emsp13| unicode:: U+02004 .. THREE-PER-EM SPACE
.. |emsp14| unicode:: U+02005 .. FOUR-PER-EM SPACE
.. |ensp|   unicode:: U+02002 .. EN SPACE
.. |female| unicode:: U+02640 .. FEMALE SIGN
.. |ffilig| unicode:: U+0FB03 .. LATIN SMALL LIGATURE FFI
.. |fflig|  unicode:: U+0FB00 .. LATIN SMALL LIGATURE FF
.. |ffllig| unicode:: U+0FB04 .. LATIN SMALL LIGATURE FFL
.. |filig|  unicode:: U+0FB01 .. LATIN SMALL LIGATURE FI
.. |flat|   unicode:: U+0266D .. MUSIC FLAT SIGN
.. |fllig|  unicode:: U+0FB02 .. LATIN SMALL LIGATURE FL
.. |frac13| unicode:: U+02153 .. VULGAR FRACTION ONE THIRD
.. |frac15| unicode:: U+02155 .. VULGAR FRACTION ONE FIFTH
.. |frac16| unicode:: U+02159 .. VULGAR FRACTION ONE SIXTH
.. |frac23| unicode:: U+02154 .. VULGAR FRACTION TWO THIRDS
.. |frac25| unicode:: U+02156 .. VULGAR FRACTION TWO FIFTHS
.. |frac35| unicode:: U+02157 .. VULGAR FRACTION THREE FIFTHS
.. |frac45| unicode:: U+02158 .. VULGAR FRACTION FOUR FIFTHS
.. |frac56| unicode:: U+0215A .. VULGAR FRACTION FIVE SIXTHS
.. |hairsp| unicode:: U+0200A .. HAIR SPACE
.. |hearts| unicode:: U+02665 .. BLACK HEART SUIT
.. |hellip| unicode:: U+02026 .. HORIZONTAL ELLIPSIS
.. |hybull| unicode:: U+02043 .. HYPHEN BULLET
.. |incare| unicode:: U+02105 .. CARE OF
.. |ldquor| unicode:: U+0201E .. DOUBLE LOW-9 QUOTATION MARK
.. |lhblk|  unicode:: U+02584 .. LOWER HALF BLOCK
.. |loz|    unicode:: U+025CA .. LOZENGE
.. |lozf|   unicode:: U+029EB .. BLACK LOZENGE
.. |lsquor| unicode:: U+0201A .. SINGLE LOW-9 QUOTATION MARK
.. |ltri|   unicode:: U+025C3 .. WHITE LEFT-POINTING SMALL TRIANGLE
.. |ltrif|  unicode:: U+025C2 .. BLACK LEFT-POINTING SMALL TRIANGLE
.. |male|   unicode:: U+02642 .. MALE SIGN
.. |malt|   unicode:: U+02720 .. MALTESE CROSS
.. |marker| unicode:: U+025AE .. BLACK VERTICAL RECTANGLE
.. |mdash|  unicode:: U+02014 .. EM DASH
.. |mldr|   unicode:: U+02026 .. HORIZONTAL ELLIPSIS
.. |natur|  unicode:: U+0266E .. MUSIC NATURAL SIGN
.. |ndash|  unicode:: U+02013 .. EN DASH
.. |nldr|   unicode:: U+02025 .. TWO DOT LEADER
.. |numsp|  unicode:: U+02007 .. FIGURE SPACE
.. |phone|  unicode:: U+0260E .. BLACK TELEPHONE
.. |puncsp| unicode:: U+02008 .. PUNCTUATION SPACE
.. |rdquor| unicode:: U+0201D .. RIGHT DOUBLE QUOTATION MARK
.. |rect|   unicode:: U+025AD .. WHITE RECTANGLE
.. |rsquor| unicode:: U+02019 .. RIGHT SINGLE QUOTATION MARK
.. |rtri|   unicode:: U+025B9 .. WHITE RIGHT-POINTING SMALL TRIANGLE
.. |rtrif|  unicode:: U+025B8 .. BLACK RIGHT-POINTING SMALL TRIANGLE
.. |rx|     unicode:: U+0211E .. PRESCRIPTION TAKE
.. |sext|   unicode:: U+02736 .. SIX POINTED BLACK STAR
.. |sharp|  unicode:: U+0266F .. MUSIC SHARP SIGN
.. |spades| unicode:: U+02660 .. BLACK SPADE SUIT
.. |squ|    unicode:: U+025A1 .. WHITE SQUARE
.. |squf|   unicode:: U+025AA .. BLACK SMALL SQUARE
.. |star|   unicode:: U+02606 .. WHITE STAR
.. |starf|  unicode:: U+02605 .. BLACK STAR
.. |target| unicode:: U+02316 .. POSITION INDICATOR
.. |telrec| unicode:: U+02315 .. TELEPHONE RECORDER
.. |thinsp| unicode:: U+02009 .. THIN SPACE
.. |uhblk|  unicode:: U+02580 .. UPPER HALF BLOCK
.. |ulcrop| unicode:: U+0230F .. TOP LEFT CROP
.. |urcrop| unicode:: U+0230E .. TOP RIGHT CROP
.. |utri|   unicode:: U+025B5 .. WHITE UP-POINTING SMALL TRIANGLE
.. |utrif|  unicode:: U+025B4 .. BLACK UP-POINTING SMALL TRIANGLE
.. |vellip| unicode:: U+022EE .. VERTICAL ELLIPSIS
```

Document size limit reached. Remaining files not included.

Summary

Total files processed: 146
Total characters added: 802,914